

Inside the LLM

From Tokens to Systems to Research

Vijey Shrivathsan Vasudevan and Shri Venkatakrisnan Vasudevan

Publication Candidate - September 2026

Contents

Edition and Review Notice	1
Preface	2
How to Use This Book	3
Part 1: What Is This Machine?	4
1 The LLM in One Picture	5
1.1 The machine we are building	5
1.2 1. Prompt: the input context	5
1.3 2. Tokenizer: text becomes integer IDs	5
1.4 3. Embeddings: IDs become vectors	6
1.5 4. Transformer: context changes the representation	6
1.6 5. Logits: one score per vocabulary entry	6
1.7 6. Sampling: choosing is separate from scoring	7
1.8 7. Output tokens: append and repeat	7
1.9 8. What the model predicts	7
1.10 Four ways to follow the same operation	7
1.11 First experiment	8
1.12 What comes next	8
1.13 References	8
2 Training vs Inference	10
2.1 Training versus inference	10
2.2 Parameters versus activations	11
2.3 Loss versus generated text	11
2.4 Learning versus execution	11
2.5 Trace one training step	12
2.6 Four perspectives	12
2.7 Exit check	13
2.8 References	13
3 Anatomy of the Modern LLM Stack	14
3.1 Applications	14
3.2 Agents and RAG in the stack	14
3.3 Inference server	14
3.4 Model runtime	15
3.5 PyTorch and the compiler	15
3.6 Tensor operations	15
3.7 GPU kernels	15
3.8 GPU hardware	16
3.9 Memory	16

3.10	Silicon	16
3.11	Follow one request	16
3.12	Diagnostic exercise	16
3.13	Four perspectives	17
3.14	References	17
4	Building Our Tiny Laboratory	18
4.1	Python	18
4.2	NumPy	18
4.3	PyTorch	19
4.4	Notebooks	19
4.5	Profiling tools	19
4.6	CPU baseline	19
4.7	Seeds and environment records	20
4.8	Minimal experiment protocol	20
4.9	Four perspectives	20
4.10	Exit artifact	20
4.11	References	21
Part 2:	Tensors, Math and Computation	22
5	Tensors and Resource Accounting	23
5.1	Scalars	23
5.2	Vectors	23
5.3	Matrices	23
5.4	Tensors	23
5.5	Shapes	23
5.6	Dimensions	24
5.7	Indexing	24
5.8	Transpose	24
5.9	Batch dimensions	24
5.10	Broadcasting	24
5.11	Dtypes	24
5.12	Element counts	24
5.13	Bytes	25
5.14	FLOPs	25
5.15	Matrix multiplication	25
5.16	Resource ledger	25
5.17	Exit problem	25
5.18	References	25
6	Geometry and Number Formats	27
6.1	Dot products	27
6.2	Cosine similarity	27
6.3	Magnitude	27
6.4	Basis	27
6.5	Subspaces	27
6.6	Orthogonality	28
6.7	Projections	28
6.8	Linear transformations	28
6.9	FP32	28
6.10	FP16	28
6.11	BF16	28
6.12	FP8	28
6.13	INT8	29
6.14	INT4	29

6.15	Overflow	29
6.16	Underflow	29
6.17	Numerical stability	29
6.18	Exit problem	29
6.19	References	29
7	Probability, Logits and Loss	30
7.1	Conditional probability	30
7.2	Likelihood	30
7.3	Log-probability	30
7.4	Entropy	30
7.5	KL divergence	31
7.6	Softmax	31
7.7	Logits	31
7.8	Cross-entropy	31
7.9	Log-sum-exp	31
7.10	Worked example	31
7.11	Exit derivation	31
7.12	References	32
8	Gradients, Graphs and Autodiff	33
8.1	Derivatives	33
8.2	Gradients	33
8.3	Chain rule	33
8.4	Computational graphs	33
8.5	Automatic differentiation	34
8.6	Reverse-mode autodiff	34
8.7	Finite-difference checks	34
8.8	The micro-autograd engine	34
8.9	Four perspectives	34
8.10	Exit check	34
8.11	References	35
Part 3:	Neural Networks Before Transformers	36
9	Linear Layers and Nonlinearities	37
9.1	Linear layers	37
9.2	Biases	37
9.3	Nonlinearities	37
9.4	ReLU	37
9.5	GELU	38
9.6	Parameter shapes and cost	38
9.7	Four perspectives	38
9.8	Exit check	38
9.9	A nonlinear test case	38
9.10	References	38
10A	Tiny Network, Forward and Backward	39
10.1	Building a tiny neural network	39
10.2	Forward propagation	39
10.3	Backpropagation by hand	39
10.4	Gradient shapes	40
10.5	Four perspectives	40
10.6	Exit check	40
10.7	Implementation discipline	40
10.8	References	40

11 Optimization and State	41
11.1 SGD	41
11.2 Momentum	41
11.3 AdamW	41
11.4 Optimizer states	41
11.5 Learning-rate schedules	42
11.6 Weight decay	42
11.7 Four perspectives	42
11.8 Exit check	42
11.9 Ordering one training step	42
11.10 References	42
12 Initialization and Training Stability	43
12.1 Initialization	43
12.2 Gradient norms	43
12.3 Clipping	43
12.4 Warmup	43
12.5 Vanishing gradients	44
12.6 Exploding gradients	44
12.7 Stability diagnostics	44
12.8 Four perspectives	44
12.9 Exit check	44
12.10 References	44
Part 4: Turning Language Into Numbers	45
13 Tokenization Algorithms	46
13.1 Why machines need tokens	46
13.2 Characters	46
13.3 Words	46
13.4 Subwords	46
13.5 BPE from scratch	47
13.6 WordPiece	47
13.7 SentencePiece	47
13.8 Byte-level tokenization	47
13.9 Four perspectives	47
13.10 Exit check	47
13.11 References	48
14 Vocabulary and Token IDs	49
14.1 Vocabulary	49
14.2 Token IDs	49
14.3 Special tokens	49
14.4 Unknown tokens	50
14.5 Tokenizer-model compatibility	50
14.6 Four perspectives	50
14.7 Exit check	50
14.8 References	50
15 Embedding Matrices and Geometry	51
15.1 Embedding matrices	51
15.2 Static embeddings	51
15.3 Contextual embeddings	51
15.4 Representation geometry	51
15.5 Embedding lookup and gradients	52
15.6 Four perspectives	52
15.7 Exit check	52

15.8References	52
16Position and Sequence Order	53
16.1Position and sequence order	53
16.2Sinusoidal positions	53
16.3Learned positions	53
16.4Relative-position methods	53
16.5RoPE introduction	54
16.6Worked invariance check	54
16.7Four perspectives	54
16.8Exit check	54
16.9References	54
Part 5: Attention From First Principles	55
17Sequence Modeling and the Attention Idea	56
17.1Sequence modeling bottlenecks	56
17.2RNNs	56
17.3LSTMs	56
17.4The attention idea	57
17.5Content-based retrieval	57
17.6Four perspectives	57
17.7Exit check	57
17.8References	57
18Queries, Keys, Values and Scaling	58
18.1Query	58
18.2Key	58
18.3Value	58
18.4QKV projections	58
18.5Scaled dot-product attention	58
18.6Square-root scaling	59
18.7Attention weights	59
18.8Four perspectives	59
18.9Exit check	59
18.10References	59
19Causal and Multi-Head Attention	61
19.1Causal attention	61
19.2Causal masks	61
19.3Multi-head attention	61
19.4Head concatenation	62
19.5Output projection	62
19.6Worked shape trace	62
19.7Four perspectives	62
19.8Exit check	62
19.9References	62
20Attention Shapes and Compute Cost	63
20.1Attention tensor shapes	63
20.2Attention compute cost	63
20.3Quadratic sequence cost	63
20.4Attention memory accounting	64
20.5Four perspectives	64
20.6Exit check	64
20.7References	64

Part 6: Build a Transformer	65
21 Residual Streams and Normalization	66
21.1 Learning outcome	66
21.2 Prerequisites	66
21.3 Worked implementation	68
21.4 Four recurring perspectives	68
21.5 Lab and exit check	68
21.6 Exercises	69
21.7 References	69
22 Feed-Forward Networks and Complete Blocks	70
22.1 Learning outcome	70
22.2 Prerequisites	70
22.3 Failure cases	72
22.4 Four recurring perspectives	72
22.5 Lab and exit check	72
22.6 Exercises	72
22.7 References	72
23 Stacking, Vocabulary Projection and Tying	73
23.1 Learning outcome	73
23.2 Prerequisites	73
23.3 Worked trace	74
23.4 Four recurring perspectives	74
23.5 Lab and exit check	75
23.6 Exercises	75
23.7 References	75
24 Build Our GPT From Scratch	76
24.1 Learning outcome	76
24.2 Prerequisites	76
24.3 Parameter and compute sanity check	77
24.4 Four recurring perspectives	77
24.5 Lab and exit check	77
24.6 Exercises	78
24.7 References	78
Part 7: Train the Thing	79
25 Sequences, Batches and Pretraining	80
25.1 Learning outcome	80
25.2 Prerequisites	80
25.3 Four recurring perspectives	82
25.4 Lab and exit check	82
25.5 Exercises	82
25.6 References	82
26 The Training Step and Schedule	83
26.1 Learning outcome	83
26.2 Prerequisites	83
26.3 Canonical step order	85
26.4 Four recurring perspectives	85
26.5 Lab and exit check	85
26.6 Exercises	85
26.7 References	86

27 Mixed Precision and Checkpoint Anatomy	87
27.1 Learning outcome	87
27.2 Prerequisites	87
27.3 Memory example	89
27.4 Four recurring perspectives	89
27.5 Lab and exit check	89
27.6 Exercises	89
27.7 References	89
28 Debugging and Training Our Tiny Model	90
28.1 Learning outcome	90
28.2 Prerequisites	90
28.3 Failure ledger	91
28.4 Four recurring perspectives	91
28.5 Lab and exit check	92
28.6 Exercises	92
28.7 References	92
Part 8: Modern LLM Architecture	93
29 Transformer Families	94
29.1 Learning outcome	94
29.2 Prerequisites	94
29.3 Comparison	95
29.4 Four recurring perspectives	96
29.5 Lab and exit check	96
29.6 Exercises	96
29.7 References	96
30 Modern Positions, Heads and MLPs	97
30.1 Learning outcome	97
30.2 Prerequisites	97
30.3 Four recurring perspectives	98
30.4 Lab and exit check	99
30.5 Exercises	99
30.6 References	99
31 Mixture of Experts	100
31.1 Learning outcome	100
31.2 Prerequisites	100
31.3 Four recurring perspectives	102
31.4 Lab and exit check	102
31.5 Exercises	102
31.6 References	102
32 Alternative Architectures and Real Configs	103
32.1 Learning outcome	103
32.2 Prerequisites	103
32.3 Four recurring perspectives	105
32.4 Lab and exit check	106
32.5 Exercises	106
32.6 References	106
Part 9: Data, Scaling and Evaluation	107
33 Data Sources and Cleaning	108
33.1 Learning outcome	108

33.2	Prerequisites	108
33.3	Training data sources	108
33.4	Cleaning	108
33.5	Filtering	109
33.6	Deduplication	109
33.7	Dataset provenance	109
33.8	Four recurring perspectives	110
33.9	Lab and exit check	110
33.10	Exercises	110
33.11	References	110
34	Mixtures, Tokens and Synthetic Data	111
34.1	Learning outcome	111
34.2	Prerequisites	111
34.3	Dataset mixtures	111
34.4	Token budgets	111
34.5	Synthetic data	112
34.6	Data quality	112
34.7	Contamination	112
34.8	Split design	112
34.9	Four recurring perspectives	112
34.10	Lab and exit check	113
34.11	Exercises	113
34.12	References	113
35	Evaluation and Meaningful Experiments	114
35.1	Learning outcome	114
35.2	Prerequisites	114
35.3	Evaluation	114
35.4	Perplexity	114
35.5	Downstream tasks	115
35.6	Reasoning benchmarks	115
35.7	Human evaluation	115
35.8	Experimental design	115
35.9	Uncertainty introduction	115
35.10	Four recurring perspectives	116
35.11	Lab and exit check	116
35.12	Exercises	116
35.13	References	116
36	Scaling Laws	117
36.1	Learning outcome	117
36.2	Prerequisites	117
36.3	Scaling laws	117
36.4	Parameters vs data vs compute	117
36.5	Compute budgets	118
36.6	Scaling fits	118
36.7	Extrapolation limits	118
36.8	Four recurring perspectives	118
36.9	Lab and exit check	118
36.10	Exercises	118
36.11	References	119
Part 10	Post-Training and Reasoning	120
37	Pretraining, Continued Pretraining and SFT	121
37.1	Learning outcome	121

37.2 Prerequisites	121
37.3 Pretraining vs post-training	121
37.4 Continued pretraining	122
37.5 Supervised fine-tuning	122
37.6 Chat templates	122
37.7 Loss masking	122
37.8 Four recurring perspectives	123
37.9 Lab and exit check	123
37.10 Exercises	123
37.11 References	123
38 Parameter-Efficient Adaptation	124
38.1 Learning outcome	124
38.2 Prerequisites	124
38.3 PEFT	124
38.4 Adapters	124
38.5 Prompt tuning	125
38.6 Prefix tuning	125
38.7 LoRA	125
38.8 QLoRA	125
38.9 Memory worked example	125
38.10 Four recurring perspectives	126
38.11 Lab and exit check	126
38.12 Exercises	126
38.13 References	126
39 Preferences and Reinforcement Learning	127
39.1 Learning outcome	127
39.2 Prerequisites	127
39.3 Preference data	127
39.4 Reward models	127
39.5 RLHF	128
39.6 PPO	128
39.7 DPO-type objectives	128
39.8 Policy KL control	128
39.9 Four recurring perspectives	129
39.10 Lab and exit check	129
39.11 Exercises	129
39.12 References	129
40 Reasoning, Test-Time Compute and Distillation	130
40.1 Learning outcome	130
40.2 Prerequisites	130
40.3 RL with verifiable rewards	130
40.4 Reasoning models	130
40.5 Why generation can produce reasoning-like behavior	131
40.6 Test-time compute	131
40.7 Distillation	131
40.8 Verifier limits	131
40.9 Four recurring perspectives	132
40.10 Lab and exit check	132
40.11 Exercises	132
40.12 References	132
Part 11: GPU Fundamentals	133

41 Why GPUs and How They Execute	134
41.1 Learning outcome	134
41.2 CPU vs GPU	134
41.3 Why GPUs	134
41.4 GPU anatomy	134
41.5 SMs	135
41.6 Threads	135
41.7 Warps	135
41.8 Blocks	135
41.9 CUDA cores	136
41.10 Tensor Cores	136
41.11 Registers	136
41.12 Four recurring perspectives	136
41.13 Lab and exit check	136
41.14 References	136
42 Memory Hierarchy and Movement	137
42.1 Learning outcome	137
42.2 Memory hierarchy	137
42.3 Shared memory and SRAM	137
42.4 Caches	137
42.5 HBM	138
42.6 RAM	138
42.7 SSD	138
42.8 HBM bandwidth	138
42.9 Coalescing	138
42.10 Four recurring perspectives	139
42.11 Lab and exit check	139
42.12 References	139
43 Throughput, Intensity and Rooflines	140
43.1 Learning outcome	140
43.2 Compute throughput	140
43.3 Arithmetic intensity	140
43.4 Compute-bound vs memory-bound	141
43.5 Roofline model	141
43.6 Four recurring perspectives	141
43.7 Lab and exit check	142
43.8 References	142
44 Profiling Transformer Bottlenecks	143
44.1 Learning outcome	143
44.2 Decode memory bottlenecks	143
44.3 FLOPs and memory profiling	143
44.4 Bandwidth utilization	144
44.5 Latency measurement	144
44.6 Synchronization	144
44.7 Four recurring perspectives	145
44.8 Lab and exit check	145
44.9 References	145
Part 12: Kernels and the Software Stack	146
45 Kernels, GEMM and Tiling	147
45.1 Prerequisites and learning outcomes	147
45.2 GPU kernels	147
45.3 GEMM	147

45.4	Tiling creates reuse	148
45.5	Tensor Core MMA	148
45.6	Kernel launch overhead	148
45.7	Kernel fusion	148
45.8	Four perspectives and exit check	148
45.9	Primary references	148
46	CUDA, Triton and CUTLASS	150
46.1	Prerequisites and learning outcomes	150
46.2	CUDA: explicit control	150
46.3	Triton: blocked programs	150
46.4	CUTLASS-level concepts	150
46.5	Simple Triton kernel	150
46.6	GPU profiling	151
46.7	Four perspectives, failures, and exit check	151
46.8	Primary references	151
47	Graphs, Compilers and Runtimes	152
47.1	From Python to device code	152
47.2	PyTorch eager execution	152
47.3	Computational graph capture	152
47.4	TorchDynamo	152
47.5	Intermediate representation	152
47.6	torch.compile	153
47.7	Inductor	153
47.8	Generated kernels	153
47.9	Compiler graph breaks	153
47.10	Four perspectives and exit check	153
47.11	Primary references	153
48	CUDA Graphs and Execution Overhead	155
48.1	Why a fast kernel can make a slow program	155
48.2	CUDA Graphs	155
48.3	Dispatch overhead	155
48.4	Graph replay	155
48.5	Shape specialization	156
48.6	Compilation amortization	156
48.7	Measurement, perspectives, and exit check	156
48.8	A concrete bucket policy	156
48.9	Primary references	156
Part 13	LLM Inference From First Principles	157
49	Load Weights and Trace a Forward Pass	158
49.1	Goal and notation	158
49.2	Loading model weights	158
49.3	Prompt processing	158
49.4	GPU inference	159
49.5	End-to-end real-model forward-pass tracing	159
49.6	Four perspectives and debugging	159
49.7	Primary references	160
50	Prefill, Decode and KV Cache	161
50.1	Two phases of one computation	161
50.2	Prefill	161
50.3	Autoregressive decode	161
50.4	KV cache	161

50.5	Why K and V, but not historical Q?	162
50.6	Context length	162
50.7	Correctness lab	162
50.8	Four perspectives and failures	162
50.9	Primary references	162
51	Sampling and Decoding Policies	163
51.1	From logits to a sequence	163
51.2	Sampling	163
51.3	Greedy decoding	163
51.4	Temperature	163
51.5	Top-k	164
51.6	Top-p	164
51.7	Beam search	164
51.8	Stopping conditions	164
51.9	Worked comparison and four perspectives	164
51.10	Primary references	165
52	Inference Memory and Latency Accounting	166
52.1	Build a budget before measuring	166
52.2	Inference memory accounting	166
52.3	Weights plus KV plus activations plus overhead	166
52.4	Time to first token	166
52.5	Inter-token latency	167
52.6	TPOT	167
52.7	Throughput	167
52.8	Memory per token	167
52.9	FLOPs per token	167
52.10	Measurement protocol and four perspectives	167
52.11	Primary references	168
Part 14	Making Inference Fast	169
53	FlashAttention and Paged KV Memory	170
53.1	Two different memory problems	170
53.2	Why naive inference is slow	170
53.3	FlashAttention	170
53.4	Online softmax and IO scheduling	170
53.5	KV cache layout	171
53.6	Paged KV memory	171
53.7	PagedAttention	171
53.8	Four perspectives and exit check	171
53.9	Primary references	171
54	Caching, Batching and Prefill Scheduling	173
54.1	The scheduler is part of model performance	173
54.2	Prefix caching	173
54.3	Prefix reuse	173
54.4	Continuous batching	173
54.5	Chunked prefill	174
54.6	Cache correctness	174
54.7	Simulation and four perspectives	174
54.8	Primary references	174
55	Speculative Decoding and Dynamic Execution	176
55.1	Spend less target-model time per accepted token	176
55.2	Speculative decoding	176

55.3	Draft-verifier design	176
55.4	Speculative-model architectures	177
55.5	Sparsity	177
55.6	Pruning	177
55.7	Early exit	177
55.8	Layer skipping	177
55.9	CUDA Graph optimization	177
55.10	Exactness, perspectives, and exit check	177
55.11	Primary references	178
56	Quantization Across the Inference Path	179
56.1	Representing useful signal with fewer bits	179
56.2	Quantization error and a worked matrix	179
56.3	Post-training quantization	179
56.4	Quantization-aware training	179
56.5	GPTQ	180
56.6	AWQ	180
56.7	Weight quantization	180
56.8	Activation quantization	180
56.9	KV quantization	180
56.10	FP8 inference	180
56.11	INT8 inference	180
56.12	INT4 inference	181
56.13	GUF formats	181
56.14	Four perspectives, validation, and exit check	181
56.15	Primary references	181
Part 15	Distributed LLMs	182
57	Parallel Training Strategies	183
57.1	Learning outcome	183
57.2	Prerequisites	183
57.3	Why one accelerator stops being enough	183
57.4	Data parallelism	183
57.5	DDP	184
57.6	ZeRO	184
57.7	FSDP	184
57.8	Sharded optimizer state	184
57.9	Four recurring perspectives	185
57.10	Lab and exit check	185
57.11	Primary references	185
58	Partitioning Model Computation	186
58.1	Learning outcome	186
58.2	Prerequisites	186
58.3	Tensor parallelism	186
58.4	Pipeline parallelism	186
58.5	Sequence parallelism	187
58.6	Context parallelism	187
58.7	Expert parallelism	187
58.8	Four recurring perspectives	187
58.9	Lab and exit check	188
58.10	Primary references	188
59	Collectives and Interconnects	189
59.1	Learning outcome	189
59.2	Prerequisites	189

59.3 Broadcast	189
59.4 All-reduce	189
59.5 All-gather	189
59.6 Reduce-scatter	190
59.7 All-to-all	190
59.8 NCCL	190
59.9 PCIe	190
59.10 NVLink	190
59.11 NVSwitch	190
59.12 InfiniBand	191
59.13 Four recurring perspectives	191
59.14 Lab and exit check	191
59.15 Primary references	191
60 Distributed Execution and Failure Recovery	192
60.1 Learning outcome	192
60.2 Prerequisites	192
60.3 Distributed training failures	192
60.4 Distributed inference	192
60.5 Stragglers	193
60.6 Multi-GPU execution	193
60.7 Checkpoint recovery	193
60.8 Four recurring perspectives	193
60.9 Lab and exit check	194
60.10 Primary references	194
Part 16: Serving LLMs as a System	195
61 Anatomy of an Inference Server	196
61.1 Learning outcome	196
61.2 Prerequisites	196
61.3 Inference server architecture	196
61.4 HTTP requests	196
61.5 Request queues	197
61.6 Scheduling	197
61.7 Dynamic batching	197
61.8 KV allocation	197
61.9 Streaming	198
61.10 Four recurring perspectives	198
61.11 Lab and exit check	198
61.12 Primary references	198
62 Tenants, Models and Residency	199
62.1 Learning outcome	199
62.2 Prerequisites	199
62.3 Multi-tenant serving	199
62.4 Multi-model serving	199
62.5 Memory residency	200
62.6 Model loading	200
62.7 Model offloading	200
62.8 GPU-RAM-SSD hierarchy	200
62.9 Four recurring perspectives	201
62.10 Lab and exit check	201
62.11 Primary references	201
63 Disaggregation and Runtime Comparisons	202
63.1 Learning outcome	202

63.2	Prerequisites	202
63.3	Prefill/decode disaggregation	202
63.4	vLLM internals	202
63.5	TensorRT-LLM internals	203
63.6	llama.cpp architecture comparison	203
63.7	Transfer overhead	203
63.8	Four recurring perspectives	203
63.9	Lab and exit check	204
63.10	Primary references	204
64	Production Serving and GPU Clusters	205
64.1	Learning outcome	205
64.2	Prerequisites	205
64.3	Throughput vs latency	205
64.4	Tokens per second	205
64.5	Requests per second	206
64.6	p95 and p99 latency	206
64.7	Serving SLOs	206
64.8	Observability	206
64.9	Autoscaling	206
64.10	Failure recovery	206
64.11	GPU cluster scheduling	207
64.12	Four recurring perspectives	207
64.13	Lab and exit check	207
64.14	Primary references	207
Part 17	LLMs on Your Laptop	208
65	CPU and Edge Inference	209
65.1	Learning outcome	209
65.2	Prerequisites	209
65.3	Edge constraints	209
65.4	CPU inference	209
65.5	SIMD	210
65.6	CPU vectorization	210
65.7	CPU threading	210
65.8	Four recurring perspectives	210
65.9	Lab and exit check	211
65.10	Primary references	211
66	Hybrid Memory and Local Loading	212
66.1	Learning outcome	212
66.2	Prerequisites	212
66.3	GPU offloading	212
66.4	CPU-GPU hybrid inference	212
66.5	Unified memory	213
66.6	Shared memory	213
66.7	Quantized formats	213
66.8	GGUF	213
66.9	Memory mapping	213
66.10	Startup and model loading	214
66.11	Four recurring perspectives	214
66.12	Lab and exit check	214
66.13	Primary references	214
67	Local Runtimes and Accelerators	215
67.1	Learning outcome	215

67.2	Prerequisites	215
67.3	llama.cpp internals	215
67.4	OpenVINO	215
67.5	NPUs	216
67.6	Accelerator backends	216
67.7	Kernel coverage	216
67.8	Four recurring perspectives	216
67.9	Lab and exit check	217
67.10	Primary references	217
68	Packaging, Power and Thermal Limits	218
68.1	Learning outcome	218
68.2	Prerequisites	218
68.3	Power limits	218
68.4	Thermal limits	218
68.5	Model packaging	219
68.6	Offline deployment	219
68.7	Edge distillation	219
68.8	Dynamic edge computation	219
68.9	Four recurring perspectives	220
68.10	Lab and exit check	220
68.11	Primary references	220
Part 18:	Retrieval, Memory and Agents	221
69	Embeddings, Contrastive Learning and Vector Indexes	222
69.1	Embeddings revisited	222
69.2	Sentence embeddings	222
69.3	Document embeddings	222
69.4	Contrastive embedding training	223
69.5	Vector search	223
69.6	Approximate nearest neighbors	223
69.7	HNSW	223
69.8	IVF	223
69.9	Product quantization	224
69.10	Vector databases	224
69.11	Four perspectives	224
69.12	Lab and exit check	224
69.13	References	224
70	RAG and Retrieval Quality	226
70.1	RAG	226
70.2	Chunking strategies	226
70.3	Retrieval strategies	226
70.4	Reranking	227
70.5	Retrieval evaluation	227
70.6	Grounding	227
70.7	Four perspectives	227
70.8	Lab and exit check	228
70.9	References	228
71	Memory, Tools and Structured Output	229
71.1	Short-term memory	229
71.2	Persistent memory	229
71.3	Long-term memory	229
71.4	Tool calling	230
71.5	Structured outputs	230

71.6	Constrained decoding	230
71.7	Memory invalidation	230
71.8	Four perspectives	230
71.9	Lab and exit check	231
71.10	References	231
72	Agents and Multi-Model Economics	232
72.1	Agents	232
72.2	Planning	232
72.3	Multi-agent systems	232
72.4	Multi-LLM systems	233
72.5	Agent compute economics	233
72.6	Routing and fallback	233
72.7	Four perspectives	233
72.8	Lab and exit check	233
72.9	References	234
Part 19:	Inside the Model: Representations and Intervention	235
73	Hidden States, Layers and Attention Heads	236
73.1	Hidden states	236
73.2	Residual stream analysis	236
73.3	Representation geometry analysis	237
73.4	What individual layers learn	237
73.5	Layer specialization	237
73.6	Attention-head analysis	237
73.7	Worked measurement protocol	237
73.8	Four perspectives	238
73.9	Lab and exit check	238
73.10	References	238
74	Features, Probes and Mechanisms	239
74.1	Neurons	239
74.2	Features	239
74.3	Probing	239
74.4	Sparse autoencoders	239
74.5	Mechanistic interpretability	240
74.6	Causal interventions	240
74.7	Evidence ladder	240
74.8	Four perspectives	240
74.9	Lab and exit check	241
74.10	References	241
75	Steering, Editing and Representation Engineering	242
75.1	Activation steering	242
75.2	Representation engineering	242
75.3	Model editing	243
75.4	Intervention side effects	243
75.5	Worked protocol	243
75.6	Four perspectives	243
75.7	Lab and exit check	244
75.8	References	244
76	Merging, Stitching and Representation Compatibility	245
76.1	Model merging	245
76.2	Layer stitching	245
76.3	Cross-model alignment	246

76.4	Normalization mismatch	246
76.5	Latent bridge cost	246
76.6	Established result versus hypothesis	246
76.7	Four perspectives	246
76.8	Lab and exit check	247
76.9	References	247
Part 20:	Frontier Architecture and Research	248
77	Efficient Memory and Adaptive Computation	249
77.1	Efficient KV memory	249
77.2	KV-cache compression	249
77.3	KV-cache eviction	249
77.4	Activation compression	250
77.5	Dynamic computation	250
77.6	Conditional layer execution	250
77.7	Adaptive computation	250
77.8	Worked budget	250
77.9	Four perspectives	250
77.10	Lab and exit check	251
77.11	References	251
78	Routable Intelligence and Neural Handoffs	252
78.1	Neural routing	252
78.2	Model cascades	252
78.3	Latent communication	252
78.4	Cross-model representations	253
78.5	Neural handoffs	253
78.6	Sparse expert intervention	253
78.7	Silent expert intervention	253
78.8	LLM dead-call elimination	253
78.9	Computation as a routable resource	254
78.10	Minimal experiment	254
78.11	Four perspectives	254
78.12	Lab and exit check	254
78.13	References	254
79	Research Design and Reproduction	255
79.1	Forming research questions	255
79.2	Literature search	255
79.3	Prior-art search	255
79.4	Reading papers critically	255
79.5	Designing a research experiment	256
79.6	Choosing baselines	256
79.7	Benchmark selection	256
79.8	Reproducing papers	256
79.9	Reproducibility	256
79.10	Scientific vs engineering claims	257
79.11	Worked preregistration	257
79.12	Four perspectives	257
79.13	Lab and exit check	257
79.14	References	257
80	Evidence, Ablations and Publication	259
80.1	Ablations	259
80.2	Statistical significance	259
80.3	Confidence intervals	260

80.4	Uncertainty	260
80.5	Profiling research claims	260
80.6	FLOPs-memory-latency measurement	260
80.7	Writing papers	260
80.8	arXiv	260
80.9	Workshops	261
80.10	Conferences	261
80.11	Peer review	261
80.12	Finding unanswered questions	261
80.13	Claim-evidence table	261
80.14	Four perspectives	262
80.15	Lab and exit check	262
80.16	References	262
A	Exercise and Exit-Check Solutions	263
A.1	Part I — Orientation	263
A.2	Part II — Mathematical Foundations	264
A.3	Part III — Neural Networks	264
A.4	Part IV — Tokens and Representations	265
A.5	Part V — Attention	265
A.6	Part VI — The Transformer	266
A.7	Part VII — Training Systems	266
A.8	Part VIII — Architectures	267
A.9	Part IX — Data and Evaluation	268
A.10	Part X — Adaptation and Reasoning	268
A.11	Part XI — Hardware Performance	269
A.12	Part XII — Kernels and Runtime	269
A.13	Part XIII — Inference	270
A.14	Part XIV — Efficient Inference	270
A.15	Part XV — Distributed Execution	271
A.16	Part XVI — Serving	272
A.17	Part XVII — Local Inference	272
A.18	Part XVIII — Retrieval and Agents	273
A.19	Part XIX — Interpretability and Composition	273
A.20	Part XX — Research Frontier	274
B	Working glossary	275
C	Consolidated chapter references	276
C.1	1. The LLM in One Picture	276
C.2	2. Training vs Inference	276
C.3	3. Anatomy of the Modern LLM Stack	276
C.4	4. Building Our Tiny Laboratory	276
C.5	5. Tensors and Resource Accounting	277
C.6	6. Geometry and Number Formats	277
C.7	7. Probability, Logits and Loss	277
C.8	8. Gradients, Graphs and Autodiff	277
C.9	9. Linear Layers and Nonlinearities	277
C.10	10. A Tiny Network, Forward and Backward	277
C.11	11. Optimization and State	277
C.12	12. Initialization and Training Stability	277
C.13	13. Tokenization Algorithms	278
C.14	14. Vocabulary and Token IDs	278
C.15	15. Embedding Matrices and Geometry	278
C.16	16. Position and Sequence Order	278
C.17	17. Sequence Modeling and the Attention Idea	278

C.1818. Queries, Keys, Values and Scaling	278
C.1919. Causal and Multi-Head Attention	278
C.2020. Attention Shapes and Compute Cost	278
C.2121. Residual Streams and Normalization	279
C.2222. Feed-Forward Networks and Complete Blocks	279
C.2323. Stacking, Vocabulary Projection and Tying	279
C.2424. Build Our GPT From Scratch	279
C.2525. Sequences, Batches and Pretraining	279
C.2626. The Training Step and Schedule	279
C.2727. Mixed Precision and Checkpoint Anatomy	279
C.2828. Debugging and Training Our Tiny Model	279
C.2929. Transformer Families	280
C.3030. Modern Positions, Heads and MLPs	280
C.3131. Mixture of Experts	280
C.3232. Alternative Architectures and Real Configs	280
C.3333. Data Sources and Cleaning	280
C.3434. Mixtures, Tokens and Synthetic Data	280
C.3535. Evaluation and Meaningful Experiments	280
C.3636. Scaling Laws	281
C.3737. Pretraining, Continued Pretraining and SFT	281
C.3838. Parameter-Efficient Adaptation	281
C.3939. Preferences and Reinforcement Learning	281
C.4040. Reasoning, Test-Time Compute and Distillation	281
C.4141. Why GPUs and How They Execute	281
C.4242. Memory Hierarchy and Movement	281
C.4343. Throughput, Intensity and Rooflines	281
C.4444. Profiling Transformer Bottlenecks	282
C.4545. Kernels, GEMM and Tiling	282
C.4646. CUDA, Triton and CUTLASS	282
C.4747. Graphs, Compilers and Runtimes	282
C.4848. CUDA Graphs and Execution Overhead	282
C.4949. Load Weights and Trace a Forward Pass	282
C.5050. Prefill, Decode and KV Cache	282
C.5151. Sampling and Decoding Policies	283
C.5252. Inference Memory and Latency Accounting	283
C.5353. FlashAttention and Paged KV Memory	283
C.5454. Caching, Batching and Prefill Scheduling	283
C.5555. Speculative Decoding and Dynamic Execution	283
C.5656. Quantization Across the Inference Path	284
C.5757. Parallel Training Strategies	284
C.5858. Partitioning Model Computation	284
C.5959. Collectives and Interconnects	284
C.6060. Distributed Execution and Failure Recovery	284
C.6161. Anatomy of an Inference Server	284
C.6262. Tenants, Models and Residency	285
C.6363. Disaggregation and Runtime Comparisons	285
C.6464. Production Serving and GPU Clusters	285
C.6565. CPU and Edge Inference	285
C.6666. Hybrid Memory and Local Loading	285
C.6767. Local Runtimes and Accelerators	285
C.6868. Packaging, Power and Thermal Limits	285
C.6969. Embeddings, Contrastive Learning and Vector Indexes	285
C.7070. RAG and Retrieval Quality	286
C.7171. Memory, Tools and Structured Output	286
C.7272. Agents and Multi-Model Economics	286
C.7373. Hidden States, Layers and Attention Heads	286

C.7474. Features, Probes and Mechanisms	286
C.7575. Steering, Editing and Representation Engineering	286
C.7676. Merging, Stitching and Representation Compatibility	287
C.7777. Efficient Memory and Adaptive Computation	287
C.7878. Routable Intelligence and Neural Handoffs	287
C.7979. Research Design and Reproduction	287
C.8080. Evidence, Ablations and Publication	287

Edition and Review Notice

Publication candidate, September 2026. Copyright © 2026 Vijey Shrivathsan Vasudevan and Shri Venkatakrisnan Vasudevan. All rights reserved unless a component states a separate license.

This edition is provided for education and research. It is not a security guarantee, production deployment prescription, or substitute for reviewing the official documentation of the exact framework, model, accelerator, driver, runtime, and dataset version in use.

The repository distinguishes three forms of assurance:

1. **Automated validation** checks coverage, anchors, manuscript invariants, executable tests, and publication artifacts.
2. **AI-assisted internal review** checks consistency, calculations, terminology, citations, and obvious technical defects while recording known limitations.
3. **Independent review** requires a qualified person outside the drafting process. It has not occurred unless a dated record explicitly names that review.

Preface

Large language models are often explained as either equations without systems or systems without equations. Inside the LLM connects the two. It follows four recurring objects through the stack: the token that carries the data, the gradient that changes the model, the byte that pays the hardware cost, and the request that turns a model into a service.

The book begins with tensors and automatic differentiation, constructs a small Transformer, and then follows the same computation through training, kernels, distributed execution, inference serving, local deployment, retrieval, agents, interpretability, and research design. The examples intentionally separate mathematical mechanisms from framework conveniences so that readers can identify what must remain invariant when implementations change.

This working edition was developed with AI-assisted drafting, checking, and typesetting under the direction of the named authors. Automated checks establish structural coverage and executable invariants; they do not replace expert judgment. Technical claims should be evaluated against the cited primary sources and the reader's target software and hardware versions.

How to Use This Book

Each chapter states an outcome, prerequisites, the mechanism, four operational perspectives, and an exit check. Readers new to the field should follow the numbered order through the first ten parts. Readers focused on systems can begin with Parts 11-17 after reviewing the tensor, attention, Transformer, and inference foundations. Researchers can use Parts 18-20 as a map, but should retain the evaluation and systems chapters: a novel mechanism without a fair measurement protocol is merely an anecdote with equations.

The four recurring perspectives are:

- **Follow the Token:** what representation exists at each boundary and which other tokens can influence it.
- **Follow the Gradient:** which parameters receive learning signal, what state the optimizer retains, and where training can become unstable.
- **Follow the Byte:** where weights, activations, caches, gradients, and optimizer state reside and move.
- **Follow the Request:** how online execution schedules, isolates, observes, and recovers a user's workload.

Notation is local to each chapter unless explicitly carried forward. Shapes use B for batch size, T for sequence length, D for model width, H for attention heads, and V for vocabulary size. Binary memory quantities use KiB, MiB, and GiB; decimal device specifications use KB, MB, and GB. FLOP counts state the multiply-add convention when it affects comparisons.

The companion repository contains executable reference implementations, milestone labs, tests, editable figures, coverage records, and reproducible PDF/HTML build commands. Measurements marked as simulated are teaching models, not hardware benchmark results.

Part 1: What Is This Machine?

Chapter 1

The LLM in One Picture

Status: **Draft — not yet technically reviewed.**

The purpose of this chapter is to let you trace a piece of text through a language model without treating any stage as magic. We will use a decoder-only Transformer as our running example. Other language-model families arrive in Chapter 29. You do not need calculus yet; you need to distinguish a symbol, an array of numbers and a probability distribution.

1.1 The machine we are building

Imagine sending the text “The sky is” to a model. Its next output could be “blue,” but the model does not first construct an English sentence and then reveal it. In the autoregressive setup used here, one forward pass produces scores for possible next tokens. A decoding policy selects a token. The selected token is appended to the context, and the process repeats.

This already separates three jobs: encoding the input, computing predictions and deciding what to emit. A tokenizer, a neural network and a decoding policy perform those jobs. An application may add messages, tools, retrieval and a user interface around them. Those surrounding components matter, but they are not interchangeable with the model’s weights.

1.2 1. Prompt: the input context

The prompt is the input presented to the model. In a chat application, the visible message may be only part of that input: a template may include role markers and previous messages. For now, our entire prompt is “The sky is.”

Be careful with the word context. At the text level it means the material supplied to the model. Later, at the tensor level, it also determines a sequence length. We will write that length as T . T counts tokens, not words or characters.

1.3 2. Tokenizer: text becomes integer IDs

A tokenizer converts text into a sequence of token IDs. For illustration, suppose our tokenizer produces [17, 42, 9]. These numbers are invented, not the encoding of any published tokenizer. A real tokenizer might split the same text differently, including spaces inside token pieces.

An ID is a lookup index. ID 42 is not twice as meaningful as ID 21. Treating token IDs as ordinary scalar measurements would impose a numerical relationship that the vocabulary does not promise.

Tokenization also defines the available output pieces. Some pieces correspond to complete words, some to word fragments or bytes, and some to special control markers. We will implement a tokenizer in Chapter 13 rather than assuming every token is a word.

1.4 3. Embeddings: IDs become vectors

Suppose the vocabulary has V entries and each embedding has D numbers. The token embedding table then has shape $[V, D]$. For every input ID, the model selects one row. Our three input IDs become an array of shape $[3, D]$.

With a batch dimension B , the usual conceptual shape is $[B, T, D]$. For $B=1$, $T=3$ and $D=4$, there are twelve numbers. $D=4$ is deliberately tiny; it makes the array easy to picture. We are describing dimensions, not recommending a useful model size.

The lookup vectors are learned parameters. They are not a hand-written dictionary of concepts. Position information also has to enter the computation, although the mechanism depends on the architecture. Some models add positional vectors; others apply position-dependent transformations inside attention. Chapter 16 separates these choices.

1.5 4. Transformer: context changes the representation

Transformer blocks transform the vectors. In our causal model, the representation at a position can use that position and earlier positions, but not later ones. Attention mixes information across the allowed positions. Feed-forward layers transform information within each position; residual connections and normalization support the overall computation.

The result is a sequence of contextual hidden states. The vector associated with “is” now depends on the preceding context, rather than being only the embedding-table row for that token. That is why we distinguish an input embedding from a contextual hidden state.

We will not pretend that each coordinate corresponds to a neat human concept such as “sky-ness.” A vector’s usefulness is established by what the model does with it. Later chapters investigate internal representations with experiments instead of assigning meanings by intuition.

For this chapter, imagine each block accepts $[B, T, D]$ and returns $[B, T, D]$. Keeping the outside shape constant does not mean the inside is cheap: attention projections, score arrays and expanded feed-forward layers may have different shapes and costs.

1.6 5. Logits: one score per vocabulary entry

At the final position, the model has a hidden vector of length D . A vocabulary projection turns it into V scores, called logits. Logits are unrestricted real numbers. They are not probabilities and need not sum to one.

Consider a deliberately tiny output vocabulary with four choices:

Token	Logit	Softmax probability, rounded
blue	2.0	0.644
green	1.0	0.237
clear	0.0	0.087
end marker	-1.0	0.032

These illustrative scores were chosen by hand. Softmax converts them to positive values whose sum is one. Its formula is $p[i] = \exp(z[i]) / \sum(\exp(z[j]))$. Subtracting the largest logit

before exponentiation gives the same mathematical distribution and avoids unnecessarily large exponentials. The accompanying CPU lab demonstrates this.

A training forward pass commonly produces logits at every position, with conceptual shape $[B, T, V]$. For choosing the next token after our prompt, only the last position's distribution is needed. Implementations may exploit that distinction to avoid work.

1.7 6. Sampling: choosing is separate from scoring

Greedy decoding selects the highest-scoring token: “blue” in the table. Sampling instead draws from a distribution, so it can select “green” even though “blue” is more likely. Temperature and filtering policies can modify the distribution before the draw. We will derive their behavior in Chapter 51.

The distinction matters when debugging. Two different outputs do not by themselves establish that the model's parameters changed. The decoding policy or its random draws may differ. Conversely, a fluent output is not proof that a claim is true: the scores predict token continuations, not a guaranteed factual verdict.

1.8 7. Output tokens: append and repeat

After selecting “blue,” append its ID to the sequence. The next prediction is conditioned on this longer sequence. Generation continues until a stopping condition is met, such as an end marker or a length limit. The output IDs are decoded to text for display.

A simple implementation can recompute the full growing sequence at every step. A more efficient implementation reuses cached state. Chapter 50 explains precisely what a KV cache stores and why it is valid. You do not need the cache to understand the logical operation: select a token, extend the context, predict again.

Tokens and displayed words need not align one-for-one. A fragment may complete a word from an earlier token. This is one reason tokens per second and words per second are different measurements.

1.9 8. What the model predicts

At step t , the model represents $P(x_t \mid x_{<t})$: a conditional distribution for the next token given previous tokens. For a fixed token sequence, the probability chain rule gives $P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{<t})$. This factorization is exact; the modeling choice is how the network parameterizes each conditional distribution. Training and generation use the relationship in different ways.

During training, known sequences supply target next tokens. A loss measures how poorly the predictions fit those targets, and gradients guide parameter updates. During ordinary inference, the parameters remain fixed while the input and temporary state change. Inference can produce new text without performing a training update.

This is the central distinction to carry forward: hidden states can change for every request while the learned weights remain the same.

1.10 Four ways to follow the same operation

Follow the Token. Text becomes IDs, lookup vectors, contextual states, vocabulary scores and another token ID. Track which representation is present at each stage.

Follow the Gradient. During training, a loss on predicted tokens leads back through the vocabulary projection and Transformer to trainable parameters. Ordinary inference does not need this backward pass. We will build it in Chapter 8 before relying on a framework’s automatic differentiation.

Follow the Byte. A tensor’s shape and numeric format determine its ideal storage. Our `[1,3,4]` array contains twelve values; at two bytes each it uses 24 bytes for its payload. Actual programs also allocate weights, temporary arrays, caches and bookkeeping. Counting only one visible array does not give total memory use.

Follow the Request. A user request can wait in a queue before the model runs. Its tokens may then be processed alongside other requests and streamed back incrementally. Thus a fast matrix multiplication does not automatically imply a fast user-visible response.

1.11 First experiment

From the project root, run:

```
python code/resource_lab.py
```

The program does not run a trained Transformer. It makes the score-to-probability-to-token step executable and checks the first resource calculations. Change the logits and observe both the greedy choice and the probability distribution. Add 10,000 to every logit; the stable softmax should preserve the probabilities.

Then answer these questions before reading the answers:

1. Does token ID 42 mean something is quantitatively larger than token ID 17?
2. Where does `[B,T,D]` become scores over the vocabulary?
3. If sampling emits a different token on another run, must the weights have changed?
4. How many payload bytes does `[1024,4096]` require in BF16?
5. Does appending a token require retraining the model?

Answers: (1) No; IDs are indices. (2) At the vocabulary projection or LM head. (3) No; randomness and decoding choices can change the output. (4) 8,388,608 bytes, which is 8 MiB; this excludes metadata and other tensors. (5) No; the next forward computation uses the extended context with the same weights.

1.12 What comes next

Chapter 2 separates the complete training and inference lifecycles. Chapter 5 turns shape and byte tracking into a habit. The rest of the book progressively replaces each box in this chapter with equations, runnable implementations and measured behavior.

Editorial note: this is an original introductory draft with a locally tested numerical example. It has not received an independent human technical review.

1.13 References

- Vaswani et al., [Attention Is All You Need](#), 2017 — Transformer architecture and scaled dot-product attention.
- Sutskever, Vinyals, and Le, [Sequence to Sequence Learning with Neural Networks](#), 2014 — autoregressive sequence factorization and decoding.

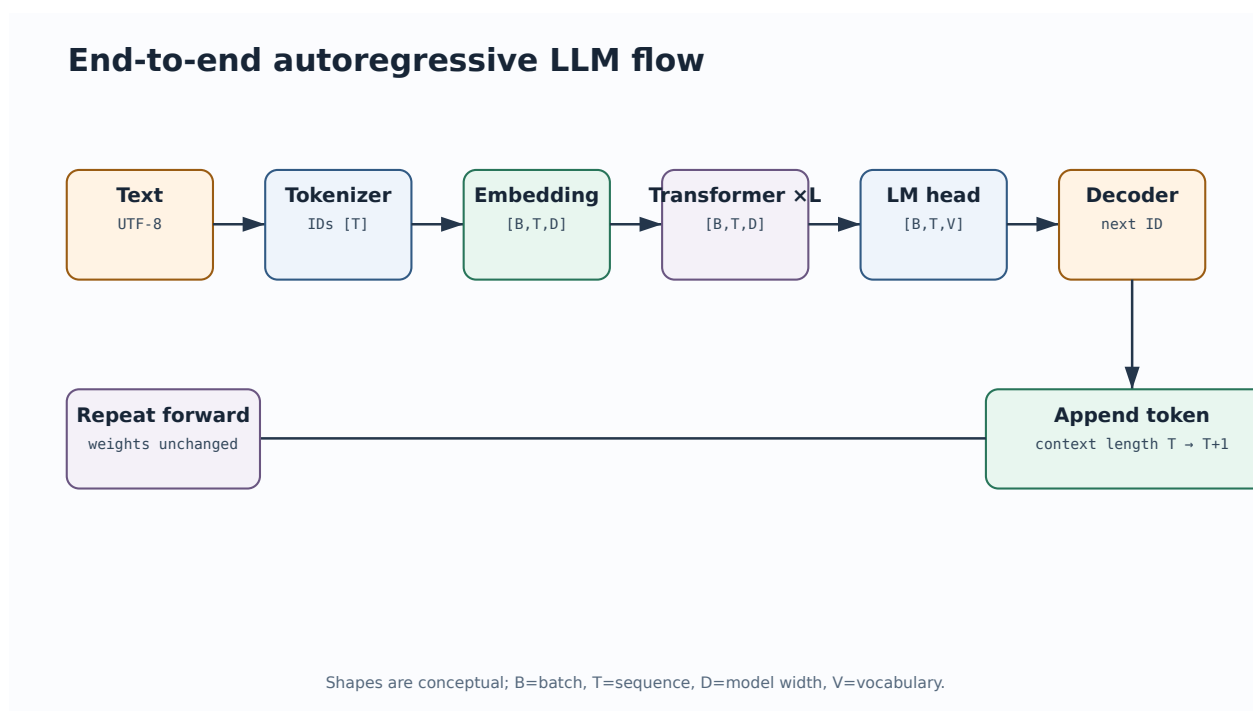


Figure 1.1: End-to-end language-model flow.

Chapter 2

Training vs Inference

Status: **Draft — not yet independently reviewed.**

A language model has two main operating modes. **Training** changes learned parameters so future predictions improve under an objective. **Inference** uses fixed parameters to compute predictions for new input. Both modes can run the same forward equations; their state, memory needs, and purpose differ.

2.1 Training versus inference

Let a model with parameters θ receive tokens x and return logits $f_\theta(x)$. During training, known target tokens y let us compute a scalar loss:

$$L(\theta) = \text{CrossEntropy}(f_\theta(x), y).$$

A backward pass computes gradients $\nabla_\theta L$. An optimizer uses those gradients and its own state to produce new parameters. One simplified SGD update is

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta L,$$

where η is the learning rate. AdamW is more involved, but the defining point remains: training includes a parameter update.

Inference stops after the forward computation and a decoding decision. The weights normally remain unchanged. The prompt grows during autoregressive generation, and temporary activations and caches change, but those changes do not mean the model learned.

Stage	Training	Autoregressive inference
Input	Training sequences and targets	Prompt plus generated tokens
Forward pass	Yes	Yes
Loss	Usually	Usually absent
Backward graph	Yes for paths to trainable parameters; frozen subgraphs may avoid it	Unnecessary
Optimizer step	Yes	No
Persistent weight change	Yes	No
Sampling	Optional for evaluation	Common for generation

The word evaluation creates a common ambiguity. Evaluating validation loss uses inference-like execution because parameters do not update, even though the output is a loss rather than generated prose. Benchmark generation is also evaluation. The mode is defined by state changes and graph requirements, not by whether a human sees text.

2.2 Parameters versus activations

Parameters are persistent learned tensors: embedding tables, attention projections, feed-forward matrices, normalization scales, and similar values. They are shared across requests until a training or editing procedure changes them.

Activations are values computed for particular inputs: embeddings selected for this batch, attention results, residual states, and logits. A new request creates new activations. Training often retains selected activations or equivalent information because the backward pass needs them. This is a major reason a model that fits for inference can require far more memory for training.

Training also introduces persistent optimizer state and transient gradients. For Adam-like optimization, each trainable parameter commonly has a gradient and two moment estimates in addition to the parameter itself. Exact storage depends on precision, sharding, optimizer variant, and implementation, so “four times the model size” is a rough story rather than a universal law.

Inference has different growing state. During generation, a decoder can retain attention keys and values for processed positions. This KV cache is request-specific. It can become the dominant memory consumer at long contexts or high concurrency while the model weights remain fixed.

Classify each tensor by asking two questions:

1. Does it survive after this request or training step?
2. Does it describe learned behavior, optimizer history, or this input’s computation?

That avoids vague labels such as “model memory,” which can hide weights, caches, allocator reservations, and temporary workspaces inside one number.

2.3 Loss versus generated text

A loss is a numerical objective. Generated text is the consequence of repeatedly selecting tokens from predicted distributions. These are related, but neither substitutes for the other.

Suppose the correct next token has probability 0.8. Its negative log-likelihood is about 0.223. If its probability is 0.2, the loss is about 1.609. Training can use this smooth signal even when the greedy choice is unchanged. Conversely, a model can generate a plausible sentence while assigning poor probabilities across a broad held-out corpus.

Loss is normally computed with teacher forcing: at each training position, the model receives the true preceding tokens. During free-running generation, it receives its own selected tokens. One early mistake can therefore change all later contexts. This difference helps explain why a modest validation-loss improvement and a visible generation improvement need not move together.

Perplexity, benchmark scores, task success, factuality, safety, latency, and human preference measure different properties. “The loss went down” is evidence about the specified objective and data distribution; it is not a universal quality certificate.

2.4 Learning versus execution

Execution means running an existing computation. Learning means changing persistent state according to data and an objective. A retrieval system can improve an answer by adding documents without changing model parameters. A longer prompt can alter behavior without training. A

tool result can correct a calculation without being stored in the model. These are execution-time interventions.

Fine-tuning changes some or all trainable parameters. LoRA training changes small adapter parameters while the base weights remain frozen. Prompt tuning may train persistent prompt vectors. In-context learning changes the model's behavior through context while leaving its parameters fixed. The same word learning is sometimes used for all of these; the state table tells you what actually happened.

Framework controls must also be kept separate. In PyTorch, `model.eval()` changes the behavior of modules such as dropout or batch normalization; it does **not** disable gradient recording. `torch.no_grad()` and inference mode control autograd recording. PyTorch's autograd documentation states that the dynamic graph is rebuilt on each iteration and that inference mode avoids additional tracking with restrictions on later autograd use. These are execution controls, not optimizer steps.

2.5 Trace one training step

For a batch of token sequences:

1. Read parameters θ_k and optimizer state.
2. Build activations and logits in the forward pass.
3. Compare logits with targets to compute loss.
4. Traverse the recorded graph backward to accumulate parameter gradients.
5. Apply clipping or scaling if configured.
6. Update optimizer state and produce θ_{k+1} .
7. Clear or reset gradients before the next accumulation window.

For one inference request:

1. Read fixed parameters θ .
2. Tokenize the prompt and build activations.
3. Produce logits and select a token.
4. Append the token and update request-local cache state.
5. Repeat until a stopping condition.
6. Release request-local state.

The same matrix multiplications can appear in both lists. The backward graph, optimizer, and persistent update make training a different system.

2.6 Four perspectives

Follow the Token: training consumes known next-token targets; generation feeds selected tokens back as new input.

Follow the Gradient: only training needs the gradient path to parameters. Frozen parameters can participate in a forward and still receive no stored gradient.

Follow the Byte: training retains backward information, gradients, and optimizer state; inference retains request activations and often a KV cache.

Follow the Request: a production request should not silently mutate shared weights. Online adaptation, when desired, must be an explicit isolated workflow with versioning and rollback.

2.7 Exit check

For each item, label it persistent or request/step-local, then say whether it changes during ordinary inference: model weights, prompt IDs, hidden activations, KV cache, parameter gradients, Adam moments, random-generator state. The answer depends on the boundary you choose, so state the boundary. For a single request: weights and optimizer state are persistent; prompt IDs, activations, and KV cache are local; gradients are absent; random state advances if sampling uses it.

2.8 References

- [PyTorch autograd mechanics](#) — dynamic graphs, saved tensors, grad modes, and the distinction between evaluation mode and gradient control. Accessed 2026-09-18.

Chapter 3

Anatomy of the Modern LLM Stack

Status: **Draft — not yet independently reviewed.**

A slow or incorrect answer can originate far above or far below the Transformer. Treating the whole product as “the model” makes diagnosis almost impossible. This chapter builds a layer map and follows one request through it.

3.1 Applications

The application owns the user-visible contract: authentication, conversation state, permissions, input validation, presentation, and product-specific policy. It may construct a prompt from several messages and stream partial output. If the UI freezes after receiving tokens, changing attention kernels will not fix it.

The application also decides what counts as success. A code assistant may value compilable patches; a search assistant may require citations; a drilling assistant may require offline operation and bounded latency. Model quality must be evaluated inside this contract.

3.2 Agents and RAG in the stack

Retrieval-augmented generation adds a data path: query construction, search, filtering, reranking, and context assembly. An agent adds a control path: choose an action, validate arguments, execute a tool, observe the result, and decide whether to continue. Both eventually create model inputs, but many failures occur before or after the model call.

A missing document may be an indexing or retrieval failure. A correct document that never fits the context may be a packing failure. A malformed tool call may reflect model output, schema design, or validation. Record these stages separately.

3.3 Inference server

The inference server accepts requests and turns them into scheduled model work. It tokenizes or accepts token IDs, manages queues, forms batches, allocates KV-cache blocks, invokes the runtime, samples output, and streams tokens. Under load, queue delay may dominate time to first token even when a single isolated request is fast.

The server balances competing goals: low latency for each user, high accelerator utilization, bounded memory, fairness, and failure isolation. Continuous batching lets new requests enter

between decode iterations. Prefix caching can reuse work, but only when token sequences and relevant model state match correctly.

3.4 Model runtime

The runtime implements model execution for a chosen device and numeric format. It loads weights, chooses operator implementations, manages caches and temporary buffers, and may partition computation across devices. PyTorch eager mode, compiled PyTorch, llama.cpp, vLLM, and TensorRT-LLM expose different abstractions, but all must ultimately produce numerically acceptable outputs from the same model definition.

A runtime can change latency or memory without changing the conceptual architecture. It can also change numerical results slightly through precision, reduction order, or fused operations. Correctness therefore means agreement within defined tolerances and task-level quality, not necessarily bitwise identity.

3.5 PyTorch and the compiler

In an eager framework, Python requests tensor operations one by one. Compilation can capture a region of those operations, transform an intermediate representation, fuse work, and generate device code. Unsupported data-dependent behavior can break a graph into smaller regions, reducing optimization opportunities.

Compilation cost is real. A first request may spend time tracing, compiling, and warming kernels; later requests reuse the result. Honest latency reports separate cold start, compile time, warmup, and steady-state execution.

3.6 Tensor operations

The Transformer is expressed through operations such as matrix multiplication, normalization, elementwise functions, indexing, reductions, and data movement. Shapes and dtypes determine legal operations and much of their cost. $[B, T, D] @ [D, 4D]$ is a feed-forward projection; $[B, H, T, Dh] @ [B, H, Dh, T]$ creates attention scores. The formulas are architecture; their execution plan is systems engineering.

Tensor operations are a useful debugging boundary. If logits are already wrong after one layer, an HTTP timeout setting is irrelevant. If tensor outputs are correct but the streamed text is truncated, the error is above the runtime.

3.7 GPU kernels

A kernel is a program executed across many device threads. Framework operations may launch one kernel, several kernels, or a fused kernel that performs multiple conceptual operations. Launch overhead, memory access, synchronization, and shape specialization can matter as much as the arithmetic count.

Kernel names in a profiler are evidence about what ran, not a complete explanation. To connect them to the model, map each kernel back to an operator, tensor shapes, transferred bytes, and the request phase: prefill or decode.

3.8 GPU hardware

Accelerators supply parallel arithmetic units, registers, on-chip shared memory and caches, plus high-bandwidth device memory. Matrix engines achieve large throughput only when shapes, precision, and data movement allow them to remain busy. Decode often performs little work per byte of weight read, while large prefill matrices can expose more parallelism.

Peak specifications are ceilings under particular conditions. Application throughput also includes scheduling gaps, kernels that do not use matrix engines, communication, and idle capacity caused by small batches.

3.9 Memory

Memory is a hierarchy rather than one capacity number. Model weights may reside in device memory, host RAM, or mapped storage. Active tensors move through caches, shared memory and registers. The serving layer separately manages KV-cache pages and allocator reservations.

“Out of memory” can mean the weights do not fit, a long-context cache exhausted its pool, temporary workspace peaked, or fragmentation prevented a sufficiently large allocation. Record allocated, reserved, active, and peak memory when the runtime exposes them.

3.10 Silicon

At the bottom, transistors implement storage and arithmetic under limits imposed by area, power, heat, signaling, and manufacturing. Software cannot repeal these constraints. Quantization reduces the bits moved and may enable faster hardware paths; it helps only when the full stack has suitable kernels and acceptable accuracy.

Silicon details matter when they change a system decision: supported precision, memory capacity, interconnect bandwidth, power envelope, or thermal throttling. A block diagram becomes useful when it predicts a measurable bottleneck.

3.11 Follow one request

A request enters the application, which assembles messages and retrieved context. The server queues it, tokenizes it, and reserves cache space. The runtime loads or references resident weights and invokes framework or compiled operators. Operators launch kernels that move bytes through the hardware hierarchy and execute arithmetic. Logits return upward; the server samples and streams a token; the application renders it. Decode repeats this loop.

Measure at the boundaries. Application latency, queue time, tokenization time, prefill time, time per output token, tool latency, and rendering time answer different questions. A single end-to-end stopwatch tells you that the user waited; it does not tell you what to fix.

3.12 Diagnostic exercise

Classify each symptom before proposing a remedy:

Symptom	First layers to inspect
Relevant document never reaches prompt	Retrieval, reranking, context assembly
First request slow; later identical requests fast	Loading, compilation, cache warmup
Latency explodes only under concurrency	Queueing, scheduler, KV allocation, batching

Symptom	First layers to inspect
Correct tokens arrive but UI updates late	Streaming transport and application rendering
Single-token decode saturates memory bandwidth	Runtime kernels, weight/KV movement, hardware
Tool executes destructive malformed arguments	Agent policy, schema validation, permissions

This table points to the first investigation, not a guaranteed root cause. Instrumentation must confirm it.

3.13 Four perspectives

Follow the Token: trace text, token IDs, hidden tensors, logits, sampled IDs, and displayed text.

Follow the Gradient: training extends below the model into backward kernels and above it into data and objective design; ordinary serving omits that path.

Follow the Byte: locate weight, activation, cache, communication, and storage movement at each layer.

Follow the Request: include queueing, scheduling, tools, cancellation, streaming, and failure recovery—not only forward compute.

3.14 References

- Kwon et al., [Efficient Memory Management for Large Language Model Serving with Page-dAttention](#), 2023 — KV-cache paging and serving throughput.
- Ansel et al., [PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation](#), 2024 — capture, graph breaks, and compilation in PyTorch 2.
- Williams, Waterman, and Patterson, [Roofline: An Insightful Visual Performance Model for Multicore Architectures](#), 2009 — relating arithmetic throughput to memory traffic.

Chapter 4

Building Our Tiny Laboratory

Status: **Draft — not yet independently reviewed.**

The book’s laboratory has one purpose: make claims executable. We begin with a CPU-only standard-library baseline so every reader can run the first experiments. NumPy, PyTorch, profilers, accelerators, and notebooks are added when a chapter needs them and pinned when introduced.

4.1 Python

Use Python 3.10 or newer for the starter project. Run commands from the repository root so relative paths are stable. A virtual environment isolates project packages from unrelated applications:

```
python -m venv .venv
```

Activate it with `.venv/bin/activate` on common POSIX shells or `.venv\Scripts\Activate.ps1` in PowerShell. Activation is a convenience, not a requirement; invoking the environment’s Python executable directly is more explicit. Python’s documentation describes virtual environments as disposable and non-portable, so record dependencies and recreate them rather than copying the environment directory.

The current starter has no third-party dependencies. That is deliberate: `python code/resource_lab.py` should run before package installation succeeds. Later chapters will add a lockable research environment for NumPy, PyTorch, plotting, and optional GPU tools.

4.2 NumPy

NumPy will be our transparent array workbench. It makes shapes, strides, broadcasting, matrix multiplication and dtype effects visible without the training framework’s module system. We will use it for small reference implementations, never as proof that a GPU kernel behaves the same way.

When NumPy is introduced, print its version and array dtype in experiment output. Default dtypes can differ across libraries and operations. An unnoticed float64 reference compared with a float16 implementation can make accuracy and memory conclusions misleading.

4.3 PyTorch

PyTorch will carry the main training implementation because it combines tensor operations, automatic differentiation, neural-network modules, compilation, and device backends. We will first build important pieces ourselves, then compare them with framework implementations.

Version pins matter. Operator semantics, compiler behavior, supported dtypes, and kernels change. Every measured result must record the PyTorch version, device backend, and model configuration. A code snippet that was correct under one pinned revision is not a timeless benchmark.

Keep three controls distinct:

- `model.train()` and `model.eval()` control module behavior.
- `grad mode`, `no_grad`, and `inference mode` control autograd recording.
- the optimizer step changes parameters.

Confusing them is a reliable way to produce silent evaluation errors.

4.4 Notebooks

Notebooks are useful for explanation and exploration, but hidden state makes them poor sole records of an experiment. A cell can depend on variables created by an earlier execution order that the displayed document no longer reflects.

Put reusable logic in importable modules. A notebook should orchestrate an experiment, visualize results, and state conclusions. Before accepting a result, restart the kernel and run all cells from top to bottom. For automated validation, expose a script or test that does not require clicking through a notebook.

4.5 Profiling tools

Begin with the least invasive measurement that answers the question. Wall-clock timing tells you user-visible duration. Operator profilers attribute framework time. System profilers reveal CPU scheduling and device activity. Kernel profilers expose launches, occupancy and memory behavior.

Every profiler perturbs execution. Warmup, asynchronous devices, compilation, clock changes, and background work can overwhelm the effect being measured. Later GPU labs require synchronization around timed regions and separate cold from steady-state measurements.

A profile is a trace of a workload under stated conditions. It does not prove that the same bottleneck exists at another batch size, sequence length, concurrency, dtype, or device.

4.6 CPU baseline

Run the included baseline:

```
python code/resource_lab.py
```

It demonstrates stable softmax, seeded sampling, tensor payload storage, linear-layer FLOP accounting, and KV-cache byte accounting. It labels its logits as illustrative because they do not come from a trained model.

Then run the tests:

```
python -m unittest discover -s tests -v
```

A CPU reference gives us something portable to compare against. It is not a performance proxy for a GPU. When an accelerated implementation arrives, compare correctness first, then performance under a workload that actually uses the target hardware.

4.7 Seeds and environment records

A seed controls a pseudorandom stream; it does not freeze every source of variation. Parallel reductions, nondeterministic kernels, library algorithms, compiler decisions, data order, and hardware can still differ. Record the seed because it enables investigation, not because it guarantees identical floating-point bits.

Create an environment record:

```
python code/environment_record.py > build/environment.json
```

The record includes Python, platform, processor information, project revision when available, and installed packages. It intentionally avoids environment variables because they can contain credentials. Add model, dataset and accelerator metadata to each experiment record rather than pretending one machine-level file describes every run.

4.8 Minimal experiment protocol

1. State the hypothesis and the metric before running.
2. Record code revision, environment, input shapes, dtypes, seeds, and workload.
3. Establish a correctness reference and tolerance.
4. Separate setup, compilation, warmup, measurement, and teardown.
5. Repeat enough times to expose variability; report the aggregation method.
6. Save raw observations, including failures.
7. Change one factor at a time or use an explicit experimental design.
8. Report what the result does and does not support.

For example, “the fused implementation is 1.3× faster” is incomplete. Faster for which shapes, device, dtype, warmup, number of repetitions, and correctness tolerance? Did compilation time count? Did quality change? The laboratory exists to force those questions into the record.

4.9 Four perspectives

Follow the Token: save small inputs and expected outputs so transformations can be inspected.

Follow the Gradient: use finite differences or a trusted reference on smooth toy cases before trusting training.

Follow the Byte: derive ideal payload size, then measure process or device memory and explain the gap.

Follow the Request: measure end-to-end behavior separately from isolated operators and preserve queueing/concurrency conditions.

4.10 Exit artifact

Run the baseline and tests, create `build/environment.json`, and write a short experiment note containing: hypothesis, command, expected result, actual result, and one limitation. If another person cannot identify what ran, the experiment is not reproducible enough for this book.

4.11 References

- [Python virtual environments](#) — creation, isolation, activation, and portability constraints. Accessed 2026-09-18.
- [PyTorch autograd mechanics](#) — graph recording and grad-mode semantics. Accessed 2026-09-18.

Part 2: Tensors, Math and Computation

Chapter 5

Tensors and Resource Accounting

Status: **Draft — not yet independently reviewed.**

A tensor is an array plus a contract: shape, numeric type, layout, device, and meaning. Most LLM bugs become easier once those facts are written beside every operation.

5.1 Scalars

A scalar has one value: a loss, learning rate, or temperature. A framework may store it as a zero-dimensional tensor with shape `[]`. Scalar does not mean unimportant: one loss value can summarize millions of token predictions.

5.2 Vectors

A vector is a one-dimensional array. A token representation of width D has shape `[D]`; a vocabulary-logit vector has shape `[V]`. Its coordinates only gain meaning from the space and operation using them.

5.3 Matrices

A matrix has two axes. A weight $W \in \mathbb{R}^{D_{in} \times D_{out}}$ maps an input row vector from width D_{in} to D_{out} . An embedding table `[V,D]` is also a matrix, but lookup selects rows instead of multiplying by the integer token ID.

5.4 Tensors

Tensor is the general term for an array with any number of axes. A batch of hidden states commonly has `[B,T,D]`: batch, token position, hidden width. Names matter more than rank. Both an image and attention scores may be rank four while representing completely different things.

5.5 Shapes

Shape records the length of each axis. For `X[B,T,D]` and `W[D,F]`, `X @ W` produces `[B,T,F]`. Write this before code. Shape agreement catches wrong transposes and accidental mixing of heads or batches.

5.6 Dimensions

Dimension can mean the number of axes or an axis length. This book uses **rank** for the number of axes and names such as hidden width for axis lengths. A $[2, 8, 64]$ tensor has rank three and 1,024 elements.

5.7 Indexing

$X[b, t, d]$ chooses one scalar. $X[b, t, :]$ chooses the hidden vector at one token. Slicing may produce a view sharing storage rather than a copy; mutating a view can therefore alter its base. We avoid in-place mutation until ownership is explicit.

5.8 Transpose

Transposing swaps axes without changing element values. Attention uses QK^T : $[T, D_h] @ [D_h, T] \rightarrow [T, T]$. A transpose may change strides instead of physically rearranging bytes. Some kernels need contiguous layouts and may trigger a copy later.

5.9 Batch dimensions

Batch axes describe independent examples or requests processed together. Batched matrix multiplication applies the compatible operation across leading axes. A batch increases parallel work, but training batches, prefill batches, and continuously changing decode batches have different scheduling behavior.

5.10 Broadcasting

Broadcasting aligns shapes from the trailing axes. Dimensions are compatible when equal or when one is 1. Adding bias $[F]$ to output $[B, T, F]$ reuses the same bias across batch and token axes. Broadcasting is conceptually expanded; a good implementation need not materialize copies. An accidental $[T, 1] + [T]$ yields $[T, T]$, a classic silent memory bug.

5.11 Dtypes

A dtype specifies representation and nominal bits per element. It affects range, precision, supported hardware paths, and storage. Never infer dtype from a model name or file extension. Mixed-precision systems may store weights in one format, accumulate in another, and quantize caches separately.

5.12 Element counts

For dense shape $(n_1, \dots, n_k)$, element count is $N = \prod_i n_i$. $[1024, 4096]$ contains 4,194,304 elements. Zero-length axes make the product zero; sparse tensors require a different accounting model.

5.13 Bytes

Ideal dense payload bytes equal $\text{elements} \times \text{bits}/8$. BF16 uses 2 bytes, so the preceding tensor needs 8,388,608 bytes, or 8 MiB. Actual memory can include headers, padding, allocator rounding, alignment, packed-format metadata, workspaces, copies, gradients, and cached blocks. State whether GB means 10^9 bytes or GiB means 2^{30} .

5.14 FLOPs

For multiplying $[M,K] @ [K,N]$ by the textbook algorithm, there are MKN multiplications and $MN(K-1)$ additions. We approximate this as $2MKN$ FLOPs, dropping the lower-order MN term. Some sources count one fused multiply-add as one operation rather than two; always state the convention. FLOPs estimate arithmetic, not time.

5.15 Matrix multiplication

For $X[B,T,D] @ W[D,F]$, flattening the leading axes gives $M = BT$, so output is $[B,T,F]$ and cost is approximately $2BTDF$ FLOPs. With $B=1, T=128, D=256, F=1024$, the estimate is 67,108,864 FLOPs. Input, weights, and output must also move through memory; later roofline analysis asks which resource limits execution.

5.16 Resource ledger

For every important tensor, record: semantic axes, shape, dtype, elements, ideal bytes, lifetime, owner, and operation count. Apply it to attention score $[B,H,T,T]$: its quadratic T^2 term is immediately visible. FlashAttention later changes the materialization schedule; it does not change the mathematical attention result.

5.17 Exit problem

Let X be BF16 $[4,512,1024]$ and W be BF16 $[1024,4096]$. The output is $[4,512,4096]$. Ideal payloads are 4 MiB, 8 MiB, and 16 MiB. The projection costs about $2(4)(512)(1024)(4096) = 17,179,869,184$ FLOPs. These figures exclude bias, allocator overhead, accumulation details, and backward work.

5.18 References

- [NumPy documentation](#) — multidimensional arrays, indexing, vectorization, and broadcasting. Accessed 2026-09-18.

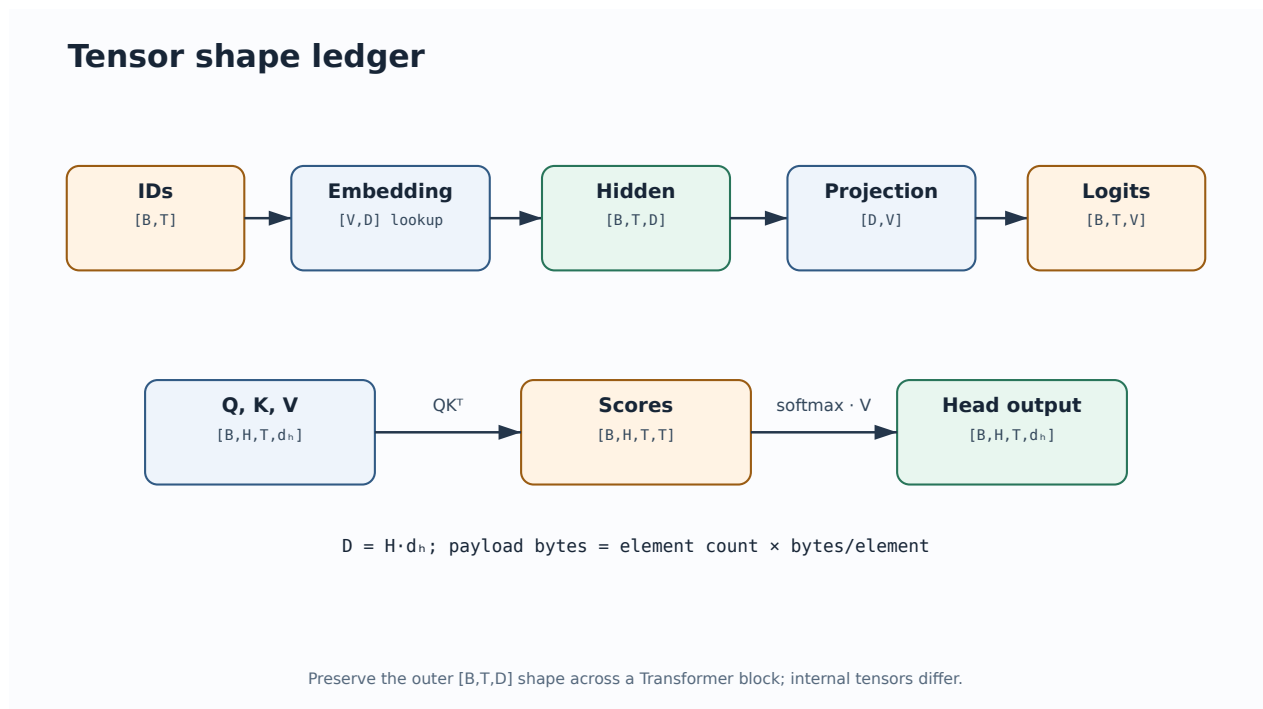


Figure 5.1: Tensor-shape ledger through the model.

Chapter 6

Geometry and Number Formats

Status: **Draft — not yet independently reviewed.**

Geometry explains how representations interact. Number formats explain how approximately a machine can store and compute those interactions.

6.1 Dot products

For vectors $a, b \in \mathbb{R}^D$, $a \cdot b = \sum_i a_i b_i$. It combines alignment and magnitude. Attention uses dot products between queries and keys; linear layers compute many dot products at once.

6.2 Cosine similarity

Cosine similarity divides the dot product by both magnitudes: $\cos(a, b) = a \cdot b / (\|a\| \|b\|)$. It compares direction and is undefined for a zero vector. Normalizing vectors makes cosine equal their dot product.

6.3 Magnitude

The Euclidean norm is $\|a\|_2 = \sqrt{\sum_i a_i^2}$. If $a=[1,0]$, $b=[10,1]$, and $c=[1,1]$, dot-product ranking may favor b because it is large, while cosine asks which direction is closer. Neither metric is universally correct; training and retrieval design decide what magnitude should mean.

6.4 Basis

A basis is a set of independent vectors that can express every vector in a space. Coordinates depend on the chosen basis. Rotating a representation and compensating in the next linear map can preserve behavior, which is why reading individual hidden coordinates as fixed concepts is risky.

6.5 Subspaces

A subspace is closed under vector addition and scalar multiplication. The columns of a matrix span its column space. A low-rank adapter restricts an update to a lower-dimensional factorization,

connecting a practical fine-tuning method to subspace structure.

6.6 Orthogonality

Vectors are orthogonal when their dot product is zero. Orthogonality expresses perpendicular directions under the chosen inner product; it does not automatically mean statistical independence or semantic unrelatedness.

6.7 Projections

The projection of x onto nonzero u is $(x \cdot u)/(u \cdot u) u$. Probes and steering methods often identify a direction and project activations onto or away from it. A measured direction can correlate with a feature without being its unique causal representation.

6.8 Linear transformations

A matrix maps vectors while preserving addition and scalar multiplication. Rank describes the dimension of the output subspace reachable through the map. Transformer projections learn different coordinate systems for queries, keys, values, and feed-forward features.

6.9 FP32

IEEE 754 binary32 allocates one sign bit, eight exponent bits, and 23 explicitly stored fraction bits (24 bits of precision for normal values because of the implicit leading bit). It offers much wider range and finer precision than 16-bit formats at twice their payload size. A workload labeled “FP32” can still use fused operations, reduced-precision hardware modes, or library-specific accumulation behavior; inspect operations rather than relying on the label.

6.10 FP16

FP16 saves memory and can use fast matrix hardware, but its exponent range is much narrower than FP32 or BF16. Small gradients can underflow and large values can overflow. Loss scaling shifts gradients into a representable range; it cannot repair every unstable computation.

6.11 BF16

BF16 keeps an exponent width comparable to FP32 with fewer fraction bits. It trades precision for range, which often makes training easier than FP16 while retaining two-byte storage. Accumulation may still occur in FP32.

6.12 FP8

FP8 denotes families of eight-bit floating formats with different exponent/fraction tradeoffs. Scaling and hardware support are part of the method. Saying “FP8” without naming the format and scaling policy is incomplete.

6.13 INT8

Integer quantization maps real values to an integer grid using scale and possibly zero point. Per-tensor scaling is cheap but sensitive to outliers; per-channel or grouped scaling can reduce error at metadata and kernel complexity cost.

6.14 INT4

Four-bit weights can roughly halve packed payload relative to INT8, but scales, zeros, group metadata, padding, and unpacking remain. A nominal 4-bit model is not exactly $\text{parameter_count}/2$ bytes on disk or in memory.

6.15 Overflow

Overflow occurs when magnitude exceeds the representable range, often producing infinity. $\exp(\text{large_logit})$ is a familiar trigger. Subtracting the maximum logit before exponentiation prevents this without changing softmax.

6.16 Underflow

Underflow maps very small magnitudes to subnormal values or zero. Multiplying many probabilities underflows quickly, motivating sums of log-probabilities. A zeroed gradient cannot guide an update.

6.17 Numerical stability

Stable algorithms compute the intended mathematical quantity while avoiding avoidable range and cancellation problems. Associativity fails in floating point: $(a+b)+c$ may differ from $a+(b+c)$. Parallel reductions therefore need tolerances, not blind bitwise equality.

6.18 Exit problem

Compare query $q=[1,0]$ with $a=[2,0]$ and $b=[1,1]$. Dot products are 2 and 1; cosines are 1 and about 0.707. Both rank a first, but for $c=[100,100]$, dot product favors c while cosine still favors a . Then explain why stable softmax subtracts the maximum and why packed INT4 memory includes more than weight nibbles.

6.19 References

- IEEE, [IEEE Standard for Floating-Point Arithmetic \(IEEE 754-2019\)](#), 2019.
- Micikevicius et al., [Mixed Precision Training](#), 2017.

Chapter 7

Probability, Logits and Loss

Status: **Draft — not yet independently reviewed.**

A language model returns a conditional distribution. Probability gives us the vocabulary for interpreting it; log space makes the computation practical.

7.1 Conditional probability

$P(x_t \mid x_{<t})$ is the probability assigned to the next token given preceding tokens. The chain rule factorizes a sequence: $P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{<t})$, where $x_{<1}$ denotes the chosen start condition. The equality is a probability identity; an autoregressive model approximates the conditional factors.

7.2 Likelihood

Given observed training tokens, likelihood treats model parameters as variable and asks how much probability the model assigns to the data. Maximum-likelihood training chooses parameters that increase this value. High likelihood on training data alone may reflect memorization.

7.3 Log-probability

Products of many probabilities become sums of log-probabilities: $\log P(x_{1:T}) = \sum_t \log P(x_t \mid x_{<t})$. Sums are numerically safer and easier to optimize. Log-probabilities are nonpositive because probabilities do not exceed one.

7.4 Entropy

Entropy $H(p) = -\sum_i p_i \log p_i$ measures uncertainty inside one distribution. A point mass has zero entropy; a uniform distribution over V tokens has entropy $\log V$. Entropy is not factuality: a confidently wrong model has low entropy.

7.5 KL divergence

$D_{KL}(p\|q) = \sum_i p_i \log[p_i/q_i]$ measures the expected log-ratio under p . It is nonnegative but asymmetric and is not a distance metric. In post-training, KL penalties can discourage a policy from moving too far from a reference under a chosen direction and sampling distribution.

7.6 Softmax

For logits z , $p_i = \exp(z_i) / \sum_j \exp(z_j)$. Adding the same constant to every logit leaves probabilities unchanged. Subtract $m = \max z$ before exponentiation to stay in range.

7.7 Logits

Logits are unnormalized scores. Their differences determine odds: $p_i/p_j = \exp(z_i - z_j)$. Absolute offsets do not matter to softmax. Temperature divides logits before softmax; it changes relative sharpness without retraining weights.

7.8 Cross-entropy

For target distribution y , cross-entropy is $H(y, p) = -\sum_i y_i \log p_i$. With a one-hot target class k , it becomes $-\log p_k$. Averaging token losses requires a declared denominator and mask; padding and ignored prompt tokens can otherwise corrupt comparisons.

7.9 Log-sum-exp

Combining softmax and negative log-likelihood yields stable token loss

$$L = -z_k + m + \log \sum_j \exp(z_j - m).$$

This avoids first constructing tiny probabilities. The derivative with respect to logit z_i is $p_i - y_i$: raise the target logit and reduce competing logits in proportion to their predicted probability.

7.10 Worked example

For logits $[2, 1, 0]$, stable exponentials are $[1, e^{-1}, e^{-2}]$, probabilities are about $[0.665, 0.245, 0.090]$, and loss for target 0 is about 0.408. Greedy accuracy is one for this example. Changing logits to $[2, 1.9, 0]$ keeps greedy accuracy one but increases target loss: likelihood detects reduced confidence that accuracy hides.

Sampled accuracy is noisier still. A token with probability 0.665 is omitted about one-third of draws. Sampling evaluates a decoding process; cross-entropy evaluates assigned probability to known targets.

7.11 Exit derivation

Prove shift invariance by replacing every z_i with $z_i + c$ and cancelling e^c . Then derive the one-hot loss above. Finally explain three distinct facts: entropy belongs to one distribution, cross-entropy

compares targets with predictions, and KL compares two normalized distributions in a direction-sensitive way.

7.12 References

- Shannon, [A Mathematical Theory of Communication](#), 1948 — entropy and information measures.
- Goodfellow, Bengio, and Courville, [Deep Learning, Chapter 3](#), 2016 — probability, cross-entropy, and KL divergence.

Chapter 8

Gradients, Graphs and Autodiff

Status: **Draft — not yet independently reviewed.**

Training becomes less mysterious when `backward()` is treated as organized chain-rule bookkeeping.

8.1 Derivatives

A derivative measures local sensitivity. For $f(x) = x^2$, $df/dx = 2x$. At $x = 3$, a small change Δx changes the output by approximately $6\Delta x$. The approximation improves as the change shrinks, until floating-point cancellation interferes.

8.2 Gradients

For scalar loss L and parameter vector θ , the gradient collects partial derivatives: $\nabla_{\theta} L = [\partial L / \partial \theta_1, \dots]$. It points toward steepest local increase under the Euclidean metric; gradient descent moves opposite it.

8.3 Chain rule

If $y = f(x)$ and $L = g(y)$, then $dL/dx = (dL/dy)(dy/dx)$. Computation graphs apply this locally. For $L = (ab + c)^2$, each operation needs only its inputs and the gradient arriving from downstream.

8.4 Computational graphs

A graph node represents a value; an edge records dependency through an operation. Shared values require gradient accumulation. If a contributes through two paths, its total derivative is the sum of both path contributions. A topological order ensures downstream gradients arrive before a node propagates backward.

8.5 Automatic differentiation

Automatic differentiation applies analytic derivative rules for primitive operations to the executed graph, while the values and derivative calculations remain subject to floating-point rounding. At nondifferentiable points, a library must choose a convention or report that no derivative exists. Autodiff differs from symbolic differentiation, which manipulates formulas, and numerical differentiation, which perturbs inputs.

8.6 Reverse-mode autodiff

Reverse mode is efficient when many parameters influence one scalar loss. The forward pass records parents and local backward rules. The backward pass seeds loss gradient 1 and traverses nodes in reverse topological order, accumulating contributions into parents. PyTorch uses reverse automatic differentiation and rebuilds its dynamic graph each iteration.

8.7 Finite-difference checks

A centered check estimates $df/dx \approx [f(x+h) - f(x-h)]/(2h)$. Too-large h creates truncation error; too-small h creates rounding and cancellation error. Check smooth toy inputs away from nondifferentiable points, use float64 where possible, and compare relative as well as absolute error.

8.8 The micro-autograd engine

Run:

```
python code/micrograd.py
```

For $L=(ab+c)^2$ at $a=2, b=-3, c=10$, the intermediate $ab+c=4$, so $L=16$. Analytically: $dL/da=2(4)b=-24$, $dL/db=2(4)a=16$, and $dL/dc=8$. The program checks these values and finite differences.

The engine accumulates with $+=$. Replacing accumulation with assignment fails when a value has multiple children. It also rebuilds the graph for each expression; reusing it after mutating values would no longer describe the executed computation.

8.9 Four perspectives

Follow the Token: token losses at many positions reduce to a scalar objective.

Follow the Gradient: the scalar seed travels backward through loss, logits, blocks, and parameters.

Follow the Byte: saved forward values trade memory for the ability to compute local derivatives; checkpointing later recomputes some values instead.

Follow the Request: ordinary serving omits this graph. Accidentally recording it wastes memory even if no optimizer step occurs.

8.10 Exit check

Modify the program to use $L=(a*a)+(a*b)$. Confirm that $a.grad=2a+b$, demonstrating path accumulation. Test finite differences at several values. A passing example supports the implemented

primitives; it does not validate a future tensor engine, broadcasting rules, or nondifferentiable operations.

8.11 References

- [PyTorch autograd mechanics](#) — dynamic graph construction, saved tensors, reverse-mode behavior, and grad controls. Accessed 2026-09-18.

Part 3: Neural Networks Before Transformers

Chapter 9

Linear Layers and Nonlinearities

Status: **Draft — not yet independently reviewed.**

A neural network begins with a modest operation: mix input coordinates, shift the result, then bend it. The bend is what lets depth create functions a single affine map cannot express.

9.1 Linear layers

For an input row vector $x \in \mathbb{R}^{D_{in}}$, a layer computes $z = xW + b$, with $W \in \mathbb{R}^{D_{in} \times D_{out}}$. A batch $X \in \mathbb{R}^{B \times D_{in}}$ produces $Z \in \mathbb{R}^{B \times D_{out}}$. Each output coordinate is a learned weighted sum of every input coordinate.

Strictly, $xW + b$ is affine when $b \neq 0$, though libraries call it linear. Two such layers without an activation collapse: $(xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$. Depth alone adds no new function class.

9.2 Biases

The bias moves a decision boundary away from the origin. For one feature, $z = 2x - 3$ crosses zero at $x = 1.5$; without the bias it must cross at zero. In batched code the same bias vector is broadcast across rows. A common bug is to give each example its own bias, changing both meaning and parameter count.

9.3 Nonlinearities

An activation $a = \phi(z)$ prevents adjacent affine maps from collapsing. With enough hidden units, bent regions can approximate curved functions and disconnected decision regions. It does not create information: if a projection destroyed a distinction, an activation cannot recover it.

9.4 ReLU

$\text{ReLU}(z) = \max(0, z)$ is cheap and has derivative 1 for positive inputs and 0 for negative inputs. It creates piecewise-linear networks. At zero, frameworks choose a subgradient convention. A unit that remains negative for all data receives zero upstream gradient and may become a “dead” unit.

Worked example: $x = [2, -1]$, $W = \begin{bmatrix} 1 & -2 \\ 3 & 1 \end{bmatrix}$, and $b = [0, 1]$ give $z = [-1, -4]$ and ReLU output $[0, 0]$. The shape is correct, but this particular example sends no gradient through either activation.

9.5 GELU

GELU scales rather than sharply gates: $\text{GELU}(x) = x\Phi(x)$, where Φ is the standard-normal CDF. Negative values can survive weakly, and the derivative changes smoothly. Transformer implementations often use an exact or tanh-based approximation; mixing variants can cause small reproducibility differences.

9.6 Parameter shapes and cost

The layer has $D_{in}D_{out} + D_{out}$ trainable scalars. With $D_{in} = 768$, $D_{out} = 3072$, BF16 parameters occupy about 4.72 MB (decimal), before gradients or optimizer state. A forward batch costs roughly $2BD_{in}D_{out}$ FLOPs if a multiply and add count as two.

9.7 Four perspectives

Follow the Token: one token state of width D_{in} becomes a state of width D_{out} . **Follow the Gradient:** $\partial L / \partial W = X^T(\partial L / \partial Z)$ and bias gradients sum across the batch. **Follow the Byte:** weights are reused across examples; their movement can dominate small batches. **Follow the Request:** inference applies identical learned parameters to each request and stores no optimizer state.

9.8 Exit check

Prove algebraically that three affine layers collapse to one. Then sketch a two-ReLU construction for a one-dimensional “tent” function. Explain why a wrong bias shape might run successfully under broadcasting while learning the wrong model.

9.9 A nonlinear test case

The XOR dataset makes the limitation visible: inputs $(0, 0)$ and $(1, 1)$ share class 0, while $(0, 1)$ and $(1, 0)$ share class 1. No single line separates those pairs. A two-layer network can create hidden half-spaces and combine them into the required disconnected region. When testing an implementation, compare a linear classifier against the same training loop with a hidden activation, repeat across seeds, and report both optimization failures and successful fits. One lucky run is not an architectural proof; the geometry establishes impossibility for the linear model, while repeated execution checks the code.

9.10 References

- [PyTorch Linear](#) and [activation functions](#). Accessed 2026-09-19.
- Hendrycks and Gimpel, [Gaussian Error Linear Units](#), 2016.

Chapter 10

A Tiny Network, Forward and Backward

Status: **Draft — not yet independently reviewed.**

We now make learning concrete with a two-layer scalar network. Every number is small enough to check by hand.

10.1 Building a tiny neural network

Let $x = 2$, target $y = 0$, and $h = \text{ReLU}(w_1x + b_1)$, $\hat{y} = w_2h + b_2$, $L = \frac{1}{2}(\hat{y} - y)^2$. Choose $w_1 = 0.5, b_1 = 0, w_2 = -1, b_2 = 2$. These four scalars are model state. The intermediate activation and loss belong to this execution.

This network has one hidden unit only so the arithmetic stays visible. Real networks replace scalars with matrices, but the graph and local derivative rules are the same.

10.2 Forward propagation

Compute in program order: $z_1 = 1$, $h = 1$, $\hat{y} = 1$, and $L = 0.5$. A forward pass is evaluation; it neither discovers a gradient nor updates a parameter. For a batch, inputs become $X[B, D_{in}]$, hidden states $H[B, D_h]$, and outputs $Y[B, D_{out}]$. Vectorization changes execution efficiency, not the per-example function.

10.3 Backpropagation by hand

Begin at the loss: $\partial L / \partial \hat{y} = \hat{y} - y = 1$. Therefore $\partial L / \partial w_2 = h = 1$, $\partial L / \partial b_2 = 1$, and $\partial L / \partial h = w_2 = -1$. Because $z_1 = 1 > 0$, ReLU's local derivative is 1. Thus $\partial L / \partial w_1 = (-1)(1)x = -2$ and $\partial L / \partial b_1 = -1$.

With learning rate 0.1, one SGD step gives $w_1 = 0.7, b_1 = 0.1, w_2 = -1.1, b_2 = 1.9$. Re-evaluating yields $h = 1.5, \hat{y} = 0.25$, and $L = 0.03125$: a lower loss for this example. The decrease is evidence for this step, not a guarantee for arbitrary learning rates.

10.4 Gradient shapes

Every parameter gradient has the parameter's shape. For $Z = XW + b$, upstream $G = \partial L / \partial Z$ gives $\partial L / \partial W = X^T G$, $\partial L / \partial X = G W^T$, and $\partial L / \partial b = \sum_{i=1}^B G_i$. Shape reasoning catches transpose and reduction errors before numerical debugging.

Gradients accumulate when a parameter is reused. If a weight participates at several sequence positions, its gradient is the sum of all those paths. Accidentally overwriting rather than accumulating produces a plausible but incorrect update.

10.5 Four perspectives

Follow the Token: the example becomes a hidden activation and a prediction. **Follow the Gradient:** sensitivity travels in reverse through the exact executed path. **Follow the Byte:** backward needs saved x, z_1, h or must recompute them. **Follow the Request:** training requests retain a graph and mutate parameters after aggregation; serving requests normally do neither.

10.6 Exit check

Repeat the calculation with $b_1 = -2$, making $z_1 = -1$. Which first-layer gradients become zero, and why? Verify the original derivatives with centered finite differences. State what happens if the loss is summed rather than averaged over a batch.

10.7 Implementation discipline

A trustworthy tiny-network lab should make parameter mutation explicit. First clear accumulated gradients. Then run the forward graph, reduce the loss, run backward once, optionally clip gradients, update each parameter under a no-gradient context, and only then begin the next graph. Record loss both before and after the update on the same example. Compare analytical gradients with finite differences in float64 and avoid ReLU's kink during the check. A correct scalar example does not validate broadcasting, batched reductions, or device kernels, so add shape assertions as the implementation grows.

There are two useful failure injections. Replace gradient accumulation by assignment and reuse a weight twice; the shared-path test should fail. Then update a parameter before all backward computations finish; the saved forward values no longer match the mutated model. These bugs explain why mature frameworks track graph versions and separate parameter updates from differentiation.

10.8 References

- [PyTorch autograd mechanics](#). Accessed 2026-09-19.

Chapter 11

Optimization and State

Status: **Draft — not yet independently reviewed.**

Backpropagation supplies a direction. An optimizer decides how parameter state changes over time.

11.1 SGD

Plain stochastic gradient descent applies $\theta_{t+1} = \theta_t - \eta g_t$. If $\theta = 3$, $g = 4$, and $\eta = 0.1$, the new value is 2.6. “Stochastic” refers to estimating the full-data gradient from a sample or minibatch. A learning rate that is too small wastes steps; one that is too large can jump across a basin and diverge.

11.2 Momentum

Momentum filters noisy gradients: $v_t = \beta v_{t-1} + g_t$ and $\theta_{t+1} = \theta_t - \eta v_t$ under one common convention. Definitions differ by scaling and sign, so compare equations rather than optimizer names. Momentum accelerates persistent directions but can overshoot when the useful direction changes.

11.3 AdamW

Adam tracks exponential averages $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ and $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$. Bias-corrected $\hat{m}_t$ and $\hat{v}_t$ compensate for zero initialization, and the adaptive update divides by $\sqrt{\hat{v}_t} + \epsilon$. AdamW then applies decoupled weight decay separately. Adaptive scaling helps coordinates with different gradient scales, but it does not remove schedule tuning.

11.4 Optimizer states

SGD needs no persistent tensor beyond parameters unless momentum is enabled. Momentum adds one parameter-sized buffer. AdamW usually adds two, often stored in FP32 even when model weights use BF16. Training memory may also include parameters, gradients, master weights, activations, and communication buffers. “A 7B model needs 14 GB” describes BF16 weights only, not a training job.

11.5 Learning-rate schedules

A schedule makes η depend on step. Constant, step, cosine, and linear decay encode different assumptions. Cosine decay from peak η_{max} to η_{min} over progress $p \in [0, 1]$ uses $\eta = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos \pi p)$. Schedulers must step at the intended unit—optimizer step, not every microbatch when using accumulation.

11.6 Weight decay

Decoupled decay applies $\theta \leftarrow (1 - \eta\lambda)\theta$ separately from the gradient update. L2 regularization adds $\lambda\theta$ to the gradient; the two coincide for plain SGD but generally differ for adaptive methods. Biases and normalization scales are often excluded by policy; the policy should be explicit.

11.7 Four perspectives

Follow the Token: a batch produces the loss estimate that drives one update. **Follow the Gradient:** gradients are transient measurements; optimizer moments are persistent history. **Follow the Byte:** Adam’s two FP32 moments can outweigh low-precision weights. **Follow the Request:** training systems coordinate accumulation, clipping, optimizer step, scheduler step, and zeroing in a strict order.

11.8 Exit check

For two parameters in BF16 with FP32 gradients and Adam moments, write a byte formula and state whether master FP32 weights are assumed. Explain why `zero_grad` is necessary when a framework accumulates gradients. Compare coupled L2 and AdamW on one symbolic step.

11.9 Ordering one training step

With gradient accumulation, divide or otherwise normalize the loss consistently across microbatches, accumulate gradients, unscale them if mixed precision is active, detect nonfinite values, clip if configured, call the optimizer, advance the schedule, and clear gradients. Changing this order changes behavior. Clipping scaled gradients uses the wrong threshold; advancing the schedule per microbatch shortens the schedule; clearing between microbatches defeats accumulation. A skipped optimizer step after overflow should also skip scheduler advancement under the usual step-based interpretation.

To compare optimizers, hold model, data order, token budget, schedule budget, precision, and evaluation fixed. Reporting only the lowest training loss rewards overfitting and extra compute. Optimizer choice is part of a full experimental configuration, including epsilon, betas, decay exclusions, batch size, and gradient normalization.

11.10 References

- Kingma and Ba, [Adam](#), 2014.
- Loshchilov and Hutter, [Decoupled Weight Decay Regularization](#), 2017.
- [PyTorch optimizer documentation](#). Accessed 2026-09-19.

Chapter 12

Initialization and Training Stability

Status: **Draft — not yet independently reviewed.**

A valid training program can still fail because signals shrink, explode, overflow, or change too abruptly. Stability is observed through measurements, not inferred from the absence of an exception.

12.1 Initialization

If units in the same layer start with identical incoming and outgoing parameters, the architecture and data treat them symmetrically: they receive identical gradients and remain redundant. Random weight initialization breaks that symmetry, while its scale aims to preserve activation and gradient variance. Xavier-style scaling suits roughly symmetric activations; He scaling uses weight variance near $2/fan_{in}$ for ReLU. Biases may still be initialized identically because randomized weights already distinguish the units. These rules are approximations based on independence and distribution assumptions, not universal guarantees.

12.2 Gradient norms

The global L2 norm is $\|g\|_2 = \sqrt{\sum_i g_i^2}$. Log global and per-layer norms alongside parameter norms and update-to-weight ratios. A single global number can hide one broken layer; per-layer traces reveal where failure begins.

12.3 Clipping

Global norm clipping with threshold c rescales $g \leftarrow g \min(1, c/\|g\|_2)$. For $g = [3, 4]$ and $c = 2$, the norm falls from 5 to 2 and the gradient becomes $[1.2, 1.6]$. Clipping limits a damaging step; chronic clipping usually signals a deeper problem such as scale, data, precision, or learning rate.

12.4 Warmup

Warmup raises the learning rate gradually during early steps while activations and optimizer moments are poorly calibrated. Linear warmup to η_{max} over N optimizer steps uses $\eta_t = \eta_{max} t/N$. Count optimizer steps consistently when gradient accumulation is active.

12.5 Vanishing gradients

Backprop multiplies local Jacobians. Repeated factors below one can shrink sensitivity exponentially, leaving early layers nearly unchanged. Saturating sigmoid or tanh units amplify this risk. Residual connections, normalization, suitable initialization, and gated recurrence create shorter or better-conditioned paths.

12.6 Exploding gradients

Repeated factors above one can create enormous norms, overflow, NaNs, and destructive updates. Check the first nonfinite tensor rather than only the eventual loss. Lowering the learning rate cannot repair an overflow that already occurred in the forward pass.

12.7 Stability diagnostics

Track loss, learning rate, gradient norms before clipping, clipping fraction, activation mean/variance, parameter norms, update ratios, nonfinite counts, and throughput. Reproduce with a fixed small batch; overfit that batch; then add scale and mixed precision one change at a time. Data corruption can mimic optimizer instability, so inspect tokens and targets too.

12.8 Four perspectives

Follow the Token: malformed or extreme inputs can poison activation statistics. **Follow the Gradient:** identify the earliest layer where norms collapse or spike. **Follow the Byte:** lower precision narrows numerical range and loss scaling changes how gradients occupy it. **Follow the Request:** the training loop should fail fast on nonfinite values and checkpoint enough state for reproduction.

12.9 Exit check

Given per-layer norms $[0.9, 0.8, 0.02, 0.0001]$, propose a diagnostic sequence without claiming the cause from norms alone. Calculate the clipped version of $[6, 8]$ at threshold 5. Explain why increasing epsilon may mask symptoms while altering optimization.

12.10 References

- Glorot and Bengio, [Understanding the difficulty of training deep feedforward neural networks](#), 2010.
- He et al., [Delving Deep into Rectifiers](#), 2015.
- [PyTorch gradient clipping](#). Accessed 2026-09-19.

Part 4: Turning Language Into Numbers

Chapter 13

Tokenization Algorithms

Status: **Draft — not yet independently reviewed.**

Models consume integers from a finite vocabulary. Tokenization defines the reversible contract between text and those integers, and its choices affect sequence length, multilingual coverage, code handling, cost, and compatibility.

13.1 Why machines need tokens

Text arrives as encoded bytes, while a model predicts among a fixed number of categories. A tokenizer segments text and maps each segment to an ID. The decoder reverses IDs into segments and reconstructs text. Token boundaries need not match linguistic words: unbelievable might become un, believ, able.

13.2 Characters

A character vocabulary is compact and handles unseen words, but sequences grow long. “Transformers” has 12 ASCII characters and potentially more Unicode code points or bytes under other scripts. Unicode normalization matters: visually identical strings can have different code-point sequences.

13.3 Words

Word tokens shorten common prose and offer intuitive boundaries, but vocabularies explode across inflections, spelling, identifiers, and languages. An out-of-vocabulary word must collapse to an unknown token or be decomposed by another scheme. Punctuation and whitespace rules become part of the model contract.

13.4 Subwords

Subwords trade vocabulary size against sequence length. Frequent strings become single tokens; rare strings decompose into smaller known units. The learned inventory reflects corpus frequencies, so a tokenizer trained mostly on English may represent another language inefficiently. Token count is therefore a property of both text and tokenizer.

13.5 BPE from scratch

Byte Pair Encoding begins with small symbols and repeatedly merges the most frequent adjacent pair. For a corpus containing `low`, `lower`, and `lowest`, an early merge might join `l+o`, then `lo+w`. Encoding applies learned merges in rank order. Training and encoding require deterministic tie-breaking; otherwise two implementations may produce incompatible vocabularies.

A minimal trainer counts adjacent pairs weighted by word frequency, selects one pair, replaces non-overlapping occurrences, and repeats until the vocabulary budget is reached. Naive rescanning is educational but slow. Production trainers maintain efficient pair statistics and carefully specify pre-tokenization, normalization, byte fallback, and merge order.

13.6 WordPiece

WordPiece also builds subwords but historically chooses additions using a likelihood-related criterion rather than raw pair frequency. Common implementations mark continuation pieces, such as `play + ##ing`. The exact training algorithm is implementation-specific; the vocabulary plus segmentation procedure determines compatibility.

13.7 SentencePiece

SentencePiece trains directly from raw sentences and treats whitespace as an ordinary symbol, commonly rendered `_`. It supports BPE and unigram language-model tokenization. The unigram method begins with candidates and removes pieces while minimizing likelihood damage, enabling alternative segmentations during training.

13.8 Byte-level tokenization

Starting from 256 byte values guarantees coverage of arbitrary UTF-8 input. Learned merges recover compact common strings. Coverage does not guarantee efficiency: rare scripts or binary-like data may expand heavily, and a Unicode character can span several byte tokens. Decode must concatenate bytes before UTF-8 interpretation; decoding each token independently can create invalid fragments.

13.9 Four perspectives

Follow the Token: raw bytes become segments and IDs. **Follow the Gradient:** discrete segmentation is fixed during ordinary model training; gradients update embeddings, not merge choices. **Follow the Byte:** tokenization changes T , which changes activation, attention, and KV-cache cost. **Follow the Request:** the server must use the exact normalization, special-token policy, vocabulary, and merge model expected by the weights.

13.10 Exit check

Perform three BPE merges on `low lower lowest` with stated frequencies and tie-breaking. Verify `decode(encode(text)) == text` for whitespace, emoji, composed/decomposed accents, and an unseen script. Explain why round-trip success alone does not establish efficient tokenization.

13.11 References

- Sennrich, Haddow, and Birch, [Neural Machine Translation of Rare Words with Subword Units](#), 2016.
- Kudo and Richardson, [SentencePiece](#), 2018.
- Schuster and Nakajima, [Japanese and Korean Voice Search](#), 2012.

Chapter 14

Vocabulary and Token IDs

Status: **Draft — not yet independently reviewed.**

A vocabulary is an ordered mapping, not merely a bag of strings. Its row numbers connect tokenizer output to embedding and output-projection rows.

14.1 Vocabulary

Suppose the vocabulary is $\{0:<\text{pad}>, 1:\text{cat}, 2:\text{dog}, 3:\text{s}\}$. Its size $V = 4$ sets the embedding table's first dimension and the number of language-model logits. Adding a token creates a new category; removing or reordering one changes the semantics of an existing row. The tokenizer model, normalization rules, pre-tokenizer, merge ranks, and special-token configuration together form the full artifact.

14.2 Token IDs

An ID is an index, not a magnitude. ID 100 is not semantically “greater” than ID 5. Feeding raw IDs as continuous scalar features would invent an ordering; embedding lookup uses the ID to select a row. For a batch of IDs $I[B, T]$, lookup into $E[V, D]$ produces $X[B, T, D]$.

Two tokenizers may emit different IDs for the same visible token. Even when both contain cat, its row can differ. Human-readable token strings therefore cannot establish model compatibility.

14.3 Special tokens

Special tokens encode protocol events such as beginning/end of text, padding, unknown input, separators, roles, or tool boundaries. Some are allowed only when inserted by a chat template. Treating user text that resembles a control token as the control token can create injection or parsing errors. Tokenizers distinguish ordinary encoding from explicit special-token recognition.

Padding is usually masked so it does not affect attention or loss. End-of-sequence often terminates generation, but the runtime's stopping policy decides behavior. A token's name alone does not enforce semantics.

14.4 Unknown tokens

Word-level and some subword tokenizers use <unk> when no decomposition exists. Multiple distinct strings then become the same ID, irreversibly losing information. Byte fallback avoids unknown text by representing its encoded bytes, though at potentially high token cost. An unknown token appearing frequently in production is a monitoring signal for data or configuration drift.

14.5 Tokenizer-model compatibility

Imagine swapping IDs 1 and 2 above while leaving weights unchanged. Text cat now retrieves the row learned for dog, and the logit formerly decoded as cat becomes dog. Tensor shapes still match; outputs become wrong without a clean exception. Compatibility requires the exact token-to-ID map, special IDs, added tokens, normalization, chat template, and vocabulary size used for training or adaptation.

Vocabulary expansion is possible: resize input embeddings and output head, initialize new rows, and train them. It is a model change, not a harmless tokenizer edit. With tied input/output weights, both interfaces change together. Shrinking a vocabulary requires remapping or discarding rows and can invalidate checkpoints and adapters.

14.6 Four perspectives

Follow the Token: a string segment becomes one stable integer and selects one row. **Follow the Gradient:** only rows referenced by a batch receive gradients through the lookup operation, though a tied or separate output softmax can touch all vocabulary rows; optimizer weight decay can also change a row with zero data gradient. **Follow the Byte:** embeddings and LM head scale as $V \times D$, so vocabulary expansion has a measurable memory cost. **Follow the Request:** client templates and server tokenizers must agree on control tokens and stop IDs.

14.7 Exit check

Construct a four-token vocabulary and embedding table. Permute two IDs without moving rows and describe the failure. Then permute both IDs and rows consistently. List the artifacts you would hash in a production model package to detect mismatch.

14.8 References

- [Hugging Face tokenizer summary](#) and [special tokens](#). Accessed 2026-09-19.

Chapter 15

Embedding Matrices and Geometry

Status: **Draft — not yet independently reviewed.**

An embedding layer turns a category index into a learned vector. That vector is an initial state, not a permanent dictionary definition.

15.1 Embedding matrices

An embedding table $E \in \mathbb{R}^{V \times D}$ contains one D -dimensional row for each vocabulary item. IDs $[2, 5, 2]$ return rows $[E_2, E_5, E_2]$, producing shape $[3, D]$. Mathematically, lookup equals multiplying a one-hot vector by E , but gathering rows avoids materializing enormous sparse one-hot tensors.

Initialization is usually random. Meaningful structure emerges because gradient updates make rows useful for prediction. Rows can encode many entangled regularities; assigning a single human meaning to a coordinate is rarely justified.

15.2 Static embeddings

A static token embedding returns the same table row for the same ID before context processing. In a simple word-vector model, bank might have one vector even in “river bank” and “bank loan.” Static geometry can capture distributional similarity, but polysemy is compressed into one point.

15.3 Contextual embeddings

After Transformer layers, each occurrence has a hidden state conditioned on surrounding tokens. The two occurrences of bank begin with the same token row but diverge as attention and MLPs mix context. Calling every hidden state an “embedding” is common but ambiguous; specify whether you mean input-table rows, layer activations, pooled sequence vectors, or a separately trained retrieval model.

15.4 Representation geometry

Cosine similarity compares direction: $\cos(a, b) = a \cdot b / (\|a\| \|b\|)$. Euclidean distance also depends on magnitude. Nearest neighbors can expose useful structure, but similarity is evidence about geometry under a metric, not proof that the model “understands” a concept. An invertible rotation

changes coordinates while preserving dot products, showing why individual dimensions have no stable universal semantics.

Anisotropy, frequency effects, and normalization can distort neighbor searches. Always state the layer, pooling method, normalization, metric, corpus, and model revision when reporting embedding results.

15.5 Embedding lookup and gradients

If IDs $[2, 5, 2]$ yield upstream gradients g_1, g_2, g_3 , then row 2 receives $g_1 + g_3$, row 5 receives g_2 , and unused rows receive zero from this lookup. Repeated-token accumulation is the sparse analogue of shared-parameter gradient accumulation. Dense output heads may still generate gradients for every vocabulary row when weights are tied.

For $V = 50,000$, $D = 4096$, a BF16 table stores 409.6 MB decimal. Lookup performs little arithmetic relative to bytes moved, so it is memory-oriented. During distributed training, sharding or sparse access patterns complicate the simple local picture.

15.6 Four perspectives

Follow the Token: ID i selects E_i , then context transforms that state layer by layer. **Follow the Gradient:** repeated IDs accumulate into shared rows. **Follow the Byte:** table size is VD elements and lookup is a gather with irregular access. **Follow the Request:** tokenization and checkpoint revision determine which semantic row a request retrieves.

15.7 Exit check

Given IDs $[1, 1, 3]$ and three upstream vectors, write the nonzero row gradients. Compare cosine and Euclidean neighbors before and after scaling one vector by ten. Explain why a high probe score from one layer would be correlational rather than automatically causal.

15.8 References

- [PyTorch Embedding](#). Accessed 2026-09-19.
- Mikolov et al., [Efficient Estimation of Word Representations in Vector Space](#), 2013.
- Peters et al., [Deep contextualized word representations](#), 2018.

Chapter 16

Position and Sequence Order

Status: **Draft — not yet independently reviewed.**

Self-attention based only on token content is permutation equivariant: rearranging input rows rearranges outputs in the same way. Language needs order, so the model must receive positional information.

16.1 Position and sequence order

“Dog bites person” and “person bites dog” contain the same token multiset but express different relationships. If input states are only token embeddings and attention has no positional signal, the mechanism cannot distinguish the permutations beyond corresponding row order. Position can enter by addition, attention-score bias, or transformations of queries and keys.

16.2 Sinusoidal positions

The original Transformer adds fixed sinusoids: $PE(pos, 2i) = \sin(pos/10000^{2i/D})$ and $PE(pos, 2i + 1) = \cos(pos/10000^{2i/D})$. Different frequencies provide a structured code for each position. Because phase differences express offsets, relative positions can be derived through linear relationships. No positional parameters are learned, though extrapolation beyond training length is not automatically reliable.

16.3 Learned positions

A learned table $P[T_{max}, D]$ adds row P_t to the token at position t . It is flexible and simple, but adds parameters and has no trained rows beyond T_{max} . Extending the table by random initialization does not teach the model to use longer contexts; interpolation and continued training are model changes requiring evaluation.

16.4 Relative-position methods

Relative methods modify attention using distance $i - j$ rather than only absolute i and j . A learned or fixed bias can favor nearby or direction-specific positions. T5-style relative biases bucket distances so many far offsets share parameters. ALiBi adds head-specific linear penalties

based on distance. These schemes change attention scores directly and often avoid a full $T \times D$ position table.

16.5 RoPE introduction

Rotary Position Embedding rotates pairs of query and key coordinates by a position-dependent angle. The dot product between a query at i and key at j then depends on their relative phase $i - j$. Values are not rotated in the standard construction. For one 2-D pair, $R(\theta p)[x_1, x_2]^T$ uses the familiar rotation matrix; multiple coordinate pairs use different frequencies.

RoPE consumes little persistent memory, but context extension depends on frequency behavior and training. Scaling recipes modify positions or frequencies and must be validated for quality across both short and long contexts.

16.6 Worked invariance check

Take content vectors a, b and swap their rows. Without positions, their pairwise score matrix swaps rows and columns consistently. Add distinct P_0, P_1 before projections and the states are no longer simply the same multiset. This demonstrates the missing information rather than claiming any one positional method is universally best.

16.7 Four perspectives

Follow the Token: the same token gets a different state or attention relation at each position. **Follow the Gradient:** learned tables and biases receive gradients; fixed sinusoids and fixed rotations do not. **Follow the Byte:** learned positions store $T_{max}D$ parameters, while score biases may materialize or generate T^2 relations. **Follow the Request:** cached decode must assign monotonically correct position indices, including after prefix reuse or batching.

16.8 Exit check

Show why content-only self-attention is permutation equivariant. Calculate one 2-D rotation at positions 1 and 3 and relate its dot product to their offset. List two reasons a model can fail beyond its training context even when its positional formula is defined there.

16.9 References

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Shaw, Uszkoreit, and Vaswani, [Self-Attention with Relative Position Representations](#), 2018.
- Su et al., [RoFormer](#), 2021.

Part 5: Attention From First Principles

Chapter 17

Sequence Modeling and the Attention Idea

Status: **Draft — not yet independently reviewed.**

A sequence model must combine information across positions while respecting order and, for generation, causality. Recurrence and attention offer different routes through that problem.

17.1 Sequence modeling bottlenecks

To predict the next word in “The keys to the cabinet are...,” a model must preserve which noun controls the verb across intervening words. A fixed-width state may need to carry many facts, and a serial dependency path makes distant learning harder. Any architecture also faces compute and memory limits; attention moves the bottleneck rather than abolishing it.

17.2 RNNs

A basic recurrent network updates $h_t = \phi(W_x x_t + W_h h_{t-1} + b)$ and emits from h_t . The same parameters are reused at every step. This naturally summarizes a prefix and supports streaming with constant state size, but training unfolds a chain of length T . Later states cannot be computed before earlier ones, limiting parallelism. Gradients multiply through recurrent Jacobians, encouraging vanishing or explosion.

With $h_t = 0.5h_{t-1} + x_t$ and $h_0 = 0$, inputs $[2, 0, 0]$ leave states $[2, 1, 0.5]$: the first input survives but decays. This scalar example makes the compression explicit.

17.3 LSTMs

An LSTM adds a cell state and learned gates controlling writing, forgetting, and reading. The cell update $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ creates a more direct gradient path when forget gates stay near one. It improves long-range learning but remains sequential and does not guarantee indefinite memory. Gates can saturate, state capacity is finite, and all past information is still compressed into fixed-width vectors.

17.4 The attention idea

Attention lets each position build a weighted mixture of source states. Rather than asking one recurrent state to preserve everything, the current position scores candidate states, normalizes the scores, and retrieves their values. In self-attention, source and destination positions belong to the same sequence.

This shortens the dependency path between distant tokens: one attention layer can connect them directly. During training, positions can be processed in parallel. The tradeoff is that dense attention compares many pairs and typically has quadratic score storage or work in sequence length.

17.5 Content-based retrieval

Suppose a query produces scores $[2, 0, -1]$ for three earlier states. Softmax yields approximately $[0.844, 0.114, 0.042]$. If scalar values are $[10, 4, -2]$, the retrieved result is about 8.81. The weights are data-dependent and differentiable, so training can shape both the addressing scheme and retrieved content.

Attention weights are not automatically faithful explanations of model decisions. They show coefficients in one operation; residual paths, value vectors, later layers, and nonlinearities also matter. Causal claims require interventions, not attractive heatmaps.

17.6 Four perspectives

Follow the Token: recurrence repeatedly rewrites one summary; attention retrieves from many position states. **Follow the Gradient:** recurrent paths traverse time steps, while attention offers shorter paths but couples many token pairs. **Follow the Byte:** RNN inference holds compact state; attention may retain per-token KV state. **Follow the Request:** recurrent decoding is inherently serial; Transformer prefill parallelizes positions, while autoregressive decode remains sequential across generated tokens.

17.7 Exit check

Calculate the scalar RNN states for $[1, 2, 3]$ with coefficient 0.25. Then choose three values and reproduce the weighted retrieval above. State one advantage and one cost of each mechanism without declaring either universally superior.

17.8 References

- Elman, [Finding Structure in Time](#), 1990.
- Hochreiter and Schmidhuber, [Long Short-Term Memory](#), 1997.
- Bahdanau, Cho, and Bengio, [Neural Machine Translation by Jointly Learning to Align and Translate](#), 2014.

Chapter 18

Queries, Keys, Values and Scaling

Status: **Draft — not yet independently reviewed.**

Attention separates what a position seeks, what each source advertises, and what each source contributes. Those roles are queries, keys, and values.

18.1 Query

A query is the destination position’s search representation. It is not an English question; it is a learned vector whose dot products with keys drive routing. Different layers can learn different matching rules.

18.2 Key

A key is a source position’s address-like representation. A large query-key dot product indicates compatibility under the learned projections. Similar content vectors can have dissimilar keys if the layer learns to route them differently.

18.3 Value

A value is the information mixed when its key is selected. Keys decide where to read; values decide what is returned. Keeping them separate lets a source advertise one property while contributing another representation.

18.4 QKV projections

Given hidden states $X[B, T, D]$, learned matrices produce $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. In one head, $W_Q, W_K \in \mathbb{R}^{D \times d_k}$ and $W_V \in \mathbb{R}^{D \times d_v}$. Implementations often fuse the three matrix multiplications into one projection and split the result; the mathematical roles remain distinct.

18.5 Scaled dot-product attention

For one sequence and head,

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right), \quad O = AV,$$

where softmax acts across source positions and M is an optional mask or bias. Let $q = [1, 0]$, keys $k_1 = [1, 0]$, $k_2 = [0, 1]$, and values $v_1 = [2, 0]$, $v_2 = [0, 4]$. Unscaled scores are $[1, 0]$; after division by $\sqrt{2}$, softmax is approximately $[0.670, 0.330]$, giving output $[1.340, 1.320]$.

18.6 Square-root scaling

If query and key coordinates are independent with mean zero and variance one, their dot product has variance approximately d_k . Dividing by $\sqrt{d_k}$ keeps score variance near one. Without scaling, large dimensions push softmax toward saturation, producing tiny derivatives for most entries. Learned distributions violate the simple assumptions, but the variance argument explains the design.

18.7 Attention weights

Each row of A is nonnegative and sums to one over allowed sources. The output is therefore a convex combination of value rows for that head. Masked entries should receive zero probability. Numerically stable softmax subtracts the row maximum before exponentiating; using a finite mask value that is insufficiently negative in a low-precision dtype can leak probability. A row in which every source is masked has no valid normalized attention distribution and must be handled explicitly; naively applying softmax to all $-\infty$ inputs produces an undefined $0/0$ normalization and typically NaNs.

Weights depend on both query and keys. Changing the value vectors leaves weights unchanged but changes the output; changing a key can reroute every query that compares with it. Gradients flow through both routing and content paths.

18.8 Four perspectives

Follow the Token: each destination constructs a query, compares with source keys, then mixes values. **Follow the Gradient:** loss changes query/key projections through weights and value projections through the mixture. **Follow the Byte:** Q, K, V are linear in T , while the dense score matrix is quadratic. **Follow the Request:** prefill builds all positions together; decode creates one new query and compares it with cached keys.

18.9 Exit check

Recompute the worked example with values swapped, then with keys swapped. Which changes routing and which changes only retrieved content? Derive the score variance under independent unit-variance coordinates. Verify every tensor shape for $T = 4$, $d_k = 2$, $d_v = 3$.

18.10 References

- Vaswani et al., [Attention Is All You Need](#), 2017.

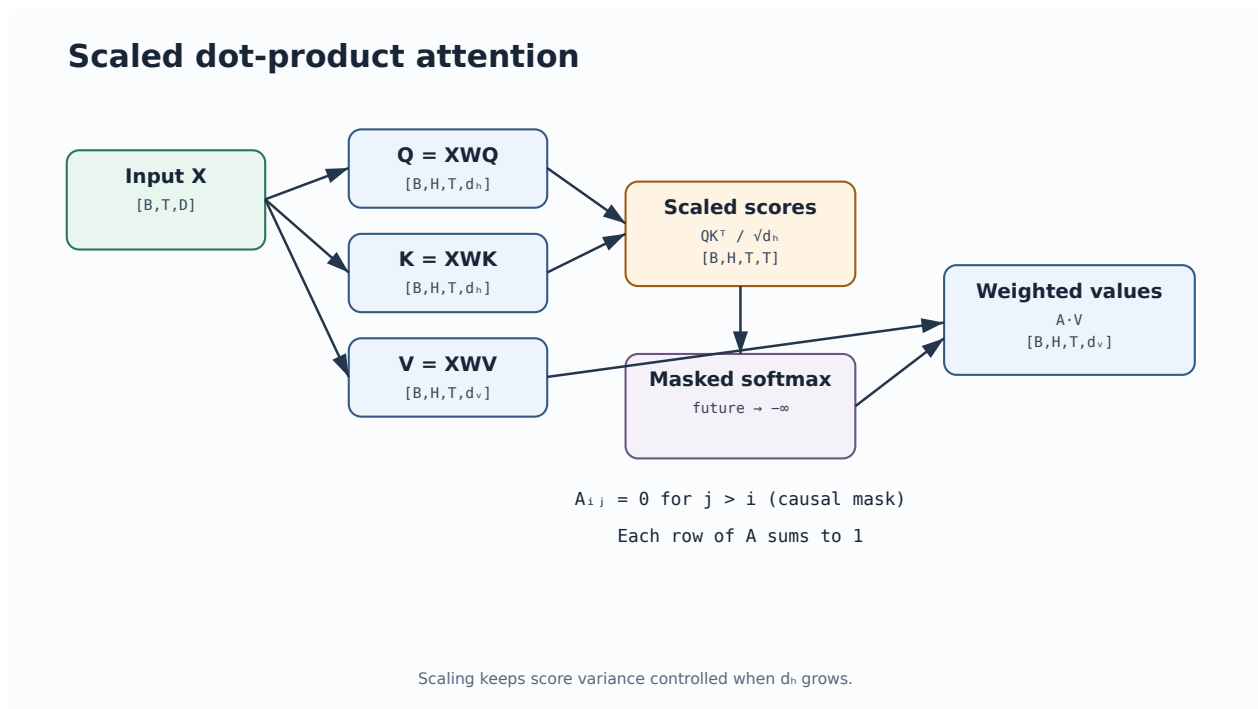


Figure 18.1: Scaled dot-product attention.

Chapter 19

Causal and Multi-Head Attention

Status: **Draft — not yet independently reviewed.**

Language-model training exposes an entire sequence for efficiency while enforcing the information boundary of left-to-right generation. Causal masks provide that boundary; multiple heads provide several learned routing spaces in parallel.

19.1 Causal attention

At position i , a decoder-only language model may attend only to positions $j \leq i$. Thus the representation used to predict token $i + 1$ cannot inspect that target or later tokens. Training can still compute every position simultaneously because the mask constrains information flow within the matrix operation.

19.2 Causal masks

The additive mask has $M_{ij} = 0$ when $j \leq i$ and $-\infty$ otherwise. It is added before softmax, making forbidden probabilities zero. For $T = 3$, allowed rows are $[1, 0, 0]$, $[1, 1, 0]$, and $[1, 1, 1]$ conceptually. A mask with the wrong diagonal can prevent self-attention; a transposed inequality can expose the future.

A decisive test changes future token embeddings while holding the prefix fixed. Outputs at earlier positions must remain unchanged within numerical tolerance. Merely inspecting a triangular matrix is weaker because broadcasting and axis mistakes can apply it incorrectly.

19.3 Multi-head attention

With H heads and head width d_h , projections reshape Q, K, V from $[B, T, Hd_h]$ to $[B, H, T, d_h]$. Each head computes its own $[T, T]$ routing matrix. Heads can specialize in different patterns, though specialization should be measured rather than assumed and individual heads can be redundant.

When $D = Hd_h$, the combined projection width matches the residual width. Grouped-query and multi-query variants later change the number of KV heads, but ordinary multi-head attention uses the same head count for Q, K, and V.

19.4 Head concatenation

Per-head outputs $O_h[B, T, d_h]$ are concatenated along the feature axis to $O_{cat}[B, T, Hd_h]$. Concatenation preserves every coordinate; it does not average heads. A reshape must transpose axes correctly—reinterpreting $[B, H, T, d_h]$ directly as $[B, T, Hd_h]$ can interleave positions and heads.

19.5 Output projection

The matrix $W_O[Hd_h, D]$ mixes concatenated head features back into the residual width. Without it, heads occupy fixed disjoint coordinate blocks. The projection lets downstream computation combine their contributions and is part of attention’s parameter and FLOP cost.

19.6 Worked shape trace

For $B = 2, T = 5, D = 8, H = 2, d_h = 4$: projected Q, K, V each reshape to $[2, 2, 5, 4]$; scores and weights are $[2, 2, 5, 5]$; head outputs are $[2, 2, 5, 4]$; transpose and concatenate to $[2, 5, 8]$; output projection preserves $[2, 5, 8]$.

19.7 Four perspectives

Follow the Token: each token retrieves several mixtures, one per head. **Follow the Gradient:** the causal mask blocks computational paths from future tokens, so their gradients cannot influence earlier states through attention. **Follow the Byte:** every head contributes a score matrix; fused kernels may avoid storing it in full. **Follow the Request:** during decode, a single new query attends to cached prefix keys and values and needs no future mask entries.

19.8 Exit check

Write the 4×4 causal mask. Design the future-token mutation test. Trace shapes for $B = 3, T = 7, D = 12, H = 3$. Explain why attention visualizations alone cannot establish what each head causally contributes.

19.9 References

- Vaswani et al., [Attention Is All You Need](#), 2017.
- [PyTorch scaled dot-product attention](#). Accessed 2026-09-19.

Chapter 20

Attention Shapes and Compute Cost

Status: **Draft — not yet independently reviewed.**

The formula for attention is short; its tensors can be enormous. Shape and resource accounting turn “quadratic” from a slogan into an engineering estimate.

20.1 Attention tensor shapes

Define batch B , sequence length T , model width D , heads H , and head width $d_h = D/H$. Standard multi-head Q, K, V have shape $[B, H, T, d_h]$. Scores and probabilities have shape $[B, H, T, T]$. Multiplying probabilities by V returns $[B, H, T, d_h]$, then concatenation returns $[B, T, D]$.

Write these shapes beside code. Most attention bugs are axis, mask-broadcast, or reshape errors that may preserve the total element count and avoid a runtime exception.

20.2 Attention compute cost

QKV projections cost roughly $6BTD^2$ FLOPs and output projection about $2BTD^2$, using two FLOPs per multiply-add. Score formation QK^T costs about $2BHT^2d_h = 2BT^2D$; multiplying weights by V costs another $2BT^2D$. Thus the main dense-attention matrix products cost about $4BT^2D$, excluding softmax and masking.

For $B = 1, T = 2048, D = 4096$, the two attention matrix multiplications require roughly $4(2048^2)(4096) = 68.7$ billion FLOPs per layer. The four projections require about $8(2048)(4096^2) = 274.9$ billion FLOPs. Under these assumptions the two terms are equal when $T = 2D$; at this context and width, projections still exceed attention matmuls. This comparison omits biases, softmax, normalization, and feed-forward work.

20.3 Quadratic sequence cost

Doubling T doubles projection work but quadruples pairwise attention work. Dense attention creates T^2 query-key relations per head. “Attention is $O(T^2D)$ ” omits constants, projections, batch, heads, and the distinction between training/prefill and single-token decode, so use it as scaling guidance rather than a complete performance model.

During cached decode for one new token, the query compares with T cached keys, so per-step attention is linear in current context. Generating many tokens still repeats that growing linear work. Hardware utilization, memory traffic, and kernel launch overhead determine elapsed time; FLOPs alone do not.

20.4 Attention memory accounting

A materialized score tensor uses BHT^2 elements. With $B = 2, H = 32, T = 4096$, that is 1,073,741,824 elements: about 2.15 GB decimal in BF16 for one tensor. Probabilities, dropout masks, saved activations, and gradients can multiply this during training. Q, K, and V together use $3BTD$ elements, linear in T .

FlashAttention changes the IO schedule and avoids writing the full score/probability matrix to HBM, while computing exact attention up to normal floating-point differences. It does not change the dense pairwise arithmetic's asymptotic T^2 nature. This distinction—stored intermediates versus operations—is crucial.

20.5 Four perspectives

Follow the Token: every destination relates to every allowed source. **Follow the Gradient:** training may need Q, K, V and normalization statistics or recomputation for backward. **Follow the Byte:** materialized scores can dominate activation memory, and memory-efficient kernels tile them on-chip. **Follow the Request:** prefill exposes a $T \times T$ workload, while one decode step exposes $1 \times T$ attention plus weight reads and KV-cache traffic.

20.6 Exit check

For $B = 4, T = 1024, D = 2048, H = 16$, derive every tensor shape, BF16 QKV bytes, score bytes, projection FLOPs, and attention-matmul FLOPs. Repeat after doubling T . State whether each quantity doubles or quadruples, and state what your calculation omits.

20.7 References

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Dao et al., [FlashAttention](#), 2022.

Part 6: Build a Transformer

Chapter 21

Residual Streams and Normalization

Status: **Drafted — technical and editorial review pending.**

Part 6: Build a Transformer

21.1 Learning outcome

Implement both normalization placements and inspect residual scales.

21.2 Prerequisites

CH-20.

Let $X \in \mathbb{R}^{B \times T \times D}$ denote the current hidden state. A Transformer block must transform it without erasing the information already present, while keeping numerical scales trainable through many layers. Residual addition preserves a direct path; normalization controls scale. Those two mechanisms are simple enough to fit on one line and consequential enough to decide whether a deep model trains.

21.2.1 Residual connections

Coverage ID: T-21-01.

A residual sublayer computes

$$Y = X + F(X; \theta).$$

The shapes of X and $F(X)$ must match exactly. For $B = 2, T = 4, D = 8$, both contain 64 elements. The addition has no learned parameters. It lets a sublayer learn a correction rather than reconstruct the entire representation. Using column-vector gradients and $J_F = \partial F / \partial X$, backpropagation gives

$$\nabla_X L = (I + J_F)^\top \nabla_Y L,$$

so the identity term provides a short gradient path. This does not guarantee healthy gradients—the other branch can still destabilize them—but it makes exact information and gradient preservation possible when F is initially small.

A common bug is adding tensors with accidentally broadcast dimensions. Assert equality rather than accepting broadcasting at residual boundaries.

21.2.2 Residual streams

Coverage ID: T-21-02.

The **residual stream** is the D -dimensional state carried through the stack. Attention writes a context-dependent update into it; the MLP writes a token-local update. For pre-normalized blocks:

$$X' = X + \text{Attention}(\text{Norm}(X)), \quad X'' = X' + \text{MLP}(\text{Norm}(X')).$$

Calling the stream a shared workspace is useful, provided we do not imply that individual coordinates have stable human meanings. Rotating the basis can preserve the computation while changing every coordinate. The stream is also where later probes and activation interventions operate, which is why scale and normalization placement matter.

21.2.3 LayerNorm

Coverage ID: T-21-03.

LayerNorm normalizes each token independently across its D features. For token vector $x \in \mathbb{R}^D$,

$$\mu = \frac{1}{D} \sum_i x_i, \quad \sigma^2 = \frac{1}{D} \sum_i (x_i - \mu)^2,$$

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

With $x = [1, 2, 3]$, $\mu = 2$, variance = $2/3$, and the unscaled output is approximately $[-1.225, 0, 1.225]$. $\gamma, \beta \in \mathbb{R}^D$ contribute $2D$ parameters. Normalizing across batch or time is a different operation and leaks dependence between examples or positions.

Compute mean and variance in adequate precision. A small epsilon prevents division by zero; changing it can affect low-precision behavior.

21.2.4 RMSNorm

Coverage ID: T-21-04.

RMSNorm removes mean subtraction:

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{D^{-1} \sum_i x_i^2 + \epsilon}}.$$

It usually has D learned scale parameters and no bias. For $[1, 2, 3]$, RMS is $\sqrt{14/3} \approx 2.160$, giving $[0.463, 0.926, 1.389]$ before γ . RMSNorm fixes root-mean-square scale but does not center the vector. It is therefore not interchangeable with LayerNorm after training.

21.2.5 Pre-norm

Coverage ID: T-21-05.

Pre-norm applies normalization before the sublayer: $Y = X + F(N(X))$. The residual path from Y to X is an exact identity, which generally makes deep optimization easier. The stream itself is not normalized after every addition, so its magnitude may grow with depth; a final normalization before the LM head is common.

21.2.6 Post-norm

Coverage ID: T-21-06.

Post-norm computes $Y = N(X + F(X))$. Every forward state is normalized immediately, but the direct backward route now passes through the normalization Jacobian. The original Transformer used post-norm; many decoder LLMs use pre-norm or related variants for stable deep training. Loading weights into the wrong layout changes the function even when all tensor shapes match.

21.3 Worked implementation

```
class PreNormBlock(nn.Module):
    def forward(self, x):
        x = x + self.attn(self.norm1(x))
        x = x + self.mlp(self.norm2(x))
        return x
```

```
class PostNormBlock(nn.Module):
    def forward(self, x):
        x = self.norm1(x + self.attn(x))
        x = self.norm2(x + self.mlp(x))
        return x
```

Log the RMS of the stream and both updates per layer. A useful diagnostic is `update_rms / stream_rms`; abrupt spikes often precede divergence.

21.4 Four recurring perspectives

- **Follow the Token:** one token remains a D -vector while attention and the MLP add different updates.
- **Follow the Gradient:** residual identities shorten the backward route; norm placement changes its Jacobian.
- **Follow the Byte:** normalization reads and writes BSD elements and reduction statistics; unfused kernels add memory traffic.
- **Follow the Request:** inference applies the same block layout encoded by the checkpoint; a layout mismatch corrupts every generated token.

21.5 Lab and exit check

Implement both variants with identical sublayers. Track stream RMS, update RMS, and gradient norm for 24 layers on the same batch. Explain why identical weights do not make the two layouts equivalent. Verify that each normalization treats batch and time positions independently.

21.6 Exercises

1. Compute LayerNorm and RMSNorm for $x = [-1, 1]$ with unit scale, zero bias, and $\epsilon = 0$. **Check:** both return $[-1, 1]$. Then repeat for $x = [1, 3]$ and explain why only LayerNorm is invariant to adding the same constant to every coordinate.
2. Let $F(x) = Ax$ for a two-dimensional vector. Derive the residual Jacobian and identify a matrix A for which the residual path still has a zero singular value. **Check:** the Jacobian is $I + A$; $A = -I$ is the simplest counterexample to the claim that residual connections guarantee nonvanishing gradients.

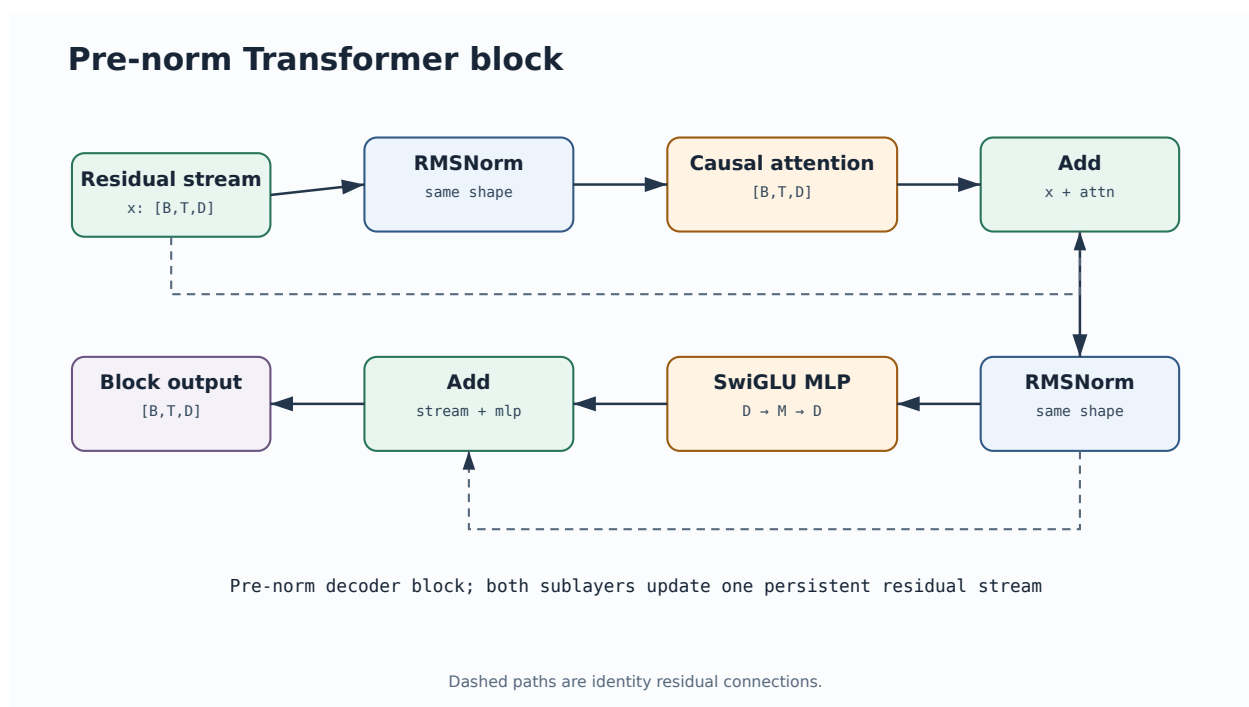


Figure 21.1: Pre-normalized Transformer block.

21.7 References

- He et al., [Deep Residual Learning for Image Recognition](#), 2015.
- Ba, Kiros, and Hinton, [Layer Normalization](#), 2016.
- Zhang and Sennrich, [Root Mean Square Layer Normalization](#), 2019.
- Vaswani et al., [Attention Is All You Need](#), 2017.

Chapter 22

Feed-Forward Networks and Complete Blocks

Status: **Drafted — technical and editorial review pending.**

Part 6: Build a Transformer

22.1 Learning outcome

Build a block with explicit shapes and a parameter-count breakdown.

22.2 Prerequisites

CH-21.

Attention moves information between positions. The feed-forward sublayer transforms each position independently. This division—communication, then computation—is the repeating structure of a decoder Transformer.

22.2.1 Feed-forward networks

Coverage ID: T-22-01.

For $X \in \mathbb{R}^{B \times T \times D}$, a conventional position-wise FFN is

$$\text{FFN}(X) = \phi(XW_1 + b_1)W_2 + b_2,$$

where $W_1 \in \mathbb{R}^{D \times M}$, $W_2 \in \mathbb{R}^{M \times D}$, and M is the intermediate width. The same weights are applied to all BT token vectors. With $D = 512$, $M = 2048$, weights contribute $2DM = 2,097,152$ parameters; biases add 2,560. The two matrix multiplies cost roughly $4BTDM$ FLOPs if one multiply-add counts as two FLOPs.

Calling the FFN “position-wise” does not mean it ignores context: its input already contains context mixed by attention. It simply performs no new cross-position communication.

22.2.2 GELU vs SwiGLU

Coverage ID: T-22-02.

GELU smoothly gates by input magnitude:

$$\text{GELU}(z) = z\Phi(z),$$

usually approximated in code. A gated unit uses two input projections. SwiGLU is commonly written

$$\text{SwiGLU}(X) = (\text{SiLU}(XW_g) \odot XW_u) W_d, \quad \text{SiLU}(z) = z\sigma(z).$$

The gate projection W_g decides how much of the “up” projection W_u passes elementwise. Shapes are $W_g, W_u \in \mathbb{R}^{D \times M}$ and $W_d \in \mathbb{R}^{M \times D}$. A common implementation error is swapping conventions from a checkpoint; multiplication is commutative, but applying SiLU to the wrong branch is not.

22.2.3 MLP expansion

Coverage ID: T-22-03.

The intermediate width creates a larger feature workspace. A GELU FFN near $M = 4D$ has about $8D^2$ weights. A SwiGLU FFN has $3DM$, so choosing $M \approx (8/3)D$ approximately matches that parameter count. Hardware-friendly widths are often rounded to a multiple such as 256.

For $D = 4096$: a $4D$ GELU FFN has $2 \cdot 4096 \cdot 16384 \approx 134.2$ million weights. A SwiGLU with $M = 11008$ has $3 \cdot 4096 \cdot 11008 \approx 135.3$ million. “Smaller expansion” therefore does not imply fewer parameters once the third matrix is included.

22.2.4 Complete Transformer block

Coverage ID: T-22-04.

A pre-norm decoder block is:

$$\begin{aligned} A &= X + \text{MHA}(N_1(X); \text{causal mask}), \\ Y &= A + \text{MLP}(N_2(A)). \end{aligned}$$

Every boundary has shape $[B, T, D]$. For standard MHA, four $D \times D$ projections contribute about $4D^2$ weights. Add a SwiGLU MLP with $3DM$, plus small normalization vectors. At $D = 768$, $M = 2048$, the block has roughly $4(768)^2 + 3(768)(2048) = 7.08$ million main weights.

```
class Block(nn.Module):
    def forward(self, x):
        assert x.ndim == 3 and x.shape[-1] == self.width
        x = x + self.attn(self.norm1(x), causal=True)
        x = x + self.mlp(self.norm2(x))
        return x
```

Dropout, biases, positional method, norm type, and attention variant belong in the configuration because they change behavior or checkpoint compatibility.

22.3 Failure cases

Wrong causal masks leak labels. A missing output projection prevents heads from being re-combined. Applying a single normalization to both residual branches changes the architecture. Materializing unnecessary intermediate tensors can dominate runtime even though the formula is correct.

22.4 Four recurring perspectives

- **Follow the Token:** attention gathers context; the MLP transforms the resulting token vector independently.
- **Follow the Gradient:** each residual branch receives gradients through both its update and identity paths.
- **Follow the Byte:** MLP weights often exceed attention weights; decode repeatedly streams them from accelerator memory.
- **Follow the Request:** each generated token traverses every block, making block-level inefficiency multiply by depth and output length.

22.5 Lab and exit check

Implement one block, print every parameter shape, and reconcile the framework total with a hand calculation. For $(B=2, T=16, D=128, H=4, M=352)$, assert every residual boundary remains $[2, 16, 128]$. Explain why SwiGLU needs three matrices and why its expansion ratio differs from a two-matrix GELU FFN.

22.6 Exercises

1. For a bias-free block with $D = 128$, $M = 352$, standard MHA, and SwiGLU, calculate the main weight count. **Check:** $4D^2 + 3DM = 200,704$.
2. Give a concrete input shape that a tensor library could broadcast against $[2, 16, 128]$ but that must be rejected at a residual boundary. Explain why successful broadcasting is not evidence of architectural correctness.

22.7 References

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Hendrycks and Gimpel, [Gaussian Error Linear Units](#), 2016.
- Shazeer, [GLU Variants Improve Transformer](#), 2020.

Chapter 23

Stacking, Vocabulary Projection and Tying

Status: **Drafted — technical and editorial review pending.**

Part 6: Build a Transformer

23.1 Learning outcome

Trace hidden states to vocabulary logits and verify shared-weight gradients.

23.2 Prerequisites

CH-22.

A single block refines representations once. A language model stacks blocks, normalizes the final residual state, and converts each position into scores over the vocabulary.

23.2.1 Stack the blocks

Coverage ID: T-23-01.

Let $X_0 = E[token] + P[position]$ and $X_{\ell+1} = \text{Block}_\ell(X_\ell)$ for $\ell = 0 \dots L-1$. Every $X_\ell \in \mathbb{R}^{B \times T \times D}$, but layers do not share parameters unless explicitly designed to. A final normalization produces $H = N_f(X_L)$.

Depth increases sequential computation: block $\ell + 1$ cannot start until block ℓ produces its state for that microbatch. Activation memory during training also scales roughly with $L B T D$ before checkpointing. Do not reuse one Python module L times accidentally; that creates weight sharing rather than a stack of independent blocks.

23.2.2 Vocabulary projection

Coverage ID: T-23-02.

The vocabulary projection maps $H \in \mathbb{R}^{B \times T \times D}$ to logits $Z \in \mathbb{R}^{B \times T \times V}$:

$$Z = H W_{vocab}^\top + b, \quad W_{vocab} \in \mathbb{R}^{V \times D}.$$

For ($B=2, T=128, D=768, V=50\{, \}000$), logits contain 12.8 million values: about 25.6 MB in BF16 (decimal units). Training need not retain every full-precision softmax probability if a fused cross-entropy kernel is used.

23.2.3 LM head

Coverage ID: T-23-03.

The projection module is called the **language-modeling head**. Logits are unnormalized scores, not probabilities. For next-token training, logit vector ($Z_{\{b,t,: \}}$) is compared with target token ($x_{\{b,t+1 \}}$). During generation, only the last needed position is sampled after prefill, avoiding vocabulary projection for unused positions where the runtime permits.

A bias is optional. Checkpoint loaders must match whether it exists. Temperature and top-p belong to decoding after logits; they are not part of the trained head.

23.2.4 Weight tying

Coverage ID: T-23-04.

Input embeddings use $E \in \mathbb{R}^{V \times D}$. Weight tying sets $W_{vocab} = E$, so token lookup and output scoring share one parameter tensor. This saves VD parameters and couples the geometry used to read and predict tokens. For $V = 50,000, D = 768$, that saves 38.4 million parameters, or 76.8 MB of BF16 weights.

The shared tensor receives two gradient contributions:

$$\nabla_E L = \nabla_E^{input} L + \nabla_E^{output} L.$$

In PyTorch, use the same Parameter object, not copied values:

```
self.token_embedding = nn.Embedding(V, D)
self.lm_head = nn.Linear(D, V, bias=False)
self.lm_head.weight = self.token_embedding.weight
assert self.lm_head.weight is self.token_embedding.weight
```

Calling `.clone()` creates a different tensor and breaks tying. Optimizer parameter lists must not update a shared parameter twice.

23.3 Worked trace

With token IDs $[4, 1, 9]$, embedding lookup produces $[1, 3, D]$. After L blocks the shape remains $[1, 3, D]$; the head returns $[1, 3, V]$. The final vector at position 1 predicts token 9. The logit for vocabulary item j is the dot product between hidden state h and embedding row E_j when weights are tied.

23.4 Four recurring perspectives

- **Follow the Token:** an ID becomes an embedding, is repeatedly refined, then is compared with every vocabulary embedding.
- **Follow the Gradient:** tied embeddings accumulate input-path and output-path signals into one parameter.
- **Follow the Byte:** the $V \times D$ table is large; tying removes a full duplicate but logits can still be large.
- **Follow the Request:** prefill may compute logits selectively; decode requires the head once per generated step.

23.5 Lab and exit check

Build tied and untied versions. Verify object identity, parameter counts, and that one backward pass gives the shared weight a gradient reflecting both uses. Trace shapes from $[B, T]$ IDs to $[B, T, V]$ logits and identify which slice predicts each label.

23.6 Exercises

1. For $V = 32,000$ and $D = 4096$, calculate the parameters and BF16 bytes saved by tying.
Check: 131,072,000 parameters and 262,144,000 bytes (262.144 MB decimal).
2. With IDs $[4, 1, 9]$, list the two next-token training pairs and identify the logit positions used.
Check: position 0 predicts ID 1 and position 1 predicts ID 9; no internal target follows position 2.

23.7 References

- Press and Wolf, [Using the Output Embedding to Improve Language Models](#), 2016.
- Inan, Khosravi, and Socher, [Tying Word Vectors and Word Classifiers](#), 2016.

Chapter 24

Build Our GPT From Scratch

Status: **Drafted — executable lab included; technical and editorial review pending.**

Part 6: Build a Transformer

24.1 Learning outcome

Assemble a runnable tiny GPT with no pretrained-model wrapper.

24.2 Prerequisites

CH-23.

This milestone joins the mechanisms already derived. “From scratch” means we use tensor primitives and ordinary neural-network modules rather than loading a pretrained Transformer or delegating the architecture to a model wrapper. Autograd may still compute derivatives.

24.2.1 GPT implementation

Coverage ID: T-24-01.

A minimal GPT contains token and position representations, L decoder blocks, a final norm, and an LM head. The reference implementation is `code/tiny_gpt_core.py`. Its attention explicitly forms Q, K, and V, applies a causal mask, merges heads, and projects the result. This favors inspectability over kernel efficiency.

```
x = token_embedding(ids) + position_embedding(position_ids)
for block in blocks:
    x = block(x)
logits = lm_head(final_norm(x))
```

24.2.2 Decoder-only model assembly

Coverage ID: T-24-02.

Decoder-only means every position attends only to itself and earlier positions, and the model predicts the next token at every training position. For IDs $I \in \mathbb{N}^{B \times T}$: embeddings are $[B, T, D]$; Q/K/V are $[B, H, T, d_h]$; attention scores are $[B, H, T, T]$; logits are $[B, T, V]$. The causal invariant is: changing input position $j > i$ cannot change logits at position i in evaluation mode.

24.2.3 Model configuration

Coverage ID: T-24-03.

Configuration is an architectural contract:

vocab_size V	context_length C
width D	layers L
attention_heads H	intermediate_width M
normalization	activation/gating
position method	dropout/bias/tie_embeddings

Require $D \bmod H = 0$, input $T \leq C$, IDs in $[0, V)$, and valid padding/mask semantics. A tiny example uses $V = 64, C = 32, D = 48, L = 2, H = 4, M = 128$. Its small size supports CPU tests; it says nothing about large-model quality.

24.2.4 Forward-pass invariants

Coverage ID: T-24-04.

The implementation should enforce:

1. output shape is $[B, T, V]$;
2. residual shape stays $[B, T, D]$;
3. all probabilities are finite after softmax;
4. causal isolation holds in `eval()` mode;
5. tied weights share storage when enabled;
6. loss uses logits at positions $0..T-2$ against IDs at $1..T-1$;
7. inputs exceeding context length fail loudly.

A causality test runs two inputs that share a prefix but differ later. Their logits through the last shared prediction position must match within floating-point tolerance. Dropout must be disabled, otherwise randomness can imitate a causality failure.

24.3 Parameter and compute sanity check

Ignoring biases, learned positions, and norms, standard MHA plus SwiGLU contributes per layer approximately $4D^2 + 3DM$ weights. Embedding/head contribute VD when tied or $2VD$ when untied. A rough dense forward estimate is about two FLOPs per weight used per token plus attention score/value work; it is a reasoning estimate, not a measured runtime.

24.4 Four recurring perspectives

- **Follow the Token:** ID $\rightarrow$ embedding $\rightarrow$ residual updates $\rightarrow$ vocabulary logits $\rightarrow$ next ID.
- **Follow the Gradient:** cross-entropy sends error through the head, every block, and the embedding table.
- **Follow the Byte:** parameters persist; activations and attention matrices scale with batch and context.
- **Follow the Request:** prompt tokens run together in prefill; generated tokens later run one step at a time with cached state.

24.5 Lab and exit check

Run `python code/tiny_gpt_core.py`. Confirm shape, finite logits, parameter count, tied storage, and causal isolation. Then deliberately remove the causal mask and observe the isolation test fail. Explain why passing a shape test alone cannot establish correctness.

24.6 Exercises

1. For the tiny configuration $V = 64, C = 32, D = 48, L = 2, H = 4, M = 128$, state the shapes of Q , attention scores, and logits for $B = 3, T = 20$. **Check:** $[3, 4, 20, 12]$, $[3, 4, 20, 20]$, and $[3, 20, 64]$.
2. Design a causality test in which two inputs share positions 0 through 5 and differ afterward. Specify exactly which logits must agree. **Check:** in evaluation mode, logits at positions 0 through 5 agree within numerical tolerance.

24.7 References

- Radford et al., [Improving Language Understanding by Generative Pre-Training](#), 2018.
- Vaswani et al., [Attention Is All You Need](#), 2017.
- PyTorch documentation for `nn.Module`, pinned by the environment record for executable results.

Part 7: Train the Thing

Chapter 25

Sequences, Batches and Pretraining

Status: **Drafted — technical and editorial review pending.**

Part 7: Train the Thing

25.1 Learning outcome

Construct shifted labels and packed sequences without unintended cross-example attention.

25.2 Prerequisites

CH-24.

A model becomes a language model through the examples and objective presented to it. Data preparation determines which predictions are rewarded, which tokens can see one another, and how efficiently the training budget is used.

25.2.1 Preparing training sequences

Coverage ID: T-25-01.

Tokenize documents, insert explicitly defined boundary tokens, concatenate or pack according to policy, and cut windows no longer than context C . Given tokens [BOS, the, cat, sat, EOS], the input is [BOS, the, cat, sat] and labels are [the, cat, sat, EOS]. Padding labels normally use an ignore index so they contribute neither loss nor gradient.

Splitting data after making overlapping windows can leak almost identical text into train and validation. Split at the document or source level before window construction and deduplicate across splits.

25.2.2 Sequence packing

Coverage ID: T-25-02.

Packing fills a fixed-length row with multiple short examples to reduce padding. Suppose capacity is 8 and examples are [A1,A2,A3] and [B1,B2]. The packed tokens may occupy one row, but ordinary causal attention would let B tokens attend to A tokens. If examples are meant to be

independent, use a block-diagonal causal mask and reset or consistently define position IDs. Also mask the loss transition $A3 \rightarrow B1$; otherwise the model learns an artificial boundary.

Packing policy depends on the training goal. Continuous pretraining corpora may intentionally treat document separators as part of one stream. Instruction tuning usually isolates examples. “Packed” therefore describes storage, not semantics.

25.2.3 Batches

Coverage ID: T-25-03.

A batch tensor has shape $[B, T]$, and the model returns $[B, T, V]$. All examples in the batch are processed before an optimizer step, but they remain logically independent unless the attention mask says otherwise. Token count $B \times T$ is often a better workload measure than sequence count.

25.2.4 Mini-batches

Coverage ID: T-25-04.

Full-corpus gradients are impractical, so training estimates them from mini-batches. Sampling introduces gradient noise. Shuffling should be reproducible yet different each epoch; distributed samplers must avoid unintended duplication. Bucketing by length can reduce padding but may correlate neighboring batches with content or source, so shuffle at an appropriate level.

25.2.5 Next-token prediction

Coverage ID: T-25-05.

For valid label positions $\mathcal{M}$, the objective is

$$L = -\frac{1}{|\mathcal{M}|} \sum_{(b,t) \in \mathcal{M}} \log p_{\theta}(x_{b,t+1} \mid x_{b,\leq t}).$$

A length- T row yields at most $T - 1$ internal predictions. Verify the shift using readable tokens before training; an off-by-one label bug can still produce a decreasing loss on repetitive data.

25.2.6 Full pretraining

Coverage ID: T-25-06.

Pretraining repeatedly samples token batches from a large mixture, computes next-token loss, updates parameters, evaluates held-out data, and checkpoints state. “One epoch” may be meaningless when the corpus is streamed or sampled by mixture weights; token budget and optimizer steps are clearer. The learned model reflects the dataset, tokenizer, mixture, ordering, objective, and optimization procedure—not architecture alone.

25.2.7 Batch size and sequence length dynamics

Coverage ID: T-25-07.

Increasing B increases independent sequences; increasing T increases context and causal dependencies. Both raise tokens per step, but attention compute grows approximately with BT^2D , while token-wise layers scale near $BT D^2$. Doubling T can therefore more than double cost. Larger batches reduce gradient noise and the number of optimizer steps per token budget; learning-rate behavior may need retuning. Longer sequences also change the distribution of positional distances, so matching total tokens does not make two runs equivalent.

25.3 Four recurring perspectives

- **Follow the Token:** each token becomes both an input and, shifted by one position, a supervision target.
- **Follow the Gradient:** only unmasked label positions contribute; batching averages their signals.
- **Follow the Byte:** padding wastes activation and attention storage; packing improves utilization but needs correct masks.
- **Follow the Request:** pretraining uses full sequence batches, while production prompts have variable lengths and different scheduling constraints.

25.4 Lab and exit check

Write a collator that packs two examples, returns IDs, positions, attention mask, and loss mask, and visually print all four. Assert that changing A cannot change logits for B under independent-example packing. Reconcile the count of valid labels with the loss denominator.

25.5 Exercises

1. Pack examples of lengths 3 and 2 into capacity 8. Count valid next-token labels when cross-example transitions and padding are masked. **Check:** $(3 - 1) + (2 - 1) = 3$.
2. At fixed B and D , double T from 512 to 1024. By what factor do token-wise work and the quadratic attention-score term grow? **Check:** approximately $2\times$ and $4\times$, respectively.

25.6 References

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Brown et al., [Language Models are Few-Shot Learners](#), 2020.

Chapter 26

The Training Step and Schedule

Status: **Drafted — technical and editorial review pending.**

Part 7: Train the Thing

26.1 Learning outcome

Show equivalence of accumulated and full-batch gradients under matched conditions.

26.2 Prerequisites

CH-08, CH-11, CH-24, CH-25.

A training step is a state transition. Parameters, optimizer buffers, scheduler counters, and random-number generators may all change. Separating the phases makes bugs observable.

26.2.1 Training forward pass

Coverage ID: T-26-01.

Given IDs and masks, the model produces logits $Z \in \mathbb{R}^{B \times T \times V}$. Training mode may activate dropout. Autograd records operations and saves selected tensors needed backward. Labels are data, not model inputs, even when a convenience API accepts them in the same call.

26.2.2 Training loss

Coverage ID: T-26-02.

Flatten valid logits to $[N, V]$ and labels to $[N]$, where N is the number of supervised tokens. Cross-entropy should be computed with log-sum-exp stabilization. Track both mean loss and token count: averaging per microbatch and then averaging microbatches is biased when their valid-token counts differ.

26.2.3 Training backward pass

Coverage ID: T-26-03.

`loss.backward()` accumulates gradients into parameter `.grad` buffers. It does not update parameters. Clear gradients intentionally—usually before the accumulation window—and inspect norms

before the optimizer step. Retaining the graph without need grows memory; detaching the loss before backward removes the graph entirely.

26.2.4 Optimizer state updates in memory

Coverage ID: T-26-04.

AdamW stores first and second moment tensors m, v for each parameter w :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

After bias correction, the optimizer updates w , while decoupled weight decay applies separately. With FP32 parameters, gradients, m , and v , persistent training state can approach 16 bytes per parameter before mixed-precision copies or distributed sharding. `optimizer.step()` mutates parameters and buffers; `zero_grad()` releases or zeroes gradient storage.

26.2.5 Learning rates

Coverage ID: T-26-05.

The learning rate controls update scale, not loss scale. Too large diverges; too small wastes compute or settles poorly. Schedules may be step-based or token-based. Record which, because variable batch sizes make those clocks disagree.

26.2.6 Warmup

Coverage ID: T-26-06.

Warmup increases the learning rate from a small value over early steps or tokens. Early moment estimates and representations are poorly calibrated, making full-size updates risky. A linear warmup to $\eta_{\max}$ over W steps is $\eta_s = \eta_{\max}(s + 1)/W$ for $s < W$. Off-by-one errors can begin at zero or overshoot the intended peak.

26.2.7 Gradient accumulation

Coverage ID: T-26-07.

To emulate one batch using K equal-size microbatches, backpropagate `loss / K` for each, then step once. More generally, accumulate **summed token losses** and divide by total valid tokens. Exact equivalence requires the same examples, stochastic masks, batch-dependent operations, precision, and reduction order. Dropout and floating-point associativity can prevent bitwise equality even when the estimator is correct.

26.2.8 Gradient clipping

Coverage ID: T-26-08.

Global-norm clipping rescales all gradients when

$$\|g\|_2 = \sqrt{\sum_p \|g_p\|_2^2} > c, \quad g \leftarrow g \frac{c}{\|g\|_2 + \epsilon}.$$

Clip after unscaling mixed-precision gradients and after completing accumulation, before `optimizer.step()`. Frequent clipping signals an unstable regime; it is a guardrail, not a cure.

26.3 Canonical step order

zero gradients $\rightarrow K \times$ (forward $\rightarrow$ token-summed loss $\rightarrow$ scaled backward)
 $\rightarrow$ unscale $\rightarrow$ inspect/clip $\rightarrow$ optimizer step $\rightarrow$ scheduler step $\rightarrow$ log

26.4 Four recurring perspectives

- **Follow the Token:** valid target tokens determine the denominator and effective batch.
- **Follow the Gradient:** backward accumulates; clipping transforms; the optimizer consumes and clears nothing automatically.
- **Follow the Byte:** gradients and Adam moments rival or exceed weight memory.
- **Follow the Request:** training schedules optimize corpus throughput; inference has no optimizer state or backward graph.

26.5 Lab and exit check

Compare one batch of eight examples against four microbatches of two using dropout disabled. Weight both by valid token counts. Assert parameter gradients are numerically close, then enable dropout and explain any difference. Print parameter, gradient, and optimizer-buffer bytes.

26.6 Exercises

1. Two microbatches contain 80 and 20 valid tokens with mean losses 2.0 and 4.0. Compute the correct combined mean. **Check:** $(80 \cdot 2 + 20 \cdot 4)/100 = 2.4$, not 3.0.
2. A gradient has norm 12 and the clipping threshold is 3. Ignoring ϵ , compute the multiplier. **Check:** $3/12 = 0.25$.

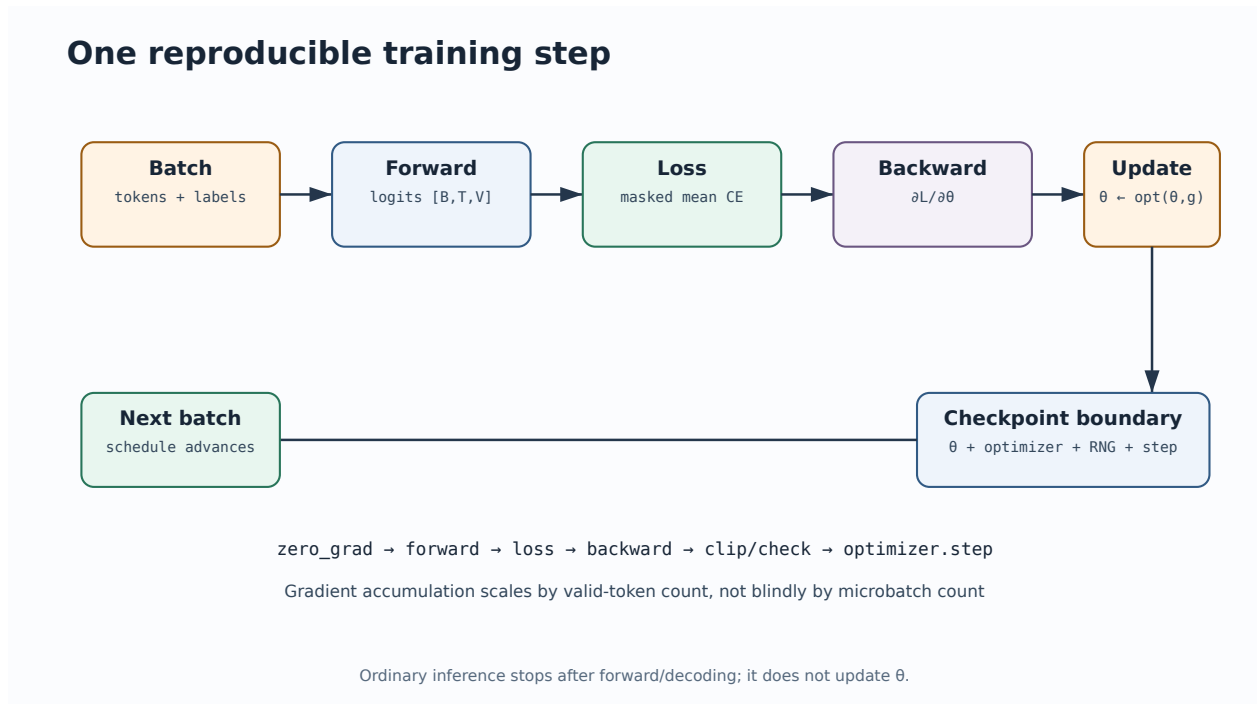


Figure 26.1: Training and reproducibility loop.

26.7 References

- Kingma and Ba, [Adam: A Method for Stochastic Optimization](#), 2014.
- Loshchilov and Hutter, [Decoupled Weight Decay Regularization](#), 2017.
- Pascanu, Mikolov, and Bengio, [On the Difficulty of Training Recurrent Neural Networks](#), 2013.

Chapter 27

Mixed Precision and Checkpoint Anatomy

Status: **Drafted — technical and editorial review pending.**

Part 7: Train the Thing

27.1 Learning outcome

Resume a run with matching next-step behavior and explain memory tradeoffs.

27.2 Prerequisites

CH-26.

Training must fit in memory and survive interruption. Mixed precision reduces cost; activation checkpointing trades compute for memory; durable checkpoints preserve the entire state transition, not merely model weights.

27.2.1 Mixed-precision training

Coverage ID: T-27-01.

Mixed precision uses lower-precision formats for selected matrix operations and activations while retaining sensitive reductions or master state in higher precision. FP16 has limited exponent range; BF16 retains FP32-like exponent width with fewer fraction bits. The exact autocast policy is framework and hardware dependent, so record versions and device. “BF16 training” does not mean every tensor is BF16.

27.2.2 Loss scaling

Coverage ID: T-27-02.

Small FP16 gradients can underflow to zero. Multiply loss by scale S , backpropagate scaled gradients, then divide gradients by S before clipping and stepping. Dynamic scaling lowers S when Inf/NaN gradients occur and may raise it after stable steps. BF16 often needs no loss scaling due to its wider exponent range. Clipping before unscale uses the wrong norm.

27.2.3 Activation checkpointing

Coverage ID: T-27-03.

Ordinary backward stores intermediates from each layer. Activation checkpointing stores selected boundary states and recomputes missing forward operations during backward. Memory falls; FLOPs and wall time rise. Random operations require preserved RNG behavior, otherwise recomputation uses different dropout masks and yields an invalid gradient. This mechanism is unrelated to saving a durable checkpoint despite the shared name.

27.2.4 Checkpoints

Coverage ID: T-27-04.

A resumable training checkpoint is a versioned bundle written atomically. Save to a temporary path, flush, then rename; incomplete writes should never appear valid. Include step/token counters, data position or sampler state, software/config identifiers, and integrity metadata in addition to the components below.

27.2.5 Weights

Coverage ID: T-27-05.

Model parameters reproduce the learned function in evaluation mode when paired with the architecture. They do not reproduce the next training step alone. Shared weights should remain shared after loading, and dtype conversion must be explicit.

27.2.6 Config

Coverage ID: T-27-06.

Store all shape- and behavior-defining fields: (V,C,D,L,H,M), normalization, positions, biases, dropout, tying, and special-token IDs. Code defaults drift. A checkpoint without its config is an archaeological artifact.

27.2.7 Tokenizer state

Coverage ID: T-27-07.

Save vocabulary, merge/model rules, normalization and pre-tokenization rules, added tokens, and special-token mappings. The same text under a different tokenizer produces different IDs; matching vocabulary size is insufficient.

27.2.8 Optimizer state

Coverage ID: T-27-08.

Adam moments and per-parameter step counters determine the next update. Omitting them restarts optimization dynamics even if weights match. Parameter ordering or names must map buffers to the correct tensors.

27.2.9 Scheduler state

Coverage ID: T-27-09.

Save the scheduler's current step/token counter, phase, and any best-metric state. Reconstructing from filename can be wrong after skipped steps, accumulation changes, or partial epochs.

27.2.10 RNG state

Coverage ID: T-27-10.

Save Python, NumPy, CPU framework, and each accelerator RNG state as applicable. Also preserve distributed sampler epoch/state. Exact replay can still be prevented by nondeterministic kernels or different hardware, but missing RNG state guarantees dropout and sampling divergence.

27.3 Memory example

One billion BF16 parameters occupy about 2 GB (decimal). FP32 master weights add 4 GB, FP32 gradients 4 GB, and two FP32 Adam moments 8 GB: roughly 18 GB before activations and allocator overhead. Implementations and sharding strategies vary, so measure the actual process.

27.4 Four recurring perspectives

- **Follow the Token:** autocast changes arithmetic representation, not token meaning or labels.
- **Follow the Gradient:** scaling protects small gradients; recomputation recreates the graph; optimizer buffers shape updates.
- **Follow the Byte:** low-precision activations save memory; optimizer state often dominates persistent training storage.
- **Follow the Request:** inference checkpoints may need only weights/config/tokenizer, while exact training resume needs the whole state bundle.

27.5 Lab and exit check

Train two steps, save, run a third step and record loss and parameters. Reload after step two and repeat the third step. With deterministic settings, compare loss, sampled batch, dropout behavior, parameters, optimizer buffers, and scheduler count. List any platform limitation preventing exact equality.

27.6 Exercises

1. A scaled backward pass uses $S = 1024$ and produces a stored gradient value 51.2. What value should clipping inspect after unscale? **Check:** 0.05.
2. List the minimum additional state needed to reproduce the next AdamW step beyond model weights. **Check:** optimizer moments and counters, scheduler state, batch/sampler position, applicable RNG states, and the exact configuration/software behavior.

27.7 References

- Micikevicius et al., [Mixed Precision Training](#), 2017.
- Chen et al., [Training Deep Nets with Sublinear Memory Cost](#), 2016.
- PyTorch reproducibility and automatic mixed precision documentation, version pinned for execution.

Chapter 28

Debugging and Training Our Tiny Model

Status: **Drafted — technical and editorial review pending.**

Part 7: Train the Thing

28.1 Learning outcome

Overfit a tiny batch, then train with a held-out split and document failures.

28.2 Prerequisites

CH-27.

Training combines data, masks, numerics, optimization, and mutable state. Debug it as a pipeline of falsifiable invariants rather than staring at one loss curve.

28.2.1 Debugging training

Coverage ID: T-28-01.

Begin with the smallest deterministic case. Print decoded inputs and shifted labels. Assert IDs, shapes, masks, dtypes, devices, and finite values. Establish a random-loss baseline: with V roughly uniform classes, cross-entropy begins near $\log V$. Overfit one tiny batch before scaling; if the model cannot memorize it, more data and GPUs only make the bug expensive.

Log valid tokens, learning rate, loss, gradient norm, parameter/update norm, tokens per second, and memory. Save the exact config and seed. Change one variable per experiment.

28.2.2 NaNs

Coverage ID: T-28-02.

NaNs can originate in data, logits, normalization, softmax, loss, gradients, or optimizer state. Locate the **first** nonfinite tensor with hooks or anomaly detection. Frequent causes include excessive learning rate, FP16 overflow, division by tiny values, all-masked attention rows, and invalid labels. Once a NaN enters Adam moments, restoring finite weights alone may not recover the run.

28.2.3 Exploding gradients

Coverage ID: T-28-03.

Plot global and per-layer gradient norms before clipping. A sudden rise can reflect bad batches, unstable residual scale, excessive learning rate, or incorrect loss normalization. Clipping avoids one destructive step but can conceal repeated instability. Compare update norm to parameter norm to learn whether nominal learning rate translates into extreme movement.

28.2.4 Bad initialization

Coverage ID: T-28-04.

Initialization that is too large produces saturated logits and growing residuals; too small can make updates or signals negligible. Inspect activation mean/RMS and gradient RMS by depth on the first batch. Output/head and residual projection initialization conventions can be architecture-specific. Never assume a generic initializer reproduces a published model.

28.2.5 Overfitting

Coverage ID: T-28-05.

Memorizing one batch is a debugging success. Memorizing the training split while held-out loss rises is a generalization failure. Track both under `eval()` with fixed data. Data leakage may make validation look artificially good; split before overlapping chunks and audit duplicates.

28.2.6 Train a tiny language model

Coverage ID: T-28-06.

Use a small corpus with a documented train/validation split. Fit tokenizer only on allowed training data if the experiment requires strict isolation. First overfit one batch until loss falls sharply. Then reset from initialization, train on the full training split, evaluate at fixed token intervals, and save the best and latest checkpoints. Generate samples from a fixed prompt and fixed seed as a qualitative smoke test; samples do not replace held-out loss.

A minimal loop is:

```
for step, batch in enumerate(loader):
    optimizer.zero_grad(set_to_none=True)
    logits = model(batch.ids)
    loss = token_cross_entropy(logits[:, :-1], batch.ids[:, 1:], batch.mask)
    loss.backward()
    grad_norm = clip_grad_norm_(model.parameters(), limit)
    optimizer.step(); scheduler.step()
```

28.3 Failure ledger

Record symptom, first failing invariant, evidence, hypothesis, intervention, and outcome. “Lowered LR and it worked” is weaker than showing the first nonfinite tensor was an FP16 attention score and that a precision change eliminated it under the same batch.

28.4 Four recurring perspectives

- **Follow the Token:** decode inputs and targets; many apparent model bugs are corrupt examples or shifts.

- **Follow the Gradient:** trace finiteness and scale from loss back through layers before the optimizer mutates state.
- **Follow the Byte:** leaks often come from retained graphs, saved activations, or unbounded logging—not model weights.
- **Follow the Request:** training correctness is tested with batches; generation tests causal execution and decoding as a user would observe it.

28.5 Lab and exit check

Pass four gates: uniform-loss sanity, one-batch overfit, held-out evaluation, and checkpoint resume. Intentionally inject a shifted-label bug and an excessive learning rate; document how each appears in metrics. Report seeds, versions, dataset hash, and limits of reproducibility.

28.6 Exercises

1. For vocabulary size 256, compute the uniform-prediction loss in nats. **Check:** $\log 256 \approx 5.545$.
2. Training loss falls while validation loss rises. Name two distinct hypotheses and one discriminating test for each. **Check:** examples include ordinary overfitting versus split leakage; regularization/data scaling tests the first, while cross-split duplicate auditing tests the second.

28.7 References

- Goodfellow, Bengio, and Courville, [Deep Learning](#), optimization chapters, 2016.
- PyTorch autograd anomaly detection and reproducibility documentation, version pinned for execution.

Part 8: Modern LLM Architecture

Chapter 29

Transformer Families

Status: **Drafted — technical and editorial review pending.**

Part 8: Modern LLM Architecture

29.1 Learning outcome

Compare masks and objectives across three Transformer families.

29.2 Prerequisites

CH-28.

The Transformer is a component family, not one fixed model. Encoder-only, decoder-only, and encoder-decoder systems differ primarily in information flow and training objective. Those differences determine what can be cached, what can be generated, and which representation serves a downstream task.

29.2.1 GPT to Llama-type architectures

Coverage ID: T-29-01.

Early GPT-style models established the decoder-only, causal, next-token recipe. Llama-type models retain that recipe while commonly replacing learned absolute positions with RoPE, LayerNorm with RMSNorm, GELU FFNs with SwiGLU, and in later variants full multi-head KV projections with grouped-query attention. These are concrete engineering choices, not a new Transformer family. Always read the pinned config: “Llama-like” is too vague to load weights or count memory.

29.2.2 Decoder-only Transformers

Coverage ID: T-29-02.

A decoder-only stack uses causal self-attention. At position (t) , Q comes from token (t) , while K and V come from positions $(0..t)$. Pretraining predicts (x_{t+1}) . This alignment directly supports open-ended generation and KV-cached decode. It can also perform classification or extraction when cast as generation or by adding a task head.

29.2.3 Encoder-only Transformers

Coverage ID: T-29-03.

Encoder self-attention is usually bidirectional: every non-padding token can attend to every other input token. This is ideal for contextual representations of a known sequence but cannot autoregressively generate without adding a causal decoder or iterative procedure. Padding masks hide absent tokens; they are distinct from causal masks.

29.2.4 BERT

Coverage ID: T-29-04.

BERT pretrains an encoder by corrupting selected tokens and predicting them from bidirectional context, alongside the original next-sentence prediction objective. A [CLS] representation is often fed to task heads. Because masked tokens appear during pretraining but not ordinary use, masked-language modeling differs from left-to-right likelihood. Later encoders changed objectives, but the core BERT example makes the family distinction clear.

29.2.5 Encoder-decoder Transformers

Coverage ID: T-29-05.

An encoder maps source IDs of length S to $H_e \in \mathbb{R}^{B \times S \times D}$ using bidirectional attention. A causal decoder processes target prefix length T , alternating self-attention with cross-attention to H_e . This separates source understanding from target generation and naturally supports translation, summarization, and conditional generation.

29.2.6 T5

Coverage ID: T-29-06.

T5 expresses tasks as text-to-text and pretrains with span corruption: contiguous spans are replaced by sentinel tokens, and the decoder generates the missing spans. Its original design uses relative position biases and an encoder-decoder stack. “T5” should not be reduced to ordinary token masking; sentinel-based span targets define its example construction.

29.2.7 Cross-attention

Coverage ID: T-29-07.

For decoder state $X_d \in \mathbb{R}^{B \times T \times D}$ and encoder state $H_e \in \mathbb{R}^{B \times S \times D}$:

$$Q = X_d W_Q, \quad K = H_e W_K, \quad V = H_e W_V,$$

$$\text{CrossAttn}(X_d, H_e) = \text{softmax}(QK^\top / \sqrt{d_h})V.$$

Scores have shape ([B,H,T,S]). The decoder’s causal mask applies to decoder self-attention, not ordinarily across the source; a source padding mask still applies. During autoregressive decode, encoder K/V can be computed once and reused.

29.3 Comparison

Family	Self-attention visibility	Common objective	Native output
Encoder-only	bidirectional	corruption/masked token	representations
Decoder-only	causal	next token	continuation
Encoder-decoder	encoder bidirectional; decoder causal	conditional/span generation	source-conditioned sequence

29.4 Four recurring perspectives

- **Follow the Token:** visibility masks determine which surrounding tokens can shape each representation.
- **Follow the Gradient:** objectives supervise corrupted tokens, next tokens, or conditional targets differently.
- **Follow the Byte:** encoder-decoder models store source states and cross-attention K/V in addition to decoder state.
- **Follow the Request:** classification may use one encoder pass; decoder generation repeats sequential decode; source encoding is reusable.

29.5 Lab and exit check

For a four-token source and three-token target, draw all permitted entries in encoder self-attention, decoder self-attention, and cross-attention masks. State which K/V tensors can be cached during generation and why. Compare the labels produced by BERT masking, GPT shifting, and T5 span corruption on one sentence.

29.6 Exercises

1. Give the materialized score shapes for encoder self-attention, decoder self-attention, and cross-attention with $B = 2, H = 8, S = 5, T = 3$. **Check:** $[2, 8, 5, 5]$, $[2, 8, 3, 3]$, and $[2, 8, 3, 5]$.
2. Explain why the decoder's cross-attention keys and values can be reused across generated positions but its self-attention cache grows. **Check:** the encoded source is fixed, whereas each emitted target token adds a new decoder key and value.

29.7 References

- Devlin et al., [BERT](#), 2018.
- Raffel et al., [Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer](#), 2019.
- Touvron et al., [LLaMA](#), 2023.

Chapter 30

Modern Positions, Heads and MLPs

Status: **Drafted — technical and editorial review pending.**

Part 8: Modern LLM Architecture

30.1 Learning outcome

Derive KV dimensions and parameter counts for three head-sharing configurations.

30.2 Prerequisites

CH-16, CH-19, CH-29.

Modern decoder models often keep the overall block while changing how position enters attention, how K/V are shared, and how the MLP gates features. These changes target quality, training behavior, and especially decode memory traffic.

30.2.1 RoPE deep dive

Coverage ID: T-30-01.

Rotary position embeddings rotate pairs of Q and K coordinates by a position-dependent angle. For one pair and position (p),

$$R(p\theta) \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} \cos p\theta & -\sin p\theta \\ \sin p\theta & \cos p\theta \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}.$$

Because $R(p\theta)^\top R(q\theta) = R((q-p)\theta)$, the dot product between rotated Q at p and K at q depends on relative displacement. Values are not normally rotated. Frequencies span coordinate pairs; model-specific base, scaling, and partial rotary dimensions are checkpoint contracts. Extending context by changing them is an intervention requiring evaluation, not a free extrapolation.

30.2.2 MHA

Coverage ID: T-30-02.

With H query heads and $d_h = D/H$, multi-head attention produces Q, K, and V each with H heads. Per cached token per layer, K and V store $2Hd_h = 2D$ elements. Projection weights for Q/K/V total $3D^2$ when all project from D to D .

30.2.3 MQA

Coverage ID: T-30-03.

Multi-query attention retains H query heads but uses one shared K head and one shared V head. Q has shape $[B, H, T, d_h]$; K/V have $[B, 1, T, d_h]$ and broadcast logically across query heads. Cache per token per layer falls to $2d_h$ elements, a factor H below MHA. Sharing can affect quality or capacity; it is not simply lossless compression.

30.2.4 GQA

Coverage ID: T-30-04.

Grouped-query attention uses H_{kv} K/V heads, where $1 < H_{kv} < H$ and normally $H \bmod H_{kv} = 0$. Each K/V group serves H/H_{kv} query heads. Cache storage is $2H_{kv}d_h$ elements per token per layer. MHA is the special case $H_{kv} = H$; MQA is $H_{kv} = 1$.

Example: $D = 4096, H = 32, d_h = 128$. MHA caches 8,192 K/V elements per token-layer; GQA with $H_{kv} = 8$ caches 2,048; MQA caches 256. At BF16 those are 16,384, 4,096, and 512 bytes respectively.

Projection parameters also change: Q remains $D \times D$, while K and V each become $D \times (H_{kv}d_h)$. The output projection remains $D \times D$.

30.2.5 Modern MLPs

Coverage ID: T-30-05.

Many current decoders use bias-free SwiGLU-like MLPs with an “up,” “gate,” and “down” projection. RMSNorm and pre-norm frequently accompany them, but these choices are independent. Fused kernels may compute two input projections together; the mathematical parameter count remains three matrices.

30.2.6 SwiGLU parameter accounting

Coverage ID: T-30-06.

For width D and intermediate M , SwiGLU weights total $3DM$. Compare a two-matrix GELU FFN at $M_g = 4D$: $2D(4D) = 8D^2$. Equal parameter count gives $3DM_s \approx 8D^2$, so $M_s \approx (8/3)D$, before rounding and biases. At $D = 4096, M_s = 11008$, the total is 135,266,304 weights.

30.3 Four recurring perspectives

- **Follow the Token:** RoPE changes Q/K phase; GQA changes which K/V representation each query head reads.
- **Follow the Gradient:** shared K/V heads aggregate learning signals from multiple query heads.
- **Follow the Byte:** GQA/MQA chiefly shrink persistent KV cache and K/V projection sizes; decode benefits strongly.
- **Follow the Request:** long-context, high-concurrency serving amplifies KV savings, while prefill still performs substantial attention compute.

30.4 Lab and exit check

For ($D=2048, H=16$), calculate projection parameters and BF16 KV bytes per token-layer for ($H_{\{kv\}}=16,4,1$). Verify that (D/H) is integral and each query head maps to exactly one K/V group. Implement a two-dimensional RoPE rotation and numerically show its dot product depends on position difference.

30.5 Exercises

1. For $D = 2048, H = 16, d_h = 128$, compute Q/K/V projection weights for $H_{kv} = 4$. **Check:** Q has 4,194,304 weights; K and V each have 1,048,576.
2. Compute BF16 KV bytes per token-layer for $H_{kv} = 16, 4, 1$. **Check:** 8192, 2048, and 512 bytes.

30.6 References

- Su et al., [RoFormer: Enhanced Transformer with Rotary Position Embedding](#), 2021.
- Shazeer, [Fast Transformer Decoding: One Write-Head is All You Need](#), 2019.
- Ainslie et al., [GQA: Training Generalized Multi-Query Transformer Models](#), 2023.

Chapter 31

Mixture of Experts

Status: **Drafted — technical and editorial review pending.**

Part 8: Modern LLM Architecture

31.1 Learning outcome

Simulate top-k routing and measure imbalance and token overflow.

31.2 Prerequisites

CH-30.

A mixture-of-experts layer replaces one dense MLP with many candidate MLPs and activates only a small subset per token. It expands parameter capacity without using every parameter on every token, while creating routing, communication, and reliability problems.

31.2.1 MoE

Coverage ID: T-31-01.

For token state $x \in \mathbb{R}^D$, a router emits expert logits $r = xW_r \in \mathbb{R}^E$. After softmax and top- k selection $\mathcal{T}(x)$,

$$y = \sum_{e \in \mathcal{T}(x)} g_e(x) \text{Expert}_e(x).$$

Experts are typically MLPs with separate parameters. Attention may remain dense. “A trillion-parameter MoE” therefore does not execute a trillion parameters per token, though all weights must be stored or distributed.

31.2.2 Sparse activation

Coverage ID: T-31-02.

With $E = 64, k = 2$, only two experts process each token. Active expert compute scales with k , while parameter storage scales with E . Router work, dispatch, gathering, and communication remain. Sparse activation means conditional computation, not sparse weight matrices.

31.2.3 Expert routing

Coverage ID: T-31-03.

Routing turns a flat set of $N = BT$ token vectors into per-expert buffers. Stable implementations track original token indices, selected expert, gate weight, and rank. After expert computation, outputs are weighted and scattered back. Top-k is discrete; training uses gradients through selected gate weights and auxiliary mechanisms, not ordinary gradients through the selection boundary.

A router can collapse, sending most tokens to a few experts. Small input or numerical changes near a top-k boundary may change dispatch abruptly.

31.2.4 Capacity

Coverage ID: T-31-04.

A common capacity per expert is

$$C = \left\lceil \text{capacity factor} \cdot \frac{Nk}{E} \right\rceil.$$

For $N = 1024, E = 8, k = 2$, balanced load is 256 assignments per expert. With factor 1.25, $C = 320$. Assignments above capacity must be dropped, rerouted, or handled by overflow logic. Each choice changes output and cost. Capacity computed per device or microbatch can behave differently at small batch sizes.

31.2.5 Load balancing

Coverage ID: T-31-05.

Measure assignment counts, gate-probability mass, maximum-to-mean load, coefficient of variation, entropy, and overflow. Equal counts are not the sole goal: experts should specialize without stranding capacity. Batch composition affects observed balance, so report distributions across steps rather than one average.

31.2.6 Router losses

Coverage ID: T-31-06.

Auxiliary losses encourage balanced probability and assignment. One family penalizes correlation between fraction of assignments f_e and mean router probability p_e , scaled by $E \sum_e f_e p_e$. Exact definitions differ by model. Too weak allows collapse; too strong can prioritize uniformity over task loss. Router z-losses may penalize large log-sum-exp values to stabilize logits.

31.2.7 Expert parallelism introduction

Coverage ID: T-31-07.

When experts live on different devices, tokens travel to their selected experts and outputs return, commonly through all-to-all collectives. Compute may be sparse while network traffic is dense and irregular. Placement, topology, token balance, and capacity determine utilization. Expert parallelism will be developed in Part XV; here the key point is that routing becomes a distributed systems operation.

31.3 Four recurring perspectives

- **Follow the Token:** router logits choose expert destinations; outputs are gathered back into the original sequence order.
- **Follow the Gradient:** task loss trains chosen experts and gates; auxiliary loss shapes routing statistics.
- **Follow the Byte:** all expert weights consume storage, while only selected weights and activations execute; distributed dispatch moves token vectors.
- **Follow the Request:** small or decode batches can underutilize experts and make communication dominate latency.

31.4 Lab and exit check

Simulate ($N=1000, E=8, k=2$) routing with biased logits. Compute capacity, assignment histogram, coefficient of variation, entropy, and overflow for factors 1.0 and 1.25. Compare dropping versus rerouting and name the resulting semantic difference. Repeat with a small decode batch and explain utilization.

31.5 Exercises

1. For $N = 1000, E = 8, k = 2$, calculate per-expert capacity at factors 1.0 and 1.25. **Check:** 250 and 313 after applying the ceiling.
2. One expert receives 340 assignments when capacity is 313. How many assignments overflow, and why is the token-drop rate not determined until the routing ranks are inspected? **Check:** 27 assignments overflow; a token may lose one of two routes or all routes depending on which assignments are dropped.

31.6 References

- Shazeer et al., [Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer](#), 2017.
- Fedus, Zoph, and Shazeer, [Switch Transformers](#), 2021.
- Lepikhin et al., [GShard](#), 2020.

Chapter 32

Alternative Architectures and Real Configs

Status: **Drafted — technical and editorial review pending.**

Part 8: Modern LLM Architecture

32.1 Learning outcome

Read a pinned model config and independently estimate its cost.

32.2 Prerequisites

CH-31.

Quadratic full attention is powerful but not mandatory. This chapter surveys structured attention, recurrent/state-space mechanisms, and hybrids, then turns a real configuration into parameter, memory, and compute estimates.

32.2.1 Sparse attention

Coverage ID: T-32-01.

Sparse attention restricts the $T \times T$ score pattern. If each query attends to $w \ll T$ keys, score work and storage can approach $O(Tw)$ rather than $O(T^2)$. The actual speedup requires kernels and layouts that exploit the pattern; storing a dense matrix with many masked entries saves nothing.

32.2.2 Sliding-window attention

Coverage ID: T-32-02.

A causal sliding window lets position t attend to positions $\max(0, t - w + 1), \dots, t$. Per-layer receptive field is limited, but stacking layers lets information propagate farther. KV eviction becomes possible for layers using only the window. The model cannot directly retrieve an arbitrarily old token through that layer.

32.2.3 Local attention

Coverage ID: T-32-03.

Local attention is the broader idea of neighborhood-limited connectivity and may be causal or bidirectional, one-dimensional or blocked. Boundary handling and dilation define which positions interact. It offers locality and regularity but can miss long-range dependencies.

32.2.4 Global attention

Coverage ID: T-32-04.

Selected global tokens or periodic full-attention layers create long-range routes. A pattern combining local windows with g global positions costs roughly $O(T(w + g))$. Which tokens become global is a modeling decision; excessive global connectivity restores quadratic cost.

32.2.5 State-space models

Coverage ID: T-32-05.

A state-space model updates a fixed-size state recurrently,

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t + Dx_t,$$

with modern variants making parts input-dependent and implementing training with parallel scans or convolutions. Recurrence gives linear sequence scaling and constant-size inference state with respect to context length, but the state compresses history rather than retaining every K/V vector for direct lookup.

32.2.6 Recurrent hybrids

Coverage ID: T-32-06.

Recurrent hybrids add gating, convolution, attention, or recurrent state to balance retrieval and efficient streaming. “Recurrent” describes sequential state evolution at inference; training may still use parallel algorithms. Evaluate quality, state size, kernel maturity, and latency rather than assuming asymptotic complexity predicts performance.

32.2.7 Hybrid models

Coverage ID: T-32-07.

A hybrid can interleave attention and state-space layers or combine local and global mechanisms. Attention layers provide content-addressable access; recurrent layers process long streams cheaply. The optimal ratio depends on tasks and hardware. Interfaces still carry ([B,T,D]) tensors, making composition easy at the shape level, though training dynamics remain architecture-specific.

32.2.8 Reading a real model config

Coverage ID: T-32-08.

Pin the exact configuration file or release. Extract (V,D,L,H,H_{kv},M,C), norm type, positional settings, tying, biases, and architecture class. As a worked configuration, use the published Llama 2 7B architecture values: (D=4096,L=32,H=32,H_{kv}=32,M=11008,V=32000). Treat the following as estimates; implementation details such as biases and tying must be checked against the pinned artifact.

32.2.9 Parameters

Coverage ID: T-32-09.

Per layer, standard attention has approximately $4D^2 = 67.11$ million weights. SwiGLU has $3DM \approx 135.27$ million. Together that is about 202.38 million; across 32 layers, 6.476 billion. Add embeddings/head and norms to approach the advertised scale. Advertised names are rounded, so derive from tensors when exactness matters.

32.2.10 Attention dimensions

Coverage ID: T-32-10.

Head dimension is $d_h = D/H = 128$. Q/K/V under MHA each project to 4096 values per token. Score shape for batch B and context T is $[B, 32, T, T]$ when materialized, motivating IO-aware kernels.

32.2.11 KV heads

Coverage ID: T-32-11.

Here $H_{kv} = 32$, so this configuration uses MHA. A config with GQA would replace K/V projection output width D with $H_{kv}d_h$. Never infer KV heads from query-head count.

32.2.12 MLP size

Coverage ID: T-32-12.

The intermediate width 11008 gives expansion $11008/4096 \approx 2.6875$. Because SwiGLU has three matrices, this is close in parameter count to a two-matrix $4\times$ FFN.

32.2.13 Memory footprint

Coverage ID: T-32-13.

Seven billion parameters require roughly 14 GB at two bytes each, 28 GB at four bytes, before allocator metadata, activations, KV cache, temporary workspaces, or framework overhead. Decimal GB and binary GiB differ. Quantized files also include scales and metadata, so “4-bit” does not imply exactly 0.5 byte per parameter in memory.

For MHA, BF16 KV cache per token is $L \times 2 \times H_{kv} \times d_h \times 2$ bytes: $32 \times 2 \times 32 \times 128 \times 2 = 524,288$ bytes per token across layers, about 512 KiB, before batching and overhead.

32.2.14 FLOPs per token

Coverage ID: T-32-14.

A common dense-model estimate is roughly two FLOPs per active weight per token for the matrix multiplies, about 14 GFLOPs/token for 7B active parameters, plus attention terms and other operations. During training, forward plus backward is often estimated near six FLOPs per parameter-token, but activation recomputation and implementation details change it. During prefill, attention adds terms growing with context; decode rereads weights and attends over cached positions. Always state whether the estimate is training, prefill, or decode.

32.3 Four recurring perspectives

- **Follow the Token:** sparse attention restricts explicit history; state-space recurrence compresses history; hybrids combine both.

- **Follow the Gradient:** parallel scans and sparse patterns change backward execution even when layer interfaces match.
- **Follow the Byte:** configuration fields determine weight and state/KV storage; asymptotic savings require supporting kernels.
- **Follow the Request:** prefill and streaming decode stress different mechanisms, so architecture comparisons require matched workloads and quality.

32.4 Lab and exit check

Download or record one exact official config revision, then compute (d_h), attention/MLP parameters, weight bytes by dtype, and KV bytes for ($B=1, T=4096$). Compare your sum with framework tensor counts and explain every discrepancy. For one sparse or state-space alternative, name information it cannot access as directly as full attention.

32.5 Exercises

1. Using the Llama 2 7B values in this chapter, compute BF16 KV storage at $B = 1, T = 4096$. **Check:** $524,288 \times 4096 = 2,147,483,648$ bytes, exactly 2 GiB before overhead.
2. A causal window has $w = 512$ and $L = 24$ layers. Give an upper bound on how far information can propagate through stacked local-attention layers and state why this is not equivalent to one-layer direct retrieval. **Check:** roughly $L(w - 1) = 12,264$ positions through successive hops, subject to boundary and implementation details.

32.6 References

- Beltagy, Peters, and Cohan, [Longformer](#), 2020.
- Gu and Dao, [Mamba: Linear-Time Sequence Modeling with Selective State Spaces](#), 2023.
- Touvron et al., [Llama 2](#), 2023.

Part 9: Data, Scaling and Evaluation

Chapter 33

Data Sources and Cleaning

Status: **Drafted.**

Part 9: Data, Scaling and Evaluation

33.1 Learning outcome

Create a small corpus with reproducible filtering and exact/near-duplicate checks, while retaining enough provenance to explain why every document is present.

33.2 Prerequisites

CH-25 and CH-28. Sequence construction and split leakage are developed there; this chapter moves upstream to the corpus pipeline.

33.3 Training data sources

An LLM learns from token sequences, but those sequences begin as artifacts made for other purposes: web pages, books, source code, papers, conversations, manuals, and licensed or commissioned datasets. A source is more than a topic label. It has an acquisition method, timestamp, rights basis, language distribution, document boundaries, and selection process. Those properties shape the model. Web crawls offer breadth but contain spam and duplication; books offer sustained prose but a narrower distribution; code has executable structure but licenses and generated copies require care; instruction data teaches interaction patterns but may be small and stylistically repetitive.

Build a source manifest before processing content. A useful row contains `source_id`, URI or collection name, retrieval date, content hash, rights basis, parser version, language hint, and raw byte count. Keep raw and derived datasets logically distinct. A later parser fix must not silently redefine the old dataset. The central engineering rule is lineage: a training token should be traceable to a source record and transformation version.

33.4 Cleaning

Cleaning converts an artifact into a faithful document. For HTML, that may mean decoding bytes, removing navigation and scripts, extracting visible text, preserving headings and code blocks,

normalizing Unicode, and recording extraction failures. Cleaning should remove representation noise without rewriting the author’s meaning.

Order matters. Decode before Unicode normalization; parse markup before collapsing whitespace; preserve document and paragraph boundaries before tokenization. Keep counters for every transformation. If 18% of a domain becomes empty after extraction, that is a pipeline incident. Aggressive cleaning can be destructive: lowercasing harms case-sensitive code, ASCII-only rules delete most languages, and flattening tables can scramble relationships. Inspect before/after samples for every rule.

33.5 Filtering

Filtering decides whether a document or span belongs in the training distribution. Deterministic filters may use length, repeated-character ratio, language identification, parser confidence, boilerplate fraction, or secrets and personal-data detectors. Learned classifiers can rank quality, but their own training distribution imports preferences and blind spots.

Treat a filter as a versioned decision function that emits a reason, not merely a Boolean. Measure acceptance rates by source, language, topic, and time. A global 60% acceptance rate can hide a filter that keeps 90% of English and 5% of Tamil. Quality filtering is optimization under coverage constraints, not a universal scalar.

33.6 Deduplication

Exact deduplication hashes a canonical representation. Near deduplication finds substantial overlap. A common pipeline tokenizes shingles, computes MinHash signatures, and uses locality-sensitive hashing to produce candidate pairs; exact similarity is checked only for candidates.

For token-shingle sets A and B , Jaccard similarity is

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

If two pages have 90 shared 5-grams and 10 unique 5-grams each, their union has 110 and $J = 90/110 \approx 0.818$. A threshold of 0.8 marks them as near duplicates. The threshold and shingle size are part of the dataset definition.

Cluster duplicates before assigning splits. Otherwise a paraphrased training item can leak into evaluation. Distinguish unwanted repetition from legitimate recurrence: identical license headers should not force unrelated code files into one giant cluster.

33.7 Dataset provenance

Publish a dataset card or internal equivalent containing sources, versions, transformations, removal rules, known limitations, composition statistics, and hashes of produced shards. Pin code and configuration. Record random seeds for probabilistic sampling. One design is immutable raw objects plus an append-only ledger:

```
raw_hash -> parse:v3 -> language_filter:v2 -> pii_filter:v5
          -> dedup_cluster:8f31 -> shard:train-0042
```

If a source must later be removed, lineage identifies affected shards and checkpoints. Without it, “we removed the data” is difficult to demonstrate.

33.8 Four recurring perspectives

- **Follow the Token:** raw bytes become decoded text, normalized spans, tokenizer IDs, and packed sequences.
- **Follow the Gradient:** preprocessing has no gradient, but inclusion determines which token losses later contribute gradients.
- **Follow the Byte:** raw copies, parsed text, signatures, and tokenized shards can multiply storage.
- **Follow the Request:** provenance connects an observed response back to candidate training sources and transformations.

33.9 Lab and exit check

Collect 100 documents from two permitted sources. Produce a manifest, clean them, emit filter reason codes, remove exact duplicates, and cluster near duplicates. Report document and byte counts after each stage and inspect ten accepted and ten rejected items. Exit check: can another person reproduce the final shard from the manifest and pinned code?

33.10 Exercises

1. Two documents have 75 shared shingles and 25 unique shingles each. Compute Jaccard similarity. **Check:** the union has 125 shingles, so $J = 0.60$.
2. Explain why deduplicating separately inside train and validation is insufficient. **Check:** cross-split duplicates survive; clustering must precede assignment if the cluster is the unit of independence.

33.11 References

- Lee et al., [Deduplicating Training Data Makes Language Models Better](#), 2021.
- Dodge et al., [Documenting Large Webtext Corpora](#), 2021.
- Penedo et al., [The RefinedWeb Dataset for Falcon LLM](#), 2023.

Chapter 34

Mixtures, Tokens and Synthetic Data

Status: **Drafted.**

Part 9: Data, Scaling and Evaluation

34.1 Learning outcome

Design a defensible token allocation, construct splits by unit of independence, and detect evaluation leakage.

34.2 Prerequisites

CH-33.

34.3 Dataset mixtures

A mixture specifies the probability that the next training example comes from each component. Raw availability should not decide this automatically. If a corpus contains 90% web prose and 1% code, sampling in those proportions may undertrain code. Upsampling code to 20% changes both capabilities and the number of epochs seen by that component.

Let component i contain D_i tokens and receive probability p_i . Over budget T , expected sampled tokens are $p_i T$, and expected epochs are $p_i T / D_i$. With $T = 10\text{B}$, a 100M-token curated corpus at $p = 0.05$ is seen five times. Mixtures may be fixed, staged, or adaptive. Always save realized counts, not just intended probabilities.

34.4 Token budgets

A token budget is the number of tokens processed, including repetitions. It connects data to compute: for a dense decoder Transformer, a planning approximation is roughly $6ND$ FLOPs for N non-embedding parameters and D training tokens. The constant varies with architecture and implementation, so profile before procurement.

Allocate the budget in a table with available tokens, sampling weight, expected sampled tokens, expected epochs, and rationale. Account for tokenizer differences: “one billion tokens” is not language-neutral. Packing efficiency matters too; padding consumes compute while teaching nothing.

34.5 Synthetic data

Synthetic data is generated or transformed by models, programs, simulators, or rules. It can target rare skills, create verifiable exercises, translate material, or attach explanations. Its value comes from controlled coverage and verification.

A robust pipeline separates generator and judge, retains generator version and prompt, validates answers with an external oracle where possible, and samples failures for human review. Execute code tests in a sandbox and recompute arithmetic. Common failures are self-imitation, collapsed style, plausible wrong answers, and amplification of generator bias. As of September 2026, claims that synthetic data either inevitably poisons models or universally solves scarcity remain too broad.

34.6 Data quality

Quality is conditional on intended behavior. A novel is poor data for exact API usage; terse compiler errors may be excellent debugging data. Define measurable dimensions: correctness, relevance, diversity, freshness, formatting integrity, and source trust. Track quality by slice and validate it against downstream experiments. Loss alone is insufficient: repetitive or leaked data can be easy to predict.

34.7 Contamination

Contamination occurs when evaluation content, answers, or close variants influence training. Exact matching catches only easy cases. Search normalized n-grams, near duplicates, problem templates, translations, and solution text. Date splits help only when timestamps are reliable.

Leakage inflates an estimate without improving the underlying capability. If a benchmark problem appears verbatim, a correct response may measure recall. If only a solution pattern appears, the boundary is murkier; report methods and thresholds instead of claiming binary purity.

34.8 Split design

Random row splits are often wrong because rows are not independent. Split by author, repository, website, conversation, patient, or deduplication cluster—whichever unit carries shared signal. Repeatedly inspecting test failures turns the test set into development data; freeze it or introduce a fresh final set.

For temporal products, train before a cutoff and test afterward. For code, split repositories rather than files. For conversations, keep each thread together. Document exclusions and slice counts.

34.9 Four recurring perspectives

- **Follow the Token:** a source weight becomes a sampled document, tokenized sequence, and position in a packed batch.
- **Follow the Gradient:** mixture weights scale how frequently each distribution supplies gradient signals.

- **Follow the Byte:** token budgets determine stored shards and total training traffic.
- **Follow the Request:** split design must resemble future requests; leakage can make offline scores irrelevant.

34.10 Lab and exit check

Given components of 8B, 2B, 500M, and 100M unique tokens, allocate a 12B-token run and compute expected epochs. Plant five exact and five paraphrased benchmark leaks; test hash and shingle detectors. Explain false positives and false negatives.

34.11 Exercises

1. A 200M-token component has mixture weight 0.08 in a 15B-token run. Compute its expected sampled tokens and epochs. **Check:** 1.2B sampled tokens and six expected epochs.
2. Give one contamination detector likely to miss translated benchmark items and one stronger follow-up. **Check:** exact hashes miss translations; multilingual semantic retrieval followed by human or model-assisted adjudication is stronger but has false positives.

34.12 References

- Gao et al., [The Pile](#), 2020.
- Longpre et al., [A Pretrainer's Guide to Training Data](#), 2023.
- Carlini et al., [Quantifying Memorization Across Neural Language Models](#), 2022.

Chapter 35

Evaluation and Meaningful Experiments

Status: **Drafted.**

Part 9: Data, Scaling and Evaluation

35.1 Learning outcome

Report a paired baseline comparison with defined metrics, controlled inference, slice analysis, and uncertainty.

35.2 Prerequisites

CH-28, CH-33, and CH-34.

35.3 Evaluation

Evaluation estimates behavior under a stated distribution and protocol. Begin with the decision the result supports. Model selection needs sensitivity; a release gate needs thresholds tied to risk; research needs a falsifiable hypothesis.

Pin checkpoint, tokenizer, prompt template, decoding settings, tool environment, dataset revision, scoring code, and hardware/software versions when measuring performance. Report quality, latency, throughput, memory, reliability, and relevant safety properties. A single average hides regressions.

35.4 Perplexity

For M evaluated tokens with average negative log-likelihood

$$L = -\frac{1}{M} \sum_{t=1}^M \log p(x_t | x_{<t}),$$

perplexity is $\text{PPL} = e^L$. If $L = 2$, perplexity is about 7.39. It is the geometric mean inverse probability assigned to the observed next token. Compare it only under the same tokenizer, token rules, and corpus. Lower perplexity can coexist with worse instruction following.

35.5 Downstream tasks

Task evaluations test code, retrieval, summarization, or domain QA. Exact match fits objective short answers, not every semantic task. Execute code against hidden tests; separate retrieval recall from answer generation; check summary factuality.

Choose slices before seeing results: language, difficulty, context length, domain, and request type. Macro averaging gives slices equal weight; micro averaging weights examples. Report which one you use.

35.6 Reasoning benchmarks

Reasoning benchmarks mix reasoning, knowledge, parsing, and format compliance. Verifiable final answers are useful, but generated rationales do not prove hidden computation followed that prose. Test alternate forms, distractors, and out-of-distribution compositions. Hold inference budget fixed: 64 samples versus one sample compares different systems.

Benchmark saturation and exposure make claims time-sensitive. As of September 2026, check the exact benchmark and model revisions plus contamination evidence.

35.7 Human evaluation

For contextual outputs, write a rubric with observable criteria, randomize order, blind model identity, and permit ties. Use multiple raters on a subset and report agreement. Pairwise comparison is easier than absolute scoring but remains vulnerable to position, verbosity, and style biases. Preference is not truth: raters can prefer confident errors, so add factual checks and domain experts.

35.8 Experimental design

Write the hypothesis and primary metric first. Choose a baseline differing only in the mechanism studied. Hold data, training budget, inference settings, and scoring constant. Run multiple seeds when variance can change the result. Use paired comparisons on identical items. Record attempted variants; selecting the best among many and calling it one test is multiple-comparison bias.

35.9 Uncertainty introduction

A point estimate without uncertainty invites false precision. For accuracy on n independent items, rough standard error is $\sqrt{\hat{p}(1 - \hat{p})/n}$. For paired systems, bootstrap items and recompute the difference; cluster-bootstrap when examples share a repository or conversation.

If A scores 73% and B 74% on 200 items, the gap may be noise. Report an interval for $B - A$. Statistical significance also need not imply practical value; predefine the smallest useful effect.

35.10 Four recurring perspectives

- **Follow the Token:** scoring must identify conditioned tokens and tokens included in loss.
- **Follow the Gradient:** evaluation disables gradients, but using tests to choose updates transfers information into selection.
- **Follow the Byte:** retained logits dwarf scalar metrics; stream statistics unless diagnostics require them.
- **Follow the Request:** include prompts, tools, retries, queueing, and stop conditions when evaluating the system.

35.11 Lab and exit check

Compare two fixed models on 300 examples. Pin decoding, report paired accuracy difference with a bootstrap 95% interval, break results into predetermined slices, inspect disagreements, and measure latency with warmups. State one supported conclusion and one unsupported conclusion.

35.12 Exercises

1. Compute perplexity for average NLL 1.5 nats. **Check:** $e^{1.5} \approx 4.48$.
2. Model A and B are evaluated on the same 500 prompts. Explain why bootstrapping paired item-level differences is preferable to treating their aggregate scores as independent. **Check:** pairing retains covariance from shared item difficulty and estimates uncertainty in the actual per-item comparison.

35.13 References

- Liang et al., [Holistic Evaluation of Language Models](#), 2022.
- Dror et al., [Testing Statistical Significance in NLP](#), 2018.
- Mitchell et al., [Model Cards for Model Reporting](#), 2018.

Chapter 36

Scaling Laws

Status: **Drafted.**

Part 9: Data, Scaling and Evaluation

36.1 Learning outcome

Fit a small scaling experiment, allocate a compute budget, and disclose extrapolation uncertainty.

36.2 Prerequisites

CH-34 and CH-35.

36.3 Scaling laws

Scaling laws are empirical regularities connecting loss to model size, data, and compute. A common form is

$$L(N) = L_{\infty} + AN^{-\alpha}.$$

Taking logs linearizes the reducible part only if L_{∞} is known. Fit the original objective or jointly estimate the floor; subtracting an arbitrary floor can manufacture a straight line. Scaling laws summarize a measured regime, not a physical guarantee.

Kaplan et al. (2020) found smooth power laws over their range. Hoffmann et al. (2022) revisited compute-optimal allocation with more runs and concluded model size and token count should grow roughly together under their assumptions. Different recommendations reflect data, fits, and optimization; a “Chinchilla ratio” is not universal.

36.4 Parameters vs data vs compute

More parameters increase capacity and per-token cost. More unique appropriate data supplies evidence. More compute can buy either. Under fixed compute, an oversized model trained on too few tokens may be undertrained; a small model may see many epochs but hit capacity.

For dense Transformer planning, $C \approx 6ND$ FLOPs is useful. With $N = 1\text{B}$ and $D = 20\text{B}$, this gives 1.2×10^{20} FLOPs. It excludes system inefficiency. Wall time is $C/(uP)$, where P is peak FLOP/s and u achieved utilization; measure u .

36.5 Compute budgets

Budget pilot runs, search, failures, evaluation, checkpointing, and inference after training. A training-optimal model can be deployment-poor if latency or memory is unacceptable. Use proxy runs to calibrate tokens/second, utilization, loss curves, and input bottlenecks. Maintain rerun reserve and predeclare stopping limits.

36.6 Scaling fits

Train a grid varying N and D enough to identify both effects. Keep data, tokenizer, optimizer quality, and evaluation consistent. Fit held-out loss, inspect residuals, and bootstrap across runs. Report estimates, intervals, observed range, and fit error.

For sizes 10M, 30M, 100M, and 300M at several budgets, changing architecture and data only for the largest model destroys the scale-only comparison. Curved residuals indicate misspecification.

36.7 Extrapolation limits

Predictions beyond the measured range compound uncertainty. Data-regime changes, architecture transitions, optimizer instability, precision limits, and capability thresholds can break a fit. Show observations and extrapolations separately, offer plausible ranges, and refit after each step. As of September 2026, results from one architecture and corpus remain evidence about that setup rather than a timeless recipe.

36.8 Four recurring perspectives

- **Follow the Token:** D counts processed tokens, including repeats; composition can change while the scalar stays fixed.
- **Follow the Gradient:** batch size and scale change optimization noise; a poor optimizer can resemble a scaling limit.
- **Follow the Byte:** parameters, optimizer states, activations, and checkpoints constrain feasible scale.
- **Follow the Request:** serving volume can make a smaller, more-trained model economically superior.

36.9 Lab and exit check

Train three sizes at three budgets on one corpus. Fit a power law, visualize residuals, bootstrap parameters, and predict a held-out run. Report prediction error and interval. Add an inference-cost term and see whether model selection changes.

36.10 Exercises

1. Estimate training FLOPs for $N = 3 \times 10^9$ parameters and $D = 60 \times 10^9$ tokens using $6ND$.
Check: 1.08×10^{21} FLOPs.

2. At sustained throughput $uP = 2 \times 10^{17}$ FLOP/s, convert that estimate to idealized wall time.
Check: 5,400 seconds, or 1.5 hours; this deliberately simplified result excludes failures, evaluation, and pipeline stalls.

36.11 References

- Kaplan et al., [Scaling Laws for Neural Language Models](#), 2020.
- Hoffmann et al., [Training Compute-Optimal Large Language Models](#), 2022.
- Bian et al., [Scaling Inference-Efficient Language Models](#), 2025.

Part 10: Post-Training and Reasoning

Chapter 37

Pretraining, Continued Pretraining and SFT

Status: **Drafted.**

Part 10: Post-Training and Reasoning

37.1 Learning outcome

Specify a continued-pretraining and supervised fine-tuning pipeline with correct templates and loss masks.

37.2 Prerequisites

CH-25 through CH-28 and CH-33 through CH-36.

37.3 Pretraining vs post-training

Pretraining learns broad statistical structure, usually through next-token prediction over large heterogeneous corpora. Given tokens $x_1, \dots, x_T$, the causal loss is

$$\mathcal{L} = - \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_{\leq t}).$$

Every eligible position supplies a label. The resulting base model can complete text, but it has not necessarily learned the conversational contract “follow this user’s instruction.” Post-training changes behavior using narrower data and objectives: supervised demonstrations, preferences, rewards, safety examples, or domain adaptation. These stages do not install a separate reasoning module; they modify the same parameters or attached modules.

Keep checkpoint names precise. “Base,” “continued pretrained,” “instruction tuned,” and “preference optimized” describe different distributions and should not be used interchangeably. Evaluate after each stage so later gains do not hide earlier regressions.

37.4 Continued pretraining

Continued pretraining resumes the language-model objective on new material: a domain, language, recent period, or revised mixture. It is useful when vocabulary and knowledge distributions differ materially. It can also cause catastrophic forgetting or shift the model's style.

Use a conservative learning rate, mix some general replay data when retention matters, and compare domain and general validation loss. If the tokenizer poorly represents the new domain, adding tokens changes embedding and output matrices and requires careful initialization. Domain documents that contain instructions are still treated as sequences; continued pretraining does not automatically teach which spans are user requests versus answers.

Example: adapt a 7B base model to maintenance manuals. Pack complete document sections, mask padding, retain source boundaries, and monitor both manual perplexity and general benchmarks. If domain loss improves but instruction following later degrades, the problem may be mixture or stage ordering rather than insufficient capacity.

37.5 Supervised fine-tuning

Supervised fine-tuning (SFT) trains on demonstrations of desired responses. A record may contain system instructions, user messages, assistant responses, and tool events. Teacher forcing supplies the correct previous tokens during training. The model learns response format and behavior patterns, but it can also memorize stylistic artifacts and errors.

Curate for correctness, coverage, and diversity. A small high-quality set can outperform a larger repetitive one. Mix multi-turn and refusal examples only in proportions justified by deployment. Monitor length: long answers contribute more token losses and can dominate unless examples or tokens are reweighted.

37.6 Chat templates

A chat template serializes roles into tokens. A conceptual record might become:

```
<bos><system>You are concise.<eot>
<user>What is 6*7?<eot>
<assistant>42<eot>
```

The exact tokens are tokenizer- and model-specific. Training and inference must use the same template, including beginning, end, tool, and generation-prompt tokens. Duplicating a BOS token or omitting the assistant marker changes conditioning. Treat templates as versioned model artifacts and test round-trip rendering.

37.7 Loss masking

An attention mask controls what positions can be read; a loss mask controls which target positions contribute gradients. They solve different problems. For assistant-only SFT, system and user tokens remain visible as context but receive label -100 (or equivalent) so cross-entropy ignores them. Assistant response tokens are labels.

For the sequence above, a simplified mask is:

```
tokens: system... user... assistant: 4 2 <eot>
loss:      0...    0...          1 1  1
```


Mask padding and decide explicitly whether tool outputs, assistant headers, and end tokens are trained. An off-by-one shift can train the model to predict the first response token from the wrong prefix. Unit-test token IDs, rendered text, labels, and the count of supervised positions.

37.8 Four recurring perspectives

- **Follow the Token:** source text becomes a chat serialization; context tokens may be visible while excluded from loss.
- **Follow the Gradient:** pretraining updates on nearly every non-padding target; assistant-only SFT updates only selected spans.
- **Follow the Byte:** full fine-tuning stores gradients and optimizer state for the entire model, often far beyond weight memory.
- **Follow the Request:** the exact deployed template determines whether learned role boundaries are reproduced.

37.9 Lab and exit check

Render three multi-turn records with a pinned tokenizer. Print token IDs, roles, attention masks, and labels. Assert that only intended assistant spans contribute loss, including after truncation and packing. Draft a continued-pretraining mixture with general replay and define domain-retention gates.

37.10 Exercises

1. A rendered example has 120 context tokens, 80 assistant tokens, and 10 padding tokens. Under assistant-only loss, how many targets may contribute before accounting for the one-token shift? **Check:** 80, provided the response and end-token policy define all 80 as supervised.
2. Explain why setting user-token labels to an ignore index must not also hide those tokens in the attention mask. **Check:** they provide conditioning context even though they do not directly contribute cross-entropy terms.

37.11 References

- Ouyang et al., [Training Language Models to Follow Instructions with Human Feedback](#), 2022.
- Wei et al., [Finetuned Language Models Are Zero-Shot Learners](#), 2021.
- Hugging Face, [Chat templates documentation](#) (version-sensitive; accessed September 2026).

Chapter 38

Parameter-Efficient Adaptation

Status: **Drafted.**

Part 10: Post-Training and Reasoning

38.1 Learning outcome

Choose an adaptation method and count its trainable parameters, persistent storage, and optimizer memory.

38.2 Prerequisites

CH-27 and CH-37.

38.3 PEFT

Parameter-efficient fine-tuning (PEFT) freezes most base parameters and trains a small set of new or selected parameters. It reduces optimizer and gradient memory, enables many task variants to share one base checkpoint, and can simplify distribution. It does not eliminate forward/backward activation memory, and quality depends on where and how capacity is inserted.

Compare methods by trainable count, inference overhead, mergeability, task-switching cost, and quality. “One percent trainable” does not mean one percent of total training memory or compute.

38.4 Adapters

A bottleneck adapter adds a residual module such as

$$h' = h + W_{up} \sigma(W_{down} h),$$

where $W_{down} \in \mathbb{R}^{r \times d}$ and $W_{up} \in \mathbb{R}^{d \times r}$. Ignoring biases, one adapter has $2dr$ trainable parameters. For $d = 4096$, $r = 64$, that is 524,288 parameters per insertion point. Adapters add inference operations unless folded into an equivalent linear transform, which nonlinear bottleneck adapters generally cannot be.

38.5 Prompt tuning

Prompt tuning learns m virtual embedding vectors of dimension d prepended to the input. It trains md parameters: 20 virtual tokens at $d = 4096$ require 81,920. The base model stays frozen. Virtual tokens consume context positions and participate in attention. Prompt tuning can work well at sufficient model scale but provides limited capacity for large behavioral shifts.

38.6 Prefix tuning

Prefix tuning learns continuous key/value-like states for multiple layers rather than only input embeddings. It influences attention at each adapted layer but increases KV memory and per-request prefix processing or storage. Implementations differ: some directly learn prefixes; others use a small network to generate them. State the parameterization before counting.

38.7 LoRA

Low-Rank Adaptation represents a weight update as

$$W' = W + \frac{\alpha}{r}BA,$$

with frozen $W \in \mathbb{R}^{d_{out} \times d_{in}}$, $A \in \mathbb{R}^{r \times d_{in}}$, and $B \in \mathbb{R}^{d_{out} \times r}$. Trainable count is $r(d_{in} + d_{out})$. For a 4096-by-4096 projection with $r = 16$, that is 131,072 parameters versus 16,777,216 in W —0.78125%.

LoRA can target attention, MLP, or other projections. Rank, targets, scaling, dropout, and initialization matter. At inference, a LoRA update can often be merged into weights; hot-swapping many adapters may instead use separate kernels. Combining adapters by adding deltas is not guaranteed to preserve either behavior.

38.8 QLoRA

QLoRA stores the frozen base in a low-bit quantized representation while backpropagating into LoRA parameters. Its original formulation used 4-bit NormalFloat, double quantization, and paged optimizers. Computation still dequantizes blocks into a compute dtype; it is not “training in four-bit arithmetic.” Adapter weights and optimizer states use higher precision.

Quantization saves base-weight memory, but activations, temporary dequantized tiles, gradients for adapters, and optimizer state remain. Hardware kernels determine speed; smaller storage can even be slower if dequantization is inefficient. Validate final quality and peak allocated memory on the exact stack.

38.9 Memory worked example

For 100M trainable parameters with AdamW, FP32 parameters, gradients, and two optimizer moments can approach 16 bytes per parameter, or 1.6 GB, before activations. A LoRA configuration with 1M trainable BF16 parameters and FP32 moments uses roughly 2 MB weights + 2 MB gradients + 8 MB moments = 12 MB, though framework copies and master weights may add more. Measure, do not infer from trainable count alone.

38.10 Four recurring perspectives

- **Follow the Token:** prompt and prefix methods alter the states seen by every token; LoRA changes projections applied to them.
- **Follow the Gradient:** gradients stop at frozen weights but flow through the frozen computation into trainable modules.
- **Follow the Byte:** optimizer savings can be dramatic; activations and frozen-weight reads remain.
- **Follow the Request:** adapter selection becomes serving state, affecting batching, cache identity, and latency.

38.11 Lab and exit check

Adapt one linear layer with LoRA, verify the count $r(d_{in} + d_{out})$, and compare outputs before and after merging. Train on a tiny mapping, log trainable bytes and peak memory, then reload base plus adapter and reproduce output. Explain what must be invalidated in a prefix cache when the adapter changes.

38.12 Exercises

1. Apply rank-8 LoRA to Q and V projections in each of 32 layers with $d_{in} = d_{out} = 4096$. Compute the trainable count. **Check:** $32 \times 2 \times 8 \times (4096 + 4096) = 4,194,304$.
2. State which listed methods consume additional sequence or cache capacity at inference. **Check:** prompt tuning consumes input positions; prefix tuning consumes per-layer attention-prefix/KV capacity; merged LoRA need not add sequence length or a separate matmul.

38.13 References

- Housby et al., [Parameter-Efficient Transfer Learning for NLP](#), 2019.
- Li and Liang, [Prefix-Tuning](#), 2021.
- Hu et al., [LoRA](#), 2021.
- Dettmers et al., [QLoRA](#), 2023.

Chapter 39

Preferences and Reinforcement Learning

Status: **Drafted.**

Part 10: Post-Training and Reasoning

39.1 Learning outcome

Compare the data, objective, and update path for reward-model/PPO-style RLHF and direct preference optimization.

39.2 Prerequisites

CH-26, CH-35, and CH-37.

39.3 Preference data

A preference record contains a prompt x and responses y_w and y_l , where a rater or rule prefers the winner. Collection quality depends on response diversity, randomized order, a rubric, rater expertise, and treatment of ties. Preferences are conditional: “more helpful” can conflict with “shorter,” “safer,” or “more factual.” Preserve criterion-level labels where possible instead of compressing every judgment prematurely.

Sampling only obviously good versus obviously bad responses teaches little near the decision boundary. Include difficult pairs, audit disagreement, and split by prompt source or conversation to prevent leakage.

39.4 Reward models

A reward model maps (x, y) to scalar $r_\phi(x, y)$. A common Bradley–Terry objective is

$$\mathcal{L}_{RM} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

It learns ordering, not an absolute unit. Adding a constant to all rewards changes nothing. Reward models can exploit length, format, or artifacts and can be confidently wrong off distribution.

Evaluate held-out pair accuracy, calibration where meaningful, and adversarial slices; never treat reward as ground truth.

39.5 RLHF

In the classic pipeline, SFT initializes a policy, preference data trains a reward model, and reinforcement learning updates the policy to produce higher-reward outputs while remaining near a reference. The loop samples prompts, generates responses, scores them, estimates advantages, and updates parameters. This is on-policy and operationally expensive because generation sits inside training.

The reward model becomes part of the environment. Optimizing it too hard invites reward hacking. Periodically inspect raw generations and evaluate independently of the training reward.

39.6 PPO

Proximal Policy Optimization limits destructive policy jumps using a clipped probability-ratio objective. For token actions, ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{old}(a_t|s_t)$. The surrogate is

$$\mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)].$$

LLM PPO typically also trains a value function and adds a KL penalty to a reference. Correct implementation requires masks for variable lengths, consistent old log-probabilities, advantage normalization choices, and stable generation. PPO clipping does not guarantee a hard trust region, and token-level credit from a sequence-level reward is noisy.

39.7 DPO-type objectives

Direct Preference Optimization avoids a separately deployed reward model and on-policy rollouts by optimizing policy likelihood ratios on fixed preference pairs. One form minimizes

$$-\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} \right] \right).$$

Sequence log-probabilities sum token log-probabilities under appropriate masks. DPO is simpler operationally, but remains sensitive to preference quality, length effects, reference choice, and distribution shift. “No reward model” does not mean no reward assumptions; the derivation encodes a particular preference model and KL-regularized optimum.

39.8 Policy KL control

KL control penalizes movement from a reference policy:

$$\mathbb{E}[r(x, y)] - \beta D_{KL}(\pi_\theta(\cdot|x) \parallel \pi_{ref}(\cdot|x)).$$

Too little control allows exploitation and drift; too much prevents learning. Implementations often estimate token-level sampled KL rather than the full distributional KL. Log both the estimator and response behavior. A target-KL controller may adapt β , but it cannot ensure the model stays safe or correct.

39.9 Four recurring perspectives

- **Follow the Token:** a response is a sequence of actions; masks determine which log-probabilities enter rewards and objectives.
- **Follow the Gradient:** RM gradients train ϕ ; PPO gradients use sampled advantages; DPO gradients contrast winner and loser likelihood ratios.
- **Follow the Byte:** PPO may hold policy, reference, reward, and value models plus rollout buffers; DPO usually needs policy, reference, and paired batches.
- **Follow the Request:** prompt distribution, sampling policy, and rubric decide which deployment behaviors get optimized.

39.10 Lab and exit check

For one preference pair, compute reward-model loss and DPO loss from supplied scalar rewards and sequence log-probabilities. Draw PPO's generation-to-update path and list every frozen and trainable model. Change response length and show how summed versus averaged log-probabilities alter the comparison.

39.11 Exercises

1. If $r(y_w) = 2.0$ and $r(y_l) = 0.5$, compute the reward-model loss. **Check:** $-\log \sigma(1.5) \approx 0.201$.
2. For $A_t > 0$, compare PPO's unclipped and clipped terms when $r_t = 1.4$ and $\epsilon = 0.2$. **Check:** the minimum uses $1.2A_t$; clipping limits this positive-advantage incentive but does not impose a hard policy-distance bound.

39.12 References

- Christiano et al., [Deep Reinforcement Learning from Human Preferences](#), 2017.
- Schulman et al., [Proximal Policy Optimization Algorithms](#), 2017.
- Ouyang et al., [Training Language Models to Follow Instructions with Human Feedback](#), 2022.
- Rafailov et al., [Direct Preference Optimization](#), 2023.

Chapter 40

Reasoning, Test-Time Compute and Distillation

Status: **Drafted.**

Part 10: Post-Training and Reasoning

40.1 Learning outcome

Compare sampling and search budgets at fixed cost, explain verifiable-reward training, and evaluate a distilled student without anthropomorphic claims.

40.2 Prerequisites

CH-35 and CH-37 through CH-39.

40.3 RL with verifiable rewards

Some tasks provide an external checker: unit tests, a symbolic answer, proof verifier, or simulator outcome. Reinforcement learning with verifiable rewards (RLVR) generates candidate trajectories, computes a reproducible reward, and increases probability of successful behavior. It avoids some subjective reward-model errors, but correctness of the checker becomes the specification.

Sparse success rewards give weak credit assignment. Curriculum design, multiple samples, outcome baselines, and process signals can help. A model may exploit bugs, formatting shortcuts, or weak tests, so hold out checkers and adversarial cases. Research moved quickly through 2024–2026; implementation claims here should always name the paper, model, and revision date.

40.4 Reasoning models

“Reasoning model” is a behavioral label for systems trained or configured to spend more computation on multi-step tasks. It does not establish consciousness, symbolic internals, or faithful introspection. Relevant mechanisms include curated derivations, RLVR, search, self-consistency, verifiers, and longer generated scratchpads.

Measure final correctness, robustness to perturbation, compute used, and failure modes. Long output is not automatically good reasoning. Models can produce fluent rationalizations after arriving at an answer through other internal computation.

40.5 Why generation can produce reasoning-like behavior

Autoregressive generation turns one difficult mapping into a sequence of conditional steps. Each emitted token becomes additional context, functioning as writable external state. Training data contains algorithms, worked examples, arguments, and code, so next-token learning can acquire patterns that decompose problems. Attention and nonlinear layers transform internal representations at every step.

This explanation needs no claim that the model “thinks like a person.” The visible chain can support computation, but it can also be redundant, copied, or unfaithful. Causal tests—forcing intermediate values, perturbing steps, or using hidden scratchpads—are stronger than reading prose as a trace of internal causation.

40.6 Test-time compute

Inference can spend extra compute through longer trajectories, multiple samples, tree search, iterative revision, tools, or verifier ranking. Compare systems at fixed FLOPs, latency, energy, or monetary cost. Pass@ k increases the chance that at least one of k samples succeeds; it does not solve selection unless an oracle or verifier identifies the answer.

If independent samples succeed with probability p , oracle pass@ k is $1 - (1 - p)^k$. With $p = 0.3$ and $k = 4$, it is about 0.760, but independence is usually false and a real selector is imperfect. Parallel sampling improves wall time only when hardware capacity exists.

40.7 Distillation

Distillation trains a student from teacher outputs, probabilities, features, or selected trajectories. With logits, a temperature-softened loss can match distributions. Let $\tau > 0$ be the distillation temperature and $p_\tau = \text{softmax}(z/\tau)$:

$$\mathcal{L}_{KD} = \tau^2 D_{KL}(p_\tau^{\text{teacher}} \| p_\tau^{\text{student}}).$$

The τ^2 factor is the conventional compensation for the gradient-scale change caused by softening; implementations may combine this term with hard-label cross-entropy using an explicit mixing coefficient.

Sequence distillation instead trains on teacher-generated text. Reasoning distillation may select verified trajectories and then apply SFT or preference learning. It transfers demonstrated behavior, bounded by student capacity and data coverage; it does not copy an internal algorithm wholesale.

Evaluate the student against ground truth, not only teacher agreement. Track compression, latency, and calibration alongside accuracy. A teacher’s systematic errors can be amplified by large synthetic datasets.

40.8 Verifier limits

A verifier can rank outputs only on properties it observes. Outcome verifiers miss invalid intermediate logic that happens to land on the answer. Learned process verifiers inherit annotation and

distribution errors. Executable tests are incomplete specifications. Repeated selection against one verifier creates pressure to exploit it.

Use independent tests, hidden cases, verifier ensembles where justified, and periodic human audits. Report verifier false-positive and false-negative rates. When comparing search methods, account for verifier compute and selection bias.

40.9 Four recurring perspectives

- **Follow the Token:** generated intermediate tokens become new context and may act as an external work tape.
- **Follow the Gradient:** RLVR reinforces verified sequences; distillation moves student probabilities toward teacher-derived targets.
- **Follow the Byte:** longer contexts and multiple candidates multiply KV-cache and activation traffic.
- **Follow the Request:** a scheduler must budget samples, tools, verifier calls, deadlines, and stopping rules.

40.10 Lab and exit check

Run one task under equal total generation-token budgets using a single long sample, four shorter samples with majority vote, and candidates ranked by a checker. Report correctness, latency, tokens, and selection errors. Distill verified examples into a smaller model and compare it with equal-data SFT on human answers.

40.11 Exercises

1. Under the independence assumption, compute oracle pass@8 when single-sample success is $p = 0.2$. **Check:** $1 - 0.8^8 \approx 0.832$; a deployable selector generally achieves less.
2. A verifier accepts 90 correct and 15 incorrect candidates and rejects 10 correct and 85 incorrect candidates. Compute its false-positive and false-negative rates. **Check:** FPR = $15/100 = 15\%$ and FNR = $10/100 = 10\%$.

40.12 References

- Wang et al., [Self-Consistency Improves Chain of Thought Reasoning](#), 2022.
- Hinton et al., [Distilling the Knowledge in a Neural Network](#), 2015.
- Cobbe et al., [Training Verifiers to Solve Math Word Problems](#), 2021.
- Gao et al., [On Designing Effective RL Reward at Training Time for LLM Reasoning](#), 2024.

Part 11: GPU Fundamentals

Chapter 41

Why GPUs and How They Execute

Status: **Drafted.**

Part 11: GPU Fundamentals

41.1 Learning outcome

Map a tensor operation to threads, warps, blocks, streaming multiprocessors, and arithmetic units without confusing the programming model with fixed hardware.

41.2 CPU vs GPU

A CPU spends substantial silicon on sophisticated cores, large caches, branch prediction, and low-latency execution for a few instruction streams. A GPU devotes more resources to throughput across many threads and hides latency by switching among ready warps. This is a design emphasis, not a rule that CPUs are serial or GPUs are universally faster.

Small, irregular, branch-heavy work may favor CPUs. Large tensor operations expose thousands of similar operations and amortize launch and transfer costs, favoring GPUs. The host CPU still schedules work, prepares inputs, and runs control-heavy code.

41.3 Why GPUs

Transformer training and inference repeatedly apply matrix multiplication, attention, normalization, and elementwise functions across many elements. The same program acts on different data, yielding abundant parallelism. GPUs also provide high device-memory bandwidth and specialized matrix units.

Speed requires enough work. A one-token, tiny-matrix operation may underfill the machine. Shapes, dtypes, memory layout, fusion, and batch size decide whether theoretical throughput is reachable.

41.4 GPU anatomy

At a useful abstraction, an NVIDIA GPU contains streaming multiprocessors (SMs), an on-chip cache hierarchy, memory controllers connected to device memory, and engines for data movement. Each SM contains warp schedulers, register files, load/store units, scalar/vector arithmetic

pipelines often called CUDA cores, special-function units, and Tensor Cores on architectures that provide them.

Exact counts and capabilities change by architecture. Treat product tables and the matching CUDA Programming Guide as versioned sources. As of September 2026, do not carry a tile shape or throughput number from one NVIDIA architecture to another without checking its compute capability.

41.5 SMs

Blocks are assigned to SMs. An SM can host multiple blocks when registers, shared memory, thread slots, and block limits permit. Resident warps form the pool from which schedulers issue instructions. Occupancy is the ratio of active warps to the supported maximum, but maximum occupancy is not automatically maximum performance; extra registers or shared memory may reduce occupancy while enabling less memory traffic.

41.6 Threads

A CUDA thread is a logical execution instance with indices identifying its element or tile. Consider vector addition for $n = 1,000,000$: launch enough threads so thread i computes $c[i] = a[i] + b[i]$, with a bounds check for excess threads. Threads have private logical state, usually allocated in registers when possible.

The programming model promises semantics, not a permanently dedicated core per thread. A thread can execute many instructions and may wait while another warp runs.

41.7 Warps

On NVIDIA CUDA GPUs, threads execute in groups called warps, currently 32 threads in the documented programming model. A warp issues a common instruction across active lanes. If lanes take different control-flow paths, the machine executes paths with different masks, reducing useful parallel work. This divergence matters inside a warp; different warps may follow different paths independently.

41.8 Blocks

A block groups threads that can cooperate through shared memory and block-level synchronization. Blocks should be independent because the scheduler may run them in any order. A grid is the collection of blocks for one kernel launch. For 1,000,000 elements and 256 threads per block, use $\lceil 1,000,000/256 \rceil = 3907$ blocks.

Block size affects warp utilization and residency. Multiples of 32 avoid a mostly empty final warp within each full block, but the best size must be profiled.

For this particular million-element example, the final block contains 64 useful threads for block sizes 128, 256, and 512: the launch uses 7,813, 3,907, and 1,954 blocks respectively. Those 64 threads occupy two complete warps, so there is no partially active warp in the final block; the remaining warps in that block are wholly masked by the bounds check. This distinction prevents “partial block” from being confused with “partial warp.”

41.9 CUDA cores

“CUDA core” is a marketing and architectural name for scalar arithmetic lanes, not an independently programmable CPU core. They execute operations such as FP32 arithmetic according to the SM’s pipelines. Core counts alone do not predict application speed; instruction mix, clocks, memory traffic, occupancy, and issue utilization matter.

41.10 Tensor Cores

Tensor Cores accelerate matrix multiply-accumulate on tiles using supported dtypes and layouts. Framework GEMMs can use them when shapes, alignment, dtype, and libraries allow. Mixed-precision training often multiplies low-precision inputs and accumulates at higher precision. Tensor Cores do not make every operation fast: softmax, indexing, small reductions, and poorly shaped GEMMs may bottleneck elsewhere.

41.11 Registers

Registers are the fastest per-thread storage, allocated from an SM’s finite register file. High register use can reduce resident blocks; if the compiler cannot keep values in registers, spills go to local memory, which resides in device memory and is much slower despite the name. Inspect compiler resource reports and profiler counters rather than guessing.

41.12 Four recurring perspectives

- **Follow the Token:** token representations become rows and tiles distributed across threads and warps.
- **Follow the Gradient:** backward kernels expose similar parallelism but execute different operations and retain or recompute activations.
- **Follow the Byte:** registers and shared memory are scarce on-chip resources; device memory supplies much larger but slower storage.
- **Follow the Request:** launch dimensions and shapes derive from current batch, sequence, heads, and hidden width.

41.13 Lab and exit check

Map a 1,000,000-element addition and a tiled matrix multiplication onto grid, blocks, warps, and lanes. Calculate blocks for 128, 256, and 512 threads; state separately how many threads, full warps, and partially active warps do useful work in the last block. On a CUDA system, inspect kernel register use and compare several block sizes without assuming occupancy predicts the winner.

41.14 References

- NVIDIA, [CUDA C++ Programming Guide](#) (version-sensitive; accessed September 2026).
- NVIDIA, [GPU Performance Background User’s Guide](#) (accessed September 2026).

Chapter 42

Memory Hierarchy and Movement

Status: **Drafted.**

Part 11: GPU Fundamentals

42.1 Learning outcome

Draw a tensor's route through storage levels and estimate hard transfer-time lower bounds.

42.2 Memory hierarchy

The useful hierarchy spans thread registers, on-chip shared memory and caches, device memory such as HBM or GDDR, host RAM, and SSD or network storage. Capacity generally increases downward while latency and energy per access increase. The exact physical relationship varies by architecture, but the optimization goal is stable: reuse data near compute and avoid unnecessary movement.

Allocation location and access path are different questions. A tensor allocated in HBM may be fetched into L2, then L1 or shared memory, then registers before arithmetic. A performant kernel changes the schedule so each expensive load supports many operations.

42.3 Shared memory and SRAM

CUDA shared memory is programmer-managed on-chip storage shared by threads in a block. It is built from SRAM-like on-chip structures and has far lower latency and higher bandwidth than device memory. Tiled matrix multiplication loads submatrices once, synchronizes, reuses them for many multiply-accumulates, then advances.

Shared memory is finite and allocated per block. Large tiles reduce redundant loads but may lower residency. Bank conflicts can serialize accesses when lanes address the same memory bank in incompatible ways. Padding or layout changes can remove conflicts.

42.4 Caches

Hardware caches exploit locality automatically. NVIDIA designs commonly expose per-SM L1 behavior and a chip-wide L2, but sizes and policies vary. Cache hit rate alone is not a goal: a

streaming kernel may perform well with low reuse, while an apparent high hit rate can hide redundant traffic. Interpret hits with requested bytes, sectors, and runtime.

The KV cache in LLM inference is conceptually unrelated to hardware caches: it is an explicit tensor holding past keys and values, usually in device memory. Hardware caches may temporarily hold pieces of that tensor.

42.5 HBM

High Bandwidth Memory places stacked DRAM near the accelerator and uses wide interfaces to provide high aggregate bandwidth. Model weights, activations, KV state, and optimizer tensors usually reside there during computation. “HBM” is often used loosely for device memory, but some GPUs use GDDR; check the product.

HBM is large relative to on-chip SRAM and tiny relative to datasets. Reading a 140 GB BF16 checkpoint once per generated token would already imply enormous traffic; batching is one way to reuse each loaded weight across multiple tokens.

42.6 RAM

Host RAM stages datasets and weights and can hold offloaded model state. Transfers to a discrete GPU cross PCIe or another interconnect and are typically much slower than on-device memory traffic. Pinned host memory enables efficient asynchronous DMA but is a limited system resource. Unified virtual addressing does not make bandwidth uniform.

42.7 SSD

SSD stores checkpoints and dataset shards. Model startup reads files, may decompress or dequantize them, copies into RAM, and then into device memory. Memory mapping can defer pages, but first access still incurs storage traffic and page faults. Loading a 100 GB checkpoint from an SSD sustaining 5 GB/s has a best-case read time of 20 seconds before parsing and device transfer.

42.8 HBM bandwidth

Bandwidth is bytes transferred per second. A lower bound for moving B bytes through a link with sustainable bandwidth W is $t \geq B/W$. Moving 80 GB through a measured 2 TB/s path takes at least 40 ms when both quantities use decimal units. Peak datasheet bandwidth is a ceiling; access patterns, contention, ECC behavior, and controllers reduce achieved bandwidth. Do not silently divide GiB by GB/s: convert both to bytes first.

Read and write traffic both count. For vector addition of three FP32 arrays, each output element requires two 4-byte reads and one 4-byte write: at least 12 bytes for one addition, ignoring cache effects.

42.9 Coalescing

Coalescing combines memory requests from neighboring warp lanes into a small number of aligned transactions. If lane i reads $x[\text{base}+i]$, accesses are contiguous. If it reads $x[\text{base}+i*\text{large_stride}]$, the warp may require many transactions and fetch unused bytes. Transposes and tensor layouts therefore change performance without changing FLOP count.

Alignment, element size, cache-line or sector rules, and architecture determine the precise transaction count. Use profiler metrics to verify. A structure-of-arrays layout often coalesces fields better than an array of large structures.

42.10 Four recurring perspectives

- **Follow the Token:** its vector may begin on SSD, be loaded to RAM/HBM, then move through cache, shared memory, and registers.
- **Follow the Gradient:** training adds activation saves, gradient writes, optimizer reads, and parameter writes.
- **Follow the Byte:** calculate compulsory traffic first, then measure how reuse and transactions change it.
- **Follow the Request:** cold loading, cache residency, and offload determine startup and tail latency.

42.11 Lab and exit check

For a 7-billion-parameter BF16 model, calculate the 14.0 GB (about 13.0 GiB) weight payload and lower bounds for SSD-to-RAM, RAM-to-GPU, and one HBM scan using measured bandwidths. Draw the route and label every value GB or GiB. Then compare contiguous and strided vector reads in a profiler and explain transaction efficiency.

42.12 References

- NVIDIA, [CUDA C++ Best Practices Guide](#) (version-sensitive; accessed September 2026).
- NVIDIA, [CUDA C++ Programming Guide: Memory Hierarchy](#) (accessed September 2026).

Chapter 43

Throughput, Intensity and Rooflines

Status: **Drafted.**

Part 11: GPU Fundamentals

43.1 Learning outcome

Place simplified prefill and decode operations on a roofline under explicit FLOP and byte assumptions.

43.2 Compute throughput

Compute throughput is completed operations per second. Peak values assume particular dtypes, instruction forms, clocks, sparsity conditions, and full utilization. A quoted Tensor Core TOPS value cannot be compared directly with FP32 scalar FLOPs unless conventions match.

For matrix multiplication $C_{m \times n} = A_{m \times k} B_{k \times n}$, count about $2mnk$ FLOPs when multiply and add each count as one. Achieved throughput is that count divided by measured kernel time. Report shapes, dtype, and whether time includes data transfer.

43.3 Arithmetic intensity

Arithmetic intensity is operations divided by bytes transferred across the memory boundary being analyzed:

$$I = \frac{\text{FLOPs}}{\text{bytes}}.$$

The boundary matters. HBM intensity differs from L2 or shared-memory intensity. For an ideal large GEMM, each matrix is loaded or stored once, giving

$$I \approx \frac{2mnk}{s(mk + kn + mn)},$$

where s is bytes per element. Real traffic includes partial tiles, write allocation, and intermediate operations. Fusion increases intensity when it avoids materializing intermediates.

43.4 Compute-bound vs memory-bound

Let peak compute be P FLOP/s and memory bandwidth W bytes/s. The ridge point is $I^* = P/W$. Operations below it cannot feed enough arithmetic and are memory-bound under the model; above it they may be compute-bound.

Suppose $P = 300$ TFLOP/s and $W = 3$ TB/s. The ridge is 100 FLOP/byte. A kernel at 20 FLOP/byte has bandwidth ceiling $20 \times 3 = 60$ TFLOP/s. A kernel at 200 FLOP/byte has roofline ceiling 300 TFLOP/s.

Decode at batch one often has low weight reuse: producing one token reads most weights once for one sequence, so matrix-vector-like work can be memory-bound. Prefill processes many token positions together, turning projections into larger matrix multiplications that reuse weights and increase intensity. “Decode is memory-bound” is conditional: larger batches, quantization, MoE routing, cache operations, and hardware can shift the bottleneck.

43.5 Roofline model

The roofline bound is

$$\text{attainable FLOP/s} \leq \min(P, WI).$$

Plot achieved throughput against intensity on log axes. The sloped line is the bandwidth roof; the flat line is compute peak. Points far below both roofs suffer other limits: launch overhead, insufficient parallelism, dependencies, divergence, synchronization, or a different bandwidth level.

A roofline is a diagnostic model, not a performance promise. Use measured sustainable bandwidth and relevant instruction throughput when possible. End-to-end Transformer layers mix kernels; plotting only GEMM hides normalization, attention, layout conversions, and CPU gaps.

43.5.1 Worked prefill/decode comparison

Take a BF16 4096×4096 weight matrix. One token requires about $2 \times 4096^2 = 33.6$ MFLOPs and reads roughly 33.6 MB of weights, around 1 FLOP/byte before caches and output traffic. A prefill with 512 token rows performs about 17.2 GFLOPs while the same weights can be reused across rows; idealized weight-only intensity is roughly 512 FLOP/byte. Counting one read of the 4 MiB input, one read of the 32 MiB weight, and one 4 MiB output write gives 40 MiB and about 410 FLOP/byte. Real traffic lowers that number further, yet the contrast explains why prefill more readily reaches compute-oriented kernels.

At batch 16 decode, one weight read serves 16 rows, increasing idealized intensity about sixteen-fold. Continuous batching therefore changes both serving efficiency and the hardware regime.

43.6 Four recurring perspectives

- **Follow the Token:** decode handles one new position per sequence while prefill exposes many positions in parallel.
- **Follow the Gradient:** backward operations have different FLOP and byte balances and additional tensor reads/writes.

- **Follow the Byte:** intensity states how much computation each transferred byte enables at a named boundary.
- **Follow the Request:** batching and sequence phase move a request's kernels across the roofline.

43.7 Lab and exit check

Use measured bandwidth and compute peaks for one device. Calculate ideal intensity for a projection at prefill lengths 1, 32, and 512 and decode batches 1, 8, and 32. Plot roofline ceilings, then benchmark actual GEMMs. Explain points below the predicted roof.

43.8 References

- Williams, Waterman, and Patterson, [Roofline: An Insightful Visual Performance Model](#), 2009.
- NVIDIA, [Nsight Compute Roofline Analysis](#) (version-sensitive; accessed September 2026).

Chapter 44

Profiling Transformer Bottlenecks

Status: **Drafted.**

Part 11: GPU Fundamentals

44.1 Learning outcome

Measure a CPU baseline and define an honest, synchronized GPU profiling protocol that separates latency, FLOPs, and memory traffic.

44.2 Decode memory bottlenecks

At low batch, autoregressive decode repeatedly reads large weights to produce few token rows. Approximate minimum time from weights alone as weight bytes divided by sustainable bandwidth. A 14 GB quantized checkpoint at 1 TB/s requires at least 14 ms per full scan; actual execution adds KV reads, activations, sampling, launches, and imperfect bandwidth. This bound is useful because no kernel optimization can beat required traffic without reuse, compression, or changing computation.

Do not diagnose from low GPU utilization alone. Utilization metrics can mean “some kernel active,” not arithmetic saturation. Batch sweep, memory throughput, Tensor Core utilization, and kernel timeline distinguish memory limits from small launches or CPU starvation.

44.3 FLOPs and memory profiling

Start with analytic counts for major operations, then compare profiler counters. For each kernel record shape, dtype, theoretical FLOPs, requested bytes, measured device-memory bytes, duration, and launch count. Profiler FLOP metrics are architecture- and instruction-specific; tensor instructions may be counted differently from scalar ones. State the convention.

End-to-end bytes are not the sum of tensor sizes if caches and fusion reuse data. Conversely, temporary layouts can add traffic invisible in a high-level graph. Nsight Systems shows CPU/GPU timelines; Nsight Compute provides per-kernel hardware counters. PyTorch Profiler relates framework operations to kernels, with overhead that must be acknowledged.

44.4 Bandwidth utilization

Effective bandwidth is bytes attributed to the operation divided by time. Hardware bandwidth utilization uses measured memory-controller traffic relative to a sustainable or peak rate. They answer different questions. A kernel can achieve high controller bandwidth while wasting transactions on uncoalesced loads.

For FP32 vector addition over n elements, a logical-traffic estimate is $12n$ bytes: two 4-byte reads and one 4-byte write. If $n = 2^{28}$ and elapsed device time is 4 ms, that estimate is 3 GiB / 4 ms = 750 GiB/s. Call it effective logical bandwidth, not controller bandwidth; cache write policies and memory transactions can make the hardware counter differ.

Compare achieved bandwidth with a copy benchmark on the same device, dtype, clocks, and concurrency. Inspect read/write balance, cache hit rates, sectors per request, and stalls. A low percentage may be expected for a tiny kernel that cannot expose enough concurrent requests.

44.5 Latency measurement

Define boundaries: kernel latency, model-step latency, time to first token, or inter-token latency. Warm up compilation, allocator pools, caches, and clocks. Use representative shapes, fixed synchronization points, enough repetitions, and distributions such as median and p95 rather than one best sample.

GPU work launches asynchronously. A host timer around a CUDA call often measures enqueue time. Use CUDA events for device intervals or synchronize before stopping the host timer. Avoid synchronizing inside the normal hot path simply to measure it; instrumentation can destroy overlap.

For CPU baselines, control thread count, affinity if needed, warmups, input allocation, and power state. Report hardware and library versions. A fair CPU/GPU comparison includes or excludes transfer and startup consistently.

44.6 Synchronization

Synchronization establishes ordering or host visibility but can create idle gaps. Device-wide synchronization waits for all prior work; stream events express narrower dependencies. Within a block, barriers coordinate shared-memory phases. Incorrect omission causes races; excessive barriers reduce overlap.

Operations such as copying a scalar to the host, printing a CUDA tensor, or calling certain timing APIs may synchronize implicitly. Timeline profiling reveals these gaps. In distributed settings, collectives introduce further synchronization, covered later.

44.6.1 A defensible protocol

1. Pin model, inputs, batch/sequence lengths, dtype, clocks policy, framework, driver, and device.
2. Separate cold start, first compiled run, warmed steady state, prefill, and decode.
3. Warm up until timings stabilize, then collect enough repeated requests.
4. Measure end-to-end latency without profiler; collect representative profiler traces separately because profiling adds overhead.
5. Sweep batch and sequence lengths. Calculate analytic lower bounds and explain the gap.
6. Save raw traces, summary scripts, and environment metadata.

44.7 Four recurring perspectives

- **Follow the Token:** mark when each request token enters prefill, decode, sampling, and streaming.
- **Follow the Gradient:** training profiles must include backward, optimizer, communication, and any recomputation.
- **Follow the Byte:** compare compulsory bytes, measured controller traffic, and effective bandwidth.
- **Follow the Request:** end-to-end latency includes CPU scheduling and gaps that kernel-only reports omit.

44.8 Lab and exit check

Benchmark a matrix multiplication and a tiny autoregressive loop on CPU. Define the equivalent GPU protocol with warmup, CUDA events, end-to-end synchronization, and batch sweep. If CUDA is available, collect one systems timeline and one kernel report. Identify the largest interval and state what evidence would falsify your bottleneck diagnosis.

44.9 References

- NVIDIA, [Nsight Systems User Guide](#) (version-sensitive; accessed September 2026).
- NVIDIA, [Nsight Compute Profiling Guide](#) (version-sensitive; accessed September 2026).
- PyTorch, [Profiler documentation](#) (version-sensitive; accessed September 2026).

Part 12: Kernels and the Software Stack

Chapter 45

Kernels, GEMM and Tiling

Status: **Drafted**

45.1 Prerequisites and learning outcomes

You should know warps, GPU memory, arithmetic intensity, and profiling. By the end, you can explain tiled matrix multiplication, estimate traffic, and identify which intermediate writes fusion removes.

45.2 GPU kernels

A **kernel** is a function launched over many GPU threads. A launch defines a grid of blocks; a block is scheduled on one SM, and its threads execute in warps. Threads in a block may cooperate through shared memory and barriers. Blocks must be independent because their order is not guaranteed.

For vector addition, thread i loads $x[i]$ and $y[i]$, adds them, and stores $z[i]$. With FP32 that is about 12 bytes moved for one addition: extremely memory-bound. More threads do not change this ratio; they expose enough work to approach bandwidth. A correct kernel guards $i < n$ because grids are rounded up.

45.3 GEMM

GEMM computes $C_{M \times N} = A_{M \times K} B_{K \times N}$. Counting multiply and add as two FLOPs, it performs about $2MNK$ FLOPs. For $M = N = K = 4096$, that is 137.4 GFLOPs. Reading A and B and writing C once in BF16 moves about 100.7 MB, while a naive per-output loop can move orders of magnitude more.

The same traffic is 96 MiB; the distinction matters when comparing with a bandwidth reported in GiB/s. If a synchronized kernel takes 1.0 ms, the arithmetic rate is 137.4 TFLOP/s and the compulsory-traffic rate is 96 GiB/s. Neither number alone proves the bottleneck because actual traffic can exceed the compulsory minimum and the kernel may read or write C depending on its epilogue.

45.4 Tiling creates reuse

A tiled kernel assigns a block an output tile, then advances through K in chunks. It loads A and B tiles into shared memory and reuses each value for many multiply-accumulates; registers hold smaller accumulator tiles. For a 32-by-32 output tile and K chunk 32, two FP16 tiles cost 4096 bytes and support 65,536 FLOPs: 16 FLOPs/byte before output traffic. Larger tiles increase reuse but consume more shared memory and registers, potentially reducing occupancy.

45.5 Tensor Core MMA

Tensor Cores execute matrix multiply-accumulate instructions on small fragments: conceptually $D = AB + C$. Exact shapes and types depend on GPU architecture. Libraries decompose large GEMM into thread-block, warp, and instruction tiles and often accumulate in FP32 even when inputs are FP16/BF16. High peak throughput requires suitable dimensions, layout, alignment, and a pipeline that keeps operands available.

45.6 Kernel launch overhead

The CPU prepares a launch, the runtime submits it, and the GPU schedules it. Fixed cost is negligible for a large GEMM but can dominate dozens of tiny kernels. CPU timing may measure asynchronous submission rather than completed work. Warm up, synchronize outside the interval, and use GPU events or a profiler.

45.7 Kernel fusion

Suppose a linear layer writes a $[B,T,D]$ BF16 tensor, then bias, GELU, and dropout each read and write it. At $B=8, T=2048, D=4096$ the tensor is 128 MiB. Three separate elementwise passes add roughly 768 MiB of reads and writes. A fused epilogue keeps partial results in registers and writes once. Fusion can backfire if register pressure spills or extra recomputation exceeds saved traffic.

45.8 Four perspectives and exit check

- **Follow the Token:** one hidden row participates in GEMMs for Q , K , V and MLP projections.
- **Follow the Gradient:** training launches related activation- and weight-gradient GEMMs; inference has no backward pass.
- **Follow the Byte:** tiling turns repeated HBM loads into on-chip reuse; fusion removes round trips.
- **Follow the Request:** prompt and batch dimensions choose GEMM shapes and kernel efficiency.

Failures include bad edge masks, bank conflicts, uncoalesced access, spilling, and missing synchronization in benchmarks. For a non-square case such as $M=511, N=769, K=1024$, calculate $2MNK$, identify edge tiles, and name materializations a fused activation avoids. Explain why a 128-by-128 tile can lose to 64-by-64 even though its nominal reuse is greater.

45.9 Primary references

- NVIDIA, [CUDA C++ Programming Guide](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [Parallel Thread Execution ISA](#), version-sensitive; accessed 2026-09-19.

- NVIDIA, [CUTLASS documentation](#), version-sensitive; accessed 2026-09-19. Pin GPU architecture, toolkit, driver, library commit, dtype, and shapes.

Chapter 46

CUDA, Triton and CUTLASS

Status: **Drafted**

46.1 Prerequisites and learning outcomes

After Chapter 45, you can choose an abstraction level, write a masked fused kernel, validate it, and profile it honestly.

46.2 CUDA: explicit control

CUDA C++ exposes kernels, blocks, shared memory, streams, events, and architecture intrinsics. Exact control creates obligations: index arithmetic, bounds, synchronization, layout, occupancy, compilation targets, and error checking. A fused bias-plus-ReLU kernel maps one thread to one element, loads x and bias, computes $\max(x+b, 0)$, and stores once. Every thread checks the boundary. Test lengths such as 1003; powers of two can hide indexing defects.

46.3 Triton: blocked programs

Triton expresses a kernel as many instances of a **program** operating on vector blocks. `program_id` selects a block; `arange`, masked loads, and stores describe lanes. Its compiler maps operations to GPU instructions. This makes fused elementwise work and many tiled kernels shorter than CUDA while retaining explicit layouts and block sizes.

46.4 CUTLASS-level concepts

CUTLASS composes GEMM from nested device, thread-block, warp, and MMA instruction tiles. An operation specifies layouts, input and accumulator types, pipeline stages, alignment, and an **epilogue** that can fuse scaling, bias, or activation. Vendor libraries remain the default for standard GEMM. CUTLASS helps with specialized layouts, fused epilogues, and new numeric formats. A configuration copied across GPU generations may underperform.

46.5 Simple Triton kernel

```
@triton.jit
def add_relu(x, b, y, n, BLOCK: tl.constexpr):
```

```

offsets = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
mask = offsets < n
values = tl.load(x + offsets, mask=mask)
bias = tl.load(b + offsets, mask=mask)
tl.store(y + offsets, tl.maximum(values + bias, 0.0), mask=mask)

```

Here `n` is a runtime scalar while `BLOCK` is a compile-time specialization parameter. Test NaNs, negatives, multiple dtypes, small inputs, and awkward lengths. Decide and test the NaN contract explicitly because maximum behavior must match the chosen framework reference. Compare with `torch.relu(x+b)` using dtype-appropriate tolerance. Correctness comes before speed.

46.6 GPU profiling

First verify results. Warm compilation, allocator caches, and clocks. Benchmark with GPU events. Use Nsight Systems for CPU/GPU timing and Nsight Compute for kernel counters. Compare matched shapes, dtypes, and outputs. Inspect bandwidth, Tensor Core use, occupancy, transactions, register spills, and launch gaps. High occupancy can coexist with poor access; lower occupancy may win when each warp does more useful work.

46.7 Four perspectives, failures, and exit check

- **Follow the Token:** rows become lanes and tiles while semantics must match the reference.
- **Follow the Gradient:** a training operator needs backward code and gradient checks.
- **Follow the Byte:** count loads/stores and prove fusion removed an intermediate.
- **Follow the Request:** autotuning and compilation are cold costs; steady traffic sees cached kernels.

Failures include missing masks, wrong strides, overflow, races, synchronization, benchmarking compilation, and comparing different precision. Implement fused add-plus-activation, test irregular shapes, and report cold time, warmed median, p95, and environment.

Include a byte check: two input loads plus one output store imply 6 bytes per BF16 element before transaction overhead. For 16 million elements that is about 91.6 MiB; a 0.20 ms kernel corresponds to about 447 GiB/s of effective logical bandwidth. Compare that value with a same-shape copy benchmark rather than a datasheet peak.

The benchmark must include a meaningful baseline. Comparing a custom kernel only with two unfused Python operations proves little if the framework already has a fused implementation. Compare against the best available framework path, retain identical output precision, and publish cases where the custom kernel loses. Autotuning also belongs in the cost model: record how many candidates were compiled and measured, whether the cache survived process restart, and how an unseen shape behaves.

46.8 Primary references

- NVIDIA, [CUDA C++ Programming Guide](#), version-sensitive; accessed 2026-09-19.
- OpenAI, [Triton tutorials](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [CUTLASS documentation](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [Nsight Compute documentation](#), version-sensitive; accessed 2026-09-19. Pin Python, PyTorch, Triton, toolkit, driver, GPU, and commits.

Chapter 47

Graphs, Compilers and Runtimes

Status: **Drafted**

47.1 From Python to device code

The goal is to trace a function through capture, intermediate representations (IR), lowering, generated kernels, and execution.

47.2 PyTorch eager execution

Eager Python runs operator by operator. Each call performs framework dispatch, selects an implementation, and may launch GPU work. It is interactive and debuggable, but sees only local operations, which limits fusion and repeats host overhead.

47.3 Computational graph capture

Capture turns a region of dynamic execution into a graph of tensor operations plus assumptions called guards. Explicit dependencies permit fusion, constant propagation, layout selection, and memory planning. This differs from the autograd graph: one records executable tensor programs, the other derivative dependencies.

47.4 TorchDynamo

TorchDynamo observes Python frames and extracts compatible operations into FX graphs while preserving semantics through guards. A guard can assert dtype, rank, device, a global value, or a shape relationship. If later inputs violate guards, the system can recompile or fall back. Data-dependent control, unsupported extensions, and side effects can end a captured region.

47.5 Intermediate representation

FX is a relatively high-level IR; later IRs express loops, indexing, memory, and device choices. Multiple levels let one pass reason about algebra while another schedules hardware work. “The compiler optimized it” is incomplete: ask which IR, transformation, and artifact.

47.6 torch.compile

`torch.compile(model)` is the user entry point. Behavior depends on PyTorch version, backend, mode, dynamic-shape setting, model, and inputs. The first call can be much slower because capture and compilation occur. Benchmark cold startup separately and exercise all expected shape families.

47.7 Inductor

TorchInductor is the default PyTorch compiler backend for supported workloads. It lowers graphs, schedules loops, fuses work, and emits target code—often Triton for GPU regions and vectorized C++ for CPU. Calls may remain external where mature libraries win.

47.8 Generated kernels

For `layer_norm(x+bias)`, eager execution may materialize `x+b` before normalization. A compiler may fuse addition and plan buffers. Inspect code and traces: graph count, kernel count, bytes, and latency reveal whether fusion occurred. Compare numerics on representative extremes because reassociation changes rounding.

47.9 Compiler graph breaks

A graph break returns control to Python, then capture may resume. Frequent breaks reduce optimization. A **recompile storm** from varying shapes or Python values is worse. Use diagnostics and decide whether shapes should be static, symbolic, padded, or bucketed.

For example, a service that specializes independently for sequence lengths 1 through 2048 can pay 2,048 compilation costs and retain as many artifacts. Bucketing to powers of two needs 12 length families but adds padding; symbolic shapes may use fewer artifacts but can restrict scheduling choices. Record the number of unique graphs and cumulative compilation time under a production-shaped trace, not just one warmed input.

47.10 Four perspectives and exit check

- **Follow the Token:** identical tensor semantics pass through several program representations.
- **Follow the Gradient:** AOTAutograd can stage forward and backward graphs; saved tensors affect memory.
- **Follow the Byte:** fusion matters when it removes allocations and HBM traffic.
- **Follow the Request:** guards decide reuse, recompilation, or fallback.

Compile a two-layer MLP with two shape families, inspect its graph, count breaks, recompilations, guards, and kernels, and report cold versus warm time. Add a third shape that violates a guard and predict whether it recompiles or falls back before observing the trace. Failures include hiding breaks, timing compilation as inference, excessive specialization, mutation, unsupported operators, and assuming compiled is always faster.

47.11 Primary references

- PyTorch, [TorchDynamo overview](#), version-sensitive; accessed 2026-09-19.
- PyTorch, [torch.compile programming model](#), version-sensitive; accessed 2026-09-19.

- PyTorch, [FX documentation](#), version-sensitive; accessed 2026-09-19.
- PyTorch, [TorchInductor documentation](#), version-sensitive; accessed 2026-09-19. Pin PyTorch release or commit, backend, flags, CUDA/Triton, model revision, and inputs.

Chapter 48

CUDA Graphs and Execution Overhead

Status: **Drafted**

48.1 Why a fast kernel can make a slow program

When kernels take microseconds, Python, dispatch, driver submission, and synchronization gaps can dominate wall time. This chapter separates computation from orchestration.

48.2 CUDA Graphs

A CUDA Graph records GPU operations and dependencies, then instantiates an executable graph. Later iterations launch it as a unit. Capture requires graph-safe behavior: addresses and structure generally remain stable, allocations and CPU synchronization inside capture are restricted, and inputs commonly use long-lived static buffers.

48.3 Dispatch overhead

Suppose decode launches 80 kernels averaging 4 microseconds, with 3 microseconds of host gap each. Device work is 320 microseconds and gaps add 240: orchestration consumes 43% of the 560-microsecond path. Faster kernels cannot remove gaps. Fusion reduces launch count; compilation reduces framework overhead; CUDA Graphs reduce repeated submission.

48.4 Graph replay

Allocate static inputs/outputs; warm libraries and memory pools on a side stream; synchronize; capture one iteration; instantiate; copy new values into static buffers; replay; consume outputs after stream synchronization. Replay changes values, not topology. Hazards include reading before completion, mutating storage during replay, random-number semantics, and stale pointers. Compare replay with eager across multiple inputs.

48.5 Shape specialization

Requests vary in batch, prompt length, and live sequences. Systems use shape buckets, padding, multiple graphs, or leave dynamic regions uncaptured. Padding wastes compute; many variants increase warmup and memory. Decode is easier to capture than arbitrary prefill because token width is one, but changing batch membership still needs stable buffers and masks.

48.6 Compilation amortization

Let setup be C , eager latency E , and replay $R < E$. Break-even reuse is $N > C / (E - R)$. If setup costs 2 seconds and saves 0.5 ms, equality occurs at 4,000 iterations; a strict net win begins after 4,000, assuming every request hits that graph. At a 75% graph-cache hit rate, the expected saving per request is not 0.5 ms unless the miss path is measured too. Report cold start, warmup, cache hit rate, miss cost, and steady latency. A frequently reloaded service may never amortize an optimization that wins a long benchmark.

48.7 Measurement, perspectives, and exit check

Measure identical inputs across warmed eager, warmed `torch.compile`, and replay. Separate setup. Use GPU events, repeated trials, synchronization outside intervals, and p50/p95/p99. A profiler should show gaps shrinking.

- **Follow the Token:** values change while addresses and topology remain stable.
- **Follow the Gradient:** training capture must preserve backward and optimizer semantics.
- **Follow the Byte:** static pools trade persistent memory for allocation-free replay.
- **Follow the Request:** scheduling maps variable requests into captured shapes.

Design buckets for batch 1–32 and calculate break-even. Failures include capture before warmup, hidden synchronization, stale buffers, stream mistakes, excessive padding, graph-cache explosion, and reporting replay without setup.

48.8 A concrete bucket policy

One starting policy captures batch capacities 1, 2, 4, 8, 16, and 32. A batch of 11 uses the 16-slot graph and masks five rows. This bounds the number of graph variants but can waste nearly half the work just above a power of two. A traffic trace should reveal whether adding intermediate buckets pays for its compilation and memory. Measure graph-cache misses during bursts, because a policy that looks efficient after a long warmup can produce severe cold tail latency.

Graph replay also does not replace dependency management. Input copies, collective communication, and output consumers must use compatible streams and events. Capturing communication may impose additional version and topology constraints. When a request is cancelled, its masked slot and static buffers still need a defined lifecycle; otherwise stale data can leak into a later replay.

48.9 Primary references

- NVIDIA, [CUDA C++ Programming Guide: CUDA Graphs](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [CUDA Runtime API: Graph Management](#), version-sensitive; accessed 2026-09-19.
- PyTorch, [CUDA semantics: CUDA Graphs](#), version-sensitive; accessed 2026-09-19. Pin driver, toolkit, PyTorch, allocator, graph mode, and shapes.

Part 13: LLM Inference From First Principles

Chapter 49

Load Weights and Trace a Forward Pass

Status: **Drafted**

49.1 Goal and notation

We trace a decoder-only model with batch B , sequence T , width D , query heads H , KV heads H_{kv} , head width $D_h=D/H$, layers L , vocabulary V , and MLP width F .

49.2 Loading model weights

A checkpoint is more than a bag of numbers. Configuration defines architecture and shapes; tokenizer files define the token-ID contract; parameter names map stored tensors to modules; dtype and sharding determine bytes and placement. Load metadata before allocating, validate model revision and tensor shapes, and use a format that avoids executing untrusted code.

A rough weight budget is parameter count times stored bytes. Seven billion BF16 parameters require about 14 decimal GB before allocator, runtime, KV, activations, and temporary workspaces. Quantized storage can reduce capacity requirements, but kernels may unpack or dequantize values during execution.

49.3 Prompt processing

Text becomes IDs $[B, T]$. Embedding lookup produces $X [B, T, D]$. Each layer normalizes X , forms $Q [B, H, T, D_h]$, K and $V [B, H_{kv}, T, D_h]$, applies position encoding, performs causal attention, projects back to $[B, T, D]$, and applies an MLP through $[B, T, F]$. Residual additions preserve the $[B, T, D]$ stream. Final normalization and the LM head produce logits $[B, T, V]$. Generation normally needs only logits at the final valid position.

Padding needs an attention mask and correct position IDs. Left- versus right-padding changes indexing. A trace should record shape, dtype, device, stride, and live byte size at module boundaries, not print tensor contents.

49.4 GPU inference

Inference disables gradient recording and uses evaluation behavior. Parameters reside on the chosen device; inputs must follow. Automatic mixed precision, fused attention, compiler modes, and tensor parallelism alter execution without changing the model contract. Host-to-device copies, lazy library initialization, compilation, and first allocation belong to cold startup, not steady forward latency.

GPU execution is asynchronous. Moving a scalar to Python, printing a CUDA tensor, or calling certain convenience APIs can force synchronization. Profile the complete request before “optimizing” a kernel.

49.5 End-to-end real-model forward-pass tracing

Use hooks or an exported graph on a pinned, small open model. Record:

Boundary	Shape
IDs	[B,T]
embedding/residual	[B,T,D]
Q	[B,H,T,Dh]
K,V	[B,Hkv,T,Dh]
attention result	[B,H,T,Dh]
MLP expansion	[B,T,F]
logits	[B,T,V]

For $B=2, T=128, D=4096$ in BF16, one residual tensor is 2 MiB. Logits with $V=128,000$ occupy 62.5 MiB if all positions are materialized, showing why last-token-only paths matter during decode. Temporary memory can exceed boundary tensors because attention, GEMM, and compiler workspaces live inside operators.

Check the units directly: the residual contains $2 \times 128 \times 4096 = 1,048,576$ elements, or 2,097,152 bytes. The logits contain 32,768,000 elements, or 65,536,000 bytes. Reporting both element count and bytes catches accidental FP32 promotion and confusion between MB and MiB.

Verify the trace by checking parameter count against the config, confirming every layer preserves residual shape, and comparing final logits with a trusted eager run. Hooks can inhibit compilation or alter lifetimes, so profile uninstrumented execution separately.

49.6 Four perspectives and debugging

- **Follow the Token:** an ID selects an embedding, traverses L residual updates, and becomes V logits.
- **Follow the Gradient:** inference creates no autograd state; accidental grad tracking wastes memory.
- **Follow the Byte:** weights dominate residency, while activations and logits scale with B and T.
- **Follow the Request:** loading is a lifecycle cost; tokenization and transfer precede prefill.

Failures include tokenizer/model mismatch, wrong transposition, silent dtype conversion, missing eval mode, invalid positions, and comparing revisions. The exit check is a table of all boundary shapes and strides, calculated live bytes for at least residual and logits, and a numerical equality check against untraced eager output.

49.7 Primary references

- PyTorch, [inference_mode](#), version-sensitive; accessed 2026-09-19.
- Hugging Face, [Transformers model documentation](#), version-sensitive; accessed 2026-09-19.
- Hugging Face, [Transformers model loading](#), version-sensitive; accessed 2026-09-19. Pin framework versions, model commit, tokenizer commit, dtype, attention backend, and hardware.

Chapter 50

Prefill, Decode and KV Cache

Status: **Drafted**

50.1 Two phases of one computation

A causal model repeatedly predicts one next token, but serving divides work into a parallel prompt phase and a serial generation phase.

50.2 Prefill

Prefill processes all T prompt tokens together. It computes Q, K, and V for each layer, performs causal attention over the prompt, and stores K/V for later. Large GEMMs across T rows often have enough arithmetic intensity to use the GPU well. Time to first token includes queueing, tokenization/transfer, prefill, final sampling, and streaming overhead—not merely a forward pass.

50.3 Autoregressive decode

After sampling token T+1, decode feeds that one ID back. Each layer forms one new query, key, and value. The query attends to all cached keys and combines cached values. One token is emitted, then the process repeats until a stop condition. Decode has a sequential dependency across tokens; batching many requests is the primary way to expose parallel work.

50.4 KV cache

For each layer, store K and V with conceptual shape [B,Hkv,S,Dh], where S grows with sequence length. Ignoring padding and metadata, bytes are

$$2LBH_{kv}SD_hb,$$

where the leading 2 is K plus V and b is bytes per element. For L=32, Hkv=8, Dh=128, BF16, one token per sequence consumes 128 KiB. At 8,192 tokens that is 1 GiB per sequence. MHA with Hkv=32 would require four times as much.

The arithmetic is $2 \times 32 \times 8 \times 128 \times 2 = 131,072$ bytes per token. Multiplying by 8,192 gives 1,073,741,824 bytes exactly. This is logical payload only: paged allocators round to blocks and add tables, alignment, and partially filled-block waste.

Cache layout matters: layer-first, token blocks, head grouping, alignment, and page tables influence coalescing and fragmentation. Store post-position-encoding keys when the architecture requires it, and ensure cache positions match position IDs.

50.5 Why K and V, but not historical Q?

At a new step, only the **new query** asks what prior information matters. Historical queries were used to compute historical outputs, which do not change in a causal model. Historical keys must remain because the new query scores against them; historical values must remain because those scores form a weighted sum. Caching old queries provides no input to the next attention result.

Without a cache, recomputing the length-S prefix each step repeats projections and attention for unchanged tokens. With a cache, projections run only for the new token, although attention still reads K/V across S. That is why long-context decode becomes memory-intensive.

50.6 Context length

The maximum context includes prompt plus generated tokens unless the runtime supports an explicit sliding window or eviction policy. Longer S grows cache linearly and per-token attention work roughly linearly. RoPE scaling or a config advertising a longer window does not guarantee preserved model quality. Capacity, supported positions, and evaluated quality are different claims.

50.7 Correctness lab

Implement two paths on a tiny deterministic model: (1) recompute the full prefix after every sampled token; (2) prefill once and append K/V. At each step compare logits before sampling with strict FP32 tolerance or a justified reduced-precision tolerance. Use greedy decoding so randomness cannot hide a mismatch. Test T=1, multiple batches, grouped-query attention, a left-padded batch, and an irregular stop length. Assert that cache length advances by exactly one for each live sequence and does not advance for a finished row.

50.8 Four perspectives and failures

- **Follow the Token:** prompt tokens enter together; generated tokens enter one at a time.
- **Follow the Gradient:** inference caches are detached state, not training activations.
- **Follow the Byte:** each new token appends $2LHkv \cdot Dh$ elements and rereads historical state.
- **Follow the Request:** the scheduler alternates prefill work and decode iterations for active sequences.

Typical defects are off-by-one masks, wrong cache position, RoPE applied twice, cache aliasing across requests, padding contamination, and logits compared after sampling. The exit check is equality of cached and uncached logits plus a derived bytes-per-token budget.

50.9 Primary references

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Ainslie et al., [GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints](#), 2023.
- Hugging Face, [Caching](#), version-sensitive; accessed 2026-09-19. Pin model code and revision because cache structure and position handling are architecture-specific.

Chapter 51

Sampling and Decoding Policies

Status: **Drafted**

51.1 From logits to a sequence

The model produces logits z in $\mathbb{R}^V$. A decoding policy transforms or searches these scores. Keep two ideas separate: temperature/top-k/top-p change the next-token distribution; beam search maintains multiple sequence hypotheses.

51.2 Sampling

Stable probabilities are $p_i = \exp(z_i - m) / \sum_j \exp(z_j - m)$, where $m = \max(z)$. A categorical draw uses an explicit random generator and seed for reproducibility. Sampling is conditional at every step, so one early choice changes all later logits. Report prompts, seed, tokenizer, policy, and model revision.

51.3 Greedy decoding

Greedy chooses $\operatorname{argmax} z$. It is deterministic given deterministic logits and tie handling, cheap, and useful for correctness tests. It does not maximize whole-sequence probability: the locally best token can lead to a worse continuation. Repetition and bland outputs are common failure modes.

51.4 Temperature

Temperature $\tau > 0$ computes $\operatorname{softmax}(z/\tau)$. τ below one sharpens differences; above one flattens them. As τ approaches zero, behavior approaches argmax , so implementations should switch to greedy rather than divide by zero. Temperature must precede truncation when defining a policy precisely.

Example logits $[2, 1, 0]$ have probabilities approximately $[0.665, 0.245, 0.090]$ at $\tau=1$ and $[0.867, 0.117, 0.016]$ at $\tau=0.5$.

51.5 Top-k

Keep the k largest logits, set the rest to negative infinity, renormalize, then sample. It fixes candidate count regardless of uncertainty. $k=1$ equals greedy; k larger than V changes nothing. Ties and a minimum number of retained tokens need explicit behavior.

51.6 Top-p

Sort probabilities descending and keep the smallest prefix whose cumulative mass reaches p , usually retaining at least one token. The candidate count adapts: peaked distributions keep few; flat ones keep many. Implementations differ over whether the threshold-crossing token is included; document it. Filtering on logits without first computing normalized probability mass is wrong.

51.7 Beam search

Beam search maintains K partial sequences ranked by cumulative log probability, expands candidates, and retains the best. Raw sums favor short sequences because log probabilities are negative, so length penalties or normalized scores are common. Beam search suits tasks with constrained targets, but large beams do not guarantee globally optimal or human-preferred text and multiply cache/state costs.

51.8 Stopping conditions

Stop on EOS, maximum new tokens, total length, matched token sequences, cancellation, or time budget. A displayed string may not align with token boundaries, so string-stop handling must retain enough suffix and define whether matched text is returned. Batched generation needs per-sequence finished masks; completed rows must not corrupt active rows.

51.9 Worked comparison and four perspectives

For logits $[2,1,0]$, greedy selects the first token. At temperature 0.5 the probabilities are approximately $[0.867, 0.117, 0.016]$. At temperature 1, top-2 renormalizes the first two probabilities to approximately $[0.731, 0.269, 0]$. With top- $p=0.8$, the first token contributes only 0.665, so the threshold-crossing second token is included and the retained set is the first two tokens. State whether temperature is applied before top- p ; otherwise this example has more than one answer.

Draw 10,000 seeded samples from each resulting distribution and compare counts with binomial standard errors, for example $\sqrt{Np(1-p)}$ for one category. An exact match to expected counts is not required; reproducibility means the same implementation, seed, generator, device, and numeric path reproduce the same sequence.

- **Follow the Token:** the selected ID becomes the next model input.
- **Follow the Gradient:** decoding is normally outside learning; policy-gradient training is a separate process.
- **Follow the Byte:** logits $[B,V]$, sort/top- k workspaces, beam KV copies, and RNG state consume memory.
- **Follow the Request:** policies affect output length, latency, cache occupancy, and reproducibility.

Failures include unstable softmax, applying filters in an undocumented order, using different RNGs, decoding special tokens incorrectly, length-normalization errors, and stop leakage. The

exit check distinguishes distribution transformation from sequence search and reproduces seeded results.

51.10 Primary references

- Holtzman et al., [The Curious Case of Neural Text Degeneration](#), 2020.
- Hugging Face, [Generation API](#), version-sensitive; accessed 2026-09-19.
- Hugging Face, [Generation strategies](#), version-sensitive; accessed 2026-09-19. Pin Transformers, model and tokenizer revisions, generation config, seed, device, and numeric mode.

Chapter 52

Inference Memory and Latency Accounting

Status: **Drafted**

52.1 Build a budget before measuring

Capacity and speed claims need explicit workload: model revision, precision, B, prompt lengths, generated lengths, concurrency, hardware, and runtime.

52.2 Inference memory accounting

Peak device memory is not merely checkpoint size. Track persistent weights, KV cache, live activations, logits, temporary workspaces, compiler/graph pools, communication buffers, allocator fragmentation, and runtime context. Measure both logical tensor bytes and process/device peaks.

52.3 Weights plus KV plus activations plus overhead

For P parameters stored at b_w bytes, weights are Pb_w . KV is $2LBH_{kv}SD_hb_{kv}$. Boundary activation [B,T,D] costs $BTDb_a$, but peak activation depends on liveness and fused kernels. Add an empirically measured overhead term; do not invent a universal percentage.

Example: 7B BF16 weights are 14.0 GB (about 13.0 GiB). With $L=32, H_{kv}=8, D_h=128, BF16$, cache is 128 KiB per token per sequence. Four sequences at $S=8192$ use 4 GiB KV. A device marketed as 24 GB has about 22.35 GiB, leaving roughly 5.3 GiB after these logical weights and KV; it may still fail because workspaces, runtime state, and fragmentation need that remainder.

52.4 Time to first token

TTFT runs from accepted request to first emitted token. Decompose queue delay, tokenization/transfer, cache lookup, prefill, sampling, and stream/network flush. “Model TTFT” may exclude queue and network; label scope. Prompt length strongly affects prefill.

52.5 Inter-token latency

Inter-token latency is elapsed time between emitted tokens for one request. It includes scheduling gaps and may vary as batches change. Report a distribution rather than one average.

52.6 TPOT

Time per output token often means generation-phase elapsed time divided by generated tokens, commonly excluding the first token. State the formula. The reciprocal approximates per-request tokens/s only under a stable sequence; averages and reciprocals do not commute.

52.7 Throughput

Throughput can mean output tokens/s, total tokens/s, or requests/s. Batch and concurrency can improve total throughput while worsening each request’s latency. Always pair throughput with latency SLOs, arrival process, and input/output length distributions.

52.8 Memory per token

KV bytes/token/sequence equal $2LH_{kv}D_h b_{kv}$, plus page metadata and internal waste. MQA/GQA reduce Hkv. Beam search and multiple return sequences multiply logical state; prefix sharing may reduce physical duplication.

52.9 FLOPs per token

A useful dense-model approximation for one decode token is roughly twice the active parameter count for linear layers, because each weight contributes multiply-add work, plus attention over history and other operations. This is not exact: tied embeddings, MoE activation, quantization, sparsity, and dequantization change it. Prefill amortizes weights across T rows, whereas batch-one decode often streams large weight matrices for one row and becomes bandwidth-bound.

52.10 Measurement protocol and four perspectives

Record wall-clock timestamps at admission, schedule, prefill start/end, each emission, and completion. Use warm and cold runs, synchronize device measurements, and sample memory peaks. Publish p50/p95/p99 and errors/cancellations.

- **Follow the Token:** each emitted token has a queue-to-stream timeline.
- **Follow the Gradient:** inference omits gradients and optimizer state; accidental autograd is a budget bug.
- **Follow the Byte:** separate logical bytes, allocated bytes, and device/process residency.
- **Follow the Request:** batching trades individual latency for system throughput.

Failures include decimal GB/GiB confusion, omitting one of K/V or layers, using maximum context for all traffic, averaging heterogeneous prompts, excluding queueing silently, and calling reciprocal mean latency “throughput.” The exit check derives the sample budget in bytes, converts once to GiB, reserves measured non-tensor overhead, and explains why it can OOM below nominal device capacity.

52.11 Primary references

- MLCommons, [MLPerf Inference: Datacenter benchmark](#), version-sensitive; accessed 2026-09-19.
- Kwon et al., [Efficient Memory Management for Large Language Model Serving with Page-dAttention](#), 2023.
- vLLM, [official documentation](#), version-sensitive; accessed 2026-09-19. Pin engine commit and configuration, model revision, hardware clocks and power, workload, tokenizer, and measurement boundary.

Part 14: Making Inference Fast

Chapter 53

FlashAttention and Paged KV Memory

Status: **Drafted**

53.1 Two different memory problems

FlashAttention changes how attention arithmetic moves data. PagedAttention changes how a serving system allocates persistent KV state. They solve different problems and can coexist.

53.2 Why naive inference is slow

Naive attention may materialize the score matrix $S=QK^T$ and probability matrix $P=\text{softmax}(S)$, each $[B,H,T,T]$ during full attention. At $B=1,H=32,T=8192$ in BF16, one such matrix is 4 GiB. Even when enough memory exists, writing and rereading these intermediates through HBM is expensive. During decode, the matrix is only $[B,H,1,S]$, but every step rereads a growing cache; small GEMMs also use compute units poorly.

53.3 FlashAttention

FlashAttention is an **exact, IO-aware** attention algorithm up to normal floating-point rounding. It tiles Q , K , and V so score blocks are produced on chip, incorporated into a running softmax and output, and discarded rather than stored in HBM. It changes the schedule, not the attention definition.

53.4 Online softmax and IO scheduling

Softmax appears to require all scores because its denominator spans a row. Online softmax makes blocks composable. For a processed prefix, keep running maximum m , normalizer l , and weighted output accumulator o . For a new block with maximum m_b and exponent sum l_b , set $m_{\text{new}}=\max(m,m_b)$, rescale old and new contributions by $\exp(m-m_{\text{new}})$ and $\exp(m_b-m_{\text{new}})$, then combine. The final o/l equals full softmax attention.

Tiling reduces HBM traffic but introduces constraints from head width, masks, dropout, causal structure, and hardware. It does not make the quadratic arithmetic of full prefill disappear. A fallback kernel can beat it on small or unusual shapes.

53.5 KV cache layout

Decode needs efficient append of one K/V row and efficient reads across all prior positions. Layout choices order layer, block, token, KV head, and head dimension differently. A contiguous per-request allocation makes reads simple but requires predicting maximum length or reallocating/copying. Variable completion lengths create internal waste and fragmented free regions.

53.6 Paged KV memory

Paged allocation divides KV into fixed-size token blocks. Each sequence owns a logical list of blocks mapped to noncontiguous physical blocks. Growth allocates another free block; completion returns blocks; shared prefixes can reference the same physical blocks with copy-on-write semantics. Waste is bounded mostly by the final partially filled block rather than reserved maximum context.

If each block holds 16 tokens and a sequence has 129 tokens, it needs nine blocks: capacity 144, internal waste 15 tokens. Metadata and indirect addressing are the price.

That final-block waste is $15/144 = 10.4\%$ of allocated token slots for this sequence. At exactly 128 tokens it is zero; at 129 it jumps to the maximum of 15 slots for a 16-token block. Report physical blocks and logical tokens rather than an average “bytes per token” that hides this boundary effect.

53.7 PagedAttention

PagedAttention kernels follow block tables while computing attention, much as virtual memory translates logical pages. They must preserve causal order despite noncontiguous physical storage and support different sequence lengths in one batch. The original vLLM work connected this mechanism to higher serving utilization by reducing KV fragmentation and enabling sharing, but performance depends on workload and implementation.

53.8 Four perspectives and exit check

- **Follow the Token:** FlashAttention computes its exact weighted value blockwise; paging maps its historical K/V to physical blocks.
- **Follow the Gradient:** FlashAttention training uses backward kernels/recomputation; serving pages are inference state.
- **Follow the Byte:** one optimization removes T-by-T intermediates; the other reduces reserved and fragmented KV.
- **Follow the Request:** block tables grow, share, and free as requests arrive and finish.

Failures include treating FlashAttention as approximate, claiming it removes quadratic compute, confusing paging with OS swapping, stale block-table entries, reference-count bugs, and comparing unmatched attention features. Derive the 4 GiB score example, perform an online-softmax merge for two blocks and compare it numerically with a full softmax, then calculate page waste for lengths 128, 129, and 143.

53.9 Primary references

- Dao et al., [FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#), 2022.
- Dao, [FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning](#), 2023.

- Kwon et al., [Efficient Memory Management for Large Language Model Serving with PagedAttention](#), 2023.
- vLLM, [official documentation](#), version-sensitive; accessed 2026-09-19. Pin kernel version, attention backend, dtype, mask, model, block size, GPU, and workload.

Chapter 54

Caching, Batching and Prefill Scheduling

Status: **Drafted**

54.1 The scheduler is part of model performance

Requests arrive at different times with different prompt and output lengths. Static batching waits for a cohort and wastes slots when shorter members finish. Modern serving continuously changes the active batch while protecting memory and latency targets.

54.2 Prefix caching

Many requests share exact token prefixes: a system prompt, tool schema, or document. Prefix caching retains the corresponding per-layer KV blocks, so a later request begins after the cached prefix. The key must cover everything that affects K/V: model and adapter revision, exact token IDs, position treatment, relevant attention configuration, KV dtype, and often multimodal inputs or prompt metadata.

54.3 Prefix reuse

Reuse can be full or block-aligned partial matching. Radix trees or hashes locate the longest reusable prefix; reference counts protect shared blocks. Savings scale with reused prefill work, but lookup, hashing, metadata, and cache residency cost resources. Private prompt content creates isolation and eviction requirements. Text equality is insufficient if tokenization or model state differs.

54.4 Continuous batching

At each scheduling iteration, finished sequences leave and waiting sequences enter. Active decode rows execute together, raising weight reuse and throughput. The scheduler tracks per-request state, stop conditions, KV blocks, and fairness. Large batches improve throughput until memory, queue delay, or kernel scaling becomes limiting. Throughput-optimal scheduling can violate an interactive latency SLO.

Consider A and B starting together, with A finishing after two tokens and B after eight. Static batching leaves A's row idle for six steps. Continuous batching admits C into available capacity. The exact implementation may rebuild metadata rather than literally reuse a row, but the utilization effect is the same.

54.5 Chunked prefill

A long prompt can monopolize the device and delay decode tokens. Chunked prefill divides its tokens across iterations, interleaving chunks with latency-sensitive decode. It also caps temporary activation size. Smaller chunks improve fairness but can reduce prefill efficiency and add scheduling overhead. Chunk size should be chosen from measured TTFT, TPOT, throughput, and memory, not folklore.

A token-budget scheduler might allow 2,048 scheduled tokens per iteration: 32 active decode sequences consume one token each, leaving at most 2,016 prefill tokens. If two prompts each request a 1,024-token chunk, the second must be shortened to 992 tokens or deferred. Admission must ensure KV capacity for all scheduled tokens and a policy for predicted output growth; a token budget is not itself a memory budget.

54.6 Cache correctness

Cache hits are correctness claims. Validate by comparing logits with and without reuse on exact token sequences. Include model/adaptor changes, differing positions, block-boundary prefixes, cancellation, eviction, concurrent references, and hash collision handling. Hashes locate candidates; exact identity or collision-safe verification establishes equality.

Eviction policies can use recency, frequency, size, recomputation cost, tenant quota, or combinations. Evict only unreferenced blocks. A high hit rate can still lose if it retains cheap short prefixes while evicting costly long ones; saved prefill tokens or time is a better value measure.

54.7 Simulation and four perspectives

Simulate arrivals with timestamp, prompt length, target output, shared-prefix ID, and cancellation. Compare static and continuous batching with and without chunked prefill under the identical arrival trace. Report p50/p95 TTFT and TPOT, completion latency, output tokens/s, queue depth, cache hit value in saved prefill tokens, and peak physical blocks. Include one overload interval so fairness and admission behavior are observable.

- **Follow the Token:** a prompt token is computed, reused, or scheduled in a chunk.
- **Follow the Gradient:** serving caches contain no training graph and must not mutate weights.
- **Follow the Byte:** shared pages save physical KV; active sequences compete for block capacity.
- **Follow the Request:** admission, fairness, and preemption determine user-visible latency.

Failures include tenant data leakage, stale adapter cache, hash-only equality, starvation of long prompts, over-admission, head-of-line blocking, and metrics averaged across unlike requests. Exit when a deterministic simulation explains both a throughput win and a tail-latency regression.

54.8 Primary references

- Yu et al., [Orca: A Distributed Serving System for Transformer-Based Generative Models](#), 2022.

- Kwon et al., [Efficient Memory Management for Large Language Model Serving with PagedAttention](#), 2023.
- Agrawal et al., [Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve](#), 2024.
- vLLM, [Automatic Prefix Caching](#), version-sensitive; accessed 2026-09-19. Pin engine commit and configuration, block size, arrival trace, model revision, and SLO.

Chapter 55

Speculative Decoding and Dynamic Execution

Status: **Drafted**

55.1 Spend less target-model time per accepted token

Autoregressive decode normally invokes the large target once per token. Dynamic methods try to accept several tokens per expensive step or avoid unneeded computation. Every speedup claim must include quality, acceptance, batching, and overhead.

55.2 Speculative decoding

A cheaper process proposes gamma tokens. The target scores those positions in one verification pass. In exact sampling variants, an acceptance/rejection rule and correction distribution preserve the target distribution; greedy variants accept the matching prefix. Accepted tokens advance generation, then a target token resolves the first rejection or follows a fully accepted block.

Speed depends on accepted tokens per target pass, draft cost, verification efficiency, and batch effects. A high acceptance rate can still lose if the draft is slow or verification kernels are inefficient.

For an accounting example, suppose a verification pass advances by an average of 3.2 tokens and costs 8 ms, while drafting costs 3 ms per attempted block. The combined $11 \text{ ms} / 3.2 = 3.44 \text{ ms}$ per advanced token is faster than a 5 ms target-only baseline. If batching makes verification cost 14 ms, the same acceptance statistics yield 5.31 ms and lose. Accepted length is therefore an input to a latency calculation, not a speedup by itself.

55.3 Draft-verifier design

Draft and target must share compatible tokenization and conditioning. Choose gamma by measuring: too small leaves parallelism unused; too large wastes rejected work. Track accepted length distribution by prompt type and output position, target passes, end-to-end TPOT, and exact-distribution tests—not only acceptance percentage.

55.4 Speculative-model architectures

The proposer may be a smaller separate model, early-exit head, multiple-token prediction head, n-gram lookup, or tree of candidates. Self-speculative designs reuse part of the target but create cache/control complexity. Tree verification can test branches in parallel at greater temporary-memory cost. These are architectural choices beyond the generic decoding algorithm.

55.5 Sparsity

Sparse execution skips known zero or inactive work. Unstructured zeros rarely speed a dense GPU kernel without compatible storage and kernels. Structured N:M sparsity, block sparsity, and MoE routing align work into executable units, but metadata, load imbalance, and hardware support determine realized gains.

55.6 Pruning

Pruning creates sparsity by removing weights, heads, channels, layers, or experts using magnitude, sensitivity, or learned criteria. One-shot pruning is cheap but can damage quality; retraining may recover it. Report remaining parameters, actual executed operations and bytes, and downstream quality. A smaller checkpoint with dense-shaped kernels is not necessarily faster.

55.7 Early exit

Early-exit models attach prediction heads before the final layer and stop when a confidence rule fires. Calibration is difficult: entropy or margin can look confident and still be wrong. Thresholds shift compute distributions and tail behavior. Evaluate per-domain worst cases and compare against a smaller dense model at equal quality and compute.

55.8 Layer skipping

Layer skipping routes individual tokens or requests around blocks. Static removal yields a smaller model; dynamic skipping needs a router and variable execution. GPUs prefer regular batches, so divergent layer paths may erase theoretical savings unless requests are regrouped. Cache semantics also change if later layers expect missing states.

55.9 CUDA Graph optimization

Dynamic decisions conflict with fixed graph topology. Practical systems capture common verification sizes, draft lengths, batch buckets, or routing patterns and fall back elsewhere. A huge graph cache raises warmup and memory. Include cache hit rate and compilation amortization when attributing a speedup to graph replay.

55.10 Exactness, perspectives, and exit check

Test exact speculative sampling on a tiny vocabulary by enumerating or drawing enough samples to compare output frequencies with direct target sampling and confidence intervals. Include a case where the draft assigns zero probability to a token that the target can emit, exercising the correction path. For greedy mode, compare token-for-token. Then report attempted draft length,

accepted-length distribution, advanced tokens per target pass, draft time, verification time, and end-to-end latency.

- **Follow the Token:** candidates remain provisional until verified.
- **Follow the Gradient:** routers, exit heads, or pruned models require training/calibration; decoding itself is inference.
- **Follow the Byte:** drafting adds weights/KV; accepting multiple tokens amortizes target weight reads.
- **Follow the Request:** scheduler and batch size determine whether verification parallelism becomes wall-time savings.

Failures include calling approximate methods exact, ignoring draft memory, measuring batch one only, using acceptance rate without accepted length, sparse formats without kernels, and routing divergence. Exit when you can state a negative-result criterion: no speedup at matched quality and production concurrency.

55.11 Primary references

- Leviathan, Kalman, and Matias, [Fast Inference from Transformers via Speculative Decoding](#), 2023.
- Chen et al., [Accelerating Large Language Model Decoding with Speculative Sampling](#), 2023.
- NVIDIA, [CUDA C++ Programming Guide: CUDA Graphs](#), version-sensitive; accessed 2026-09-19. Pin algorithm variant, draft and target revisions, engine commit, draft length, batch and concurrency, prompts, precision, and graph policy.

Chapter 56

Quantization Across the Inference Path

Status: **Drafted**

56.1 Representing useful signal with fewer bits

Quantization maps real values to a finite codebook. For affine uniform quantization, $q = \text{clip}(\text{round}(x/s) + z)$ and reconstruction is $\hat{x} = s(q - z)$. Scale s and zero point z may be per tensor, channel, group, or token. Fewer bits reduce storage and bandwidth only when kernels execute the format efficiently.

56.2 Quantization error and a worked matrix

For symmetric signed n -bit quantization with $Q=2^{(n-1)}-1$, choose $s=\max|x|/Q$, $q=\text{round}(x/s)$, and reconstruct sq . On values $[-1.0, -0.2, 0.3, 0.9]$ with INT4, $Q=7$ and $s=1/7$, codes are $[-7, -1, 2, 6]$, reconstructing $[-1, -0.143, 0.286, 0.857]$. The maximum absolute error is about 0.057 and the MSE is about 0.00133. Four packed codes occupy 2 bytes; one FP16 scale makes this toy group 4 bytes total, only 2x smaller than its 8-byte BF16 source rather than the payload-only 4x. Real packing may add alignment and padding.

56.3 Post-training quantization

PTQ calibrates or quantizes a pretrained model without full retraining. Weight-only PTQ may need no activation dataset; activation quantization needs representative calibration. Outliers and domain shift can destroy accuracy. Calibration set, objective, group size, clipping, and kernel backend are part of the method.

56.4 Quantization-aware training

QAT simulates quantize/dequantize effects during training, often with a straight-through estimator for rounding. It can adapt weights to error and support harder activation formats, at training cost and with approximation in the backward pass. QAT is not guaranteed to beat a strong PTQ pipeline.

56.5 GPTQ

GPTQ is a layerwise post-training weight quantization method using approximate second-order information to quantize weights while compensating error in remaining weights. Results depend on calibration data, ordering, damping, group size, and implementation. The artifact still requires a matching runtime kernel.

56.6 AWQ

Activation-aware Weight Quantization observes that a small fraction of weight channels are especially important relative to activation magnitudes. It searches scaling that protects salient weights while enabling low-bit weight-only execution. It is also calibration- and backend-dependent; “AWQ 4-bit” does not uniquely specify a format or speed.

56.7 Weight quantization

Weight-only INT4 can shrink the dominant model residency and reduce weight bandwidth during decode. Scales, zero points, packing, group metadata, padding, and sometimes full-precision outliers add overhead. Dequantization may fuse into GEMM; otherwise conversion traffic can erase gains.

56.8 Activation quantization

Activations vary by token and contain outliers. INT8 schemes use static calibration or dynamic per-token/per-channel scales. Dynamic scaling costs reductions and metadata but adapts to inputs. SmoothQuant-like methods move quantization difficulty between activations and weights through equivalent scaling. Validate residual paths and normalization, not only linear layers.

56.9 KV quantization

KV cache grows with context and concurrency, so FP8/INT8/INT4 caches can unlock capacity. Error affects attention repeatedly over future tokens. Keys influence similarity scores; values influence weighted sums. Granularity, asymmetric ranges, outlier handling, and dequantization inside attention matter. Evaluate long-context quality, retrieval, and generation, not short perplexity alone.

56.10 FP8 inference

FP8 formats trade exponent range against mantissa precision, commonly E4M3- and E5M2-like variants. Hardware and frameworks may use scaling recipes and higher-precision accumulation. “FP8” must state format, scale granularity, accumulation, and which tensors remain higher precision.

56.11 INT8 inference

INT8 offers mature matrix instructions on many platforms and often supports weight-and-activation quantization. Effective compression versus BF16 is near 2x before metadata, but realized latency varies with shape and conversion overhead.

56.12 INT4 inference

INT4 halves packed weight bytes again, but representational error, packing, scale overhead, and hardware support become more important. Weight-only INT4 is more common than fully INT4 activation paths. Small batches can benefit from bandwidth reduction; large prefill may be compute-bound and see different gains.

56.13 GGUF formats

GGUF is a versioned container used by the ggml/llama.cpp ecosystem for tensors and metadata. It can hold many quantization types; the extension alone does not mean “4-bit.” Tokenizer metadata, architecture fields, tensor types, and runtime support must agree. Keep the original model license and provenance, and pin converter/runtime commits.

56.14 Four perspectives, validation, and exit check

- **Follow the Token:** its activations are scaled, rounded, accumulated, and reconstructed along the path.
- **Follow the Gradient:** PTQ has no model training; QAT inserts simulated error into forward training.
- **Follow the Byte:** count packed payload plus scales, zeros, padding, and temporary dequantized buffers.
- **Follow the Request:** lower residency raises concurrency, but conversion and quality constraints shape latency.

Quantize a tiny matrix per tensor and per row; report reconstruction metrics, physical bytes, and downstream matmul error. Then evaluate a pinned model on perplexity plus task and long-context sets at matched prompts. Failures include file-size-only accounting, comparing different kernels, calibration leakage, unspecified group size, and claiming universal quality from one benchmark.

56.15 Primary references

- Frantar et al., [GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers](#), 2022.
- Lin et al., [AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration](#), 2023.
- Xiao et al., [SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models](#), 2022.
- Open Compute Project, [8-bit Floating Point Specification](#), revision 1.0, 2023; and ggml-org, [llama.cpp source and GGUF implementation](#), version-sensitive; accessed 2026-09-19. Pin method implementation, commit, model and calibration data, quantization type and group size, hardware, and kernels.

Part 15: Distributed LLMs

Chapter 57

Parallel Training Strategies

Status: **Drafted.**

Part 15: Distributed LLMs

57.1 Learning outcome

Account for parameter, gradient and optimizer storage under replication and sharding.

57.2 Prerequisites

CH-56.

57.3 Why one accelerator stops being enough

Coverage ID: T-57-01.

A training step must accommodate persistent model state and temporary activations. For (P) parameters trained with mixed-precision Adam, a useful first estimate is 2 bytes of BF16 weights, 2 bytes of gradients, and 8 bytes for two FP32 Adam moments per parameter: roughly (12P) bytes before activations, allocator fragmentation, communication buffers, or a possible FP32 master copy. A 7-billion-parameter model therefore starts near 84 GB. Long sequences may make saved activations the limiting term even when weights fit.

In binary units, 84,000,000,000 bytes is about 78.2 GiB. If the optimizer keeps a separate FP32 master parameter copy, add 28 GB (26.1 GiB), raising this simplified total from 12 to 16 bytes per parameter. State the optimizer implementation: fused and quantized optimizers can use different state and temporary buffers.

Multiple accelerators do not create one magical memory pool. A strategy defines tensor ownership, reconstruction, and communication. Ask which state is replicated, which is sharded, which is transiently gathered, and which operation sits on the critical path.

57.4 Data parallelism

Coverage ID: T-57-02.

Every rank owns the same model and consumes different examples. For global batch (B) over (N) ranks, each commonly receives (B/N) examples. Ranks reduce local gradients and apply identical updates. It scales throughput and divides activation memory for a fixed global batch, but does not divide model state.

Silent failures include overlapping sampler shards and inconsistent loss normalization. A run can be fast while its effective data distribution or learning rate is wrong. Compare a distributed step with a single-rank reference under the same global batch.

57.5 DDP

Coverage ID: T-57-03.

Distributed Data Parallel overlaps reduction with backpropagation. When autograd completes a gradient bucket, an all-reduce can begin while earlier layers are still computing. Every rank finishes with the reduced gradient and independently performs the same optimizer step.

The 7B example still carries roughly 84 GB of model state per rank. DDP is attractive when that fits because replication simplifies execution. Bucket sizing controls overlap: tiny buckets expose launch latency; huge buckets start too late. Rank-dependent control flow can leave collectives unmatched and hang the job.

57.6 ZeRO

Coverage ID: T-57-04.

The Zero Redundancy Optimizer removes replicated state across (N) data-parallel ranks. Under the 2-byte weights, 2-byte gradients, 8-byte moments assumption, DDP uses about 12 bytes per parameter per rank. ZeRO-1 shards optimizer state: $(4+8/N)$. ZeRO-2 also shards gradients: $(2+10/N)$. ZeRO-3 shards parameters too: about $(12/N)$.

These are persistent-state estimates, not peak-memory promises. ZeRO-3 gathers parameters for computation and needs buffers; activations remain unless separately recomputed, offloaded, or sharded. Dividing total observed peak by (N) is therefore incorrect.

57.7 FSDP

Coverage ID: T-57-05.

PyTorch FSDP2 represents sharded parameters with DTensor. Before a wrapped module executes, ranks all-gather its parameter shards; after use, full parameters can be released. Backward reduce-scatters gradients so each rank retains only its shard. Wrapping boundaries control the size and lifetime of materialized weights.

FSDP trades memory for collectives. One giant unit gathers too much; tiny units create excessive calls. Prefetch can hide communication behind compute but may increase peak memory. Pin the PyTorch version and API: current documentation distinguishes FSDP2 from deprecated FSDP1.

57.8 Sharded optimizer state

Coverage ID: T-57-06.

Adam keeps first and second moments. A sharded optimizer updates only the owned parameter shard using its reduced gradient shard. For 7B parameters and eight ranks, 56 GB (52.2 GiB) of FP32 moments becomes 7 GB (6.52 GiB) per rank before overhead. Under the chapter's

assumptions, the complete ZeRO-3 persistent share is 10.5 GB (9.78 GiB) per rank; temporary all-gathers and activations sit on top.

Checkpoint metadata must map logical parameters to shards. A pile of rank-numbered files is insufficient when recovery uses another world size. Save logical tensor metadata, optimizer mapping, RNG and scheduler state, and validate resharding.

57.9 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

57.10 Lab and exit check

For $P = 7 \times 10^9$, calculate persistent state for DDP and every ZeRO stage at $N = 8$, first in bytes and then GiB. Add an explicit activation estimate and a 15% reserve applied to a clearly named subtotal. Explain why gathered modules can still cause an OOM. You pass when you can identify the tensor and collective at each FSDP boundary and explain why activation memory is separate.

57.11 Primary references

- [PyTorch FSDP2 tutorial](#), accessed 2026-09-19.
- Rajbhandari et al., [ZeRO](#), 2020.
- [PyTorch DistributedDataParallel API](#).

Chapter 58

Partitioning Model Computation

Status: **Drafted.**

Part 15: Distributed LLMs

58.1 Learning outcome

Place a model across devices and derive communication on its critical path.

58.2 Prerequisites

CH-20, CH-41, CH-57.

58.3 Tensor parallelism

Coverage ID: T-58-01.

Tensor parallelism splits matrix operations. For $(Y=XW)$, column-parallel execution partitions (W) along outputs; ranks compute slices of (Y) . A following row-parallel layer can consume those slices and reduce partial outputs. Implementations pair layouts so some intermediates remain sharded.

If a full BF16 residual $[B, T, 4096]$ must be reduced, its logical payload is $BT4096 \times 2$ bytes. At $B = 8, T = 2048$, that is 128 MiB before collective factors. This communication occurs inside blocks, so tensor parallelism favors fast intra-node links.

“Payload” is the logical tensor size, not bytes injected by each rank. A ring all-reduce communicates approximately $2(N - 1)/N$ payload bytes per rank; with four tensor-parallel ranks, a 128 MiB residual implies about 192 MiB sent/received per rank under that model. Algorithm, chunking, topology, and overlap determine observed time.

58.4 Pipeline parallelism

Coverage ID: T-58-02.

Pipeline parallelism assigns consecutive layer groups to stages. Microbatches flow between them. For (p) stages and (m) microbatches, simple forward-only utilization is approximately $(m/(m+p-1))$; too few microbatches create a bubble. Training interleaves forward and backward but retains or recomputes activations.

Stages exchange boundary activations. The slowest stage sets cadence. Equal parameter counts do not ensure balance when attention cost varies with sequence length or MoE routing is skewed.

58.5 Sequence parallelism

Coverage ID: T-58-03.

Sequence parallelism shards suitable activation operations along the token dimension, often alongside tensor parallelism. Pointwise normalization and dropout can remain local, reducing replicated activations. Attention still requires global contextual information.

Label every tensor dimension replicated or sharded. A residual $([B, T, D])$ split over (T) may need a transpose-like collective before an operation expecting (D) -shards. Confusing layout with semantics is a common bug.

58.6 Context parallelism

Coverage ID: T-58-04.

Context parallelism divides a long sequence while preserving exact attention. Ranks exchange KV blocks or partial attention statistics so each query includes allowed context. Ring algorithms stream blocks and maintain online-softmax maxima and sums rather than materializing all scores centrally.

Splitting tokens alone produces a different model if cross-rank context is omitted. Cost depends on causal masking, head layout, context length, and measured link bandwidth.

58.7 Expert parallelism

Coverage ID: T-58-05.

Expert parallelism places different MoE experts on different ranks. Routing is followed by all-to-all dispatch of token states, local expert MLPs, then all-to-all return. Traffic follows routed representations; balance follows router decisions.

A popular expert can overflow capacity or stall peers. Average GPU utilization hides the problem, so record per-expert token counts, dropped tokens, and all-to-all tail latency. Large systems compose data, tensor, pipeline, context, and expert axes; their degrees must multiply to world size and map sensibly onto topology.

58.8 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

58.9 Lab and exit check

Place a 32-layer model on eight devices using two pipeline stages and four-way tensor parallelism. For $B = 4$, $T = 1024$, $D = 4096$, a full BF16 boundary activation is 32 MiB. Calculate both that logical payload and per-rank ring traffic for the residual-sized tensor collective; identify which communication repeats each layer and which repeats at each stage boundary. Then state how microbatching changes boundary-message size and count without changing total logical examples.

58.10 Primary references

- Shoenberger et al., [Megatron-LM](#), 2019.
- Huang et al., [GPipe](#), 2019.
- Lepikhin et al., [GShard](#), 2020.

Chapter 59

Collectives and Interconnects

Status: **Drafted.**

Part 15: Distributed LLMs

59.1 Learning outcome

Match each parallel strategy to collectives and estimate transfer time.

59.2 Prerequisites

CH-58.

59.3 Broadcast

Coverage ID: T-59-01.

Broadcast copies one root tensor to all ranks. Use it for configuration or synchronized control state, not gradient averaging. Broadcasting a huge checkpoint from one root may bottleneck that host; sharded parallel reads are often better. Every rank must participate in the same communicator order.

59.4 All-reduce

Coverage ID: T-59-02.

All-reduce combines tensors elementwise and leaves the result everywhere. DDP uses it for gradients. A ring moves roughly $2(N - 1)/N$ times the tensor size per rank. For a 1 GiB buffer over eight ranks, that is about 1.75 GiB. At an effective 50 GiB/s, bandwidth time alone is 35 ms; at 50 decimal GB/s it is about 37.6 ms. Latency, protocol overhead already absent from the effective rate, and contention can add more.

59.5 All-gather

Coverage ID: T-59-03.

All-gather concatenates one shard from every rank and returns the full logical tensor to all. FSDP uses it to materialize weights before a module. The gathered tensor raises peak memory; its allocation and release timing matter alongside transfer speed.

59.6 Reduce-scatter

Coverage ID: T-59-04.

Reduce-scatter reduces corresponding elements and leaves each rank one output shard. FSDP uses it for gradients. Reduce-scatter plus all-gather decomposes the logical work of all-reduce while letting ranks retain only owned state between phases.

59.7 All-to-all

Coverage ID: T-59-05.

All-to-all sends distinct slices between all rank pairs. Expert parallelism uses it for token dispatch and return. Uneven splits and hot experts create tails. Many peer exchanges stress topology differently from a ring, so payload alone is not enough to predict time.

59.8 NCCL

Coverage ID: T-59-06.

NCCL implements GPU collectives and selects algorithms and transports from topology and run-time conditions. Applications still define communicator membership and call order. Record NCCL version, topology, rank mapping, sizes, environment overrides, and observed overlap. A collective microbenchmark cannot prove application overlap.

59.9 PCIe

Coverage ID: T-59-07.

PCIe connects GPUs, CPUs, NICs, and storage through root complexes and switches. Quoted generation bandwidth is a link maximum. Peer bandwidth depends on lane width, path, protocol overhead, and concurrent traffic. A path staging through host memory differs radically from direct peer access.

59.10 NVLink

Coverage ID: T-59-08.

NVLink provides high-bandwidth GPU peer connectivity on supported platforms. Link count and capability vary by generation and SKU. Never import a bandwidth value from a different system; measure the real peer pair in both directions.

59.11 NVSwitch

Coverage ID: T-59-09.

NVSwitch forms a switched GPU fabric and reduces point-to-point topology asymmetry. It does not abolish contention, and inter-node traffic still reaches a NIC. Concurrent jobs and collectives can share fabric resources.

59.12 InfiniBand

Coverage ID: T-59-10.

InfiniBand is a common inter-node fabric. GPUDirect RDMA can allow NIC access to GPU memory without CPU staging. End-to-end performance still depends on NIC placement, switch oversubscription, congestion, routing, and GPU-to-NIC affinity.

A first model is $t \approx \alpha s + V/B_{eff}$, with steps s , step latency α , per-rank wire bytes V , and measured effective bandwidth B_{eff} . Do not substitute logical tensor size for V without the collective's algorithmic factor. Tiny collectives are latency-bound; large ones are bandwidth-bound. Overlap and stragglers require measurement.

59.13 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

59.14 Lab and exit check

Estimate ring all-reduce time for 16 MiB, 256 MiB, and 1 GiB on eight ranks using explicit hypothetical per-step latency and effective bandwidth. Show the $2(N - 1)$ ring phases and the $2(N - 1)/N$ per-rank byte factor, keeping GiB with GiB/s. Compare intra-node and inter-node values and explain why peak link bandwidth is not effective bandwidth.

59.15 Primary references

- [NCCL collective operations](#), accessed 2026-09-19.
- [NVIDIA GPUDirect RDMA](#), accessed 2026-09-19.
- Patarasuk and Yuan, [Bandwidth Optimal All-reduce](#), 2009.

Chapter 60

Distributed Execution and Failure Recovery

Status: **Drafted.**

Part 15: Distributed LLMs

60.1 Learning outcome

Run a two-process CPU simulation and specify the real multi-GPU validation gate.

60.2 Prerequisites

CH-59.

60.3 Distributed training failures

Coverage ID: T-60-01.

Distributed jobs fail as a group. Classes include mismatched collective order or shape, transport loss, one-rank OOM, numerical divergence, and silent data corruption. A rank throwing before a collective leaves peers waiting for timeout. Logs need rank, host, device, step, last collective, tensor metadata, versions, and topology.

Correctness gates precede scale: compare one-rank and two-rank updates under matched global batch, checksum initial parameters, verify sampler disjointness, resume checkpoints, and inject one process failure.

60.4 Distributed inference

Coverage ID: T-60-02.

Inference may use data, tensor, pipeline, context, or expert parallelism. Data replicas serve independent requests with little cross-replica token traffic. Model-parallel requests communicate during each token, placing topology directly on latency. Variable output lengths complicate pipeline scheduling.

Conversation state has ownership. Routing a next turn away from its KV or prefix cache forces transfer or recomputation. All ranks also need the same tokenizer, revision, adapter, and sampling semantics.

60.5 Stragglers

Coverage ID: T-60-03.

Synchronous execution proceeds at the slowest rank. Thermal throttling, network congestion, data stalls, background load, retries, or skewed experts create stragglers. Mean kernel time can remain healthy while collective tails grow.

Collect per-rank timing around compute and collectives and compare distributions. Increasing timeouts merely hides many stragglers and wastes capacity longer.

60.6 Multi-GPU execution

Coverage ID: T-60-04.

A launcher assigns global rank, local rank, world size, and rendezvous data. Bind the intended device before model allocation. Construct process groups to match topology, such as tensor parallel ranks inside a node and corresponding data ranks across nodes.

A two-process CPU Gloo test validates rank logic and collective math. It cannot validate CUDA streams, NCCL transport, GPU peak memory, or overlap. The target gate records topology, memory, collective traces, numerical comparison, steady throughput, and failure injection on real GPUs.

60.7 Checkpoint recovery

Coverage ID: T-60-05.

A recoverable checkpoint includes parameters, optimizer and scheduler state, loss-scaler state, data position, RNG states, and sharding metadata. Write a temporary generation, validate all shards and manifest, then atomically publish completion. A directory visible before all ranks finish is not valid.

Checkpoint interval balances overhead with expected lost work. A synchronous four-minute checkpoint after every 30 minutes of useful compute adds $4/30 = 13.3\%$ overhead relative to compute time and occupies $4/34 = 11.8\%$ of resulting wall time. Async writes can hide blocking time but consume bandwidth and add consistency risks. Test resharding to another world size; rank-numbered blobs are not inherently portable.

60.8 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

60.9 Lab and exit check

Use `torch.distributed` and `Gloo` with two CPU processes. All-reduce a scalar and assert the result, then terminate one process before another collective and assert a bounded timeout rather than allowing the test to hang. Save each rank's last-step and last-collective record. Write a separate real-GPU acceptance gate covering NCCL topology, peak memory, overlap, numerics, throughput, checkpoint resume, and rank-failure recovery.

60.10 Primary references

- [PyTorch distributed overview](#), accessed 2026-09-19.
- [PyTorch Distributed Checkpoint](#), accessed 2026-09-19.
- [NCCL troubleshooting](#), accessed 2026-09-19.

Part 16: Serving LLMs as a System

Chapter 61

Anatomy of an Inference Server

Status: **Drafted.**

Part 16: Serving LLMs as a System

61.1 Learning outcome

Build a tiny continuous-batching server with cancellation and cleanup.

61.2 Prerequisites

CH-60.

61.3 Inference server architecture

Coverage ID: T-61-01.

An inference server is a state machine around a model runtime. A front end authenticates and validates; an admission controller checks capacity; a scheduler forms work; a model worker executes prefill and decode; a sampler chooses tokens; a streamer emits them; cleanup releases KV blocks. Tokenization may run in a CPU pool. Every boundary needs an owner and timeout.

The model is only one component. A fast kernel cannot rescue a queue overloaded by unbounded prompts, and a disconnected client must not leave an orphan sequence consuming KV memory.

61.4 HTTP requests

Coverage ID: T-61-02.

An HTTP request carries prompt or messages, generation limits, sampling options, model/adapters, and streaming preference. Validate maximum input, output, and combined token budgets after tokenization. Idempotency is nuanced: deterministic greedy generation may be replayable, while sampled output is not identical unless RNG state and scheduling semantics are fixed.

Return overload explicitly, typically with retry guidance, rather than accepting work into an infinite queue. Authentication and tenant quotas should run before expensive tokenization where possible.

61.5 Request queues

Coverage ID: T-61-03.

One FIFO hides distinct resource demands. Separate or classify long prefills, decodes, priority traffic, and administrative work. Queue delay equals start time minus admission time and must be measured independently from compute. Aging can prevent starvation; deadlines can reject requests that cannot meet useful service time.

A queue capacity should derive from memory and latency budgets. Length alone is insufficient: ten 100-token requests and ten 100,000-token requests are radically different liabilities.

Use token work, not request count, for a first overload check. If admitted work arrives at 12,000 token-equivalents/s while the pinned configuration sustains 10,000, no scheduling policy can make the queue stable. Little's law, $N = \lambda W$, is also a useful consistency check: at 20 admitted requests/s and 0.5 s mean time in system, about ten requests should be present on average. A large mismatch often exposes dropped measurements, retries, or work waiting outside the instrumented queue.

61.6 Scheduling

Coverage ID: T-61-04.

The scheduler chooses which sequences prefill or decode each iteration under token and KV budgets. Decode usually advances one token per active sequence; prefill can consume thousands. Chunked prefill limits monopolization by dividing a long prompt across iterations.

Fairness has a cost. Strict shortest-job-first improves average latency but can starve long contexts. Weighted fair scheduling can protect tenants. Record the policy because benchmark results are inseparable from it.

61.7 Dynamic batching

Coverage ID: T-61-05.

Dynamic batching waits briefly to combine compatible work. Continuous batching goes further: finished sequences leave and waiting sequences enter between decode iterations. Padding is reduced because sequences need not start and finish together.

Larger batches improve device utilization but raise queue delay and KV pressure. The optimum depends on prompt/output distributions, not a fixed magic size. Evaluate TTFT and inter-token latency alongside aggregate tokens per second.

61.8 KV allocation

Coverage ID: T-61-06.

Each live sequence needs per-layer key/value state. Paged allocation maps logical token blocks to noncontiguous physical blocks, reducing external fragmentation and enabling sharing. Admission must reserve enough blocks for current tokens and planned growth or deliberately support preemption.

Cancellation must decrement references for shared prefix blocks and free private blocks exactly once. Memory leaks appear as declining admission capacity over time; double-free appears as corrupted generations.

61.9 Streaming

Coverage ID: T-61-07.

Streaming sends tokens or text deltas as generation proceeds. Byte-pair tokens do not always decode to valid standalone Unicode, so the streamer needs incremental detokenization. Network backpressure must not block the model loop; use bounded per-client buffers and cancel or disconnect slow consumers by policy.

A client disconnect should propagate cancellation to the scheduler, sampler, and KV allocator. Define whether already-generated usage is billed and logged.

61.10 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

61.11 Lab and exit check

Implement a CPU simulator with arrival times, prompt lengths, output limits, a token-budget scheduler, paged KV counter, cancellation, and bounded stream queues. Assert that all pages return after every request completes or cancels. Compare FIFO and continuous batching using TTFT and completion time.

Exit questions: under overload, where is admission rejected; after a disconnect, which component owns final KV release; and can every accepted request reach exactly one terminal state (completed, cancelled, or failed)?

61.12 Primary references

- [vLLM architecture documentation](#), accessed 2026-09-19.
- Kwon et al., [Efficient Memory Management for LLM Serving with PagedAttention](#), 2023.

Chapter 62

Tenants, Models and Residency

Status: **Drafted.**

Part 16: Serving LLMs as a System

62.1 Learning outcome

Design admission and eviction policies for concurrent models within a memory budget.

62.2 Prerequisites

CH-61.

62.3 Multi-tenant serving

Coverage ID: T-62-01.

Tenants share compute, queue capacity, KV memory, and sometimes prefix caches. Isolation needs quotas on prompt/output tokens, concurrent sequences, KV blocks, and rate. Weighted scheduling makes business priority explicit; per-tenant metrics expose noisy neighbors.

Cache sharing can leak whether another tenant used a prefix through timing or content mistakes. Scope caches by trust boundary and include model, tokenizer, adapter, and relevant decoding configuration in cache keys.

Isolation must be tested, not inferred from namespaces. Construct two tenants with identical text and verify that prefix hits, logs, traces, cancellation, and quota exhaustion do not cross the intended boundary. A cache key prevents accidental aliasing only if authorization is checked before lookup and cache-hit metadata is not exposed across tenants.

62.4 Multi-model serving

Coverage ID: T-62-02.

Multiple models may occupy separate workers, time-share one device, or share a base with adapters. Routing must resolve an immutable model revision before admission. Compatibility includes tokenizer and chat template, not merely parameter shape.

Loading on demand saves residency but adds cold-start tails. Keeping every model hot can leave too little KV capacity. Model popularity, load time, size, and SLO should drive placement.

62.5 Memory residency

Coverage ID: T-62-03.

A device budget includes weights, runtime workspace, graph pools, KV cache, temporary activations, and fragmentation reserve. On a 24 GiB GPU, a 14 GiB weight image plus 2 GiB workspace and 2 GiB safety reserve leaves only 6 GiB for KV and other state.

Residency planners should use measured peaks for pinned runtime/model combinations. Marketing capacity and raw file size omit dequantization buffers, duplicated mappings, and backend workspaces.

62.6 Model loading

Coverage ID: T-62-04.

Loading includes locating artifacts, integrity verification, mapping or reading pages, deserialization, device transfer, kernel preparation, and optional graph capture/autotuning. Separate cold startup from warm startup: page cache and compiled kernels can make the second run deceptively fast.

Publish readiness only after a representative warmup and health check. If traffic reaches a half-loaded worker, failures masquerade as random availability problems.

62.7 Model offloading

Coverage ID: T-62-05.

Offloading moves inactive layers or state to host memory and restores them before use. It expands capacity at the price of transfers on the critical path. Sequential decode revisits all layers per token, so layer swapping every token is usually punishing unless transfers overlap or placement is stable.

Offload is a placement strategy, not free memory. Host RAM, pinned buffers, PCIe traffic, and CPU contention become serving resources.

62.8 GPU-RAM-SSD hierarchy

Coverage ID: T-62-06.

GPU memory offers compute-near bandwidth; RAM is larger and slower across an interconnect; SSD is persistent and slower again. A 20 GB transfer over an effective 20 GB/s PCIe path has a one-second lower bound. At 3 GB/s SSD read bandwidth it exceeds six seconds before verification and device copy.

Admission should distinguish evicting KV, unloading a model to RAM, and dropping it to disk. Each has a different recovery latency. A useful policy scores freed bytes against reload cost and near-term request probability, while pinning SLO-critical models.

For example, evicting a 6 GiB idle model that takes 0.35 s to restore may be preferable to evicting a 10 GiB model that takes 4 s, even though the second frees more memory. State the policy as a measurable objective—such as expected SLO penalty per GiB freed—and add hysteresis so alternating requests do not repeatedly unload and reload both models.

62.9 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

62.10 Lab and exit check

Given 24 GiB GPU, 64 GiB RAM, two 14 GiB models, 2 GiB workspace each, and a 6 GiB peak KV demand, design residency and eviction. Use explicit PCIe and SSD bandwidth to estimate cold transitions. Test an arrival trace with alternating models and report SLO misses.

Include one adversarial trace that alternates models just faster than their reload time. Report whether admission, hysteresis, or pinning prevents thrash and how much unused capacity the protection costs.

62.11 Primary references

- [NVIDIA Triton model management](#), accessed 2026-09-19.
- [CUDA Best Practices: data transfer](#), accessed 2026-09-19.

Chapter 63

Disaggregation and Runtime Comparisons

Status: **Drafted.**

Part 16: Serving LLMs as a System

63.1 Learning outcome

Compare pinned runtime architectures and calculate when KV transfer negates disaggregation gains.

63.2 Prerequisites

CH-50, CH-52, CH-59, CH-62.

63.3 Prefill/decode disaggregation

Coverage ID: T-63-01.

Prefill and decode have different resource profiles. Prefill processes many prompt tokens in parallel and is often compute-intensive; decode handles one new token per sequence and often streams weights/KV under a tight latency loop. Disaggregation places them on different worker pools and transfers KV state at the boundary.

Benefits include independent scaling and reduced interference. Costs include routing, KV transfer, failure coordination, and another queue. It only wins when saved interference or better utilization exceeds those costs under the real workload.

The prefill worker must hand off more than an untyped byte buffer: model revision, sequence and position metadata, KV layout, precision, and ownership of each cache block must agree. A partial or duplicated transfer needs an idempotent protocol or explicit request failure; silently decoding from incomplete KV state can yield plausible but invalid text.

63.4 vLLM internals

Coverage ID: T-63-02.

vLLM centers serving around a scheduler, block-managed KV cache, continuous batching, optimized attention kernels, and model executors. Current releases include multiple parallel modes, prefix caching, chunked prefill, graph capture/compilation, and hardware plugins. These capabilities are version- and backend-dependent.

Its PagedAttention lineage reduces KV fragmentation, while scheduling integrates new work between iterations. Do not treat a project name as one static architecture: pin commit/release, engine generation, backend, model, and flags.

63.5 TensorRT-LLM internals

Coverage ID: T-63-03.

TensorRT-LLM builds optimized inference engines around NVIDIA GPUs, with graph transformations, fused kernels, quantization choices, KV/cache management, batching, and distributed execution. Build-time and runtime profiles can specialize shapes and features for performance.

The benefit comes with platform specificity and engine lifecycle work. Unsupported model operations, changing shapes, or mismatched plugin/driver versions can trigger rebuilds or fallbacks. Benchmark the generated engine and exact software stack.

63.6 llama.cpp architecture comparison

Coverage ID: T-63-04.

llama.cpp emphasizes portable local inference through its ggml tensor library, GGUF artifacts, quantized CPU kernels, memory mapping, and optional accelerator backends. It can offload layers while retaining CPU execution. This makes it suitable for laptops and heterogeneous devices.

vLLM targets throughput-oriented serving across varied accelerators; TensorRT-LLM specializes NVIDIA deployments; llama.cpp prioritizes compact portable execution. All can evolve. Compare required hardware, model coverage, scheduling, quantization, operational complexity, latency distribution, and reproducibility rather than declaring a universal winner.

63.7 Transfer overhead

Coverage ID: T-63-05.

For L layers, H_{kv} KV heads, head dimension d , context T , and b bytes per element, KV bytes per sequence are approximately $2LTH_{kv}db$. With 32 layers, 8 KV heads, $d = 128$, $T = 8192$, and BF16, this is 1 GiB (about 1.07 GB decimal). At an effective 25 GB/s, transfer alone is about 43 ms, excluding serialization, synchronization, and queueing.

If disaggregation saves 30 ms of contention but adds 50 ms end to end, it loses on latency. It may still win fleet throughput, so evaluate both. Prefix placement, RDMA, compression, and overlap change the equation but do not erase it.

63.8 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.

- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

63.9 Lab and exit check

Pin one release each of vLLM, TensorRT-LLM, and llama.cpp. Fill a matrix for hardware, artifact, scheduling, KV management, quantization, compilation, and fallback behavior. Calculate KV transfer for at least three contexts and identify the break-even saved time.

Do not compare default commands as if they were equivalent configurations. Match model revision, precision, maximum context, prompt/output distributions, sampling, concurrency, and acceptance quality; list any feature that cannot be matched.

63.10 Primary references

- [vLLM documentation](#), accessed 2026-09-19.
- Kwon et al., [Efficient Memory Management for Large Language Model Serving with PagedAttention](#), 2023.
- [TensorRT-LLM documentation](#), accessed 2026-09-19.
- [llama.cpp repository](#), accessed 2026-09-19.

Chapter 64

Production Serving and GPU Clusters

Status: **Drafted.**

Part 16: Serving LLMs as a System

64.1 Learning outcome

Run a load test with queue delay, tail latency, errors and recovery reported.

64.2 Prerequisites

CH-63.

64.3 Throughput vs latency

Coverage ID: T-64-01.

Throughput counts completed work per time; latency follows one request. Batching can raise tokens per second while increasing queue delay and time to first token. Plot load against latency: near saturation, a small arrival increase can cause queues to explode.

Benchmark with realistic prompt/output distributions and arrival patterns. Closed-loop clients that wait before sending again hide overload; open-loop arrivals reveal it.

64.4 Tokens per second

Coverage ID: T-64-02.

Report prompt tokens per second separately from generated tokens per second because prefill and decode differ. Per-user decode rate differs from aggregate rate: 1,000 tokens/s across 100 sequences may feel like 10 tokens/s each. State whether tokenization and rejected work are included.

64.5 Requests per second

Coverage ID: T-64-03.

Requests per second is meaningful only with workload shape. One request may generate one token or a thousand. Pair RPS with input/output token distributions, concurrency, cancellation, and success rate. Capacity planning in RPS alone is a classic category error.

64.6 p95 and p99 latency

Coverage ID: T-64-04.

Percentiles expose tails hidden by averages. Report TTFT, inter-token latency or TPOT, and end-to-end completion separately at p50/p95/p99. Use enough samples for the percentile: a p99 from 100 requests is essentially one observation. Preserve coordinated-omission-resistant arrival timing.

Attach uncertainty to tail estimates. With n independent requests, an empirical p99 is an order statistic near the largest $0.01n$ observations; 100 samples cannot characterize tail shape. Bootstrap only at an independent unit and preserve temporal blocks when load varies, because treating correlated tokens or burst traffic as independent produces falsely narrow intervals.

64.7 Serving SLOs

Coverage ID: T-64-05.

An SLO defines a measured target over a window, such as 99% successful requests with TTFT below 800 ms for prompts under 2,000 tokens. Segment by declared workload class. Error budgets turn reliability into an operational decision: deployments and experiments consume budget when they cause misses.

64.8 Observability

Coverage ID: T-64-06.

Correlate request traces with queue delay, scheduler iteration, batch composition, model/adapter revision, KV occupancy, prefix hits, kernel time, GPU memory, power, errors, and cancellation. High-cardinality request IDs belong in traces, not metric labels. Logs must avoid raw sensitive prompts by default.

64.9 Autoscaling

Coverage ID: T-64-07.

GPU workers start slowly, so scaling only on GPU utilization reacts late. Queueing delay, admitted token backlog, KV pressure, and predicted demand are better signals. Scale-down must drain or migrate stateful sequences. Maintain warm capacity when cold start exceeds the SLO horizon.

64.10 Failure recovery

Coverage ID: T-64-08.

Remove unhealthy workers from routing, stop admission, and decide which in-flight requests can retry. Streaming retries can duplicate text unless the protocol includes sequence position and

the client reconciles it. Model-parallel groups usually fail as a unit. Test worker crash, network partition, OOM, and corrupt artifact.

64.11 GPU cluster scheduling

Coverage ID: T-64-09.

Cluster scheduling must place entire gangs, honor topology, GPU type, memory, and NIC affinity, and limit fragmentation. Packing small jobs improves utilization but may create interference; spreading can waste fast links. Track useful tokens per accelerator-hour and SLO attainment, not allocation percentage alone.

Hardware figures vary by exact SKU, power mode, topology, drivers, and precision. Treat vendor peaks as ceilings and publish measured results with the environment.

64.12 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

64.13 Lab and exit check

Run an open-loop load test that ramps beyond saturation. Report queue delay, TTFT, TPOT, completion, prompt/generated tokens per second, RPS, p50/p95/p99, errors, and cancellations. Kill one worker mid-test and show detection, routing removal, recovery, and user-visible impact.

Publish the offered-load trace as well as achieved throughput. Verify the accounting identity $\text{offered} = \text{completed} + \text{rejected} + \text{cancelled} + \text{still in flight}$ over the test window, with retries represented explicitly rather than disappearing into client code.

64.14 Primary references

- [OpenTelemetry specification](#), accessed 2026-09-19.
- [Kubernetes device plugins](#), accessed 2026-09-19.
- [NVIDIA DCGM documentation](#), accessed 2026-09-19.

Part 17: LLMs on Your Laptop

Chapter 65

CPU and Edge Inference

Status: **Drafted.**

Part 17: LLMs on Your Laptop

65.1 Learning outcome

Measure CPU decode with controlled thread counts and explain memory bandwidth effects.

65.2 Prerequisites

CH-64.

65.3 Edge constraints

Coverage ID: T-65-01.

Edge deployment begins with a budget: usable RAM, memory bandwidth, supported instructions, cores, power, cooling, storage, startup limit, and required context/output rate. The device may be offline, shared with a UI, or limited to a few watts. “Runs” is weaker than “meets sustained latency without exhausting memory or degrading the host.”

Model choice must include artifact size, KV growth, workspace, tokenizer, and application state. Reserve operating-system headroom. A model that uses every byte in a clean benchmark will fail when the real application allocates buffers.

65.4 CPU inference

Coverage ID: T-65-02.

Autoregressive decode repeatedly streams model weights for matrix-vector-like work. At low batch size it is frequently memory-bandwidth-bound. A simple ceiling is effective memory bandwidth divided by bytes read per token. If a 4.5 GB quantized weight image must effectively be read once per token and measured bandwidth is 45 GB/s, the optimistic ceiling is about 10 tokens/s before KV traffic and other overhead.

Prefill has more matrix-matrix reuse and can behave differently. Measure prompt processing and decode separately. Cache state, NUMA placement, memory channels, model quantization, and context length all matter.

The bandwidth division is a roofline-style upper bound, not a prediction. Weight pages may remain in last-level cache for very small models, some tensors may be reused or repacked, and KV reads grow with context. Measure bytes through hardware counters where possible; otherwise call the assumed bytes/token an estimate and reconcile it with resident-set size and observed bandwidth.

65.5 SIMD

Coverage ID: T-65-03.

Single Instruction Multiple Data executes one instruction over vector lanes. x86 AVX2/AVX-512, Arm NEON/SVE, and matrix extensions offer different widths and operations. Quantized kernels unpack values, accumulate into wider types, and apply scales. Alignment, tails, and conversion overhead can reduce theoretical gains.

Binaries should dispatch to features detected at runtime or ship conservative targets. Compiling for an instruction unavailable on the destination gives an illegal-instruction crash, not a graceful slowdown.

65.6 CPU vectorization

Coverage ID: T-65-04.

Useful vectorization depends on contiguous layout and loop structure. Packed quantized blocks should match the kernel's load and dot-product pattern. Compilers may auto-vectorize simple loops, but production runtimes often use architecture-specific kernels because dequantization and horizontal reductions need careful scheduling.

Verify generated instructions or profiler samples. A “vectorized” source expression may compile to scalar work because of aliasing, misalignment, or unsupported dtype conversion.

65.7 CPU threading

Coverage ID: T-65-05.

More threads help until memory bandwidth saturates or overhead dominates. Physical cores, simultaneous multithreading, NUMA nodes, and competing processes change the curve. Thread pools should avoid oversubscription between application, BLAS, and runtime layers.

Benchmark 1, 2, 4, ... threads with affinity controlled. Report median and tail decode latency and system responsiveness. If eight threads equal sixteen, the extra threads may only consume power. On multi-socket systems, first-touch allocation and NUMA-local execution can matter more than thread count.

65.8 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.

- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

65.9 Lab and exit check

Run one pinned quantized model and prompt with controlled context and output. Sweep thread counts, discard warmup, and report prompt tokens/s, decode tokens/s, p95 token latency, memory use, bandwidth counters if available, CPU power/temperature, and host responsiveness. Explain the saturation point with bytes per token.

Repeat one point with threads pinned within a NUMA node and one spanning nodes. Record whether the model pages were first-touched locally; otherwise a topology conclusion may actually be a memory-placement artifact.

65.10 Primary references

- [llama.cpp performance documentation and source](#), accessed 2026-09-19.
- [Intel Intrinsic Guide](#), accessed 2026-09-19.
- [Arm ACLE](#), accessed 2026-09-19.

Chapter 66

Hybrid Memory and Local Loading

Status: **Drafted.**

Part 17: LLMs on Your Laptop

66.1 Learning outcome

Compare cold/warm startup and compute a CPU-GPU placement budget.

66.2 Prerequisites

CH-65.

66.3 GPU offloading

Coverage ID: T-66-01.

A local runtime may place some layers on a discrete GPU and leave the rest on CPU. GPU-resident layers benefit from faster kernels and memory bandwidth; CPU layers avoid exceeding VRAM. Layer count is only a proxy for bytes because embeddings and output heads can be unusually large.

The boundary transfers hidden states. With stable placement, this may occur only at placement transitions per token; with active layer swapping it can occur repeatedly and dominate. Measure the actual transfer path and whether kernels overlap it.

66.4 CPU-GPU hybrid inference

Coverage ID: T-66-02.

Hybrid execution needs a placement plan for weights, KV cache, compute buffers, and graph workspaces. Keeping KV with attention compute avoids repeated cache movement. A 12 GB GPU cannot safely host 11.8 GB of weights if the runtime later needs 2 GB of KV and 1 GB workspace.

A practical planner reserves fixed workspace and safety margin, computes KV growth from model dimensions and maximum context, then places weights in remaining VRAM. Benchmark boundary choices; “maximum layers offloaded” is not always fastest if it triggers paging.

66.5 Unified memory

Coverage ID: T-66-03.

CUDA unified memory presents a shared virtual address space whose pages may migrate between CPU and GPU. It simplifies programming but does not eliminate physical location or transfer cost. Oversubscription can produce page faults and thrashing, especially when the working set alternates.

Prefetch and access-advice mechanisms may help when access is predictable. Treat unified memory as managed placement and measure migrations; do not market it as infinite VRAM.

66.6 Shared memory

Coverage ID: T-66-04.

On integrated systems, CPU and GPU may access one physical memory pool. This avoids discrete PCIe copies but both processors compete for bandwidth and capacity. “Unified” product memory also does not imply equal coherent access costs for every engine.

Reserve memory for the OS and application. Measure sustained GPU inference while the CPU performs realistic tokenization/UI work; isolated accelerator bandwidth numbers overstate the shared workload.

66.7 Quantized formats

Coverage ID: T-66-05.

A quantized artifact stores low-bit weight blocks plus scales and metadata. Format choice must match available kernels. Smaller files can improve loading and weight-stream bandwidth, but dequantization and accuracy differ by scheme, group size, and layer sensitivity.

File size is not runtime memory: mappings, KV cache, compute buffers, and possibly repacked weights add capacity. Compare task quality as well as tokens per second.

66.8 GGUF

Coverage ID: T-66-06.

GGUF is a versioned container used by the ggml ecosystem. It stores named tensors and model/tokenizer metadata and supports multiple tensor quantization types. The container is not itself one quantization algorithm. Runtime compatibility depends on architecture support, metadata expectations, and quantization kernels.

Record converter revision, source-model revision, GGUF version, and per-tensor quantization. An artifact that opens but uses the wrong tokenizer metadata can generate plausible nonsense.

66.9 Memory mapping

Coverage ID: T-66-07.

Memory mapping lets the OS map file pages into virtual memory and load them on demand. Warm starts benefit from page cache; cold starts must read storage. Mapping avoids an eager userspace copy but does not make disk reads free. Random faults can create latency spikes.

Measure after clearing or bypassing cache only when the procedure is documented and safe; otherwise label the run warm. Hashing the full artifact also adds startup I/O but may be required for integrity.

Define “cold” operationally. Dropping the operating-system page cache affects unrelated workloads and may require privilege, while reading a different large file is an unreliable substitute. A defensible report can use a freshly booted test host or direct-I/O staging, record the method, and present both cold and warm distributions rather than one stopwatch result.

66.10 Startup and model loading

Coverage ID: T-66-08.

Separate phases: executable launch, manifest validation, artifact integrity, file map/read, meta-data parse, GPU transfer, kernel initialization, graph capture, warmup, and ready signal. Times-tamp each. Cold, warm, and already-resident paths serve different operational questions.

Use an atomic readiness gate. If loading fails, keep the prior known-good version available or return a clear unavailable state; never expose a partially initialized worker.

66.11 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

66.12 Lab and exit check

For a device with 8 GiB VRAM and 32 GiB RAM, budget a 6 GiB quantized model, workspace, maximum-context KV, and 15% VRAM reserve. Choose CPU/GPU placement. Measure process start to readiness after cold storage and warm page cache, plus first-token latency. Explain every difference.

Verify the plan at the maximum declared context, not only an empty-cache startup. Include a failed-allocation test and show that readiness is withdrawn without corrupting the previous usable model.

66.13 Primary references

- [CUDA Unified Memory programming guide](#), accessed 2026-09-19.
- [GGUF specification](#), accessed 2026-09-19.
- [Linux mmap documentation](#).

Chapter 67

Local Runtimes and Accelerators

Status: **Drafted.**

Part 17: LLMs on Your Laptop

67.1 Learning outcome

Trace a pinned local runtime and identify unsupported-operation fallback.

67.2 Prerequisites

CH-66.

67.3 llama.cpp internals

Coverage ID: T-67-01.

llama.cpp loads model metadata and tensors through ggml/GGUF, builds a computation graph, allocates backend buffers, selects CPU or accelerator kernels, and evaluates prompt and decode graphs. Quantized weights are consumed by specialized kernels; model state includes a KV cache and runtime buffers. Optional server and sampling layers sit above the core graph.

The project changes quickly. Pin a commit or release, compiler flags, backend, artifact hash, context/batch settings, and command. A performance number without these is hard to reproduce.

67.4 OpenVINO

Coverage ID: T-67-02.

OpenVINO converts or reads a model representation, applies graph transformations, compiles it for a selected device, and executes inference requests through plugins. CPU, GPU, and NPU capabilities differ. Model conversion can reshape or fuse graphs, and compiled-model caching can affect startup.

Validate outputs against a reference at representative lengths and decoding settings. Report conversion revision and precision. A successful compile on CPU does not prove the same graph runs entirely on an NPU.

For numerical comparison, first compare logits or fixed-token teacher-forced outputs. Free-running sampled text diverges chaotically after a small logit perturbation and is therefore a poor equality test. Declare absolute/relative tolerances and separately measure task quality for any intentionally lossy precision change.

67.5 NPUs

Coverage ID: T-67-03.

Neural processing units target efficient tensor operations under tight power budgets. They often impose supported operation, shape, datatype, and memory constraints. LLM execution may require static buckets, quantized paths, or CPU participation for tokenization, sampling, and unsupported nodes.

Vendor TOPS is an operation-rate peak at a specified precision, not tokens per second. End-to-end decode includes weight/KV movement, host orchestration, and fallback. Measure power and sustained latency on the actual device.

67.6 Accelerator backends

Coverage ID: T-67-04.

A backend maps a logical graph to CUDA, Metal, Vulkan, SYCL, OpenCL, vendor NPUs, or CPU implementations. Selection can be per tensor or operation. Device discovery, allocation, copies, synchronization, and kernel launch are part of the backend contract.

Reproducible evaluation records driver, runtime, backend build flags, and device identifiers. “GPU enabled” may mean only some layers or operations run there.

67.7 Kernel coverage

Coverage ID: T-67-05.

Kernel coverage is the set of model operations, dtypes, shapes, and layouts the backend implements efficiently. Unsupported work may fail compilation, fall back to CPU, or insert conversions. One fallback inside every transformer block can erase accelerator gains through synchronization and copies.

Trace placement using runtime logs or profiler events and build a table: operation, count per token, chosen device, dtype, fallback reason, transfer bytes, and time. Compare outputs to a CPU reference with tolerances suitable for quantization. The acceptance criterion is complete enough coverage for the pinned model path, not a vendor compatibility logo.

67.8 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

67.9 Lab and exit check

Pin a model artifact and runtime revision. Capture one prefill and five decode steps in logs or a profiler. Produce an operation-placement table and flag every host-device copy or CPU fallback. Deliberately select one unsupported configuration, record the failure/fallback, and verify numerical output against a reference.

The report must distinguish an explicit failure, a documented fallback, and a silent fallback inferred from traces. Treat silent placement changes as a reproducibility defect even when output remains correct.

67.10 Primary references

- [llama.cpp repository](#), accessed 2026-09-19.
- [OpenVINO documentation](#), accessed 2026-09-19.
- [OpenVINO GenAI repository](#), accessed 2026-09-19.

Chapter 68

Packaging, Power and Thermal Limits

Status: **Drafted.**

Part 17: LLMs on Your Laptop

68.1 Learning outcome

Package a reproducible local demo and record sustained latency under thermal load.

68.2 Prerequisites

CH-67.

68.3 Power limits

Coverage ID: T-68-01.

Power caps determine sustainable clocks and therefore latency. A short benchmark may use burst power unavailable over a long session. Measure wall power when possible and device package power when available, stating the tool and sampling interval. Energy per generated token, joules/token, connects performance to battery and cooling.

Lowering a power limit can sometimes improve efficiency with modest latency cost. Compare a curve of tokens/s and joules/token rather than selecting maximum wattage automatically.

Compute energy over a bounded phase: $E = \int_{t_0}^{t_1} P(t) dt$ and joules/token = $E/N_{generated}$. Subtracting idle power answers incremental device energy; retaining it answers wall-energy cost. Report which definition is used, whether display and host power are included, and the uncertainty of low-rate power sensors.

68.4 Thermal limits

Coverage ID: T-68-02.

Heat accumulates until cooling reaches equilibrium. When temperature or power policy limits are reached, clocks fall. Record cold-start performance and a sustained run long enough to stabilize, with temperature, clocks, power, fan state, and token latency over time.

Laptop position, ambient temperature, power adapter, battery state, and fan mode are test conditions. A five-token demo is not evidence for an hour-long workload.

68.5 Model packaging

Coverage ID: T-68-03.

A reproducible package contains the executable/runtime, exact model and tokenizer artifacts or an explicit provision step, configuration schema, licenses, checksums, supported hardware matrix, and diagnostics. Keep model data separate from code when size or licensing demands it, but bind them with a signed manifest.

Avoid embedding writable state inside installation directories. Version logs, caches, conversation data, and models independently. Startup should verify compatibility before allocating scarce memory.

68.6 Offline deployment

Coverage ID: T-68-04.

Offline means every runtime dependency, artifact, trust root, and recovery tool is available without a network. Build on a connected staging system, generate a software bill of materials and hashes, scan, sign, then transfer through the approved channel. Installation should verify signatures and free-space requirements.

Include rollback and uninstall. Logs must avoid secrets and have bounded retention. Exercise installation on a clean offline machine; developer laptops often conceal cached libraries and drivers.

68.7 Edge distillation

Coverage ID: T-68-05.

Distillation trains a smaller student to imitate teacher outputs or internal distributions, often mixing hard labels with soft targets. It can preserve more task capability than simply choosing an untrained small model. The student still needs independent evaluation for quality, calibration, robustness, and domain shift.

Teacher-generated data can carry teacher errors and licensing/privacy constraints. Record prompts, filtering, temperature, objective, and data provenance. Distillation changes learning; quantization changes representation. They may be combined but solve different problems.

68.8 Dynamic edge computation

Coverage ID: T-68-06.

Dynamic computation adapts work to request difficulty or device state: early exit, speculative decoding, retrieval routing, model cascades, context truncation policies, or feature degradation under thermal pressure. A controller needs observable signals and safe bounds.

Optimization must include controller errors. An early-exit confidence threshold can be overconfident out of domain; a cascade can spend more energy after escalation than a single larger call.

Evaluate total quality, latency, energy, and escalation rate by workload slice. During thermal stress, prefer explicit degraded modes over unpredictable throttling.

68.9 Four recurring perspectives

- **Follow the Token:** trace the representation and its ownership across the execution boundary.
- **Follow the Gradient:** identify synchronized or sharded learning state; for inference-only paths, state explicitly that no gradient exists.
- **Follow the Byte:** calculate persistent state, temporary buffers, transfers, and the relevant bandwidth tier.
- **Follow the Request:** include admission, scheduling, cancellation, failure, and externally visible latency.

68.10 Lab and exit check

Create an offline package with manifest, hashes, configuration, license inventory, diagnostic command, rollback, and clean uninstall. On a clean target, measure startup, first token, steady decode, peak memory, package power, temperature, clocks, and latency for at least 30 minutes. Compare two power modes and document thermal steady state. Validate one corrupt-artifact and one interrupted-install recovery.

Acceptance requires that the package installs with networking disabled and an empty application cache. Re-run the artifact hash and license inventory from the transferred bundle rather than trusting the staging machine's manifest.

68.11 Primary references

- [NIST Secure Software Development Framework](#), accessed 2026-09-19.
- Hinton et al., [Distilling the Knowledge in a Neural Network](#), 2015.
- [SPDX specification](#), accessed 2026-09-19.

Part 18: Retrieval, Memory and Agents

Chapter 69

Embeddings, Contrastive Learning and Vector Indexes

Status: **Draft — not yet independently reviewed.**

An embedding system turns a search problem into geometry. Its success depends on three contracts: the encoder must place useful items near one another, the index must retrieve those neighbors efficiently, and the application must know what “near” means for its users.

69.1 Embeddings revisited

Given an encoder f , an item x becomes a vector $z = f(x) \in \mathbb{R}^D$. Similarity is commonly dot product $z_q^T z_d$, cosine similarity, or negative Euclidean distance. If vectors are unit-normalized, ranking by cosine and squared Euclidean distance is equivalent because $\|q - d\|^2 = 2 - 2q^T d$. That algebra does not make the representation semantic by decree: the training data and objective decide which distinctions the space preserves.

69.2 Sentence embeddings

A sentence encoder returns one fixed-width vector for a variable-length sentence. Mean pooling contextual token states is a strong, transparent baseline; a learned pooling token is another choice. A bi-encoder embeds queries and candidates separately, enabling offline candidate indexing. Its speed comes from compressing each text before the pair is known, so it can miss fine token-to-token interactions. A cross-encoder reads the pair jointly and is usually slower but more expressive.

For “reset the pump controller” and “restart pump control,” a domain encoder may produce nearby vectors. “Reset the account password” shares a verb but should remain farther away. Test such hard negatives rather than admiring a two-dimensional projection.

69.3 Document embeddings

Long documents rarely fit cleanly into one vector. Truncation discards evidence; pooling thousands of tokens can dilute a decisive paragraph. Common designs embed chunks, store several vectors per document, or build hierarchical document and passage vectors. Retrieval should preserve the document ID, section path, timestamps, permissions, and character offsets so the

original evidence can be reconstructed. A document embedding is one record in a retrieval data model, not a replacement for the document.

69.4 Contrastive embedding training

For a batch of matched query-document pairs (q_i, d_i) , a standard in-batch contrastive loss is

$$L_i = -\log \frac{\exp(s(q_i, d_i)/\tau)}{\sum_j \exp(s(q_i, d_j)/\tau)},$$

where s is similarity and τ is temperature. Gradients pull the positive pair together and push batch negatives away. This works only when negatives are meaningful. False negatives teach the model to separate valid matches; trivial negatives produce little learning. Hard-negative mining improves discrimination but can amplify annotation errors. Sentence-BERT demonstrated efficient sentence retrieval with siamese encoders; DPR showed the same basic pattern for open-domain passage retrieval.

The denominator shown uses documents in the batch as negatives for each query. A symmetric objective also predicts queries from documents, but it is a different loss and should be stated. Deduplicate near-identical items before batch construction and keep documents from one source entity in one data split; otherwise near-duplicate passages can become false negatives during training and leakage during evaluation.

69.5 Vector search

Exact search computes similarity to all N vectors, costing roughly $O(ND)$ operations per query. For one million 768-dimensional FP32 vectors, the raw matrix is about 3.07 GB decimal, before IDs and index overhead. A matrix multiplication can make exact search surprisingly competitive at modest scale and high batch size. Always measure it as the quality reference.

69.6 Approximate nearest neighbors

Approximate nearest-neighbor (ANN) methods visit or decode only part of the collection. They trade recall for latency, memory, construction time, and update behavior. Report $\text{recall}@k$ against exact neighbors, queries per second at stated concurrency, p50/p95 latency, index size, build time, and filtering conditions. “Fast ANN” without the achieved recall is an incomplete claim.

69.7 HNSW

Hierarchical Navigable Small World indexes organize vectors in a multi-layer proximity graph. Search begins in sparse upper layers, greedily approaches the query, then explores a candidate set in denser lower layers. Larger construction and search breadth parameters generally improve recall while consuming time and memory. HNSW supports incremental inserts well, but graph links and per-node metadata can be expensive. Deletes, filters, and changing embedding distributions require operational testing; the elegant graph sketch does not settle those costs.

69.8 IVF

An inverted-file index first assigns each vector to a coarse centroid. At query time it probes the nearest n probe lists and searches only their members. More lists reduce candidates per list; more

probes recover recall at added cost. Coarse quantizer training must represent production data. Distribution shift can create overloaded or poorly placed cells, so list-size histograms and recall by cohort matter.

69.9 Product quantization

Product quantization splits a D -dimensional vector into m subvectors and replaces each with a codebook ID. With 256 centroids per subspace, each ID takes one byte, so a vector becomes roughly m bytes plus codebooks. Distance tables let search compare compressed codes cheaply. Compression introduces distortion; IVF-PQ often retrieves a candidate set using compressed distances and reranks a smaller set with full vectors. Measure end-task quality because nearest-neighbor recall alone may hide losses on the evidence the generator needs.

69.10 Vector databases

A vector database combines an index with persistence, metadata filtering, updates, replication, access control, and query APIs. Those system properties can dominate algorithm choice. Filtering after ANN may return too few authorized results; filtering before search may fragment the index. Store the encoder name and revision with every vector, because vectors from incompatible embedding spaces are not safely comparable. Re-embedding is a data migration with dual-write, backfill, validation, and rollback concerns.

69.11 Four perspectives

- **Follow the Token:** token states are pooled into one or more vectors; ANN returns IDs that lead back to original token spans.
- **Follow the Gradient:** contrastive gradients shape neighborhoods; index construction and query execution themselves use no gradient.
- **Follow the Byte:** exact FP32 storage is $4ND$ bytes; graph edges, codes, metadata, and replicas add overhead.
- **Follow the Request:** encode query, apply authorization filters, search, optionally rerank, fetch source spans, and record retrieval telemetry.

69.12 Lab and exit check

Embed a labelled corpus and compare exact search with HNSW or IVF at several search breadths. Record recall@10, p50/p95 latency, memory, build time, and one semantic failure. Explain why cosine and Euclidean rankings agree only after normalization. A successful demo does not establish robustness under new domains, deletions, or permission filters.

Repeat the comparison with metadata filtering and with 10% of the corpus replaced by newly embedded documents. Report whether recall, latency, and index size remain stable after updates; a static index benchmark does not establish operational update quality.

69.13 References

- Reimers and Gurevych, [Sentence-BERT](#), EMNLP-IJCNLP 2019.
- Karpukhin et al., [Dense Passage Retrieval](#), EMNLP 2020.
- Malkov and Yashunin, [Efficient and Robust Approximate Nearest Neighbor Search Using HNSW](#), IEEE TPAMI 2020.

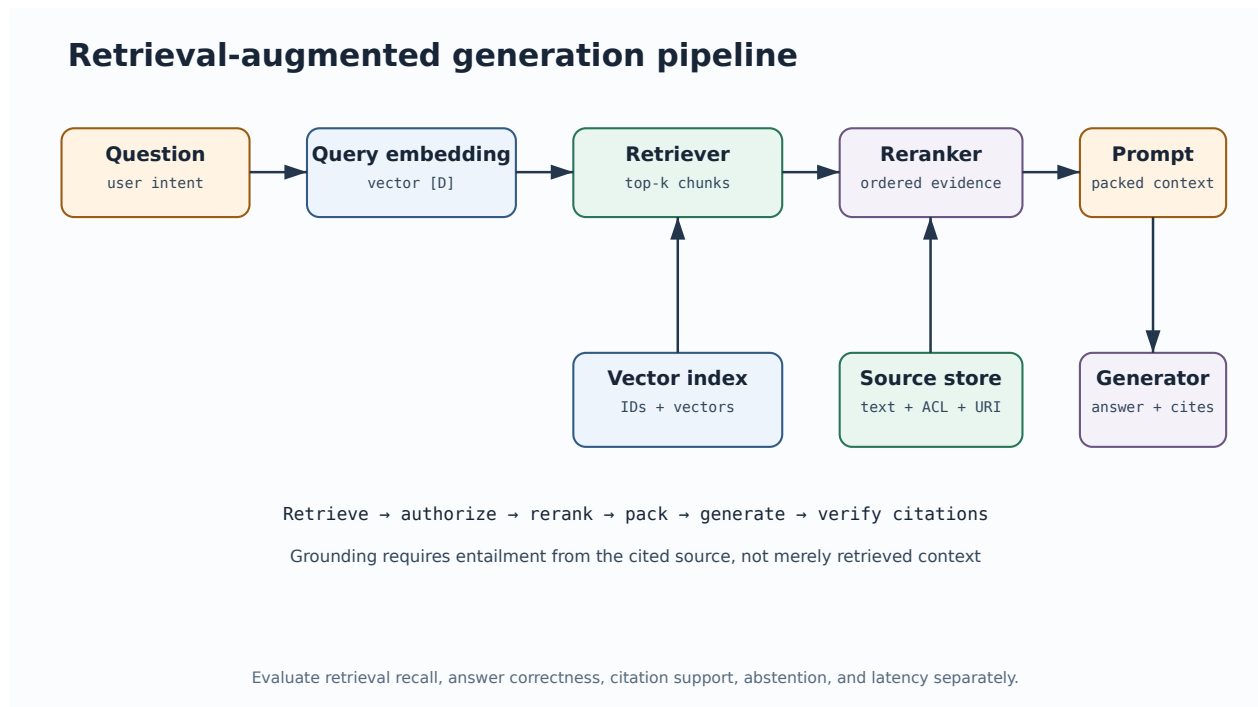


Figure 69.1: Retrieval-augmented generation pipeline.

- Jégou, Douze, and Schmid, [Product Quantization for Nearest Neighbor Search](#), IEEE TPAMI 2011.
- Johnson, Douze, and Jégou, [Billion-scale similarity search with GPUs](#), IEEE Transactions on Big Data 2021.

Chapter 70

RAG and Retrieval Quality

Status: **Draft — not yet independently reviewed.**

Retrieval-augmented generation (RAG) gives a model evidence at request time. It does not guarantee that the evidence is relevant, that the model will use it correctly, or that the resulting answer is true. Treat retrieval and generation as separately measurable stages.

70.1 RAG

A basic pipeline transforms query q into retrieval query q' , retrieves candidates D_k , constructs prompt $p(q, D_k)$, and generates answer y . Lewis et al. introduced RAG as a learned combination of parametric and retrieved non-parametric memory; production systems also use simpler frozen retrievers and prompt assembly. The practical gain is updateability and source visibility. The price is another failure surface: ingestion, indexing, query formulation, retrieval, context packing, and generation can each fail.

70.2 Chunking strategies

Fixed token windows are reproducible but can split tables or procedures. Overlap protects boundary evidence but duplicates tokens and can crowd the context. Structure-aware chunking follows headings, paragraphs, code symbols, or document sections. Semantic segmentation attempts to split on topic shifts but adds model cost and instability. Preserve parent relationships so a matched paragraph can be expanded with its heading or neighboring steps. Choose chunk size through evaluation, not a universal folklore number.

Suppose an answer requires a warning in one paragraph and a limit in the next. Independent 200-token chunks may retrieve only one. Parent-child retrieval can search small passages, then supply the containing section. The added context helps only if it does not displace more valuable evidence.

70.3 Retrieval strategies

Sparse retrieval such as BM25 rewards exact lexical overlap and handles identifiers well. Dense retrieval can match paraphrases. Hybrid retrieval combines their scores or rankings, often with reciprocal-rank fusion. Query rewriting, multi-query retrieval, hypothetical-document queries, and decomposition can expand recall, but each adds latency and may drift from intent. For

domain systems, metadata constraints—product version, date, asset, language, permission—often improve relevance more reliably than another LLM call.

70.4 Reranking

A two-stage design retrieves perhaps 50–200 candidates cheaply, then applies a cross-encoder or late-interaction model to a smaller set. Reranking improves fine-grained relevance because it evaluates query and passage together. It cannot recover an omitted candidate. Measure first-stage recall at the reranker cutoff, reranker quality, and total latency. ColBERT’s late interaction retains token-level vectors and uses maximum-similarity aggregation, occupying a middle ground between single-vector search and a full cross-encoder.

70.5 Retrieval evaluation

Build a judgement set of queries with relevant passages and explicit “no answer” cases. Useful retrieval metrics include $\text{recall}@k$, $\text{precision}@k$, mean reciprocal rank, and nDCG. Evaluate slices: exact identifiers, paraphrases, temporal questions, multi-hop questions, and permission-filtered queries. Then evaluate the complete answer with evidence attribution, correctness, completeness, abstention, and latency. A generator can answer from memorized parameters even when retrieval failed, masking a broken retriever.

Define relevance and denominators before computing metrics. $\text{Recall}@k$ is the fraction of judged relevant items retrieved in the first k ; $\text{success}@k$ merely asks whether at least one relevant item appeared. For questions requiring two independent passages, $\text{success}@k$ can look perfect while evidence recall is only one half. Pool judgements beyond one system’s results so unjudged candidates are not automatically treated as irrelevant.

An intervention test makes attribution clearer: remove the decisive passage, replace it with a conflicting passage, or shuffle irrelevant passages. If the answer never changes, claimed grounding is doubtful. If it follows malicious text blindly, instruction hierarchy is weak.

70.6 Grounding

Grounding means the response’s externally checkable claims are supported by supplied evidence. Citation presence is insufficient; the cited span must entail the claim. Require stable source IDs and offsets, distinguish quotation from inference, and make abstention valid when evidence is missing. Retrieved documents are untrusted input: they can contain obsolete content, prompt injection, or data the user may not access. Enforce access before retrieval output reaches the model and separate document text from system instructions.

70.7 Four perspectives

- **Follow the Token:** source text becomes chunks, retrieved chunks become prompt tokens, and citations map generated claims back to spans.
- **Follow the Gradient:** a frozen RAG pipeline has none at request time; retriever or generator fine-tuning changes which evidence is selected or used.
- **Follow the Byte:** overlap enlarges indexes and prompts; reranking adds activation and model-weight traffic.
- **Follow the Request:** trace ingestion version, rewritten query, candidates, scores, filters, packed context, answer, and citations as one observable record.

70.8 Lab and exit check

Create at least 30 answerable and 10 unanswerable queries over a small corpus. Compare lexical, dense, and hybrid retrieval; then add reranking. Report retrieval recall, answer correctness, citation support, abstention, and latency. Diagnose one failure as ingestion, retrieval, packing, or generation. Do not label an answer grounded merely because it sounds specific.

Add one stale-document case and one unauthorized-but-highly-relevant document. The expected result is use of the current authorized source, not merely retrieval of the semantically closest passage.

70.9 References

- Lewis et al., [Retrieval-Augmented Generation](#), NeurIPS 2020.
- Robertson and Zaragoza, [The Probabilistic Relevance Framework: BM25 and Beyond](#), 2009.
- Khattab and Zaharia, [ColBERT](#), SIGIR 2020.
- Thakur et al., [BEIR](#), ICLR 2021.
- Craswell et al., [Overview of the TREC 2020 Deep Learning Track](#), TREC 2020.

Chapter 71

Memory, Tools and Structured Output

Status: **Draft — not yet independently reviewed.**

An application’s memory is stored state selected for a future request. A model’s weights and KV cache are also forms of state, but they solve different problems. Conflating them produces systems that remember the wrong facts forever and forget the right ones at session boundaries.

71.1 Short-term memory

Short-term memory is the active conversation state: recent messages, tool results, and a working summary. Sending the entire transcript increases cost and can bury the current instruction. Sliding windows discard old details; summarization compresses them with possible loss; structured state extracts fields such as goal, constraints, and unresolved decisions. Keep provenance so a summary can distinguish user statements from model inference.

71.2 Persistent memory

Persistent memory survives sessions. It may contain explicit preferences, approved project facts, or task checkpoints. Persistence requires a schema, authorization, retention policy, and user-visible correction path. Store confidence, source, created time, last-confirmed time, and scope. “User prefers Python” should not silently become a permanent global rule because one task used Python.

71.3 Long-term memory

Long-term retrieval selects a small subset of persistent records for the current query. Similarity alone is insufficient: recency, importance, entity, project, validity interval, and access control matter. Episodic records describe events; semantic records describe consolidated facts; procedural records describe approved workflows. Consolidation can merge repetition but must preserve disagreements rather than average them into fiction.

71.4 Tool calling

A tool call is a typed request from the model to external code. A safe loop exposes names, descriptions, and argument schemas; validates proposed arguments; checks authorization; executes; returns a structured result; and lets the model continue. The model proposes actions but the application owns policy. Tool outputs are untrusted data and may contain instructions that should not override the system. Use idempotency keys for retried mutations and distinguish read operations from writes.

Idempotency needs a stable operation identity and stored outcome, not just a random key on each retry. If the connection drops after a charge succeeds, replay with the same key and arguments must return the original result or a conflict; creating a fresh key can charge twice. Exactly-once effects generally require cooperation from the external system, so document whether the application provides at-most-once attempts, idempotent replay, or reconciliation.

71.5 Structured outputs

Structured output asks the model to produce data matching a schema, such as `{query, filters, top_k}`. Schema validation catches missing fields, invalid enums, and type errors; it cannot establish that values are factually correct. Version schemas and prefer semantic field names. Optional fields need defined defaults, while unions need a discriminating tag. Parse failures should produce bounded repair or a clear error, not an infinite “please fix JSON” loop.

Validate semantic and security constraints after schema validation: `top_k` can be an integer yet exceed the service limit, and a syntactically valid path can escape an allowed directory. Bind identity and authorization from trusted application state rather than accepting model-produced tenant or user IDs.

71.6 Constrained decoding

Constrained decoders mask tokens that would make the output impossible under a grammar or schema. At each step, valid next tokens depend on the parser state; logits for invalid continuations are excluded before sampling. This can guarantee syntactic validity under supported tokenization and grammar semantics. It does not guarantee business validity: a date may be well formed but nonexistent, and an account ID may belong to another user. Validate after decoding too.

71.7 Memory invalidation

Memory becomes dangerous when facts change. Use explicit expiry for volatile data, tombstones for deletion, version or entity keys for supersession, and contradiction detection for new evidence. Retrieval should prefer current authoritative records while retaining audit history where policy permits. When a user corrects a fact, invalidate derived summaries that depended on it. Embedding deletion must propagate through replicas, caches, backups, and indexes according to the product’s retention contract.

71.8 Four perspectives

- **Follow the Token:** messages are compressed into summaries or records, selected records return as context, and constrained tokens form typed output.
- **Follow the Gradient:** ordinary memory and tool execution do not change weights; fine-tuning on memories would create a harder-to-delete channel.

- **Follow the Byte:** raw transcripts, vector copies, indexes, and tool payloads have separate storage and retention costs.
- **Follow the Request:** retrieve scoped state, construct prompt, validate tool proposal, authorize, execute, record result, and update only approved memory.

71.9 Lab and exit check

Implement a three-tool loop with JSON Schema validation, one read tool, one idempotent write tool, and one intentionally failing tool. Add persistent records with provenance and expiry. Test malformed JSON, unauthorized arguments, duplicate writes, prompt injection in a tool result, and correction of a stored fact. Explain which guarantees come from decoding, validation, authorization, and execution.

Simulate a lost response after the write commits, then retry. The test passes only if the effect occurs once and the client receives a deterministic prior result or a reconcilable status.

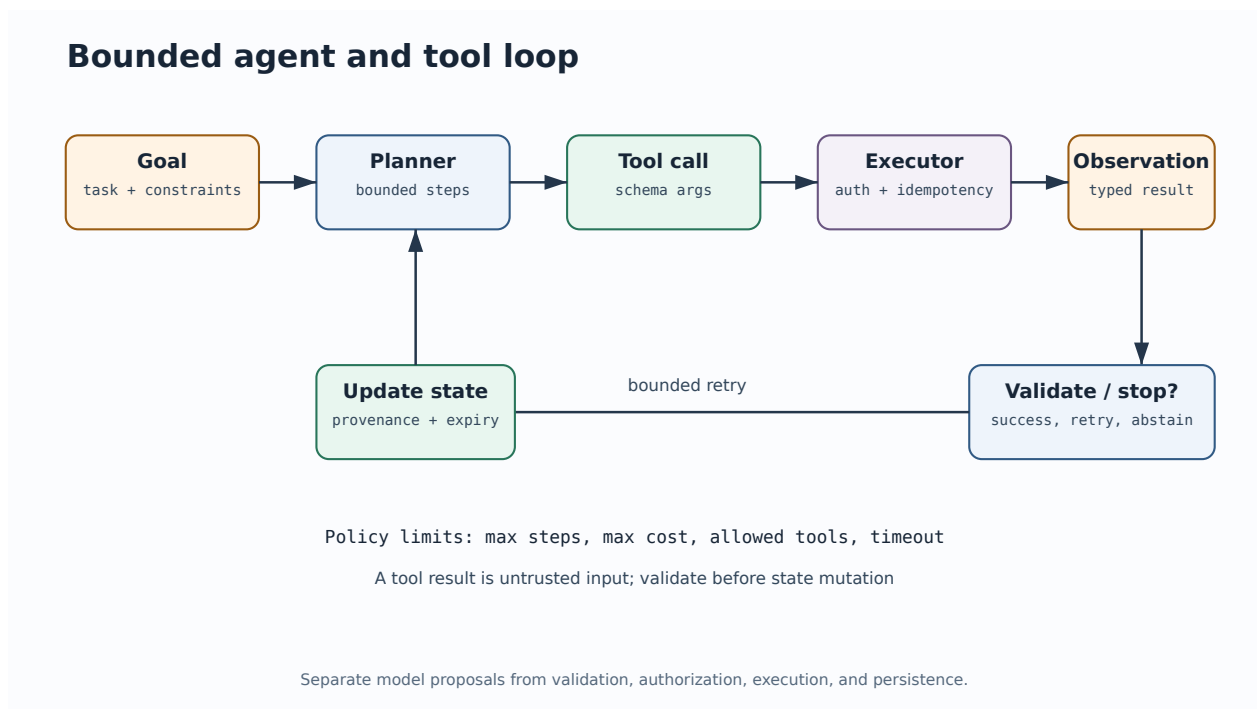


Figure 71.1: Bounded agent and tool-execution loop.

71.10 References

- Schick et al., [Toolformer](#), NeurIPS 2023.
- Poesia et al., [Synchromesh](#), ICLR 2022.
- [JSON Schema 2020-12 specification](#), accessed 2026-09-19.

Chapter 72

Agents and Multi-Model Economics

Status: **Draft — not yet independently reviewed.**

An agent is a control loop that lets a model observe state, choose an action, receive a result, and continue toward a goal. The loop—not a special kind of intelligence—is the defining engineering object. Its value must exceed the cost and new failure modes introduced by extra calls.

72.1 Agents

A minimal agent has state s_t , available actions $A(s_t)$, a policy that proposes a_t , an environment transition, and a termination rule. State may include messages, files, tool results, budgets, and a task graph. Restrict actions by capability and validate each call. Termination needs success criteria, maximum steps, cost limits, and stuck detection. Otherwise a harmless ambiguity becomes a very expensive while-loop.

72.2 Planning

Planning decomposes a goal into intermediate actions. ReAct interleaves reasoning traces and actions; other systems keep an explicit task graph or replan after observations. Plans help with long dependencies but can become stale after the first tool result. Use them as revisable state, record observable rationales such as assumptions and chosen actions, and avoid depending on private chain-of-thought as a system interface. Verify completion from environment state rather than the model's declaration.

72.3 Multi-agent systems

Multiple agents can specialize, debate, or execute tasks in parallel. They also duplicate context, propagate the same misconception, and add coordination overhead. Independent agents provide value when tasks are separable or evidence sources are diverse. A critic is useful only if it has a rubric and permission to reject; agreement among correlated models is weak evidence. Compare against one agent with equal total tokens, tools, and latency budget.

72.4 Multi-LLM systems

Different models can serve roles based on capability, latency, context length, modality, deployment location, or cost. Interfaces should pass typed artifacts—retrieval results, patches, test reports—rather than vague prose whenever possible. Model versions change, so record provider, model revision, decoding settings, prompt version, and tool schema. Cross-model escalation is a routing decision, not proof that the second model will repair the first.

72.5 Agent compute economics

For path r , expected request cost can be written

$$E[C] = \sum_r P(r)(C_{tokens,r} + C_{tools,r} + C_{compute,r} + C_{failure,r}).$$

Latency requires a critical-path calculation: parallel calls add the maximum branch latency plus coordination, while sequential calls add. Include retries, cache misses, human review, and failure remediation. Quality should be measured at a fixed budget, and cost at a fixed quality. Seven calls that equal one strong call are an orchestration tax, not a feature.

For a simple cascade, if the cheap path costs 1 unit, escalation costs another 8, and 25% of requests escalate, expected model cost is $1 + 0.25 \cdot 8 = 3$ units before tools and failures. Its escalated latency is sequential, not the probability-weighted average experienced by tail users. Publish the full path distribution so a low mean does not hide an expensive or slow cohort.

72.6 Routing and fallback

A router can use simple rules, a learned classifier, uncertainty signals, or staged escalation. Start with auditable rules and a strong single-model baseline. Evaluate routing accuracy, end-task quality, escalation rate, cost, tail latency, and failures by cohort. Fallbacks need bounded retries and must preserve state safely. A cheaper model timing out before every expensive fallback increases both cost and latency.

Use shadow evaluation before sending live traffic: replay a representative set through candidate routes and log counterfactual outcomes. Online adaptation requires guardrails because feedback is selective—only routed paths reveal their outcomes.

72.7 Four perspectives

- **Follow the Token:** each handoff serializes context; repeated summaries can lose constraints or multiply prompt tokens.
- **Follow the Gradient:** most orchestrators route frozen models; learned routers receive gradients from labelled outcomes or proxy rewards.
- **Follow the Byte:** model residency, KV caches, network payloads, and duplicated context determine whether parallelism is practical.
- **Follow the Request:** the trace must show route, tools, retries, budgets, termination reason, and final verification.

72.8 Lab and exit check

Build a router with a cheap path, a strong path, one tool, bounded retry, and explicit abstention. Compare it with the strong model alone under matched quality and workload. Report success,

cost, p50/p95 latency, calls per task, escalation, and failure recovery. Add an ablation that removes the planner or critic. Keep the component only if its marginal value survives that comparison.

Evaluate on frozen tasks with sandboxed side effects. Re-running a benchmark that mutates files, tickets, or accounts can change later trials; reset the environment or score from immutable snapshots so route order is not a confounder.

72.9 References

- Yao et al., [ReAct](#), ICLR 2023.
- Shinn et al., [Reflexion](#), NeurIPS 2023.
- Chen, Zaharia, and Zou, [FrugalGPT](#), 2023.

Part 19: Inside the Model: Representations and Intervention

Chapter 73

Hidden States, Layers and Attention Heads

Status: **Draft — not yet independently reviewed.**

A transformer exposes many tensors that correlate with behavior. Looking at them can generate hypotheses, but a colorful plot is not yet an explanation. This chapter establishes what is actually measured before later chapters intervene on it.

73.1 Hidden states

For batch B , sequence length T , and model width D , a decoder layer produces hidden states $H^{(\ell)} \in \mathbb{R}^{B \times T \times D}$. The vector at a position is contextual: it depends on earlier tokens under causal masking. Coordinates do not arrive with human labels. A direction or distributed pattern can carry information even when no individual coordinate is interpretable.

Unembedding an intermediate state with the final output matrix creates a “logit lens”: $H^{(\ell)} W_U$. It is a useful diagnostic, but earlier states may not inhabit the distribution expected by the final normalization and unembedding. Tuned lenses learn translations to reduce this mismatch, adding parameters and another source of interpretation.

For architectures with a final LayerNorm or RMSNorm, a faithful naive lens usually applies that published final normalization before W_U ; omitting it changes scale and sometimes ranking. Even with it, the lens asks what the current state would predict through the final readout, not what the remaining layers will actually compute.

73.2 Residual stream analysis

Each attention and MLP sublayer writes an update into a shared residual stream. In a simplified pre-norm block, $h' = h + A(N(h))$ and $h'' = h' + M(N(h'))$. This additive structure permits decomposition of a final logit into contributions from components under stated linearization and normalization assumptions. It does not mean components operate independently: later normalization and nonlinear modules respond to the sum.

Record where states are taken—before or after normalization, attention, MLP, or residual addition. “Layer 12 activation” is otherwise ambiguous.

73.3 Representation geometry analysis

Geometry tools compare distances, angles, subspaces, covariance, and neighborhood structure. Centered Kernel Alignment (CKA) compares representations while tolerating some transformations; canonical correlation methods compare correlated subspaces. PCA finds high-variance directions, which need not be task-causal. Results depend on token sampling, centering, normalization, prompt distribution, and layer location. A two-dimensional projection is a lossy view and should carry explained-variance or neighborhood-preservation information.

73.4 What individual layers learn

Layerwise decodability studies often find changing accessibility of lexical, syntactic, or task information. The careful claim is “a specified probe decoded label y from states at layer ℓ on dataset D .” It is stronger than raw correlation and weaker than “the layer computes concept y .” Information can be present but unused, or a powerful probe can learn the task itself. Compare with controls, random labels, simple probes, and baselines from embeddings.

73.5 Layer specialization

Specialization can mean a layer is necessary for a behavior, uniquely informative, or merely more linearly decodable. These definitions are not interchangeable. Test necessity with ablation or replacement, sufficiency by transplanting or activating the component, and specificity across tasks. Distributed computation makes sharp boundaries unlikely: removing one layer may be repaired by redundant paths, while deleting many layers can shift normalization and create out-of-distribution states.

73.6 Attention-head analysis

An attention head produces weights $P = \text{softmax}(QK^T / \sqrt{d_h} + M)$ and output PVW_O . Weight maps show routing probabilities, not the full signed contribution to logits. A large weight can carry a small value vector; a modest weight can matter through a large output projection. Analyze patterns over a dataset, then test head outputs with ablation, patching, or path-specific interventions. Induction-head research provides a concrete example of combining pattern discovery with mechanistic tests, but findings in small models do not automatically generalize to every architecture.

73.7 Worked measurement protocol

Choose a frozen model revision and 500 prompts representing both target and control behaviors. Capture states at a precisely named hook, fit a linear probe on a train split, tune on validation, and report held-out accuracy beside a majority baseline and a probe on shuffled labels. Then replace the candidate component with its mean activation or patch it from a counterfactual prompt. If decoding stays high while behavior is unchanged under intervention, the representation is readable but not shown causal.

Split by source template, entity, or document rather than by individual token when those units share lexical cues. Otherwise the probe may memorize correlated examples across the split. Fit every preprocessing transform—including centering and PCA—on the training split only.

73.8 Four perspectives

- **Follow the Token:** each position accumulates contextual updates across layers and is eventually projected to logits.
- **Follow the Gradient:** probes may learn with frozen model states; their gradients train the probe, not evidence that the base model uses the decoded feature.
- **Follow the Byte:** capturing every layer costs $BTDL$ elements and can dwarf ordinary inference memory.
- **Follow the Request:** instrumentation changes latency and storage; sample or stream activations with privacy and retention controls.

73.9 Lab and exit check

Capture three layer locations for a small open model. Compare a logit lens, linear probe, and component ablation on held-out prompts. Report shapes, hook definitions, baselines, uncertainty, and one case where a correlational signal fails a causal test. Explain why attention weights alone are not feature importance.

73.10 References

- Elhage et al., [A Mathematical Framework for Transformer Circuits](#), 2021.
- Belinkov, [Probing Classifiers](#), Computational Linguistics 2022.
- Kornblith et al., [Similarity of Neural Network Representations Revisited](#), ICML 2019.
- Olsson et al., [In-context Learning and Induction Heads](#), 2022.

Chapter 74

Features, Probes and Mechanisms

Status: **Draft — not yet independently reviewed.**

Mechanistic interpretability asks for executable accounts of how model components produce behavior. Its evidence ladder runs from observation, through prediction, to intervention. Each rung rules out more alternative explanations.

74.1 Neurons

A neuron is one scalar coordinate after a learned affine transformation and nonlinearity. Inspecting the inputs that maximize it can reveal a pattern, but polysemantic neurons may respond to unrelated contexts. Dataset search also misses rare activations. A neuron’s effect depends on its outgoing weights and the rest of the network; high activation alone does not establish importance.

74.2 Features

A feature is a hypothesized direction or pattern that represents a property relevant to computation. Features need not align with neurons. Superposition proposes that networks can represent more features than dimensions by using non-orthogonal directions when features are sparse. This is a useful model supported by toy and empirical work, not a licence to assign a clean concept name to every direction.

74.3 Probing

A probe predicts labels from frozen activations. Linear probes ask whether labels are linearly accessible; nonlinear probes blur representation and probe capability. Use selectivity controls, limited capacity, held-out data, and minimum-description-length or sample-efficiency analyses where appropriate. A successful probe establishes decodability under its setup. Causal use requires intervention.

74.4 Sparse autoencoders

A sparse autoencoder learns encoder $z = \phi(W_e h + b_e)$ and decoder $\hat{h} = W_d z + b_d$, balancing reconstruction error with sparsity. The hope is that sparse latent units separate features entangled in dense activations. Evaluate reconstruction loss, sparsity, dead latents, feature stability, and downstream behavior under reconstruction. Human labels assigned from top activations can be

incomplete or misleading. SAE features are learned model artifacts, not ground-truth atoms of thought.

Dictionary features are not automatically identifiable: different seeds, widths, penalties, or activation samples can produce different bases with similar reconstruction. Match features across runs and report stability, but do not treat matching failure as proof that no useful feature exists. Compare against PCA, random dictionaries, and reconstruction-matched baselines to isolate what sparsity contributes.

74.5 Mechanistic interpretability

A mechanism is a compact account connecting inputs, internal operations, and outputs. One workflow is: localize a behavior, identify candidate components, form a circuit hypothesis, intervene, measure specificity, and test on new examples. Faithfulness asks whether the proposed mechanism captures the model’s actual computation; completeness asks how much behavior it explains. Circuits can be approximate and context-dependent.

74.6 Causal interventions

Activation patching runs a clean and corrupted prompt, then replaces a selected corrupted activation with its clean counterpart. Recovery of the target metric localizes a causally relevant site under that corruption. Ablation replaces a component with zero, mean, resampled, or another reference value; each choice creates different distribution shifts. Path patching attempts to isolate routes between components. Interventions should include negative controls, matched prompts, multiple seeds, and tests for collateral behavior.

Report both absolute recovery and a normalized score such as $(m_{\text{patched}} - m_{\text{corrupt}})/(m_{\text{clean}} - m_{\text{corrupt}})$ when the denominator is safely nonzero. Values above one or below zero are possible and should not be clipped; they reveal overshoot or harm. Define the metric sign before inspecting patches.

Suppose changing a subject token flips a factual answer. Patch each layer-position residual state from the clean run into the corrupted run and measure logit-difference recovery. A peak identifies a location worth deeper study. It does not prove the fact “lives” there: the patch may restore an upstream signal used elsewhere.

74.7 Evidence ladder

1. **Observation:** component activates on selected examples.
2. **Association:** signal predicts behavior on held-out data.
3. **Necessity:** removing it degrades the behavior.
4. **Sufficiency:** inserting it induces or restores behavior.
5. **Mechanism:** a compositional account predicts novel interventions.

Even necessity can be obscured by redundancy, and sufficiency can reflect out-of-distribution forcing. Report the precise level reached.

74.8 Four perspectives

- **Follow the Token:** candidate features emerge at positions and travel through residual paths to output logits.
- **Follow the Gradient:** probes and SAEs learn mappings from frozen states; causal patching uses forward execution without model training.

- **Follow the Byte:** activation datasets and SAE training can require far more storage than the base weights.
- **Follow the Request:** instrumentation needs deterministic prompt pairs, hook versions, metrics, and replayable intervention traces.

74.9 Lab and exit check

Define one narrow behavior and build clean/corrupted prompt pairs. Compare a linear probe, activation patching, and ablation. Include a shuffled-label control and a non-target behavior. State the strongest justified conclusion using the evidence ladder. If the intervention damages all outputs, it has not demonstrated specificity.

74.10 References

- Alain and Bengio, [Understanding Intermediate Layers Using Linear Classifier Probes](#), 2016.
- Hewitt and Liang, [Designing and Interpreting Probes with Control Tasks](#), EMNLP-IJCNLP 2019.
- Meng et al., [Locating and Editing Factual Associations in GPT](#), NeurIPS 2022.
- Bricken et al., [Towards Monosemanticity](#), 2023; empirical research report, not peer-reviewed conference evidence.

Chapter 75

Steering, Editing and Representation Engineering

Status: **Draft — not yet independently reviewed.**

Intervening on representations can change behavior without retraining every parameter. That makes these methods attractive and risky: a small vector or weight edit may work on a demonstration while shifting unrelated behavior elsewhere.

75.1 Activation steering

A steering vector is often built from a contrast, such as the mean activation difference between positive and negative prompt sets at layer ℓ :

$$v = \mathbb{E}[h^{(\ell)} \mid P^+] - \mathbb{E}[h^{(\ell)} \mid P^-].$$

During generation, replace $h^{(\ell)}$ with $h^{(\ell)} + \alpha v$ at selected positions. Sweep α rather than reporting one flattering value. Strong coefficients can damage fluency, overwhelm prompt intent, or trigger unrelated features. Controls include random directions of matched norm, reversed vectors, alternative prompt templates, and evaluation on non-target tasks.

Name the hook precisely: adding before normalization, after attention, or after the residual update are different interventions. State whether v is normalized and whether its norm is comparable to the residual stream at that layer. Applying a fixed vector to every prompt and generated token is also different from steering only the final prompt position.

75.2 Representation engineering

Representation engineering treats population-level internal patterns as objects for measurement and control. Choices include which layer and token to read, how to construct contrasts, whether to whiten or normalize, and whether to apply a linear or learned controller. A difference-of-means direction is interpretable and cheap; PCA on contrast activations can capture a broader subspace; trained controllers are more expressive but risk dataset shortcuts.

Generalization across topics, phrasing, languages, sequence positions, and decoding policies matters more than separation on the construction set. Report the data used to discover the direction separately from data used to evaluate it.

75.3 Model editing

Model editing changes parameters to alter targeted associations or behavior. Rank-one model editing (ROME) updates selected MLP weights using a derived key-value association; MEMIT extends editing to many memories. Other methods fine-tune a small subset or use an external memory rather than altering weights. Evaluation should distinguish reliability on the exact edit, generalization to paraphrases, locality on unrelated prompts, portability to related implications, and retention after subsequent edits.

An edit that makes “X was born in Y” answer correctly but breaks other subjects is not local. An edit that works only for the training wording has not generalized. Repeated edits can interfere even when each single edit looks safe.

75.4 Intervention side effects

Side effects arise because representations and weights participate in many computations. Steering may change style, certainty, refusal, or token frequency alongside the target. Weight edits may alter semantically neighboring facts. Evaluate perplexity, broad capability suites, target-specific controls, distribution shifts, and worst-case examples. Compare with prompt-based control, retrieval, fine-tuning, and doing nothing; an internal intervention is justified only when it wins on the actual product constraint.

Causal language should match the experiment. Adding a vector and observing a behavior change establishes an effect of that intervention in that context. It does not establish that the direction is the model’s unique representation of the named concept.

75.5 Worked protocol

Construct 200 paired prompts for a narrow attribute and split them before computing the direction. Extract residual states at several layers and choose the layer using validation only. On the held-out set, sweep α and report target score, language-model loss, refusal rate, and two unrelated capabilities. Repeat with five matched random directions. Then test on a new topic and prompt format. This design exposes whether the vector controls a transferable feature or a narrow template artifact.

Correct for selection over layers and coefficients: the held-out test is evaluated once after choosing both on validation. Reporting the best test layer from the same sweep turns the test set into another tuning set.

75.6 Four perspectives

- **Follow the Token:** an additive vector alters the residual state of selected positions, which changes downstream attention, MLP updates, and logits.
- **Follow the Gradient:** basic steering uses none; model editing solves or optimizes a parameter update and permanently changes later forward passes.
- **Follow the Byte:** steering stores small vectors but adds per-token reads and adds; editing creates weight versions and rollback checkpoints.
- **Follow the Request:** log model revision, intervention ID, layer, positions, coefficient, policy decision, and outcome for reproducibility.

75.7 Lab and exit check

Build one contrastive steering vector on a small model. Use disjoint discovery, validation, and test sets; sweep strength; compare random and reversed controls; measure at least two side effects. Optionally perform a single weight edit and test exact, paraphrased, neighboring, and unrelated prompts. State only what the interventions demonstrate.

75.8 References

- Turner et al., [Activation Addition](#), 2023.
- Zou et al., [Representation Engineering](#), 2023.
- Meng et al., [ROME](#), NeurIPS 2022.
- Meng et al., [MEMIT](#), ICLR 2023.

Chapter 76

Merging, Stitching and Representation Compatibility

Status: **Draft — not yet independently reviewed.**

Two models can have the same architecture and solve the same task while using different internal coordinates. Combining them therefore requires more than matching tensor shapes.

76.1 Model merging

Weight averaging computes $\theta = \sum_i \alpha_i \theta_i$. It can work when models share initialization and remain in a connected low-loss region, as in checkpoint averaging or model soups. Independently trained networks have permutation symmetries: swapping hidden units and compensating downstream weights preserves function but makes naive averages destructive. Git Re-Basin aligns permutations before interpolation. Task-vector methods represent fine-tuning changes $\Delta_i = \theta_i - \theta_0$ and combine or remove them, but interference remains an empirical question.

Before arithmetic, verify identical parameter names, shapes, tokenizer vocabulary and ordering, positional scheme, normalization convention, and base checkpoint. Equal tensor shapes do not make two vocabulary rows semantically equivalent. For models fine-tuned from the same base, preserve the exact base hash used to define every task vector.

Measure each source, the merge, an ensemble, and a multi-task fine-tuned baseline. A merge that saves memory but loses the desired specializations may still be useful; say which objective it optimizes.

76.2 Layer stitching

A stitched model runs early layers of source A , a connector g , then later layers of target B . The simplest connector is linear: $g(h) = hW + b$. Train it while freezing both models and compare with controls such as a same-model stitch and a randomly initialized source. Stitching accuracy can measure representational compatibility, but a high-capacity connector may learn the task rather than translate representations. Limit connector capacity and report training data.

76.3 Cross-model alignment

Alignment seeks a mapping between representation spaces using paired activations from the same inputs. Orthogonal Procrustes preserves distances and solves for a rotation; linear regression also permits scaling and shear; nonlinear bridges are more expressive and less diagnostic. Match tokenization and positions carefully. Different vocabularies or sequence segmentations require semantic alignment before tensor alignment. Assess held-out reconstruction, neighborhood preservation, and downstream function after transfer.

76.4 Normalization mismatch

A target layer expects a distribution shaped by its own preceding layers. Source states can differ in mean, RMS, covariance, outlier structure, and residual scale. Even RMSNorm models can encode learned scale through surrounding weights. Log per-layer norms and covariance spectra, test pre/post-normalization hook points, and consider calibrated affine maps. A bridge trained on one prompt distribution may fail when sequence length or domain changes.

76.5 Latent bridge cost

For state $H \in \mathbb{R}^{B \times T \times D_A}$ and dense bridge $W \in \mathbb{R}^{D_A \times D_B}$, parameter count is $D_A D_B + D_B$ and application costs roughly $2BT D_A D_B$ FLOPs if one multiply-add counts as two. With $D_A = D_B = 4096$, BF16 weights alone occupy about 33.6 MB decimal; at every decoded token they add a large matrix-vector read. A low-rank bridge $W = UV$ with rank r reduces parameters to $r(D_A + D_B)$ but constrains translation.

Remote handoff also transfers $BT D_A$ elements. For one BF16 state with $B = 1, T = 2048, D_A = 4096$, that is about 16.8 MB before protocol overhead. During token-by-token decode the state is smaller, but network latency and serialization can still erase saved compute. These calculations are feasibility filters, not performance predictions.

76.6 Established result versus hypothesis

Model soups, permutation alignment, and stitching have empirical literature. Seamlessly replacing arbitrary middle layers across unrelated current LLMs is not an established general capability. Compatibility must be demonstrated for named models, hook points, distributions, and tasks. A low reconstruction error alone is insufficient; downstream behavior and cost decide usefulness.

76.7 Four perspectives

- **Follow the Token:** source token states cross a learned coordinate map before the target continues processing.
- **Follow the Gradient:** bridge training updates the connector; merging directly constructs weights and may use no gradient.
- **Follow the Byte:** merged weights can replace several resident models, while bridges add parameters, activation transfer, and possibly network traffic.
- **Follow the Request:** routing must select compatible model revisions and reject a handoff when tokenizer, hook, or bridge metadata mismatches.

76.8 Lab and exit check

Train a linear bridge between two small frozen models on paired inputs. Compare same-model and cross-model stitches, a low-rank bridge, and a nonlinear high-capacity connector. Report held-out reconstruction, task accuracy, parameter count, FLOPs, transfer bytes, and latency. If the nonlinear connector alone succeeds, do not claim the original spaces were naturally aligned.

Evaluate at least one unseen sequence length and domain. A bridge that reconstructs teacher-forced states but fails during autoregressive rollout has not established functional compatibility; report rollout error separately from one-step reconstruction.

76.9 References

- Wortsman et al., [Model Soups](#), ICML 2022.
- Ainsworth, Hayase, and Srinivasa, [Git Re-Basin](#), ICLR 2023.
- Bansal, Nakkiran, and Barak, [Revisiting Model Stitching](#), NeurIPS 2021.
- Ilharco et al., [Editing Models with Task Arithmetic](#), ICLR 2023.

Part 20: Frontier Architecture and Research

Chapter 77

Efficient Memory and Adaptive Computation

Status: **Draft — not yet independently reviewed.**

A conventional decoder stores every layer’s keys and values and executes every layer for every generated token. Those choices are simple and robust, not laws of nature. Changing them trades memory or compute for approximation risk and control overhead.

77.1 Efficient KV memory

For L layers, H_{kv} key-value heads, head width d_h , context T , batch B , and s bytes per element, a basic KV cache occupies

$$M_{KV} = 2LBTH_{kv}d_h s.$$

The factor two is keys plus values. GQA and MQA reduce H_{kv} ; lower precision reduces s ; paging reduces allocation waste but not logical KV content. Before inventing an eviction policy, calculate which factor dominates the intended workload.

77.2 KV-cache compression

Compression can quantize values, store low-rank or clustered approximations, share structure, or retain full precision for sensitive tokens and layers. Evaluate perplexity or task quality across context lengths and decode positions, not just reconstruction error. Quantization metadata and dequantization kernels consume memory and time. A nominal four-bit cache may not produce a clean 4x system reduction after scales, alignment, fragmentation, and uncompressed regions.

77.3 KV-cache eviction

Eviction retains a token subset under a memory budget. Sliding windows keep recent tokens; attention-sink approaches preserve initial tokens plus a recent window; score-based methods keep tokens judged important from past attention. Past attention is an imperfect predictor of future need, especially when a late question refers to an early detail. Policies must operate per layer or coordinate across layers, and compaction itself moves bytes. Test adversarial retrieval, long-range dependencies, and changing topics.

Eviction changes which keys and values are available, not the historical position of retained tokens. Preserve the model’s position semantics—especially rotary position indices—when compacting physical storage. Renumbering retained entries as if they were a shorter fresh sequence is a different approximation and can corrupt attention even when the selected tokens were correct.

77.4 Activation compression

Training stores activations for backward; distributed inference may transmit activations between stages or machines. Quantization, sparsification, low-rank coding, and checkpoint recomputation reduce storage or traffic. The relevant objective is end-to-end step time or request latency at matched quality. Compression can be slower when encode/decode kernels or synchronization exceed bytes saved. Error can compound across repeatedly compressed boundaries.

77.5 Dynamic computation

Dynamic computation chooses work from the input or intermediate state. Examples include early exits, conditional experts, speculative paths, token pruning, and layer skipping. A controller must estimate difficulty early enough for savings to matter. Its false-easy errors harm quality; false-hard errors lose savings. Evaluate expected compute and the worst cohorts, because average savings can conceal systematic undercomputation.

77.6 Conditional layer execution

A skip controller can bypass a transformer block or reuse a previous state. Skipping preserves tensor shape but changes the state distribution expected by later layers. Static layer pruning chooses one smaller architecture for all requests; dynamic skipping chooses per request or token and adds branching, batching fragmentation, and compilation challenges. Grouping requests by execution path can recover throughput while increasing queue delay.

77.7 Adaptive computation

Adaptive Computation Time introduced learned halting for recurrent networks; transformer variants use exit heads or confidence criteria. Confidence is not automatically calibrated, and early logits can be confidently wrong. Train or calibrate the halting policy under an explicit compute penalty, then plot quality versus actual FLOPs, bytes, and wall time. Compare with a uniformly smaller model, quantization, distillation, and fixed layer pruning.

77.8 Worked budget

Assume a 32-layer model with eight KV heads, $d_h = 128$, BF16, batch one, and 32,768 cached tokens. The basic cache is $2 \cdot 32 \cdot 32768 \cdot 8 \cdot 128 \cdot 2 \approx 4.29$ GB decimal. Halving context halves it; moving from eight to one KV head divides it by eight; 8-bit storage approximately halves element bytes before metadata. This arithmetic identifies the largest lever before benchmarking implementation details.

77.9 Four perspectives

- **Follow the Token:** each policy decides which past tokens remain addressable and how many future layers process the current token.

- **Follow the Gradient:** learned compressors and routers receive task and compute-cost gradients; heuristic eviction at inference receives none.
- **Follow the Byte:** count payload, metadata, fragmentation, compaction, and encode/decode movement rather than quoting nominal bit width.
- **Follow the Request:** scheduling must handle different cache sizes and execution paths while preserving latency SLOs and isolation.

77.10 Lab and exit check

Implement recent-window and score-based eviction on a small KV-cached decoder, plus a no-eviction baseline. Test perplexity, needle retrieval, latency, and measured cache bytes across context lengths. Separately simulate fixed and dynamic layer skipping and report batching effects. A policy passes only at matched quality on both ordinary and adversarial long-context cases.

Add a position-integrity test: compact the cache without eviction and confirm logits match the un-compacted baseline within the declared numerical tolerance. Only then attribute quality changes to the eviction policy.

77.11 References

- Graves, [Adaptive Computation Time for Recurrent Neural Networks](#), 2016.
- Xin et al., [DeeBERT](#), ACL 2020.
- Xiao et al., [Efficient Streaming Language Models with Attention Sinks](#), ICLR 2024.
- Zhang et al., [H2O: Heavy-Hitter Oracle for Efficient Generative Inference](#), NeurIPS 2023.

Chapter 78

Routable Intelligence and Neural Handoffs

Status: **Draft research agenda — hypotheses are not established results.**

Routing complete requests among models is established systems practice. Routing intermediate neural states between independently trained models is a research proposal with unresolved compatibility, quality, and economics. This chapter keeps those evidence levels separate.

78.1 Neural routing

Established: mixture-of-experts models learn routers among experts trained inside a shared architecture, and cascades route whole requests among models. **Hypothesis:** a controller could also choose a model, layer segment, or expert intervention from intermediate state. A useful router must make its decision before the avoided compute, with overhead below the savings and low false-easy error. Inputs might include prompt features, early logits, uncertainty proxies, or hidden-state summaries.

78.2 Model cascades

Cascades run a cheap model first and escalate selected cases. They provide the strongest baseline for any more exotic handoff because they require no latent compatibility. Evaluate a threshold sweep at fixed quality, including the latency of failed first attempts. Calibration drift and selective feedback are operational risks. If a cascade matches a neural handoff, prefer the simpler interface.

78.3 Latent communication

Established in bounded settings: networks trained jointly can exchange learned latent messages; split computing transmits intermediate features; multimodal systems learn projection layers. **Open for unrelated LLMs:** whether a compact latent message can preserve the information another independently trained model needs across domains. Compare latent transfer with text, logits, retrieved evidence, and structured summaries under equal bandwidth and compute.

78.4 Cross-model representations

Representations may be statistically alignable even when coordinates differ, as Chapter 76 showed. Alignment does not imply functional substitutability. A bridge must preserve the target model’s required signals, norms, position semantics, and token correspondence. Test translation on held-out distributions and use the target continuation behavior as the main metric. CKA or reconstruction can support analysis but cannot certify a handoff.

78.5 Neural handoffs

A candidate handoff runs layers $0:i$ of model A, applies bridge g , then layers $j:L_B$ of model B. Falsifiable hypothesis: for named A, B, task distribution, and budget, the stitched system reaches within a predeclared quality margin of B while reducing measured cost or latency beyond bridge and transfer overhead. Baselines are B alone, A alone, whole-request cascade, textual handoff, same-model early exit, and a bridge with shuffled pairs. Failure means missing the quality margin or failing to save resources end to end.

Autoregressive handoff has an additional state problem: B’s surviving attention layers need keys and values for all prior positions in B’s own representation space. A prefill-time handoff can run the full prefix through B’s suffix to create those caches. A mid-generation handoff must instead replay history, translate compatible KV state, or have maintained B’s cache in parallel; each option can erase the proposed saving. Any experiment that transfers only the current hidden vector while ignoring destination KV history is not testing a valid cached decoder handoff.

78.6 Sparse expert intervention

Instead of replacing a suffix, an expert might provide a sparse correction Δh at selected tokens, layers, or features. This resembles adapters or residual experts when trained jointly. The research question is whether an independently trained expert can intervene reliably through a learned interface. Measure intervention frequency, bytes, target gain, collateral effects, and whether a conventional adapter with equal parameters performs better.

78.7 Silent expert intervention

“Silent” means the expert communicates through latent state rather than user-visible text. It should not mean invisible to logs, policy, or evaluation. Hypothesis: latent advice may avoid output-token generation and preserve richer information. Counter-risk: it is harder to inspect, authorize, and debug. Require typed metadata, provenance, audit traces, and a text-handoff baseline.

78.8 LLM dead-call elimination

A dead call is an LLM invocation whose removal does not reduce task quality under a defined test distribution and budget. Detect candidates by call-graph logging, then replay tasks while bypassing, caching, replacing, or combining calls. The causal test is an ablation, not a judgement based on plausible-looking transcripts. Hypothesis: many orchestration graphs contain redundant critic, summarizer, or formatter calls. Negative result: removal materially harms a predeclared quality or safety metric.

78.9 Computation as a routable resource

The broader program treats layers, experts, models, retrieval, tools, and human review as actions with cost and expected value. This can be posed as constrained optimization: maximize expected utility subject to latency, money, energy, and risk budgets. In practice, rewards are incomplete and distribution shift is unavoidable. Start with logged, auditable rules and offline counterfactual evaluation before learned online routing.

78.10 Minimal experiment

Use two small open models with frozen revisions. Train linear and low-rank bridges on paired states from one domain. Pre-register layer pairs, datasets, quality margin, hardware, maximum bridge parameters, and latency accounting. Compare six baselines listed above. Include shuffled-pair and same-model bridges, report three or more seeds, and profile serialization separately. Do not call the method successful if GPU kernel time falls but end-to-end latency rises.

State when the route is chosen—before prefill, after prefill, or during decode—and account for destination KV construction accordingly. Report resident memory for both models even when only one executes at a time.

78.11 Four perspectives

- **Follow the Token:** define how token identities and positions align across tokenizers before any latent tensor crosses models.
- **Follow the Gradient:** bridge or router training changes only declared components unless joint training is an explicit baseline.
- **Follow the Byte:** count bridge weights, transferred activations, synchronization, KV compatibility, and duplicated resident models.
- **Follow the Request:** log routing decision, model revisions, hook locations, bridge version, fallbacks, and final verification.

78.12 Lab and exit check

Write a preregistered experiment for one neural handoff and one dead-call ablation. Include hypotheses, baselines, fixed budget, quality and systems metrics, uncertainty, negative-result criteria, and artifact plan. Then run the smallest feasible pilot. The acceptable conclusion may be that text or whole-request routing wins; that is useful evidence, not a failed research process.

78.13 References

- Jacobs et al., [Adaptive Mixtures of Local Experts](#), Neural Computation 1991.
- Teerapittayanon, McDanel, and Kung, [BranchyNet](#), 2017.
- Bansal, Nakkiran, and Barak, [Revisiting Model Stitching](#), NeurIPS 2021.
- Kang et al., [Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge](#), ASPLOS 2017.

Chapter 79

Research Design and Reproduction

Status: **Draft — not yet independently reviewed.**

Research begins when a plausible idea becomes a question that could lose. A proposal that counts every outcome as success is a product pitch, not an experiment.

79.1 Forming research questions

A useful question specifies population, intervention, comparison, outcome, and constraint. Replace “Can neural handoff work?” with: “On datasets X and Y, can a rank- r bridge from model A layer i to model B layer j match B within one accuracy point while reducing p95 latency by at least 20% on hardware H?” Name a negative result in advance. Separate mechanism questions (“why?”), empirical questions (“does it?”), and systems questions (“at what measured cost?”).

79.2 Literature search

Search by mechanism, synonyms, older terminology, and neighboring fields. Use scholarly indexes and citation graphs, then follow references backward and citing papers forward. Record queries, dates, databases, inclusion criteria, and exclusions. Search recent conference proceedings and preprints because terminology shifts quickly, while treating non-peer-reviewed work at its actual evidence level. Stop when new searches mostly repeat known clusters, not when the first supportive paper appears.

79.3 Prior-art search

Prior-art search asks whether the claimed contribution already exists, including under another name or outside LLM research. Search papers, patents, code, technical reports, theses, and product documentation. Decompose the idea into claims: architecture, objective, training method, deployment pattern, and demonstrated result. Novel combinations can still be obvious combinations; novelty is a legal and scholarly question requiring current expert review. A literature review in this book cannot certify patentability.

79.4 Reading papers critically

Read the abstract for the claimed result, methods for what was actually done, tables and figures for evidence, and appendices for missing details. Build a claim-evidence table: exact claim,

experiment, baseline, metric, uncertainty, and threat. Check data leakage, hyperparameter fairness, compute mismatch, cherry-picked tasks, and whether the metric measures the stated goal. Distinguish comparisons made in one controlled implementation from numbers copied across papers.

79.5 Designing a research experiment

Change the smallest set of variables that tests the hypothesis. Define independent variables, outcomes, controls, unit of analysis, randomization, stopping rule, and analysis before looking at results. Pilot to debug instrumentation, then freeze the protocol. Avoid tuning the proposed method on the test set while leaving baselines at defaults. If compute limits the sweep, state that constraint and choose a fair allocation.

Plan sample size from the smallest effect that would change the decision, expected variance, and intended analysis—not from a conventional round number. If formal power analysis is infeasible, simulate plausible effect and variance ranges and publish the sensitivity. Sequentially checking results and stopping when $p < 0.05$ inflates false positives unless the stopping rule and analysis explicitly support sequential testing.

79.6 Choosing baselines

Include a trivial baseline, current standard, strongest feasible competitor, and an oracle or upper bound where useful. For efficiency research, compare with smaller models, quantization, caching, batching, and hardware-aware tuning—not only an intentionally naive implementation. Match quality, data, compute budget, and optimization effort. A baseline that receives less tuning is a weak foundation for a superiority claim.

79.7 Benchmark selection

Select benchmarks that exercise the claimed mechanism and represent deployment conditions. Report contamination risk, licenses, languages, domains, prompt templates, scoring, and known limitations. One aggregate score can hide regressions; define slices and worst-case metrics. Add task-specific cases that can falsify the mechanism, such as long-range references for KV eviction. Keep a final untouched test set.

79.8 Reproducing papers

Begin with the authors' exact commit, environment, data version, checkpoint, and command when available. Reproduce one reported table cell before extending the method. Record deviations and classify outcomes: exact reproduction, approximate reproduction within uncertainty, failure due to missing artifacts, or contradiction under matched conditions. Contacting authors can resolve ambiguity, but undocumented private knowledge is itself a reproducibility limitation.

79.9 Reproducibility

A reproducibility package includes code, dependency lock, seeds, hardware and software versions, data acquisition and checksums, preprocessing, configurations, commands, raw results, and analysis scripts. Deterministic kernels may reduce variance but can alter performance. Containers capture user space, not every driver, firmware, accelerator, or external API dependency. Preserve intermediate artifacts and a machine-readable manifest.

Use terminology explicitly. Following the ACM artifact vocabulary, repeatability concerns the same team and setup, reproducibility involves a different team using the authors' artifacts, and replicability seeks the result independently. Communities use these words differently, so define the adopted convention instead of assuming it.

79.10 Scientific vs engineering claims

A scientific claim concerns general relationships, such as a routing signal predicting task difficulty across specified distributions. An engineering claim concerns an artifact under conditions, such as 18% lower p95 latency on an H100 with a named serving stack. Both matter, but evidence does not transfer automatically. A faster kernel on one shape is not a general architectural law; a statistically reliable accuracy gain may be commercially irrelevant if it doubles cost.

79.11 Worked preregistration

For KV eviction, predeclare contexts, token budgets, policies, quality metrics, three seeds, hardware, warmup, and maximum tuning trials. Primary outcome might be retrieval accuracy at a fixed 2 GB cache; secondary outcomes are p95 latency and perplexity. The negative criterion is failure to match the recent-window baseline's worst-slice accuracy. Publish the protocol before inspecting the final test results, then label exploratory analyses separately.

79.12 Four perspectives

- **Follow the Token:** datasets, tokenizers, templates, truncation, and scoring must be versioned because they define the experimental input.
- **Follow the Gradient:** training seeds, schedules, data order, checkpoint selection, and tuning budget can dominate apparent improvements.
- **Follow the Byte:** resource claims require measured memory, transfers, artifacts, and hardware counters under a reproducible workload.
- **Follow the Request:** an experiment is a replayable request pipeline with configuration, trace, outcome, and immutable provenance.

79.13 Lab and exit check

Select one paper relevant to Chapter 77 or 78. Produce a search log, prior-art matrix, claim-evidence table, reproduction plan, and preregistration with a negative-result criterion. Reproduce one central result or document precisely why it cannot be reproduced. A repository that runs only on the author's machine is an unresolved result.

79.14 References

- Pineau et al., [Improving Reproducibility in Machine Learning Research](#), JMLR 2021.
- Gundersen and Kjensmo, [State of the Art: Reproducibility in Artificial Intelligence](#), AAAI 2018.
- [NeurIPS Paper Checklist](#), current author guidance, accessed 2026-09-19.
- [ACM Artifact Review and Badging](#), accessed 2026-09-19.
- National Academies, [Reproducibility and Replicability in Science](#), 2019.

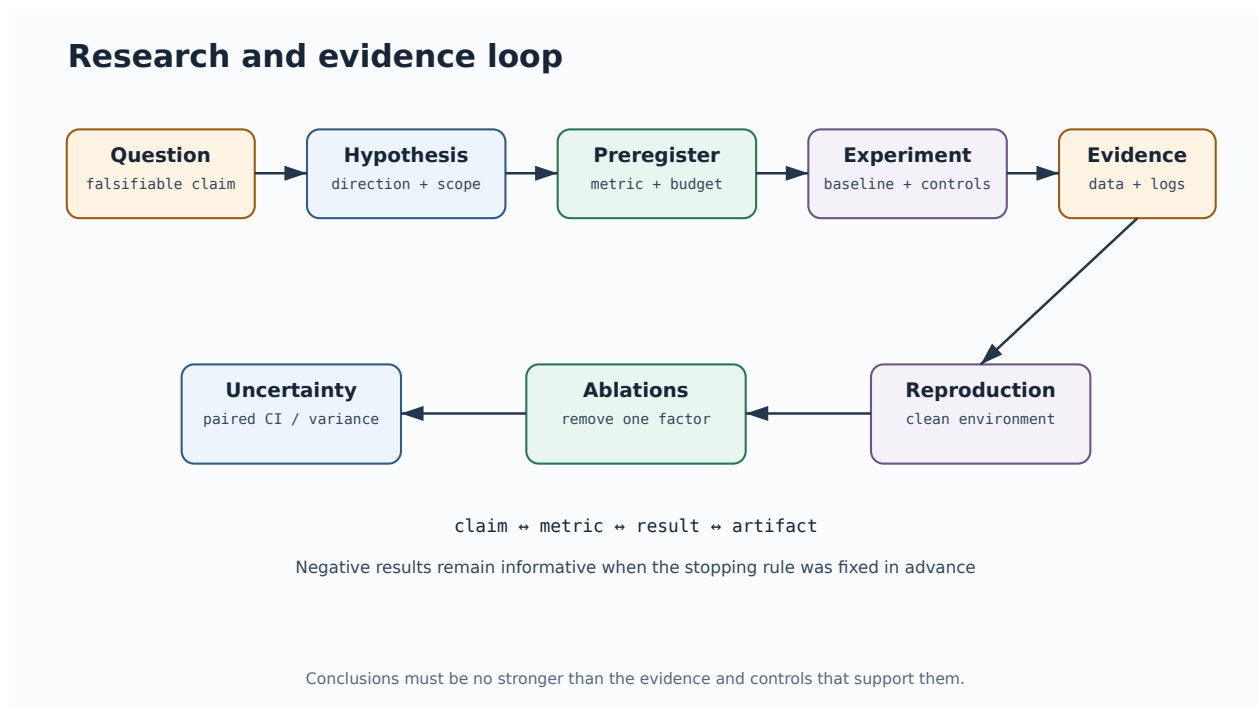


Figure 79.1: Research evidence and reproduction loop.

Chapter 80

Evidence, Ablations and Publication

Status: **Draft — not yet independently reviewed.**

The final task is to make claims no larger than the evidence. Strong work connects a controlled comparison, uncertainty, resource measurement, and a reproducible artifact to one precise conclusion.

80.1 Ablations

An ablation removes or replaces a component to estimate its marginal contribution. Compare full method, component removed, reasonable alternative, and interactions between components. Retraining after removal answers whether the architecture needs the component; removing it only at inference answers whether the trained system currently relies on it. Those are different questions. Keep tuning budgets equal, and avoid interpreting a broken implementation as evidence for the full design.

For a neural handoff, ablate the router, bridge, latent transfer, and expert intervention separately. Include a shuffled bridge and same-model stitch. If only the complete stack works, factorial ablations can reveal synergy; if nothing changes when a component disappears, its call may be dead.

80.2 Statistical significance

A statistical test quantifies incompatibility between observed data and a null model under assumptions. It does not measure practical value or probability that the hypothesis is true. Define the unit of independence: tokens from one sequence are not thousands of independent experiments, and repeated benchmark prompts may share templates. Use paired tests when methods run on the same items. Correct or constrain multiple comparisons rather than fishing among many metrics.

Report the effect estimate and uncertainty beside any test. Predeclare one primary outcome or use a justified hierarchical or multiplicity procedure; testing 20 slices and highlighting the smallest unadjusted value changes the false-positive rate. If prompts are nested in documents or users, a cluster bootstrap or hierarchical model is often more defensible than treating prompts as independent.

80.3 Confidence intervals

A confidence interval describes an estimator’s sampling uncertainty under a procedure. Bootstrap paired per-example differences when analytic assumptions are unsuitable; resample at the independent unit, such as document, user, or run. For training variance, report results across independently trained seeds, not merely repeated evaluation of one checkpoint. An interval excluding zero may still describe a uselessly small improvement.

For bounded or highly skewed metrics, state the interval method; a normal interval can extend beyond the valid range. With only a few training seeds, neither a narrow standard-error bar nor a bootstrap over benchmark examples captures optimization variance reliably. Show individual seed results and treat broad generalization claims as provisional.

80.4 Uncertainty

Separate aleatoric variation in data, epistemic uncertainty from limited knowledge, optimization variance across seeds, measurement noise, and distribution uncertainty. Report sensitivity to prompts, checkpoints, workloads, and hardware where relevant. Error bars do not cover untested populations. Use calibration metrics when probabilities drive routing, but remember that calibration can degrade after distribution shift.

80.5 Profiling research claims

Profile the complete workload after warmup and synchronize accelerators around timed regions. Separate compilation, model load, prefill, decode, transfer, bridge, queue, and sampling time. Use traces and hardware counters to explain a result, then validate with end-to-end latency. Profilers perturb execution; state their settings. Throughput measured at saturation and single-request latency answer different questions.

80.6 FLOPs-memory-latency measurement

Analytic FLOPs expose scaling; profiler FLOPs expose executed kernels; wall time exposes the system. Report counting convention, shapes, batch, sequence lengths, precision, hardware, software versions, concurrency, and quality. Measure peak allocated and resident memory where possible, plus KV and weight calculations. Include host-device and network bytes. A lower-FLOP method can be slower because it launches more kernels, moves more memory, or fragments batches.

80.7 Writing papers

Write the contribution as a verifiable claim: problem, gap, method, evidence, and limits. Methods should be reproducible; results should connect every headline claim to a table or figure; related work should explain differences rather than list citations. Put limitations beside conclusions, not buried as ceremonial text. Release negative findings when they constrain a promising direction: discovering that bridge transfer costs erase neural-handoff savings is useful systems knowledge.

80.8 arXiv

arXiv is a public preprint repository, not peer review. A posting establishes a dated, citable manuscript and can accelerate feedback. Check category, endorsement, license, anonymization

policy of intended venues, and artifact redaction before submission. Version updates transparently; do not imply that a preprint has conference acceptance.

80.9 Workshops

Workshops often welcome emerging ideas, focused communities, and preliminary results. Standards and archival status vary. They are useful for refining a research program, especially when the contribution is a careful negative result or benchmark. Read the exact call, review model, page limit, proceedings policy, and dual-submission rule.

80.10 Conferences

A conference submission targets a community’s contribution and evidence norms. Choose venue from the work, rather than reshaping claims after results to chase prestige. Check current deadlines, formatting, anonymity, ethics review, artifact tracks, and concurrent-submission rules on the official site. Acceptance is a selective review outcome, not proof that every claim is correct.

80.11 Peer review

Review tests novelty, validity, clarity, significance, and reproducibility through fallible human judgement. Respond to critiques with evidence and precise revisions. Separate fixable presentation gaps from experiments that change the conclusion. As a reviewer, disclose conflicts, protect confidentiality, and avoid demanding citations for self-promotion. Open reviews and rebuttals can improve transparency but do not remove bias or variance.

80.12 Finding unanswered questions

Unanswered questions appear where assumptions remain untested, methods fail on a slice, systems costs invalidate an algorithmic gain, or two literatures use different abstractions. Keep a discrepancy log while reproducing work. Ask which result would change a design decision, then seek the smallest decisive experiment. “Nobody used this exact acronym” is not novelty. A useful question has plausible value, measurable outcomes, feasible resources, and a result worth knowing even when negative.

80.13 Claim-evidence table

Claim	Required evidence	Common overclaim
Higher quality	Matched data/budget, strong baselines, uncertainty	Best single seed
Faster inference	End-to-end workload at matched quality	Kernel microbenchmark only
Lower memory	Measured peak plus accounting	Nominal datatype ratio
Causal mechanism	Controlled interventions and specificity	Probe or attention plot
General method	Multiple models, tasks, and shifts	One checkpoint and dataset
Novel research	Current prior-art search	New label for known method

80.14 Four perspectives

- **Follow the Token:** publish prompts, tokenization, context lengths, truncation, decoding, and scoring so evaluation inputs are reconstructible.
- **Follow the Gradient:** disclose seeds, training curves, selection rules, failed runs, and compute spent on tuning each method.
- **Follow the Byte:** reconcile theoretical memory/FLOPs with profiler counters, transfers, synchronization, and wall-clock behavior.
- **Follow the Request:** preserve the full request trace and configuration needed to reproduce both quality and service-level claims.

80.15 Lab and exit check

Produce a paper-sized evidence package: claim-evidence table, ablation grid, paired uncertainty analysis, full resource profile, limitations, and reproducibility manifest. Write one paragraph that the data supports and one stronger paragraph it does not support. Tag every source as peer-reviewed paper, preprint, technical report, documentation, or code artifact. The project is ready for submission only when a clean environment can regenerate the main table.

Before release, trace every abstract and conclusion claim to a result cell and every result cell to a command, configuration, raw artifact, and analysis step. Any broken link becomes either a repaired artifact or a narrower claim.

80.16 References

- Dodge et al., [Show Your Work: Improved Reporting of Experimental Results](#), EMNLP-IJCNLP 2019.
- Dror et al., [The Hitchhiker’s Guide to Testing Statistical Significance in NLP](#), ACL 2018.
- Agarwal et al., [Deep Reinforcement Learning at the Edge of the Statistical Precipice](#), NeurIPS 2021.
- Wasserstein, Schirm, and Lazar, [Moving to a World Beyond “ \$p < 0.05\$ ”](#), The American Statistician 2019.
- [arXiv submission help](#), accessed 2026-09-19.
- [NeurIPS Paper Checklist](#), accessed 2026-09-19.

Appendix A

Exercise and Exit-Check Solutions

These concise solutions are keyed to the numbered chapters. Numerical answers use the assumptions stated in the chapter; empirical labs instead give acceptance criteria, because benchmark values depend on the recorded environment. A result that does not include its boundary, units, configuration, and limitations is incomplete.

A.1 Part I — Orientation

A.1.1 01. The LLM in One Picture

Token IDs are categorical indices, not magnitudes. $[B, T, D]$ becomes $[B, T, V]$ at the vocabulary projection (LM head). Sampling can change output without changing weights. $[1024, 4096]$ BF16 uses $1024 \times 4096 \times 2 = 8,388,608$ bytes = 8 MiB. Appending a token extends request state; it does not retrain the model.

A.1.2 02. Training vs Inference

For one request, weights are persistent and unchanged; prompt IDs, hidden activations, and KV entries are request-local. Gradients and Adam moments are absent from ordinary inference; optimizer moments are persistent training state. RNG state advances only when the chosen decoding path consumes randomness.

A.1.3 03. Anatomy of the Modern LLM Stack

A defensible diagnosis maps the symptom to the narrowest layer: malformed roles belong to application/template logic; queue delay to serving; a graph break to compiler/runtime; a slow GEMM to the kernel; OOM to live allocations and memory policy. Confirm with boundary traces before changing a lower layer.

A.1.4 04. Building Our Tiny Laboratory

A passing artifact records the exact command, hypothesis, expected and actual result, seed, dependency/hardware versions, and one limitation. Tests must pass and `build/environment.json` must be sufficient for another reader to identify what ran; a notebook with hidden state alone is not reproducible.

A.2 Part II — Mathematical Foundations

A.2.1 05. Tensors and Resource Accounting

$X: [4, 512, 1024]$, $W: [1024, 4096]$, and $Y: [4, 512, 4096]$. Their BF16 payloads are 4, 8, and 16 MiB. The projection costs approximately $2 \text{BTD} \cdot 4096 = 17,179,869,184$ FLOPs, excluding bias, backward work, workspaces, padding, and allocator overhead.

A.2.2 06. Geometry and Number Formats

For $q=[1,0]$, dot products with $a=[2,0]$, $b=[1,1]$, and $c=[100,100]$ are 2, 1, and 100; cosines are 1, $1/\sqrt{2}$, and $1/\sqrt{2}$. Stable softmax subtracts the maximum because the common exponential factor cancels. Packed INT4 also needs scales, zero points, grouping metadata, padding, and sometimes dequantization buffers.

A.2.3 07. Probability, Logits and Loss

$\text{softmax}(z_i+c)=e^{\{z_i\}}e^c/(e^{\sum_j z_j})=\text{softmax}(z_i)$. For one-hot target y , cross-entropy is $-\sum_i y_i \log p_i = -\log p_{\text{target}}$. Entropy describes one distribution, cross-entropy scores a target against a prediction, and $\text{KL}(P||Q)$ compares normalized distributions asymmetrically.

A.2.4 08. Gradients, Graphs and Autodiff

For $L=a^2+ab$, both paths into a accumulate, so $\partial L/\partial a=2a+b$; $\partial L/\partial b=a$. Centered finite differences should converge toward these values until rounding error dominates. This tests only the implemented scalar primitives, not broadcasting, tensor views, or nondifferentiable operators.

A.3 Part III — Neural Networks

A.3.1 09. Linear Layers and Nonlinearities

Three affine maps collapse to $W=W_3W_2W_1$ and $b=W_3W_2b_1+W_3b_2+b_3$. A tent is $\text{ReLU}(x+1)-2\text{ReLU}(x)+\text{ReLU}(x-1)$. A wrongly shaped bias can broadcast without an exception, sharing or varying offsets along the wrong axis and therefore defining a different model.

A.3.2 10. A Tiny Network, Forward and Backward

With $b_1=-2$, $z_1=-1$, the first ReLU derivative is zero, so gradients through that hidden unit to its incoming weight and bias are zero. Centered differences validate the original analytic derivatives away from the ReLU kink. Summing a batch loss multiplies gradients by batch size relative to averaging.

A.3.3 11. Optimization and State

For P parameters: BF16 parameters use $2P$, FP32 gradients $4P$, and two FP32 Adam moments $8P$, totaling $14P$ bytes; add $4P$ if FP32 master weights are retained. `zero_grad` prevents unintended accumulation. Coupled L2 adds $\lambda\theta$ to the gradient before Adam normalization; AdamW applies a separate $-\eta\lambda\theta$ decay.

A.3.4 12. Initialization and Training Stability

Norms $[0.9, 0.8, 0.02, 0.0001]$ justify locating where signal collapses, then checking activations, gradients, initialization, masks, and precision at that boundary—not naming a cause from norms alone. $[6, 8]$ has norm 10, so clipping at 5 yields $[3, 4]$. Raising epsilon may suppress numerical symptoms while materially changing normalization or optimizer dynamics.

A.4 Part IV — Tokens and Representations

A.4.1 13. Tokenization Algorithms

One valid BPE answer must state corpus frequencies and a deterministic tie rule, then show the pair counts and vocabulary after each of three merges. Round trips must preserve bytes or normalized text for whitespace, emoji, both accent forms, and unseen scripts. Round-trip correctness says nothing about compression, sequence length, fairness, or model compatibility.

A.4.2 14. Vocabulary and Token IDs

If IDs 1 and 2 are permuted but embedding rows are not, both input meanings and output labels silently swap. Permuting IDs, embedding rows, and tied/output rows together preserves the function. Hash tokenizer model/config, vocabulary and merges, special-token map, model config, weight shards/index, templates, and relevant code revision.

A.4.3 15. Embedding Matrices and Geometry

For IDs $[1, 1, 3]$ with upstream vectors g_1, g_2, g_3 , only rows 1 and 3 are nonzero: $\forall E[1] = g_1 + g_2$, $\forall E[3] = g_3$. Scaling one vector by ten leaves its cosine direction unchanged but changes Euclidean distances. A probe can exploit correlated information without proving the representation causally drives behavior.

A.4.4 16. Position and Sequence Order

Without position information, permuting token rows permutes Q/K/V and therefore permutes attention outputs: the operation is equivariant. For a 2-D RoPE pair, rotations compose as $R(p) \wedge^T R(q) = R(q - p)$, so the dot product depends on relative offset. Extrapolation may fail from unseen rotation frequencies and from learned attention/content behavior outside the training distribution.

A.5 Part V — Attention

A.5.1 17. Sequence Modeling and the Attention Idea

For recurrence $h_t = 0.25h_{t-1} + x_t$, $h_0 = 0$ and inputs $[1, 2, 3]$ give 1, 2.25, and 3.5625. Attention instead forms an explicit weighted sum $\sum_i \alpha_i v_i$, with nonnegative weights summing to one. Recurrence has compact sequential state; full attention offers direct access to all positions but quadratic score work/storage in the naive form.

A.5.2 18. Queries, Keys, Values and Scaling

Swapping values changes retrieved content under unchanged weights; swapping keys changes routing weights. For independent unit-variance Q/K coordinates, the dot-product variance is d_k , hence division by $\sqrt{d_k}$. With $T=4$, $d_k=2$, $d_v=3$, Q/K are $[4, 2]$, scores and weights $[4, 4]$, V $[4, 3]$, and output $[4, 3]$ per head.

A.5.3 19. Causal and Multi-Head Attention

The permitted 4×4 mask is lower triangular: rows allow columns $\{0\}$, $\{0, 1\}$, $\{0, 1, 2\}$, $\{0, 1, 2, 3\}$; forbidden entries receive $-\infty$ before softmax. For $B=3, T=7, D=12, H=3, d_h=4$, $Q/K/V$ are $[3, 3, 7, 4]$, concatenation returns $[3, 7, 12]$. Mutating future tokens must not change earlier logits.

A.5.4 20. Attention Shapes and Compute Cost

For $B=4, T=1024, D=2048, H=16, d_h=128$; each $Q/K/V$ tensor is $[4, 16, 1024, 128]$ and 16 MiB in BF16, or 48 MiB total. Scores are $[4, 16, 1024, 1024]$ and 128 MiB in BF16. Four $D \times D$ projections cost about $8BD^2 = 137.44$ GFLOPs; QK^T plus AV costs about $4BT^2D = 34.36$ GFLOPs. Doubling T doubles projections/QKV bytes but quadruples scores and attention matmuls.

A.6 Part VI — The Transformer

A.6.1 21. Residual Streams and Normalization

Pre-norm computes $x + F(\text{norm}(x))$; post-norm computes $\text{norm}(x + F(x))$, so identical sublayer weights do not make their inputs or gradient paths equal. A correct normalization computes statistics over the feature dimension independently for every batch/time position. Report stream RMS, update RMS, and gradient norms by layer for both variants.

A.6.2 22. Feed-Forward Networks and Complete Blocks

For $(B, T, D, H, M) = (2, 16, 128, 4, 352)$, every residual boundary is $[2, 16, 128]$; per-head width is 32. A SwiGLU MLP needs two $D \rightarrow M$ projections (gate and value) and one $M \rightarrow D$ projection, hence three matrices. Its M is often smaller than the 4D width of a two-matrix GELU MLP to match parameter/compute budgets.

A.6.3 23. Stacking, Vocabulary Projection and Tying

IDs $[B, T]$ look up embeddings $[B, T, D]$, blocks preserve that shape, and the LM head produces $[B, T, V]$; logits at position t predict label $t+1$ after shifting. In a tied model, input embedding and output projection reference the same parameter object and receive the sum of gradients from both uses, reducing the parameter count by $V \cdot D$ relative to untied weights.

A.6.4 24. Build Our GPT From Scratch

A pass requires finite $[B, T, V]$ logits, a reconciled hand/framework parameter count, true tied storage, and causal isolation. Removing the causal mask should make an earlier logit change when only a future token changes. Shape tests alone cannot detect leakage, wrong normalization, bad positions, or numerically incorrect attention.

A.7 Part VII — Training Systems

A.7.1 25. Sequences, Batches and Pretraining

A correct packed collator returns token IDs, position IDs that reset per example when required, an attention mask that blocks cross-example attention, and a loss mask excluding separators/padding. Changing example A must leave B's logits unchanged. The loss denominator is the count (or weight sum) of valid target tokens, not padded sequence length.

A.7.2 26. The Training Step and Schedule

With dropout off, identical examples/order, and loss weighted by valid tokens, four microbatches of two should match one batch of eight within floating-point tolerance. Dropout consumes different masks and changes arithmetic order, so exact equality is not expected. Account separately for parameters, gradients, master weights if any, and optimizer buffers.

A.7.3 27. Mixed Precision and Checkpoint Anatomy

A complete step-two checkpoint restores model, optimizer, scheduler/scaler, data position, and every RNG state. Repeating step three should reproduce the sampled batch, dropout masks, loss, parameters, optimizer buffers, and scheduler count under deterministic kernels. Any hardware/library nondeterminism must be named rather than hidden behind a loose tolerance.

A.7.4 28. Debugging and Training Our Tiny Model

At initialization, vocabulary cross-entropy should be near $\ln V$; one batch should overfit; held-out metrics must use evaluation behavior; resume must reproduce the next step. A one-token label shift can leave loss plausible while teaching the wrong alignment; an excessive learning rate typically causes spikes/non-finite values or divergence. Record seeds, versions, and dataset hash.

A.8 Part VIII — Architectures

A.8.1 29. Transformer Families

Encoder self-attention permits all 4×4 source pairs; three-token decoder self-attention uses a 3×3 lower triangle; cross-attention permits all 3×4 target-to-source pairs. Encoder K/V can be cached during decoder generation because the source is fixed; decoder self K/V grows. BERT predicts masked positions, GPT predicts shifted next tokens, and T5 predicts sentinel-delimited removed spans.

A.8.2 30. Modern Positions, Heads and MLPs

For $D=2048, H=16, d_h=128$. Q always has D^2 parameters; K and V together have $2D(Hk_v \cdot d_h)$, and output projection has D^2 . BF16 KV per token-layer is $2 \cdot Hk_v \cdot 128 \cdot 2$ bytes: 8192 bytes for $Hk_v=16$, 2048 for 4, and 512 for 1. Each query head maps to exactly one K/V group.

A.8.3 31. Mixture of Experts

For $N=1000, E=8, k=2$, there are 2000 assignments; nominal per-expert capacity is $\text{ceil}(\text{capacity_factor} \cdot 2000 / E)$, 250 at factor 1.0 and 313 at 1.25. Report histogram, coefficient of variation, routing entropy, and assignments beyond capacity. Dropping loses routed computation; rerouting changes which expert computes the token. Small decode batches commonly underutilize experts.

A.8.4 32. Alternative Architectures and Real Configs

A passing answer pins an official config revision and derives d_h , attention/MLP tensor counts, dtype weight bytes, and KV bytes at $B=1, T=4096$; every mismatch with loaded tensors must be attributed to biases, norms, embeddings, tying, padding, or architecture-specific weights. A sparse/local/state-space alternative cannot directly access every earlier token with an arbitrary content-based edge in one layer as full attention can.

A.9 Part IX — Data and Evaluation

A.9.1 33. Data Sources and Cleaning

A reproducible shard is derivable from a manifest of permitted immutable inputs plus pinned cleaning/dedup code. The report must show document and byte counts after acquisition, filtering, exact dedup, and near-dedup, with reason codes and manual samples from accepted/rejected sets. Unrecorded source drift fails the exit check.

A.9.2 34. Mixtures, Tokens and Synthetic Data

If allocation amounts are a_i , expected epochs are a_i/u_i for unique-token counts u_i ; allocations must sum to 12B. Exact hashes should catch planted exact leaks but not paraphrases; shingles may catch overlap while creating topical false positives and missing semantic rewrites. Report both false positives and false negatives rather than one recall number.

A.9.3 35. Evaluation and Meaningful Experiments

Use paired per-example differences for the 300 shared examples and bootstrap those pairs for a 95% interval. A supported claim is restricted to the pinned prompts, decoding, slices, and measured latency boundary. An unsupported claim extrapolates to all tasks, users, or deployment loads from this sample.

A.9.4 36. Scaling Laws

Fit the stated power law to all nine size/budget runs, inspect residual structure, bootstrap runs or appropriate units, and evaluate the untouched held-out run. Report point error and interval coverage. Adding inference cost can change the selected model even when predicted training loss ordering is unchanged.

A.10 Part X — Adaptation and Reasoning

A.10.1 37. Pretraining, Continued Pretraining and SFT

For each rendered conversation, labels equal token IDs only on intended assistant spans and are ignored elsewhere, including after truncation and packing. Attention must prevent cross-example leakage. A continued-pretraining mixture should retain a stated general-data replay fraction and predefine domain and general-retention gates.

A.10.2 38. Parameter-Efficient Adaptation

LoRA for one $d_{\text{out}} \times d_{\text{in}}$ weight adds $r(d_{\text{in}} + d_{\text{out}})$ parameters through factors A and B. Before merge, $y = (W + sBA)x$; after setting $W' = W + sBA$, outputs should match within numeric tolerance. Adapter identity belongs in cache keys; changing it invalidates prefix states computed with old weights.

A.10.3 39. Preferences and Reinforcement Learning

Reward-model pair loss is $-\log \sigma(r_{\text{chosen}} - r_{\text{rejected}})$. DPO applies the same logistic form to the policy-vs-reference log-ratio margin scaled by β . PPO's path is rollout $\rightarrow$ rewards/advantages $\rightarrow$ clipped policy updates, typically with frozen reference and reward models and a trainable policy/value path. Summed sequence log-probabilities introduce a length effect that averaging changes.

A.10.4 40. Reasoning, Test-Time Compute and Distillation

All three strategies must receive the same total generation-token budget. Compare correctness, wall latency, consumed tokens, and selection failures; majority vote can waste correlated samples, while checker ranking inherits checker errors. Distillation is supported only if verified-example training beats an equal-data SFT control on held-out tasks.

A.11 Part XI — Hardware Performance

A.11.1 41. Why GPUs and How They Execute

For one million elements, block counts are $\text{ceil}(1,000,000/t)$: 7813 for 128 threads, 3907 for 256, and 1954 for 512. The final blocks contain 64, 64, and 64 valid lanes respectively; masking is required. Occupancy is only a resource constraint—memory access, registers, and instruction efficiency determine the winner.

A.11.2 42. Memory Hierarchy and Movement

For P parameters at b bytes each, weights use Pb ; the ideal transfer lower bound is bytes/bandwidth at each SSD→RAM, RAM→GPU, and HBM scan boundary. Real time is higher due to protocol, contention, page faults, and incomplete overlap. Contiguous/coalesced reads require fewer transactions than strided reads for the same useful payload.

A.11.3 43. Throughput, Intensity and Rooflines

Arithmetic intensity is FLOPs divided by bytes moved at the boundary being modeled. The roofline ceiling is $\min(\text{peak FLOP/s}, \text{intensity} \times \text{measured bandwidth})$. Prefill or larger decode batches reuse weights and raise ideal intensity. Actual points lie below the roof because of launch gaps, imperfect tiling, occupancy limits, non-GEMM work, synchronization, and unmodeled traffic.

A.11.4 44. Profiling Transformer Bottlenecks

Warm up, synchronize only outside measured intervals, use CPU wall time for end-to-end and GPU events for device spans, and sweep batches. The largest interval is a candidate bottleneck, not proof; falsification requires an intervention predicted to shrink that span without shifting the boundary or workload. Publish a systems timeline and kernel-level evidence when CUDA is available.

A.12 Part XII — Kernels and Runtime

A.12.1 45. Kernels, GEMM and Tiling

GEMM work is $2MNK$ FLOPs. A 128×128 tile can lose to 64×64 through register/shared-memory pressure, fewer resident blocks, poor edge utilization, or bank/layout effects. Fusing activation with its producer/consumer can avoid writing and rereading the intermediate activation, but only a byte trace establishes the saving.

A.12.2 46. CUDA, Triton and CUTLASS

Fused BF16 add-plus-activation logically reads two inputs and writes one output: 6 bytes/element. For 16 million elements this is 96,000,000 bytes $\approx$ 91.6 MiB; at 0.20 ms, logical bandwidth is about 447 GiB/s. A passing report covers irregular shapes, NaN contract, cold compile time, warm median/p95, environment, and the best matched framework baseline.

A.12.3 47. Graphs, Compilers and Runtimes

A passing two-shape-family experiment reports the exported/compiled graph, guards, graph breaks, generated kernel count, cold compile time, and warmed time. Fusion is valuable when it removes actual allocations/HBM traffic. A speedup claim fails if compilation is included only for one path, semantics differ, or unseen shapes trigger unreported recompilation/fallback.

A.12.4 48. CUDA Graphs and Execution Overhead

Eighty 4 μ s kernels plus eighty 3 μ s host gaps take 560 μ s, of which $240/560 \approx 43\%$ is orchestration. Break-even reuse is $N > C/(E-R)$; 2 s setup and 0.5 ms saving reaches equality at 4000 hits. Power-of-two batch buckets {1,2,4,8,16,32} bound variants; batch 11 masks five slots and wastes 5/16 capacity.

A.13 Part XIII — Inference

A.13.1 49. Load Weights and Trace a Forward Pass

The required trace is IDs $[B, T] \rightarrow$ residual $[B, T, D] \rightarrow Q [B, H, T, d_h]$, K/V $[B, Hkv, T, d_h] \rightarrow$ attention $[B, H, T, d_h] \rightarrow$ MLP $[B, T, F] \rightarrow$ logits $[B, T, V]$. For $B=2, T=128, D=4096$ BF16, one residual is 2 MiB; $V=128,000$ full logits are 62.5 MiB. Compare final logits with uninstrumented eager execution.

A.13.2 50. Prefill, Decode and KV Cache

KV bytes are $2LBHkvSd_{hb}$. With $L=32, Hkv=8, d_h=128, b=2$, one token/sequence uses 131,072 bytes = 128 KiB, and 8192 tokens use 1 GiB. Historical K/V are needed for new-query routing and retrieval; historical Q is not. Cached and full-prefix logits must match before sampling.

A.13.3 51. Sampling and Decoding Policies

For logits $[2, 1, 0]$, greedy chooses the first token; at $\tau=0.5$, probabilities are about $[0.867, 0.117, 0.016]$. Top-2 at $\tau=1$ renormalizes to $[0.731, 0.269, 0]$; top-p 0.8 keeps the first two under the include-crossing convention. Seeded frequencies should fall within a stated sampling interval. Filtering transforms a distribution; beam search compares sequences.

A.13.4 52. Inference Memory and Latency Accounting

Separate weights, KV, active tensors/workspaces, runtime pools, and reserve; KV token bytes are $2LHkv d_h b$. TTFT includes the declared admission-to-first-emission components; TPOT states whether the first token is excluded. OOM below nominal capacity can arise from fragmentation, peak temporaries, allocator reservation, concurrent requests, or non-model processes.

A.14 Part XIV — Efficient Inference

A.14.1 53. FlashAttention and Paged KV Memory

A BF16 score tensor $[1, 32, 8192, 8192]$ uses $1 \times 32 \times 8192^2 \times 2 = 4$ GiB. Online-softmax block merges track row maximum m , normalizer l , and weighted accumulator, rescaling old and new terms by $\exp(m_{old} - m_{new})$. Page waste for length S and page size P is $\text{ceil}(S/P)P - S$ slots. FlashAttention remains exact attention and still has quadratic arithmetic.

A.14.2 54. Caching, Batching and Prefill Scheduling

Use the same arrival trace for every policy and report TTFT, TPOT, completion, throughput, queue depth, cache reuse, and peak pages. Continuous batching can raise throughput yet worsen tail latency when long prefills or admission pressure delay decode. Prefix equality must include tokens, model/adaptor, positions, and relevant execution semantics—not a hash alone.

A.14.3 55. Speculative Decoding and Dynamic Execution

Exact speculative sampling must match direct target-model output frequencies statistically; greedy mode must match token-for-token. Report accepted tokens per verification pass plus end-to-end latency and added draft weight/KV memory. A valid negative conclusion is no speedup at matched quality and production concurrency, even when acceptance rate looks high.

A.14.4 56. Quantization Across the Inference Path

Per-row quantization typically lowers error for rows with different ranges but spends more scale metadata than per-tensor quantization. Physical bytes include packed values, scales, zeros, padding, and temporary buffers. A passing comparison pins group size, calibration data, kernels, prompts, and evaluates perplexity, task quality, long context, latency, and residency.

A.15 Part XV — Distributed Execution

A.15.1 57. Parallel Training Strategies

Let parameter, gradient, and optimizer-state bytes be P_b, G_b, O_b . DDP stores all three per rank. Ideal ZeRO stages shard optimizer only, then optimizer+gradients, then optimizer+gradients+parameters by N ; add unsharded activations and a 15% runtime reserve. Gather windows can still OOM because peak live parameters overlap activations, workspaces, and communication buffers.

A.15.2 58. Partitioning Model Computation

A valid placement uses two 16-layer pipeline stages and four tensor-parallel ranks per stage, consuming eight devices. A BF16 boundary $[B, T, D] = [4, 1024, 4096]$ is 32 MiB. A residual-sized collective moves the same logical 32 MiB tensor (algorithmic traffic differs). Tensor collectives recur per sharded layer; the pipeline transfer occurs at the stage boundary per microbatch/direction.

A.15.3 59. Collectives and Interconnects

For a ring all-reduce of message M on N ranks, the common ideal model is $2(N-1)\alpha + 2(N-1)M/(N \cdot BW)$. Substitute 16 MiB, 256 MiB, and 1 GiB with explicitly stated α and effective bandwidth for intra- and inter-node links. Datasheet bandwidth overpredicts performance because protocol, topology, contention, chunking, and synchronization reduce usable bandwidth.

A.15.4 60. Distributed Execution and Failure Recovery

With two ranks holding scalars 1 and 2, an all-reduce sum must yield 3 on both. Killing a rank before the next collective should produce a bounded timeout/error rather than an indefinite hang. The GPU gate separately checks NCCL topology, numerical parity, peak memory, overlap, throughput, and recovery; a CPU Gloo pass does not validate those properties.

A.16 Part XVI — Serving

A.16.1 61. Anatomy of an Inference Server

The simulator must model arrivals, token budgets, paged KV allocation/reference counts, cancellation, and bounded output queues. Its strongest invariant is zero leaked pages after all requests finish or cancel. Compare FIFO and continuous batching on identical traces; report both TTFT and completion because throughput improvements can redistribute waiting time.

A.16.2 62. Tenants, Models and Residency

Two 14 GiB models plus 2 GiB workspace each already consume 32 GiB; adding 6 GiB KV cannot fit a 24 GiB GPU. Keep at most one resident and budget workspace/KV explicitly, or reduce/offload those demands. Cold transition lower bounds are bytes divided by PCIe/SSD bandwidth; alternating-model arrivals reveal eviction thrash and SLO misses.

A.16.3 63. Disaggregation and Runtime Comparisons

Pin exact runtime releases and compare supported hardware, artifacts, scheduler, KV layout, quantization, compilation, and fallbacks. KV transfer bytes use `2LBHkvTd_hb`; transfer time is bytes/effective bandwidth plus latency. Disaggregation wins only when saved prefill/decode time and scheduling benefit exceed transfer and coordination overhead at the measured context/concurrency.

A.16.4 64. Production Serving and GPU Clusters

An open-loop ramp should show queueing rise once arrival rate exceeds sustainable service. Report queue delay, TTFT, TPOT, completion, token throughput, RPS, percentiles, errors, and cancellations without dropping failed requests. A worker-kill test passes only if detection, routing removal, capacity loss, recovery, and user-visible failures/latency are timestamped.

A.17 Part XVII — Local Inference

A.17.1 65. CPU and Edge Inference

Sweep pinned thread counts after warmup and report prompt/decode rates, p95 token latency, memory, bandwidth counters, power/temperature, and host responsiveness. Saturation is where more threads no longer improve useful bytes/s or tokens/s because memory bandwidth, cache capacity, thermals, or scheduling becomes limiting.

A.17.2 66. Hybrid Memory and Local Loading

On 8 GiB VRAM, a 15% reserve leaves 6.8 GiB before accounting for other processes. A 6 GiB model leaves only 0.8 GiB for workspace and KV, so placement must offload layers or shrink context/workspace unless their measured sum fits. Cold-vs-warm readiness differences primarily expose storage/page-cache transfer; first-token time additionally includes tokenization, transfer, and prefill.

A.17.3 67. Local Runtimes and Accelerators

A passing operation-placement table covers one prefill and five decode steps, naming each accelerator kernel and every host-device copy or CPU fallback. The deliberately unsupported case must record explicit error or fallback behavior. Numerical output must match a trusted reference within a dtype-appropriate tolerance; fast but silently different output fails.

A.17.4 68. Packaging, Power and Thermal Limits

The offline package includes hashes, config, licenses, diagnostics, rollback, clean uninstall, and recovery tests for corruption and interrupted installation. A 30-minute run must report startup, first token, steady decode, peak memory, package power, temperature, clocks, and latency. Compare power modes only after thermal steady state; initial boost is not sustained performance.

A.18 Part XVIII — Retrieval and Agents

A.18.1 69. Embeddings, Contrastive Learning and Vector Indexes

Compare approximate results against exact top-10 neighbors to compute recall@10, alongside p50/p95 latency, index memory, and build time over search breadth. For unit-normalized vectors, $\|x-y\|^2 = 2 - 2x \cdot y$, so cosine and Euclidean rankings agree; without normalization, vector norms can reorder results.

A.18.2 70. RAG and Retrieval Quality

Evaluate at least 30 answerable and 10 unanswerable queries. Retrieval recall, answer correctness, citation entailment/support, abstention, and latency are distinct metrics. Diagnose failures at ingestion, retrieval, context packing, or generation; specificity and presence of a citation do not establish grounding.

A.18.3 71. Memory, Tools and Structured Output

Constrained decoding improves syntax, schema validation checks structure/types, authorization checks permission, and execution enforces real-world semantics—none substitutes for the others. The write tool needs an idempotency key; persistent facts need provenance and expiry. Malformed JSON, unauthorized args, duplicate writes, poisoned tool output, failures, and corrections must all have tested outcomes.

A.18.4 72. Agents and Multi-Model Economics

Compare the router against strong-model-only under matched workload and quality, reporting task success, cost, latency percentiles, calls, escalation, and recovery. Bounded retry and abstention prevent unbounded cost. A planner or critic is justified only when its removal reduces marginal value beyond uncertainty while its added latency/cost is included.

A.19 Part XIX — Interpretability and Composition

A.19.1 73. Hidden States, Layers and Attention Heads

Record exact hook points and shapes at three layers. A logit lens maps states to vocabulary space; a probe predicts labels from states; ablation intervenes. If a probe is accurate but ablating/patching the signal does not selectively change behavior, the evidence is correlational. Attention weights alone omit value content and downstream computation.

A.19.2 74. Features, Probes and Mechanisms

Use disjoint clean/corrupted pairs, held-out evaluation, shuffled-label control, and a non-target behavior. A probe establishes decodability; activation patching or selective ablation can support causal involvement. If an intervention degrades all outputs, it shows nonspecific damage rather than a mechanism for the target behavior.

A.19.3 75. Steering, Editing and Representation Engineering

Build the contrastive direction only on discovery data, select strength on validation, and report final effects on test data. Random and reversed directions test specificity; at least two side-effect metrics bound collateral change. A weight edit must be checked on exact, paraphrased, neighboring, and unrelated prompts before making a locality or generalization claim.

A.19.4 76. Merging, Stitching and Representation Compatibility

Compare frozen same-model and cross-model stitches with linear, low-rank, and nonlinear bridges on held-out reconstruction and task accuracy, plus parameters, FLOPs, transfer bytes, and latency. Success only with a large nonlinear connector shows that the connector learned a translation; it does not establish that the original spaces were naturally aligned.

A.20 Part XX — Research Frontier**A.20.1 77. Efficient Memory and Adaptive Computation**

Compare recent-window and score-based KV eviction with no eviction across context lengths on perplexity, needle retrieval, latency, and measured cache bytes. Evaluate fixed and dynamic layer skipping separately and include batching divergence. A policy passes only at matched quality on ordinary and adversarial long-context cases, not merely lower memory.

A.20.2 78. Routable Intelligence and Neural Handoffs

The preregistration fixes hypothesis, baselines, budget, metrics, uncertainty, stopping/negative criteria, and artifact plan before the pilot. A dead-call ablation determines whether the handoff itself adds value beyond orchestration. If text exchange or whole-request routing wins, that is the correct result, not a failed experiment.

A.20.3 79. Research Design and Reproduction

A complete package contains a dated search log, prior-art matrix, claim-evidence links, pinned reproduction plan, preregistration, and negative-result criterion. Reproduce one central result with uncertainty, or document the precise blocker and attempted resolutions. “Runs on the author’s machine” fails until a clean environment can execute it.

A.20.4 80. Evidence, Ablations and Publication

The submission gate requires a claim-evidence table, ablation grid, paired uncertainty analysis, full resource profile, limitations, source-type tags, and a reproducibility manifest. The supported paragraph must stay within observed tasks, budgets, and intervals; the stronger rejected paragraph should expose the prohibited extrapolation. A clean environment must regenerate the main table.

Appendix B

Working glossary

Term	Meaning used in this book
Token	One element of a tokenizer-defined sequence, not necessarily a word
Token ID	Integer vocabulary index
Embedding	Vector representation; distinguish lookup embeddings from contextual states
Parameter	Learned persistent model value
Activation	Intermediate value computed for an input
Logit	Unnormalized score before probability normalization
Gradient	Derivative used to guide a parameter update
Prefill	Processing input context to prepare generation
Decode	Successive autoregressive generation steps
KV cache	Reused attention keys and values for previously processed positions
B / T / D	Batch size / token sequence length / hidden width
H / Hkv / Dh	Query head count / KV head count / per-head dimension
FLOP	Floating-point operation; this book counts multiply-add as two unless stated
MiB / GiB	2^{20} / 2^{30} bytes
TTFT	Time to first output token; specify whether queue and network time are included
TPOT	Time per output token; specify aggregation and whether prefill is excluded
SLO	Operational target for a measured service outcome

Expand this glossary as chapters are drafted; definitions here are introductory, not substitutes for the detailed chapters.

Appendix C

Consolidated chapter references

Generated from the canonical chapter reference sections. Entries remain grouped by chapter because repeated sources may support different claims. Software documentation is version-sensitive; access dates in the chapters are authoritative.

C.1 1. The LLM in One Picture

- Vaswani et al., [Attention Is All You Need](#), 2017 — Transformer architecture and scaled dot-product attention.
- Sutskever, Vinyals, and Le, [Sequence to Sequence Learning with Neural Networks](#), 2014 — autoregressive sequence factorization and decoding.

C.2 2. Training vs Inference

- [PyTorch autograd mechanics](#) — dynamic graphs, saved tensors, grad modes, and the distinction between evaluation mode and gradient control. Accessed 2026-09-18.

C.3 3. Anatomy of the Modern LLM Stack

- Kwon et al., [Efficient Memory Management for Large Language Model Serving with PagedAttention](#), 2023 — KV-cache paging and serving throughput.
- Ansel et al., [PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation](#), 2024 — capture, graph breaks, and compilation in PyTorch 2.
- Williams, Waterman, and Patterson, [Roofline: An Insightful Visual Performance Model for Multicore Architectures](#), 2009 — relating arithmetic throughput to memory traffic.

C.4 4. Building Our Tiny Laboratory

- [Python virtual environments](#) — creation, isolation, activation, and portability constraints. Accessed 2026-09-18.
- [PyTorch autograd mechanics](#) — graph recording and grad-mode semantics. Accessed 2026-09-18.

C.5 5. Tensors and Resource Accounting

- [NumPy documentation](#) — multidimensional arrays, indexing, vectorization, and broadcasting. Accessed 2026-09-18.

C.6 6. Geometry and Number Formats

- IEEE, [IEEE Standard for Floating-Point Arithmetic \(IEEE 754-2019\)](#), 2019.
- Micikevicius et al., [Mixed Precision Training](#), 2017.

C.7 7. Probability, Logits and Loss

- Shannon, [A Mathematical Theory of Communication](#), 1948 — entropy and information measures.
- Goodfellow, Bengio, and Courville, [Deep Learning, Chapter 3](#), 2016 — probability, cross-entropy, and KL divergence.

C.8 8. Gradients, Graphs and Autodiff

- [PyTorch autograd mechanics](#) — dynamic graph construction, saved tensors, reverse-mode behavior, and grad controls. Accessed 2026-09-18.

C.9 9. Linear Layers and Nonlinearities

- [PyTorch Linear](#) and [activation functions](#). Accessed 2026-09-19.
- Hendrycks and Gimpel, [Gaussian Error Linear Units](#), 2016.

C.10 10. A Tiny Network, Forward and Backward

- [PyTorch autograd mechanics](#). Accessed 2026-09-19.

C.11 11. Optimization and State

- Kingma and Ba, [Adam](#), 2014.
- Loshchilov and Hutter, [Decoupled Weight Decay Regularization](#), 2017.
- [PyTorch optimizer documentation](#). Accessed 2026-09-19.

C.12 12. Initialization and Training Stability

- Glorot and Bengio, [Understanding the difficulty of training deep feedforward neural networks](#), 2010.
- He et al., [Delving Deep into Rectifiers](#), 2015.
- [PyTorch gradient clipping](#). Accessed 2026-09-19.

C.13 13. Tokenization Algorithms

- Sennrich, Haddow, and Birch, [Neural Machine Translation of Rare Words with Subword Units](#), 2016.
- Kudo and Richardson, [SentencePiece](#), 2018.
- Schuster and Nakajima, [Japanese and Korean Voice Search](#), 2012.

C.14 14. Vocabulary and Token IDs

- [Hugging Face tokenizer summary](#) and [special tokens](#). Accessed 2026-09-19.

C.15 15. Embedding Matrices and Geometry

- [PyTorch Embedding](#). Accessed 2026-09-19.
- Mikolov et al., [Efficient Estimation of Word Representations in Vector Space](#), 2013.
- Peters et al., [Deep contextualized word representations](#), 2018.

C.16 16. Position and Sequence Order

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Shaw, Uszkoreit, and Vaswani, [Self-Attention with Relative Position Representations](#), 2018.
- Su et al., [RoFormer](#), 2021.

C.17 17. Sequence Modeling and the Attention Idea

- Elman, [Finding Structure in Time](#), 1990.
- Hochreiter and Schmidhuber, [Long Short-Term Memory](#), 1997.
- Bahdanau, Cho, and Bengio, [Neural Machine Translation by Jointly Learning to Align and Translate](#), 2014.

C.18 18. Queries, Keys, Values and Scaling

- Vaswani et al., [Attention Is All You Need](#), 2017.

C.19 19. Causal and Multi-Head Attention

- Vaswani et al., [Attention Is All You Need](#), 2017.
- [PyTorch scaled dot-product attention](#). Accessed 2026-09-19.

C.20 20. Attention Shapes and Compute Cost

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Dao et al., [FlashAttention](#), 2022.

C.21 21. Residual Streams and Normalization

- He et al., [Deep Residual Learning for Image Recognition](#), 2015.
- Ba, Kiros, and Hinton, [Layer Normalization](#), 2016.
- Zhang and Sennrich, [Root Mean Square Layer Normalization](#), 2019.
- Vaswani et al., [Attention Is All You Need](#), 2017.

C.22 22. Feed-Forward Networks and Complete Blocks

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Hendrycks and Gimpel, [Gaussian Error Linear Units](#), 2016.
- Shazeer, [GLU Variants Improve Transformer](#), 2020.

C.23 23. Stacking, Vocabulary Projection and Tying

- Press and Wolf, [Using the Output Embedding to Improve Language Models](#), 2016.
- Inan, Khosravi, and Socher, [Tying Word Vectors and Word Classifiers](#), 2016.

C.24 24. Build Our GPT From Scratch

- Radford et al., [Improving Language Understanding by Generative Pre-Training](#), 2018.
- Vaswani et al., [Attention Is All You Need](#), 2017.
- PyTorch documentation for `nn.Module`, pinned by the environment record for executable results.

C.25 25. Sequences, Batches and Pretraining

- Vaswani et al., [Attention Is All You Need](#), 2017.
- Brown et al., [Language Models are Few-Shot Learners](#), 2020.

C.26 26. The Training Step and Schedule

- Kingma and Ba, [Adam: A Method for Stochastic Optimization](#), 2014.
- Loshchilov and Hutter, [Decoupled Weight Decay Regularization](#), 2017.
- Pascanu, Mikolov, and Bengio, [On the Difficulty of Training Recurrent Neural Networks](#), 2013.

C.27 27. Mixed Precision and Checkpoint Anatomy

- Micikevicius et al., [Mixed Precision Training](#), 2017.
- Chen et al., [Training Deep Nets with Sublinear Memory Cost](#), 2016.
- PyTorch reproducibility and automatic mixed precision documentation, version pinned for execution.

C.28 28. Debugging and Training Our Tiny Model

- Goodfellow, Bengio, and Courville, [Deep Learning](#), optimization chapters, 2016.

- PyTorch autograd anomaly detection and reproducibility documentation, version pinned for execution.

C.29 29. Transformer Families

- Devlin et al., [BERT](#), 2018.
- Raffel et al., [Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer](#), 2019.
- Touvron et al., [LLaMA](#), 2023.

C.30 30. Modern Positions, Heads and MLPs

- Su et al., [RoFormer: Enhanced Transformer with Rotary Position Embedding](#), 2021.
- Shazeer, [Fast Transformer Decoding: One Write-Head is All You Need](#), 2019.
- Ainslie et al., [GQA: Training Generalized Multi-Query Transformer Models](#), 2023.

C.31 31. Mixture of Experts

- Shazeer et al., [Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer](#), 2017.
- Fedus, Zoph, and Shazeer, [Switch Transformers](#), 2021.
- Lepikhin et al., [GShard](#), 2020.

C.32 32. Alternative Architectures and Real Configs

- Beltagy, Peters, and Cohan, [Longformer](#), 2020.
- Gu and Dao, [Mamba: Linear-Time Sequence Modeling with Selective State Spaces](#), 2023.
- Touvron et al., [Llama 2](#), 2023.

C.33 33. Data Sources and Cleaning

- Lee et al., [Deduplicating Training Data Makes Language Models Better](#), 2021.
- Dodge et al., [Documenting Large Webtext Corpora](#), 2021.
- Penedo et al., [The RefinedWeb Dataset for Falcon LLM](#), 2023.

C.34 34. Mixtures, Tokens and Synthetic Data

- Gao et al., [The Pile](#), 2020.
- Longpre et al., [A Pretrainer’s Guide to Training Data](#), 2023.
- Carlini et al., [Quantifying Memorization Across Neural Language Models](#), 2022.

C.35 35. Evaluation and Meaningful Experiments

- Liang et al., [Holistic Evaluation of Language Models](#), 2022.
- Dror et al., [Testing Statistical Significance in NLP](#), 2018.
- Mitchell et al., [Model Cards for Model Reporting](#), 2018.

C.36 36. Scaling Laws

- Kaplan et al., [Scaling Laws for Neural Language Models](#), 2020.
- Hoffmann et al., [Training Compute-Optimal Large Language Models](#), 2022.
- Bian et al., [Scaling Inference-Efficient Language Models](#), 2025.

C.37 37. Pretraining, Continued Pretraining and SFT

- Ouyang et al., [Training Language Models to Follow Instructions with Human Feedback](#), 2022.
- Wei et al., [Finetuned Language Models Are Zero-Shot Learners](#), 2021.
- Hugging Face, [Chat templates documentation](#) (version-sensitive; accessed September 2026).

C.38 38. Parameter-Efficient Adaptation

- Houlshby et al., [Parameter-Efficient Transfer Learning for NLP](#), 2019.
- Li and Liang, [Prefix-Tuning](#), 2021.
- Hu et al., [LoRA](#), 2021.
- Dettmers et al., [QLoRA](#), 2023.

C.39 39. Preferences and Reinforcement Learning

- Christiano et al., [Deep Reinforcement Learning from Human Preferences](#), 2017.
- Schulman et al., [Proximal Policy Optimization Algorithms](#), 2017.
- Ouyang et al., [Training Language Models to Follow Instructions with Human Feedback](#), 2022.
- Rafailov et al., [Direct Preference Optimization](#), 2023.

C.40 40. Reasoning, Test-Time Compute and Distillation

- Wang et al., [Self-Consistency Improves Chain of Thought Reasoning](#), 2022.
- Hinton et al., [Distilling the Knowledge in a Neural Network](#), 2015.
- Cobbe et al., [Training Verifiers to Solve Math Word Problems](#), 2021.
- Gao et al., [On Designing Effective RL Reward at Training Time for LLM Reasoning](#), 2024.

C.41 41. Why GPUs and How They Execute

- NVIDIA, [CUDA C++ Programming Guide](#) (version-sensitive; accessed September 2026).
- NVIDIA, [GPU Performance Background User's Guide](#) (accessed September 2026).

C.42 42. Memory Hierarchy and Movement

- NVIDIA, [CUDA C++ Best Practices Guide](#) (version-sensitive; accessed September 2026).
- NVIDIA, [CUDA C++ Programming Guide: Memory Hierarchy](#) (accessed September 2026).

C.43 43. Throughput, Intensity and Rooflines

- Williams, Waterman, and Patterson, [Roofline: An Insightful Visual Performance Model](#), 2009.
- NVIDIA, [Nsight Compute Roofline Analysis](#) (version-sensitive; accessed September 2026).

C.44 44. Profiling Transformer Bottlenecks

- NVIDIA, [Nsight Systems User Guide](#) (version-sensitive; accessed September 2026).
- NVIDIA, [Nsight Compute Profiling Guide](#) (version-sensitive; accessed September 2026).
- PyTorch, [Profiler documentation](#) (version-sensitive; accessed September 2026).

C.45 45. Kernels, GEMM and Tiling

- NVIDIA, [CUDA C++ Programming Guide](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [Parallel Thread Execution ISA](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [CUTLASS documentation](#), version-sensitive; accessed 2026-09-19. Pin GPU architecture, toolkit, driver, library commit, dtype, and shapes.

C.46 46. CUDA, Triton and CUTLASS

- NVIDIA, [CUDA C++ Programming Guide](#), version-sensitive; accessed 2026-09-19.
- OpenAI, [Triton tutorials](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [CUTLASS documentation](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [Nsight Compute documentation](#), version-sensitive; accessed 2026-09-19. Pin Python, PyTorch, Triton, toolkit, driver, GPU, and commits.

C.47 47. Graphs, Compilers and Runtimes

- PyTorch, [TorchDynamo overview](#), version-sensitive; accessed 2026-09-19.
- PyTorch, [torch.compile programming model](#), version-sensitive; accessed 2026-09-19.
- PyTorch, [FX documentation](#), version-sensitive; accessed 2026-09-19.
- PyTorch, [TorchInductor documentation](#), version-sensitive; accessed 2026-09-19. Pin PyTorch release or commit, backend, flags, CUDA/Triton, model revision, and inputs.

C.48 48. CUDA Graphs and Execution Overhead

- NVIDIA, [CUDA C++ Programming Guide: CUDA Graphs](#), version-sensitive; accessed 2026-09-19.
- NVIDIA, [CUDA Runtime API: Graph Management](#), version-sensitive; accessed 2026-09-19.
- PyTorch, [CUDA semantics: CUDA Graphs](#), version-sensitive; accessed 2026-09-19. Pin driver, toolkit, PyTorch, allocator, graph mode, and shapes.

C.49 49. Load Weights and Trace a Forward Pass

- PyTorch, [inference_mode](#), version-sensitive; accessed 2026-09-19.
- Hugging Face, [Transformers model documentation](#), version-sensitive; accessed 2026-09-19.
- Hugging Face, [Transformers model loading](#), version-sensitive; accessed 2026-09-19. Pin framework versions, model commit, tokenizer commit, dtype, attention backend, and hardware.

C.50 50. Prefill, Decode and KV Cache

- Vaswani et al., [Attention Is All You Need](#), 2017.

- Ainslie et al., [GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints](#), 2023.
- Hugging Face, [Caching](#), version-sensitive; accessed 2026-09-19. Pin model code and revision because cache structure and position handling are architecture-specific.

C.51 51. Sampling and Decoding Policies

- Holtzman et al., [The Curious Case of Neural Text Degeneration](#), 2020.
- Hugging Face, [Generation API](#), version-sensitive; accessed 2026-09-19.
- Hugging Face, [Generation strategies](#), version-sensitive; accessed 2026-09-19. Pin Transformers, model and tokenizer revisions, generation config, seed, device, and numeric mode.

C.52 52. Inference Memory and Latency Accounting

- MLCommons, [MLPerf Inference: Datacenter benchmark](#), version-sensitive; accessed 2026-09-19.
- Kwon et al., [Efficient Memory Management for Large Language Model Serving with PagedAttention](#), 2023.
- vLLM, [official documentation](#), version-sensitive; accessed 2026-09-19. Pin engine commit and configuration, model revision, hardware clocks and power, workload, tokenizer, and measurement boundary.

C.53 53. FlashAttention and Paged KV Memory

- Dao et al., [FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#), 2022.
- Dao, [FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning](#), 2023.
- Kwon et al., [Efficient Memory Management for Large Language Model Serving with PagedAttention](#), 2023.
- vLLM, [official documentation](#), version-sensitive; accessed 2026-09-19. Pin kernel version, attention backend, dtype, mask, model, block size, GPU, and workload.

C.54 54. Caching, Batching and Prefill Scheduling

- Yu et al., [Orca: A Distributed Serving System for Transformer-Based Generative Models](#), 2022.
- Kwon et al., [Efficient Memory Management for Large Language Model Serving with PagedAttention](#), 2023.
- Agrawal et al., [Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve](#), 2024.
- vLLM, [Automatic Prefix Caching](#), version-sensitive; accessed 2026-09-19. Pin engine commit and configuration, block size, arrival trace, model revision, and SLO.

C.55 55. Speculative Decoding and Dynamic Execution

- Leviathan, Kalman, and Matias, [Fast Inference from Transformers via Speculative Decoding](#), 2023.
- Chen et al., [Accelerating Large Language Model Decoding with Speculative Sampling](#), 2023.

- NVIDIA, [CUDA C++ Programming Guide: CUDA Graphs](#), version-sensitive; accessed 2026-09-19. Pin algorithm variant, draft and target revisions, engine commit, draft length, batch and concurrency, prompts, precision, and graph policy.

C.56 56. Quantization Across the Inference Path

- Frantar et al., [GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers](#), 2022.
- Lin et al., [AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration](#), 2023.
- Xiao et al., [SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models](#), 2022.
- Open Compute Project, [8-bit Floating Point Specification](#), revision 1.0, 2023; and [ggml-org, llama.cpp source and GGUF implementation](#), version-sensitive; accessed 2026-09-19. Pin method implementation, commit, model and calibration data, quantization type and group size, hardware, and kernels.

C.57 57. Parallel Training Strategies

- [PyTorch FSDP2 tutorial](#), accessed 2026-09-19.
- Rajbhandari et al., [ZeRO](#), 2020.
- [PyTorch DistributedDataParallel API](#).

C.58 58. Partitioning Model Computation

- Shoenberger et al., [Megatron-LM](#), 2019.
- Huang et al., [GPipe](#), 2019.
- Lepikhin et al., [GShard](#), 2020.

C.59 59. Collectives and Interconnects

- [NCCL collective operations](#), accessed 2026-09-19.
- [NVIDIA GPUDirect RDMA](#), accessed 2026-09-19.
- Patarasuk and Yuan, [Bandwidth Optimal All-reduce](#), 2009.

C.60 60. Distributed Execution and Failure Recovery

- [PyTorch distributed overview](#), accessed 2026-09-19.
- [PyTorch Distributed Checkpoint](#), accessed 2026-09-19.
- [NCCL troubleshooting](#), accessed 2026-09-19.

C.61 61. Anatomy of an Inference Server

- [vLLM architecture documentation](#), accessed 2026-09-19.
- Kwon et al., [Efficient Memory Management for LLM Serving with PagedAttention](#), 2023.

C.62 62. Tenants, Models and Residency

- [NVIDIA Triton model management](#), accessed 2026-09-19.
- [CUDA Best Practices: data transfer](#), accessed 2026-09-19.

C.63 63. Disaggregation and Runtime Comparisons

- [vLLM documentation](#), accessed 2026-09-19.
- Kwon et al., [Efficient Memory Management for Large Language Model Serving with Page-dAttention](#), 2023.
- [TensorRT-LLM documentation](#), accessed 2026-09-19.
- [llama.cpp repository](#), accessed 2026-09-19.

C.64 64. Production Serving and GPU Clusters

- [OpenTelemetry specification](#), accessed 2026-09-19.
- [Kubernetes device plugins](#), accessed 2026-09-19.
- [NVIDIA DCGM documentation](#), accessed 2026-09-19.

C.65 65. CPU and Edge Inference

- [llama.cpp performance documentation and source](#), accessed 2026-09-19.
- [Intel Intrinsics Guide](#), accessed 2026-09-19.
- [Arm ACLE](#), accessed 2026-09-19.

C.66 66. Hybrid Memory and Local Loading

- [CUDA Unified Memory programming guide](#), accessed 2026-09-19.
- [GGUF specification](#), accessed 2026-09-19.
- [Linux mmap documentation](#).

C.67 67. Local Runtimes and Accelerators

- [llama.cpp repository](#), accessed 2026-09-19.
- [OpenVINO documentation](#), accessed 2026-09-19.
- [OpenVINO GenAI repository](#), accessed 2026-09-19.

C.68 68. Packaging, Power and Thermal Limits

- [NIST Secure Software Development Framework](#), accessed 2026-09-19.
- Hinton et al., [Distilling the Knowledge in a Neural Network](#), 2015.
- [SPDX specification](#), accessed 2026-09-19.

C.69 69. Embeddings, Contrastive Learning and Vector Indexes

- Reimers and Gurevych, [Sentence-BERT](#), EMNLP-IJCNLP 2019.

- Karpukhin et al., [Dense Passage Retrieval](#), EMNLP 2020.
- Malkov and Yashunin, [Efficient and Robust Approximate Nearest Neighbor Search Using HNSW](#), IEEE TPAMI 2020.
- Jégou, Douze, and Schmid, [Product Quantization for Nearest Neighbor Search](#), IEEE TPAMI 2011.
- Johnson, Douze, and Jégou, [Billion-scale similarity search with GPUs](#), IEEE Transactions on Big Data 2021.

C.70 70. RAG and Retrieval Quality

- Lewis et al., [Retrieval-Augmented Generation](#), NeurIPS 2020.
- Robertson and Zaragoza, [The Probabilistic Relevance Framework: BM25 and Beyond](#), 2009.
- Khattab and Zaharia, [ColBERT](#), SIGIR 2020.
- Thakur et al., [BEIR](#), ICLR 2021.
- Craswell et al., [Overview of the TREC 2020 Deep Learning Track](#), TREC 2020.

C.71 71. Memory, Tools and Structured Output

- Schick et al., [Toolformer](#), NeurIPS 2023.
- Poesia et al., [Synchromesh](#), ICLR 2022.
- [JSON Schema 2020-12 specification](#), accessed 2026-09-19.

C.72 72. Agents and Multi-Model Economics

- Yao et al., [ReAct](#), ICLR 2023.
- Shinn et al., [Reflexion](#), NeurIPS 2023.
- Chen, Zaharia, and Zou, [FrugalGPT](#), 2023.

C.73 73. Hidden States, Layers and Attention Heads

- Elhage et al., [A Mathematical Framework for Transformer Circuits](#), 2021.
- Belinkov, [Probing Classifiers](#), Computational Linguistics 2022.
- Kornblith et al., [Similarity of Neural Network Representations Revisited](#), ICML 2019.
- Olsson et al., [In-context Learning and Induction Heads](#), 2022.

C.74 74. Features, Probes and Mechanisms

- Alain and Bengio, [Understanding Intermediate Layers Using Linear Classifier Probes](#), 2016.
- Hewitt and Liang, [Designing and Interpreting Probes with Control Tasks](#), EMNLP-IJCNLP 2019.
- Meng et al., [Locating and Editing Factual Associations in GPT](#), NeurIPS 2022.
- Bricken et al., [Towards Monosemanticity](#), 2023; empirical research report, not peer-reviewed conference evidence.

C.75 75. Steering, Editing and Representation Engineering

- Turner et al., [Activation Addition](#), 2023.
- Zou et al., [Representation Engineering](#), 2023.
- Meng et al., [ROME](#), NeurIPS 2022.

- Meng et al., [MEMIT](#), ICLR 2023.

C.76 76. Merging, Stitching and Representation Compatibility

- Wortsman et al., [Model Soups](#), ICML 2022.
- Ainsworth, Hayase, and Srinivasa, [Git Re-Basin](#), ICLR 2023.
- Bansal, Nakkiran, and Barak, [Revisiting Model Stitching](#), NeurIPS 2021.
- Ilharco et al., [Editing Models with Task Arithmetic](#), ICLR 2023.

C.77 77. Efficient Memory and Adaptive Computation

- Graves, [Adaptive Computation Time for Recurrent Neural Networks](#), 2016.
- Xin et al., [DeeBERT](#), ACL 2020.
- Xiao et al., [Efficient Streaming Language Models with Attention Sinks](#), ICLR 2024.
- Zhang et al., [H2O: Heavy-Hitter Oracle for Efficient Generative Inference](#), NeurIPS 2023.

C.78 78. Routable Intelligence and Neural Handoffs

- Jacobs et al., [Adaptive Mixtures of Local Experts](#), Neural Computation 1991.
- Teerapittayanon, McDanel, and Kung, [BranchyNet](#), 2017.
- Bansal, Nakkiran, and Barak, [Revisiting Model Stitching](#), NeurIPS 2021.
- Kang et al., [Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge](#), ASPLOS 2017.

C.79 79. Research Design and Reproduction

- Pineau et al., [Improving Reproducibility in Machine Learning Research](#), JMLR 2021.
- Gundersen and Kjensmo, [State of the Art: Reproducibility in Artificial Intelligence](#), AAAI 2018.
- [NeurIPS Paper Checklist](#), current author guidance, accessed 2026-09-19.
- [ACM Artifact Review and Badging](#), accessed 2026-09-19.
- National Academies, [Reproducibility and Replicability in Science](#), 2019.

C.80 80. Evidence, Ablations and Publication

- Dodge et al., [Show Your Work: Improved Reporting of Experimental Results](#), EMNLP-IJCNLP 2019.
- Dror et al., [The Hitchhiker’s Guide to Testing Statistical Significance in NLP](#), ACL 2018.
- Agarwal et al., [Deep Reinforcement Learning at the Edge of the Statistical Precipice](#), NeurIPS 2021.
- Wasserstein, Schirm, and Lazar, [Moving to a World Beyond “ \$p < 0.05\$ ”](#), The American Statistician 2019.
- [arXiv submission help](#), accessed 2026-09-19.
- [NeurIPS Paper Checklist](#), accessed 2026-09-19.